```
# ==== run.py ====
# -*- coding: utf-8 -*-
"""系统启动入口。
运行：
    python run.py                启动并自动打开浏览器
    python run.py --no-browser   只启动服务，不打开浏览器
    python run.py --dev          开发模式（开启代码热重载与调试器）
    python run.py --port 5001    指定端口
默认以 127.0.0.1:5000 启动，就绪后自动打开浏览器。
"""
import os
import sys
import threading
import time
import webbrowser
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")   # 控制台中文正常显示
from app import create_app
from app.models import db
PORT = 5000
for i, a in enumerate(sys.argv):
    if a == "--port" and i + 1 < len(sys.argv):
        PORT = int(sys.argv[i + 1])
URL = "http://127.0.0.1:%d" % PORT
NO_BROWSER = "--no-browser" in sys.argv
DEV = "--dev" in sys.argv
app = create_app()
def open_browser_later(delay=1.2):
    """等服务工作起来再打开浏览器，避免打开时页面还没就绪。
    放在后台线程里，不用 sleep 阻塞服务启动；线程设为守护线程，
    用户 Ctrl+C 停止服务时不会残留。
    """
    def run():
        time.sleep(delay)
        try:
            webbrowser.open(URL)
        except Exception:
            pass            # 打开失败不影响服务本身
    t = threading.Thread(target=run, daemon=True)
    t.start()
if __name__ == "__main__":
    with app.app_context():
        db.create_all()     # 首次运行时自动建表
        from app.utils.schema import ensure_schema
        added = ensure_schema(db)     # 旧库补齐新增字段
        if added:
            print("已补充字段：%s" % "、".join(added))
    print("=" * 58)
    print("   历史遗留废弃矿山自然恢复适宜性评价系统  Ｖ１．０")
    print("=" * 58)
```

```
    print("   访问地址：%s" % URL)
    print("   演示账号：admin      密码：admin123")
    print("   停止运行：按 Ctrl+C，或直接关闭本窗口")
    print("=" * 58)
    if not NO_BROWSER and not DEV:
        print("   正在打开浏览器……")
    print()
    # 开发模式才启用热重载（重载会二次启动进程，故此时不自动开浏览器）
    if not NO_BROWSER and not DEV:
        open_browser_later()
    app.run(host="127.0.0.1", port=PORT, debug=DEV,
    use_reloader=DEV)
# ==== app/__init__.py ====
# -*- coding: utf-8 -*-
"""Flask 应用工厂。"""
from datetime import date
from flask import Flask
from flask_login import LoginManager
from flask_wtf.csrf import CSRFProtect, CSRFError
from app.config import Config
from app.models import db, User
from app.utils.constants import ROLE_LABELS
login_manager = LoginManager()
login_manager.login_view = "auth.login"
login_manager.login_message = "请先登录"
csrf = CSRFProtect()
@login_manager.user_loader
def load_user(user_id):
    return db.session.get(User, int(user_id))
def create_app(config_class=Config):
    app = Flask(__name__)
    app.config.from_object(config_class)
    db.init_app(app)
    login_manager.init_app(app)
    # CSRF 防护：WTF_CSRF_ENABLED 只对 FlaskForm 生效，页面上那些手写 POST
    表单
    # （账号增删改、项目 / 图斑删除等）以前是没校验的；同时模板里的 csrf_token()
    # 也需要它才会存在——少了这一行，保存过一次 AHP 权重后 /ahp/ 直接 500。
    csrf.init_app(app)
    # 模板里展示时间统一走 localdt：库里存 UTC，展示成本地时间
    from app.utils.timefmt import to_local
    app.jinja_env.filters["localdt"] = to_local
    # 注册视图（各模块路由）
    from app.views import auth, project, unit, standard,
    constraint, \
        ahp, indicator, evaluate, report, search
    app.register_blueprint(auth.bp)
    app.register_blueprint(project.bp)
    app.register_blueprint(unit.bp)
    app.register_blueprint(standard.bp)
```

```
app.register_blueprint(constraint.bp)
app.register_blueprint(ahp.bp)
app.register_blueprint(indicator.bp)
app.register_blueprint(evaluate.bp)
app.register_blueprint(report.bp)
app.register_blueprint(search.bp)
#  全局登录保护：除登录页与静态资源外，均需登录
from flask import request, redirect, url_for,
send_from_directory, flash
from flask_login import current_user, login_user,
logout_user
PUBLIC_ENDPOINTS = {"auth.login", "auth.register",
"static", "favicon"}
#  浏览器默认会请求根路径下的 /favicon.ico。
#  页面里虽已用 <link rel="icon"> 指向 static 目录，但出错页等场景仍会
#  回退到这个路径，补一条路由免得日志里一直刷 404。
@app.route("/favicon.ico")
def favicon():
    return send_from_directory(app.static_folder,
"favicon.ico",
                               mimetype="image/vnd.micro
soft.icon")
@app.before_request
def require_login():
    #  截图模式（仅用于生成说明书配图）：自动以管理员身份登录
    if app.config.get("SCREENSHOT_MODE"):
        if (request.endpoint not in ("auth.login",
"auth.register", "static")
                and not current_user.is_authenticated):
            admin = User.query.filter_by(role="admin").f
irst()
            if admin:
                login_user(admin)
        return None
    if request.endpoint in PUBLIC_ENDPOINTS or
request.endpoint is None:
        return None
    if not current_user.is_authenticated:
        return redirect(url_for("auth.login"))
    #  账号被停用后立即失效：不然已经登录的会话还能继续用下去
    if not current_user.active:
        logout_user()
        flash("该账号已停用，请联系管理员", "toast-error")
        return redirect(url_for("auth.login"))
    return None
#  侧边栏导航：按论文评价流程的顺序编排，分为「评价流程」与「系统管理」两组。
#  不使用模块编号，避免出现与实际流程不一致的代号。
#  每项带图标名（内联 SVG，见 templates/_macros.html 的 ICONS）
#  与可选的待办角标（该步骤还没做完的图斑数）。
FLOW_LINKS = [
```

```
        ("项目与矿山档案", "/project/", "folder"),
        ("政策与标准知识库", "/standard/", "book"),
        ("限制性指标判定", "/constraint/", "shield"),
        ("微地质单元", "/unit/", "layers"),
        ("指标体系与权重", "/ahp/", "sliders"),
        ("指标数据采集", "/indicator/", "database"),
        ("适宜性定量评价", "/evaluate/", "target"),
        ("报告与结果输出", "/report/", "file"),
    ]
    #  「用户与操作日志」已并入「个人设置」，不再是独立菜单，
    #  因此系统管理组只留一个入口。
    ADMIN_LINKS = [
        ("个人设置", "/profile", "user"),
    ]
    @app.context_processor
    def inject_nav():
        path = request.path or "/"
        def hit(prefix):
            """当前路径是否属于某个入口（/unit/ 也匹配 /unit/3）。"""
            return path.startswith(prefix.rstrip("/"))
        badges = {}
        if current_user.is_authenticated:
            from app.services import dashboard as dash
            badges = dash.nav_badges()
        def build(links):
            return [{"label": label, "href": href, "icon":
    icon,
                     "active": hit(href), "badge":
    badges.get(href, 0)}
                    for label, href, icon in links]
        groups = [
            {"title": "评价流程", "links": build(FLOW_LINKS)},
            {"title": "系统管理", "links": build(ADMIN_LINKS)},
        ]
        #  面包屑用的当前页名：按路径命中的入口取，未命中时给个通用名
        if path == "/":
            page_title = "工作台"
        elif path.startswith("/profile"):
            page_title = "个人设置"
        elif path.startswith(("/users", "/logs")):
            page_title = "账号与日志"
        elif path.startswith("/search"):
            page_title = "搜索结果"
        else:
            page_title = "工作台"
            for label, href, _icon in FLOW_LINKS:
                if hit(href):
                    page_title = label
                    break
        #  层级引导用：项目是归档单位，图斑才是评价对象。
```

```
    #  下游各页面按图斑组织数据，因此需要提示「哪些项目下还没有图斑」。
    from app.models import Project, MinePatch
    projects = Project.query.order_by(Project.id).all()
    return {
        "nav_groups": groups,
        "home_active": path == "/",
        "page_title": page_title,
        "today": date.today().strftime("%Y-%m-%d"),
        "search_q": request.args.get("q", ""),
        "role_labels": ROLE_LABELS,
        "project_count": len(projects),
        "patch_count": MinePatch.query.count(),
        "projects_without_patch": [p for p in projects
                                   if p.patches.count()
== 0],
    }
#  操作审计：统一在响应阶段记录成功的写操作（操作日志）
from app.services import audit
@app.after_request
def audit_writes(response):
    if request.method in ("POST", "PUT", "PATCH",
"DELETE"):
        audit.record(request.endpoint,
response.status_code)
    return response
#  错误页：给出可读提示，而不是默认的英文错误页
from flask import render_template
@app.errorhandler(403)
def forbidden(e):
    return render_template("error.html", code=403,
                           message="没有权限执行该操作"), 403
@app.errorhandler(404)
def not_found(e):
    return render_template("error.html", code=404,
                           message="请求的页面不存在"), 404
@app.errorhandler(413)
def too_large(e):
    """上传超过体积上限时给中文页；默认是英文的 Request Entity Too Large。"""
    limit = app.config.get("MAX_CONTENT_LENGTH") or 0
    return render_template(
        "error.html", code=413,
        message="上传的文件太大（上限 %d MB），请压缩或拆分后重试"
                % (limit // 1024 // 1024)), 413
@app.errorhandler(500)
def server_error(e):
    """未预料的错误也给中文页；生产模式下 Flask 不会带出堆栈。"""
    return render_template("error.html", code=500,
                           message="系统处理时出现异常，本次操作未生效，"
                                   "请返回重试或联系管理员"), 500
@app.errorhandler(CSRFError)
```

```
    def csrf_error(e):
        """令牌过期或来源不符时给一句人话，并把人送回上一页。
        原来校验一失败就是默认英文 400 页，或者（对手写表单）干脆什么都不发生，
        用户只会觉得「点了没反应」。回跳目标只取本站地址，避免开放重定向。
        """
        from flask import flash, redirect, url_for
        flash("页面停留时间过久或提交来源校验未通过，请重新提交一次", "error")
        referrer = request.referrer or ""
        if referrer.startswith(request.host_url):
            return redirect(referrer)
        return redirect(url_for("auth.index"))
    return app
# ==== app/config.py ====
# -*- coding: utf-8 -*-
"""应用配置。"""
import os
import secrets
from datetime import timedelta
BASE_DIR = os.path.abspath(os.path.dirname(__file__))
def _secret_key():
    """会话签名密钥：优先取环境变量，否则首次运行随机生成并存到本地文件。
    原来是一个写死在代码里的默认值，任何拿到源码的人都能自签会话 cookie。
    单机本地跑风险有限，但改成本机随机值几乎零成本。
    生成的文件已加进 .gitignore，不会进版本库。
    """
    env = os.environ.get("SECRET_KEY")
    if env:
        return env
    path = os.path.join(BASE_DIR, ".secret_key")
    try:
        if os.path.exists(path):
            with open(path, encoding="utf-8") as f:
                saved = f.read().strip()
            if saved:
                return saved
        key = secrets.token_hex(32)
        with open(path, "w", encoding="utf-8") as f:
            f.write(key)
        return key
    except OSError:
        # 目录不可写时退回固定值，保证系统仍能起来（此时会退化为原来的行为）
        return "mine-restoration-dev-key"
class Config:
    SECRET_KEY = _secret_key()
    # 单机部署，SQLite 数据库文件放在应用目录下
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE_DIR, "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    # 禁止跨站请求伪造（CSRF），登录与表单提交依赖此项
    WTF_CSRF_ENABLED = True
```

```
    # 会话 cookie：不交给脚本读取，跨站请求不带
    SESSION_COOKIE_HTTPONLY = True
    SESSION_COOKIE_SAMESITE = "Lax"
    # 模板改动立即生效，不必重启服务。
    # Flask 在非 debug 下默认把模板缓存到进程里，改完模板刷新浏览器还是旧页面，
    # 容易被误当成「改了没生效」。本系统只在本机运行，这点开销可以忽略。
    TEMPLATES_AUTO_RELOAD = True
    # 「记住我」：登录时勾选则下发持久 cookie，关闭浏览器后仍可自动登录
    REMEMBER_COOKIE_DURATION = timedelta(days=30)
    REMEMBER_COOKIE_HTTPONLY = True
    REMEMBER_COOKIE_SAMESITE = "Lax"
    # 影像与数据文件上传（遥感影像挂接、批量导入）
    UPLOAD_FOLDER = os.path.join(BASE_DIR, "static",
    "uploads")
    MAX_CONTENT_LENGTH = 32 * 1024 * 1024          # 单次上传上限
    32 MB
    ALLOWED_IMAGE_EXT = {"png", "jpg", "jpeg", "gif",
    "webp", "bmp", "tif", "tiff"}
    ALLOWED_DATA_EXT = {"csv", "xlsx", "xls"}
    # 头像只用浏览器能直接显示的格式（不含 bmp / tif），并单独限制体积
    ALLOWED_AVATAR_EXT = {"png", "jpg", "jpeg", "gif",
    "webp"}
    MAX_AVATAR_LENGTH = 4 * 1024 * 1024            # 4 MB
# ==== app/models.py ====
# -*- coding: utf-8 -*-
"""数据模型：系统全部数据表。
本文件由甲在阶段 0 一次性写完，作为两人独立开发的地基。
约定：两人只读此文件；确需改表时，由乙提需求、甲统一修改。
"""
import json
from flask_login import UserMixin
from flask_sqlalchemy import SQLAlchemy
from app.utils.timefmt import utcnow    # 替代已弃用的
    datetime.utcnow()
db = SQLAlchemy()
class User(UserMixin, db.Model):
    """用户（账户与权限）。"""
    __tablename__ = "users"
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(64), unique=True,
    nullable=False, index=True)
    password_hash = db.Column(db.String(256),
    nullable=False)
    # 职务只有两种：管理员 admin（可管理账号与他人的职务） / 操作员 operator
    role = db.Column(db.String(20), nullable=False,
    default="operator")
    active = db.Column(db.Boolean, nullable=False,
    default=True)
    created_at = db.Column(db.DateTime, default=utcnow)
    # ---- 个人资料（用户可在「个人设置」里自行维护）----
```

```
    display_name = db.Column(db.String(64), nullable=True)
    # 昵称
    email = db.Column(db.String(128), nullable=True)
    phone = db.Column(db.String(32), nullable=True)
    gender = db.Column(db.String(8), nullable=True)
    # 男 / 女 / 保密
    avatar = db.Column(db.String(256), nullable=True)
    # 头像，相对 static 的路径
    @property
    def is_admin(self):
        return self.role == "admin"
    @property
    def shown_name(self):
        """界面上显示的名字：优先用昵称，没填就用账号名。"""
        return self.display_name or self.username
    @property
    def initial(self):
        """没有头像时用首字占位。"""
        return (self.shown_name or "?")[0].upper()
    def to_dict(self):
        return {"id": self.id, "username": self.username,
                "display_name": self.display_name,
    "gender": self.gender,
                "role": self.role, "active": self.active}
class OperationLog(db.Model):
    """操作日志。"""
    __tablename__ = "operation_logs"
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id"
    ), nullable=True)
    action = db.Column(db.String(128), nullable=False)
    # 操作类型
    target_type = db.Column(db.String(64), nullable=True)
    # 对象类型
    target_id = db.Column(db.Integer, nullable=True)
    # 对象主键
    detail = db.Column(db.Text, nullable=True)
    # 操作详情
    created_at = db.Column(db.DateTime, default=utcnow,
    index=True)
class Project(db.Model):
    """评价项目（项目与矿山档案）。"""
    __tablename__ = "projects"
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(128), nullable=False)
    code = db.Column(db.String(64), nullable=True)
    # 项目编号
    description = db.Column(db.Text, nullable=True)
    status = db.Column(db.String(20), default="进行中")
    # 进行中 / 已完成 / 已归档
```

```
    created_by = db.Column(db.Integer,                                  401
    db.ForeignKey("users.id"), nullable=True)                           402
    created_at = db.Column(db.DateTime, default=utcnow)                 403
    patches = db.relationship("MinePatch",                              404
    backref="project", lazy="dynamic",                                  405
                                    cascade="all, delete-orphan")       406
class MinePatch(db.Model):                                              407
    """矿山图斑。"""                                                    408
    __tablename__ = "mine_patches"                                      409
    id = db.Column(db.Integer, primary_key=True)                        410
    project_id = db.Column(db.Integer,                                  411
    db.ForeignKey("projects.id"), nullable=False)                       412
    name = db.Column(db.String(128), nullable=False)                    413
    admin_region = db.Column(db.String(128), nullable=True)             414
    # 所在行政区                                                        415
    center_x = db.Column(db.Float, nullable=True)                       416
    # 中心经度 / 横坐标                                                 417
    center_y = db.Column(db.Float, nullable=True)                       418
    # 中心纬度 / 纵坐标                                                 419
    area_ha = db.Column(db.Float, nullable=True)                        420
    # 图斑面积（公顷）                                                  421
    mineral_type = db.Column(db.String(64), nullable=True)              422
    # 开采矿种                                                          423
    mining_method = db.Column(db.String(64), nullable=True)             424
    # 开采方式                                                          425
    stop_years = db.Column(db.Float, nullable=True)                     426
    # 停产年限                                                          427
    responsible_entity_missing = db.Column(db.Boolean,                  428
    default=True)   # 责任主体是否缺失                                 429
    # 遥感底图 / 无人机正射影像挂接                                     430
    image_path = db.Column(db.String(256), nullable=True)               431
    # 相对 static 的路径                                                432
    image_note = db.Column(db.String(256), nullable=True)               433
    # 影像说明（如来源、时相）                                          434
    image_uploaded_at = db.Column(db.DateTime,                          435
    nullable=True)                                                      436
    created_at = db.Column(db.DateTime, default=utcnow)                 437
    units = db.relationship("MicroUnit", backref="patch",               438
    lazy="dynamic",                                                     439
                                    cascade="all, delete-orphan")       440
class ReferenceEcosystem(db.Model):                                     441
    """参照生态系统：标准化基准值，与图斑绑定。"""                     442
    __tablename__ = "reference_ecosystems"                              443
    id = db.Column(db.Integer, primary_key=True)                        444
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat            445
    ches.id"), nullable=False)                                          446
    coverage_benchmark = db.Column(db.Float, default=75.0)              447
    # 植被覆盖度基准 %                                                  448
    shannon_benchmark = db.Column(db.Float, default=0.7)                449
    # Shannon-Wiener 基准                                               450
```

```
    dominant_species = db.Column(db.String(256),                    451
    nullable=True)   # 建群种                                        452
    soil_benchmark = db.Column(db.String(256),                      453
    nullable=True)     # 土壤基准                                   454
class Standard(db.Model):                                           455
    """地方标准 / 政策文件（政策与标准知识库）。"""                    456
    __tablename__ = "standards"                                     457
    id = db.Column(db.Integer, primary_key=True)                    458
    province = db.Column(db.String(32), nullable=False)             459
    # 省份                                                          460
    name = db.Column(db.String(256), nullable=False)                461
    # 名称                                                          462
    doc_number = db.Column(db.String(128), nullable=True)           463
    # 标准编号 / 文号                                               464
    publish_date = db.Column(db.String(32), nullable=True)          465
    implement_date = db.Column(db.String(32), nullable=True)        466
    # 类别：地方标准 / 政策文件                                     467
    category = db.Column(db.String(32), nullable=False,             468
    default="地方标准")                                             469
    summary = db.Column(db.Text, nullable=True)                     470
    # 特点概述                                                      471
    rules = db.relationship("StandardRule",                         472
    backref="standard",                                             473
                                cascade="all, delete-orphan")       474
class StandardRule(db.Model):                                       475
    """结构化规则：将标准条文拆解为可执行规则。"""                    476
    __tablename__ = "standard_rules"                                477
    id = db.Column(db.Integer, primary_key=True)                    478
    standard_id = db.Column(db.Integer,                             479
    db.ForeignKey("standards.id"), nullable=False)                  480
    item = db.Column(db.String(128), nullable=False)                481
    # 判定项                                                        482
    operator = db.Column(db.String(8), nullable=False)              483
    # 比较符：> >= < <= ==                                          484
    threshold = db.Column(db.String(64), nullable=True)             485
    # 阈值                                                          486
    condition_type = db.Column(db.String(32),                       487
    default="必要条件")   # 必要条件 / 充分条件                     488
    region = db.Column(db.String(64), nullable=True)                489
    # 适用分区（宁夏分区差异化）                                    490
    note = db.Column(db.Text, nullable=True)                        491
class ConstraintCheck(db.Model):                                    492
    """限制性判定记录。"""                                          493
    __tablename__ = "constraint_checks"                             494
    id = db.Column(db.Integer, primary_key=True)                    495
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat        496
    ches.id"), nullable=False)                                      497
    policy_pass = db.Column(db.Boolean, nullable=True)              498
    # 政策约束                                                      499
    location_pass = db.Column(db.Boolean, nullable=True)            500
```

```
    # 区位约束
    pollution_pass = db.Column(db.Boolean, nullable=True)
    # 土壤污染
    geo_pass = db.Column(db.Boolean, nullable=True)
    # 地质灾害
    farmland_pass = db.Column(db.Boolean, nullable=True)
    # 耕地属性
    passed = db.Column(db.Boolean, nullable=True)
    # 总体是否通过
    reason = db.Column(db.Text, nullable=True)
    # 不通过原因
    checked_at = db.Column(db.DateTime, default=utcnow)
class MicroUnit(db.Model):
    """微地质单元：岩质边坡 / 碎石边坡 / 平台。"""
    __tablename__ = "micro_units"
    id = db.Column(db.Integer, primary_key=True)
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat
    ches.id"), nullable=False)
    unit_type = db.Column(db.String(32), nullable=False)
    # 单元类型
    area = db.Column(db.Float, nullable=True)
    # 面积
    ratio = db.Column(db.Float, nullable=True)
    # 面积占比（自动计算）
    description = db.Column(db.Text, nullable=True)
    # 现场描述
class Indicator(db.Model):
    """指标定义：指标树，含角色与启用单元类型。"""
    __tablename__ = "indicators"
    id = db.Column(db.Integer, primary_key=True)
    code = db.Column(db.String(32), unique=True,
    nullable=False)  # 指标编码
    name = db.Column(db.String(64), nullable=False)
    # 指标名称
    level = db.Column(db.Integer, nullable=False)
    # 1/2/3 级
    parent_code = db.Column(db.String(32), nullable=True)
    # 父指标编码
    # 角色：土壤污染指标同时作为限制性指标与评价指标
    is_constraint = db.Column(db.Boolean, default=False)
    is_evaluative = db.Column(db.Boolean, default=True)
    unit = db.Column(db.String(32), nullable=True)
    # 单位
    ideal_value = db.Column(db.String(64), nullable=True)
    # 理想值（可配置）
    ideal_type = db.Column(db.String(32), nullable=True)
    # benefit/cost/mid/discrete
    # 启用该指标的微地质单元（逗号分隔）。岩质边坡不启用土壤指标、平台不启用地形指标
    enabled_units = db.Column(db.String(128), nullable=True)
    description = db.Column(db.Text, nullable=True)
```

```
    # 取值依据说明
    def units_list(self):
        return [s for s in (self.enabled_units or
    "").split(",") if s]
class WeightScheme(db.Model):
    """权重方案：按微地质单元类型分别保存。"""
    __tablename__ = "weight_schemes"
    id = db.Column(db.Integer, primary_key=True)
    unit_type = db.Column(db.String(32), nullable=False)
    # 所属单元类型
    name = db.Column(db.String(64), nullable=True)
    # 方案名称
    level = db.Column(db.Integer, nullable=False,
    default=1)          # 1=一级指标 2=二级指标
    is_default = db.Column(db.Boolean, default=False)
    # 是否默认（论文表 3）
    created_at = db.Column(db.DateTime, default=utcnow)
class WeightItem(db.Model):
    """权重明细：方案内的各项权重。"""
    __tablename__ = "weight_items"
    id = db.Column(db.Integer, primary_key=True)
    scheme_id = db.Column(db.Integer,
    db.ForeignKey("weight_schemes.id"), nullable=False)
    indicator_code = db.Column(db.String(32),
    nullable=False)        # 指标编码
    weight = db.Column(db.Float, nullable=False)
    # 权重值
class ExpertMatrix(db.Model):
    """专家判断矩阵：多专家打分的原始矩阵。"""
    __tablename__ = "expert_matrices"
    id = db.Column(db.Integer, primary_key=True)
    scheme_id = db.Column(db.Integer,
    db.ForeignKey("weight_schemes.id"), nullable=True)
    # 直接标注适用对象，便于无需先建权重方案即可录入专家矩阵
    unit_type = db.Column(db.String(32), nullable=True)
    # 微地质单元类型
    l1_name = db.Column(db.String(64), nullable=True)
    # 所属一级指标（二级矩阵用）
    expert_name = db.Column(db.String(64), nullable=True)
    # 专家姓名
    level = db.Column(db.Integer, nullable=False,
    default=1)          # 矩阵对应指标层级
    matrix_json = db.Column(db.Text, nullable=False)
    # 判断矩阵（ＪSON 二维数组）
    cr = db.Column(db.Float, nullable=True)
    # 该专家矩阵的一致性比例
    created_at = db.Column(db.DateTime, default=utcnow)
class IndicatorData(db.Model):
    """指标实测数据：按微地质单元组织。"""
    __tablename__ = "indicator_data"
```

```
    id = db.Column(db.Integer, primary_key=True)
    unit_id = db.Column(db.Integer, db.ForeignKey("micro_units.id"), nullable=False, unique=True)
    soil_ratio = db.Column(db.Float, nullable=True)
    # 土石比
    thickness = db.Column(db.Float, nullable=True)
    # 土层厚度 cm
    ph = db.Column(db.Float, nullable=True)
    organic_matter = db.Column(db.Float, nullable=True)
    # 有机质 g/kg
    # 重金属含量，JSON 字符串：{"Hg": 0.05, ...}，未检测项缺失或为 null
    heavy_metals = db.Column(db.Text, nullable=True)
    coverage = db.Column(db.Float, nullable=True)
    # 植被覆盖度 %
    shannon = db.Column(db.Float, nullable=True)
    # Shannon-Wiener 多样性指数
    slope = db.Column(db.Float, nullable=True)
    # 坡度 °
    aspect = db.Column(db.String(16), nullable=True)
    # 坡向：阳坡/阴阳坡/阴坡
    def heavy_metals_dict(self):
        """把重金属 JSON 解析成字典。
        字段是手工可改的文本，被脏写或导入半个 JSON 时不能让整页 500：
        解析失败按「未检测」处理（空字典），并把原文留在库里供排查。
        """
        if not self.heavy_metals:
            return {}
        try:
            data = json.loads(self.heavy_metals)
        except (ValueError, TypeError):
            return {}
        return data if isinstance(data, dict) else {}
class EvaluationResult(db.Model):
    """评价结果。"""
    __tablename__ = "evaluation_results"
    id = db.Column(db.Integer, primary_key=True)
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_patches.id"), nullable=False)
    final_score = db.Column(db.Float, nullable=True)
    # 最终评价因子 F
    grade = db.Column(db.String(32), nullable=True)
    # 适宜性等级
    measures = db.Column(db.Text, nullable=True)
    # 推荐措施
    unit_scores = db.Column(db.Text, nullable=True)
    # 各单元得分 JSON
    # 计算参数快照：本次用的权重来源、参照生态系统基准、各指标理想取值。
    # 参数现在可以在界面上改，不留快照就无法复现这一次的结果。
    params_json = db.Column(db.Text, nullable=True)
    created_by = db.Column(db.Integer,
```

```
    db.ForeignKey("users.id"), nullable=True)
    created_at = db.Column(db.DateTime, default=utcnow)
class ScoreDetail(db.Model):
    """得分明细：每一层指标的权重、标准化值、加权贡献，供追溯与复核。"""
    __tablename__ = "score_details"
    id = db.Column(db.Integer, primary_key=True)
    result_id = db.Column(db.Integer,
    db.ForeignKey("evaluation_results.id"), nullable=False)
    unit_id = db.Column(db.Integer, nullable=True)
    l1_name = db.Column(db.String(64), nullable=True)
    # 一级指标
    l2_name = db.Column(db.String(64), nullable=True)
    # 二级指标
    raw_value = db.Column(db.String(64), nullable=True)
    # 实测值
    std_value = db.Column(db.Float, nullable=True)
    # 标准化值
    weight = db.Column(db.Float, nullable=True)
    # 权重
    contribution = db.Column(db.Float, nullable=True)
    # 加权贡献
# ==== app/forms.py ====
# -*- coding: utf-8 -*-
"""表单定义（共享，由甲统一维护）。
各模块的 Flask-WTF 表单集中于此。
"""
import re
from flask_wtf import FlaskForm
from wtforms import SelectField, SubmitField, StringField,
    PasswordField, BooleanField
from wtforms.validators import (DataRequired, Length,
    Optional, EqualTo,
                                ValidationError)
from app.utils.constants import UNIT_TYPES, ASPECT_TYPES,
    GENDERS
# 邮箱格式：够用的轻量校验。
# WTForms 自带的 Email() 需要额外安装 email_validator，
# 为一个字段引入依赖不划算，这里自己用正则判断。
EMAIL_RE = re.compile(r"^[^@\s]+@[^@\s]+\.[A-Za-z]{2,}$")
def email_format(form, field):
    v = (field.data or "").strip()
    if v and not EMAIL_RE.match(v):
        raise ValidationError("邮箱格式不正确")
def email_ok(value):
    """邮箱格式判断，供不用 WTForms 的表单复用（如管理员编辑用户）。"""
    v = (value or "").strip()
    return (not v) or bool(EMAIL_RE.match(v))
class AHPSelectForm(FlaskForm):
    """选择微地质单元类型与指标层级，以确定判断矩阵规模。"""
    unit_type = SelectField("微地质单元类型",
```

```
    validators=[DataRequired()])
    level = SelectField("指标层级",
                        choices=[("1", "一级指标"), ("2",
    "二级指标")],
                        validators=[DataRequired()])
    l1 = SelectField("一级指标", validators=[],
    validate_choice=False)  # 动态填充，层级=2 时使用
    submit = SubmitField("生成判断矩阵")
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.unit_type.choices = [(t, t) for t in
    UNIT_TYPES]
class EmptyForm(FlaskForm):
    """仅用于承载 CSRF 令牌的空表单（矩阵录入等自定义表单使用）。"""
    pass
class LoginForm(FlaskForm):
    """登录表单。支持「记住我」以在关闭浏览器后保持登录状态。
    校验消息用中文写明：登录框带 novalidate，浏览器不会代填提示，
    字段为空时若不自己给消息，用户点了「登录」会以为没反应。
    """
    username = StringField("账户名称",
                           validators=[DataRequired(message=
    "请输入账户名称"),
                                       Length(max=64,

    message="账户名称不能超过 64 个字符")])
    password = PasswordField("密码",
                             validators=[DataRequired(messag
    e="请输入密码")])
    remember = BooleanField("记住我")
    submit = SubmitField("登录")
GENDER_CHOICES = [("", "不填")] + [(g, g) for g in GENDERS]
class ProfileForm(FlaskForm):
    """个人资料：昵称、性别与联系方式。字段都可留空。
    职务不在这里 —— 只能由管理员在「个人设置 · 账号」里改。
    """
    display_name = StringField("昵称", validators=[Optional(),
    Length(max=64)])
    gender = SelectField("性别", choices=GENDER_CHOICES,
    validators=[Optional()])
    email = StringField("邮箱", validators=[Optional(),
    email_format,
                                            Length(max=128)])
    phone = StringField("手机号", validators=[Optional(),
    Length(max=32)])
    submit_info = SubmitField("保存资料")
class RegisterForm(FlaskForm):
    """注册：账号与密码必填，其余选填。注册后默认是操作员。"""
    username = StringField("账号名称",
                           validators=[DataRequired("请填写账号名称
```

```
        "），                                                                751
                                                    Length(min=2, max=64,   752
                                                            message="账号名称   753
        2–64 个字符")])                                                      754
        password = PasswordField("密码",                                     755
                                    validators=[DataRequired("请设置密码     756
        ")，                                                                 757
                                                      Length(min=6,          758
        max=64,                                                              759
                                                                             760
        message="密码至少 6 位")])                                           761
        confirm = PasswordField("确认密码",                                  762
                                   validators=[DataRequired(),               763
                                                 EqualTo("password",         764
                                                                             765
        message="两次输入的密码不一致")])                                    766
        display_name = StringField("昵称", validators=[Optional(),           767
        Length(max=64)])                                                     768
        gender = SelectField("性别", choices=GENDER_CHOICES,                 769
        validators=[Optional()])                                             770
        email = StringField("邮箱", validators=[Optional(),                  771
        email_format,                                                        772
                                                      Length(max=128)])      773
        phone = StringField("手机号", validators=[Optional(),                774
        Length(max=32)])                                                     775
        submit_reg = SubmitField("注册并登录")                               776
class PasswordForm(FlaskForm):                                               777
        """修改密码：需要先验证当前密码。"""                                 778
        old_password = PasswordField("当前密码",                             779
        validators=[DataRequired()])                                         780
        new_password = PasswordField("新密码",                               781
                                        validators=[DataRequired(),          782
        Length(min=6, max=64,                                                783
                                                                             784
        message="新密码至少 6 位")])                                         785
        confirm = PasswordField("确认新密码",                                786
                                   validators=[DataRequired(),               787
                                                                             788
        EqualTo("new_password",                                              789
                                                                             790
        message="两次输入的新密码不一致")])                                  791
        submit_pwd = SubmitField("修改密码")                                 792
#  ==== app/utils/constants.py ====                                         793
#  -*- coding: utf-8 -*-                                                    794
"""评价指标常量表。                                                         795
全部数值取自论文，作为系统计算的唯一基准：                                  796
- 论文 4.2.2 节：指标标准化理想极值与取值依据                               797
- 论文 4.2.3 节 / 表 4：适宜性等级划分                                      798
- 论文表 3：按微地质单元分别确定的一级、二级指标权重                        799
- 论文 4.2.1 节：平均随机一致性指标 RI                                      800
```

```
"""                                                                    801
#  ----------------------------------------------------------------------  802
    ----------------                                                   803
#  一、微地质单元类型                                                  804
#  ----------------------------------------------------------------------  805
    ----------------                                                   806
#  论文 4.1 节："按照地形地貌划分，将其划分为平台、碎石边坡和岩质边坡  807
#  三种地质微单元类型"                                                 808
UNIT_TYPES = ("岩质边坡", "碎石边坡", "平台")                          809
#  每个单元类型启用的指标类别（论文表 3 隐含：岩质边坡无土壤指标，平台无地形指标）  810
#  供数据录入界面与评价引擎判断"该单元是否需要某类指标"               811
UNIT_ENABLED_INDICATORS = {                                            812
    "岩质边坡": ("植被指标", "地形指标"),                  # 不设土壤指标  813
    "碎石边坡": ("土壤指标", "植被指标", "地形指标"),                      814
    "平台": ("土壤指标", "植被指标"),                      # 不设地形指标  815
}                                                                      816
#  ----------------------------------------------------------------------  817
    ----------------                                                   818
#  二、指标标准化理想极值（论文 4.2.2 节）                            819
#  ----------------------------------------------------------------------  820
    ----------------                                                   821
IDEAL_SOIL_RATIO = 0.8          # 土石比理想值                         822
IDEAL_THICKNESS = 20.0          # 土层厚度理想值 (cm)                  823
IDEAL_PH_RANGE = (5.0, 9.0)     # pH 理想区间（适中型指标）            824
IDEAL_ORGANIC = 30.0            # 有机质理想值                         825
    (g/kg)，全国第二次土壤普查养分等级划分标准二等最低值               826
IDEAL_COVERAGE = 75.0           # 植被覆盖度参照生态系统 (%)           827
IDEAL_SHANNON = 0.7             # Shannon-Wiener 多样性指数参照生态系统  828
#  重金属极值 (mg/kg)，《绿化种植土壤》CJ/T 340—2016 土壤重金属含量 IV 级最高值  829
#  论文原文：镉、汞、铅、铬、砷、镍、铜、锌分别为 2、2、530、400、45、220、600、800  830
HEAVY_METALS = ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu",             831
    "Zn")                                                              832
HEAVY_METAL_LIMIT = {                                                  833
    "Cd": 2.0, "Hg": 2.0, "Pb": 530.0, "Cr": 400.0,                    834
    "As": 45.0, "Ni": 220.0, "Cu": 600.0, "Zn": 800.0,                 835
}                                                                      836
#  污染指标下 8 种重金属"重要性相当，权重平均分配"（论文 4.2.1 节）  837
HEAVY_METAL_EQUAL_WEIGHT = 1.0 / 8.0                                   838
#  坡度分界，论文 4.2.2 节引《新一轮退耕还林还草总体方案》发改西部〔2014〕1772 号  839
SLOPE_BREAK = 25.0                                                     840
#  坡向离散赋值，论文 4.2.2 节："阳坡、阴阳坡、阴坡分别赋值 0.2、0.6、1"  841
ASPECT_SCORES = {"阳坡": 0.2, "阴阳坡": 0.6, "阴坡": 1.0}              842
ASPECT_TYPES = ("阳坡", "阴阳坡", "阴坡")                              843
#  pH 适中型标准化的容差边界（区间外线性衰减，避免得分突变）          844
PH_TOL_LOW = 4.0                                                       845
PH_TOL_HIGH = 10.0                                                     846
#  ----------------------------------------------------------------------  847
    ----------------                                                   848
#  三、适宜性等级划分（论文表 4）                                      849
#  ----------------------------------------------------------------------  850
```

```
    ------------------                                                  851
GRADE_LEVELS = (                                                        852
    {"grade": "不适宜", "min": None, "max": 0.3,                        853
     "meaning": "存在限制性条件或区域生态本底较差，自然恢复影响因子严重制约自然恢复",  854
     "measure": "生态重建、人工辅助"},                                  855
    {"grade": "基本适宜", "min": 0.3, "max": 0.7,                       856
     "meaning": "生态本底良好，核心自然恢复影响因子基本满足自然恢复需求",  857
     "measure": "自然恢复为主，可辅以封闭场地和少量补种、补植、飞播等"},  858
    {"grade": "适宜", "min": 0.7, "max": None,                          859
     "meaning": "各自然恢复影响因子均能满足自然恢复需求，自然演替速度快",  860
     "measure": "自然恢复"},                                           861
)                                                                       862
def grade_of(score):                                                    863
    """按论文表 4 判定适宜性等级。"""                                   864
    for g in GRADE_LEVELS:                                              865
        lo, hi = g["min"], g["max"]                                     866
        if lo is None and score < hi:                                   867
            return g                                                    868
        if hi is None and score >= lo:                                  869
            return g                                                    870
        if lo is not None and hi is not None and lo <=                  871
    score < hi:                                                         872
            return g                                                    873
    # score 恰好等于 0.7 时落入"适宜"                                   874
    return GRADE_LEVELS[-1]                                             875
# ---------------------------------------------------------------------- 876
    ------------------                                                  877
# 四、平均随机一致性指标 RI（论文 4.2.1 节给出 n=2,3,4,5，其余为常用扩展值）  878
# ---------------------------------------------------------------------- 879
    ------------------                                                  880
RI_TABLE = {1: 0.0, 2: 0.0, 3: 0.58, 4: 0.90, 5: 1.12, 6:               881
    1.24,                                                               882
            7: 1.32, 8: 1.41, 9: 1.45, 10: 1.49}                        883
# 一致性比例阈值（论文：当 CR<0.1 时，判断矩阵满足一致性要求）          884
CR_THRESHOLD = 0.1                                                      885
# ---------------------------------------------------------------------- 886
    ------------------                                                  887
# 五、默认权重方案（论文表 3）                                          888
# ---------------------------------------------------------------------- 889
    ------------------                                                  890
# 论文 4.2.2 节："划分岩质边坡、碎石土边坡、平台三类微地质单元开展独立评价，  891
# 分别计算评价指标权重"（论文摘要、图 2 流程与表 5 均作"碎石边坡"，        892
# 系统统一采用"碎石边坡"）。权重由 10 位专家打分后经 AHP 方法确定。        893
# 下表为论文表 3 所载的归一化权重，作为系统默认方案。                    894
DEFAULT_WEIGHTS = {                                                     895
    "岩质边坡": {                                                       896
        "l1": {"植被指标": 0.45, "地形指标": 0.55},                     897
        "l2": {                                                         898
            "植被指标": {"植被覆盖度": 0.67, "生物多样性": 0.33},       899
            "地形指标": {"坡度": 0.64, "坡向": 0.36},                   900
```

```
            },
        },
        "碎石边坡":  {
            "l1":  {"土壤指标":  0.44,  "植被指标":  0.31,  "地形指标":  0.25},
            "l2":  {
                "土壤指标":  {"土石比":  0.25,  "土层厚度":  0.25,  "pH":  0.17,
                            "有机质":  0.17,  "污染指标":  0.17},
                "植被指标":  {"植被覆盖度":  0.64,  "生物多样性":  0.36},
                "地形指标":  {"坡度":  0.67,  "坡向":  0.33},
            },
        },
        "平台":  {
            "l1":  {"土壤指标":  0.54,  "植被指标":  0.46},
            "l2":  {
                "土壤指标":  {"土石比":  0.25,  "土层厚度":  0.25,  "pH":  0.17,
                            "有机质":  0.17,  "污染指标":  0.17},
                "植被指标":  {"植被覆盖度":  0.61,  "生物多样性":  0.39},
            },
        },
}
#  ------------------------------------------------------------------------
    ------------------
#  六、限制性指标（论文表  2  +  4.1  节流程）
#  ------------------------------------------------------------------------
    ------------------
#  论文摘要与  4.1  节：四项限制性指标；流程段另明确"是否为耕地"为约束条件
CONSTRAINT_ITEMS  =  ("政策",  "区位约束",  "土壤污染",  "地质灾害",  "耕地属性")
#  ------------------------------------------------------------------------
    ------------------
#  七、账号职务
#  ------------------------------------------------------------------------
    ------------------
#  只分两种：管理员可管理账号、维护基础数据并查看操作日志，操作员用业务功能。
ROLE_LABELS  =  {"admin":  "管理员",  "operator":  "操作员"}
ROLE_CODES  =  ("admin",  "operator")
DEFAULT_ROLE  =  "operator"                #  新注册账号的默认职务
ROLE_ORDER  =  ("admin",  "operator")    #  列表排序用：管理员在前
#  性别（注册与个人资料中选填）
GENDERS  =  ("男",  "女",  "保密")
#  ====  app/utils/schema.py  ====
#  -*-  coding:  utf-8  -*-
"""轻量结构迁移。
SQLite  下  `db.create_all()`  只建缺失的表，不会给已有表补列。
本模块在启动时检查目标列是否存在，缺失则  ALTER  TABLE  补上，
使旧数据库无需重建即可用上新字段。
"""
from  sqlalchemy  import  text
#  （表名，  列名，  列定义）
REQUIRED_COLUMNS  =  [
    #  遥感影像挂接
```

```
        ("mine_patches", "image_path", "VARCHAR(256)"),
        ("mine_patches", "image_note", "VARCHAR(256)"),
        ("mine_patches", "image_uploaded_at", "DATETIME"),
        # 指标维护
        ("indicators", "enabled_units", "VARCHAR(128)"),
        ("indicators", "description", "TEXT"),
        # 多专家矩阵
        ("expert_matrices", "unit_type", "VARCHAR(32)"),
        ("expert_matrices", "l1_name", "VARCHAR(64)"),
        ("expert_matrices", "cr", "FLOAT"),
        # 用户个人资料（账户页）
        ("users", "display_name", "VARCHAR(64)"),
        ("users", "email", "VARCHAR(128)"),
        ("users", "phone", "VARCHAR(32)"),
        ("users", "gender", "VARCHAR(8)"),
        ("users", "avatar", "VARCHAR(256)"),
        # 评价结果的可追溯信息：计算参数快照与计算人
        ("evaluation_results", "params_json", "TEXT"),
        ("evaluation_results", "created_by", "INTEGER"),
]
# 职务体系由「管理员 / 评价员 / 查看者」三种收敛为「管理员 / 操作员」两种，
# 旧数据里的后两种统一归为操作员。
ROLE_MIGRATION = {"evaluator": "operator", "viewer":
    "operator"}
def _add_columns(db):
    added = []
    with db.engine.connect() as conn:
        for table, column, ddl in REQUIRED_COLUMNS:
            rows = conn.execute(text("PRAGMA
    table_info(%s)" % table)).fetchall()
            if not rows:
                continue                                #
    表还不存在，create_all 会建
            existing = {r[1] for r in rows}
            if column in existing:
                continue
            conn.execute(text(
                "ALTER TABLE %s ADD COLUMN %s %s" % (table,
    column, ddl)))
            conn.commit()
            added.append("%s.%s" % (table, column))
    return added
def _migrate_roles(db):
    """把旧角色名统一为新的两种，返回改动的行数。"""
    changed = 0
    with db.engine.connect() as conn:
        rows = conn.execute(text("PRAGMA
    table_info(users)")).fetchall()
        if not rows:
            return 0
```

```
            for old, new in ROLE_MIGRATION.items():                    1001
                r = conn.execute(text("UPDATE users SET role =         1002
        :new "                                                         1003
                                      "WHERE role = :old"),            1004
                                  {"new": new, "old": old})            1005
                changed += r.rowcount or 0                             1006
            conn.commit()                                              1007
        return changed                                                 1008
def ensure_schema(db):                                                 1009
        """为已有表补齐缺失列，并做必要的数据迁移。返回变更清单。"""                1010
        try:                                                           1011
            changed = _add_columns(db)                                 1012
            n = _migrate_roles(db)                                     1013
            if n:                                                      1014
                changed.append("用户角色归一为管理员/操作员 %d 条" % n)          1015
            return changed                                             1016
        except Exception as e:                                  #      1017
        迁移失败不应阻断启动                                                 1018
            print("[schema] 结构检查跳过：%s" % e)                          1019
            return []                                                  1020
#  ==== app/utils/uploads.py ====                                      1021
#  -*- coding: utf-8 -*-                                               1022
"""上传文件处理（遥感影像、批量导入）。"""                                        1023
import os                                                              1024
import re                                                              1025
import uuid                                                            1026
from datetime import datetime                                          1027
from werkzeug.utils import secure_filename                             1028
def _safe_ext(filename):                                               1029
        return os.path.splitext(filename)[1].lower().lstrip(".")       1030
def ext_allowed(filename, allowed):                                    1031
        return _safe_ext(filename) in allowed                          1032
def save_upload(file_storage, subdir, allowed_ext,                     1033
        upload_root):                                                  1034
        """保存上传文件，返回相对 static 的路径（如 uploads/patches/xxx.png）。    1035
        文件名重命名为「时间戳_随机串.扩展名」，避免中文名与重名问题。                    1036
        """                                                            1037
        if file_storage is None or not file_storage.filename:          1038
            return None, "未选择文件"                                    1039
        ext = _safe_ext(file_storage.filename)                         1040
        if ext not in allowed_ext:                                     1041
            return None, "不支持的文件类型：.%s" % ext                      1042
        target_dir = os.path.join(upload_root, subdir)                 1043
        os.makedirs(target_dir, exist_ok=True)                         1044
        #  仅保留安全字符，并加时间戳与随机串防重名                                1045
        base = secure_filename(os.path.splitext(file_storage.fil       1046
        ename)[0]) or "file"                                           1047
        base = re.sub(r"[^A-Za-z0-9_\-]", "", base)[:40] or            1048
        "file"                                                         1049
        name = "%s_%s.%s" % (datetime.now().strftime("%Y%m%d%H%M       1050
```

```
    %S"),                                                              1051
                                uuid.uuid4().hex[:6], ext)             1052
    abs_path = os.path.join(target_dir, name)                          1053
    file_storage.save(abs_path)                                        1054
    rel = os.path.join("uploads", subdir,                              1055
    name).replace("\\", "/")                                           1056
    return rel, None                                                   1057
def remove_upload(rel_path, upload_root):                              1058
    """删除已保存的上传文件（相对 static 的路径）。                    1059
    路径必须落在上传根目录之内：rel_path 来自数据库，正常都是 save_upload 1060
    生成的，但万一被改脏（手工改库、导入异常），`../` 这种值会删掉上传根目录 1061
    之外的文件。这里用 realpath 做一次围栏校验。                       1062
    """                                                                1063
    if not rel_path:                                                   1064
        return                                                         1065
    root = os.path.realpath(upload_root)                               1066
    abs_path = os.path.realpath(                                       1067
        os.path.join(root, rel_path.replace("uploads/", "",            1068
    1)))                                                               1069
    if abs_path != root and not abs_path.startswith(root +             1070
    os.sep):                                                           1071
        return                                                         1072
    try:                                                               1073
        if os.path.isfile(abs_path):                                   1074
            os.remove(abs_path)                                        1075
    except OSError:                                                    1076
        pass                                                           1077
# ==== app/utils/charts.py ====                                        1078
# -*- coding: utf-8 -*-                                              1079
"""轻量图表。                                                         1080
用纯 SVG 输出，不依赖任何绘图库或前端脚本：                           1081
- 服务端渲染，页面与打印（PDF）中都能稳定显示                         1082
- 每个图元都带数值标签，黑白打印时仍可读（不单靠颜色区分）           1083
- 单色方案，符合界面「一个视图一个强调色」的约定                     1084
"""                                                                    1085
from html import escape                                                1086
ACCENT = "#1f3a5f"                                                     1087
ACCENT_SOFT = "#7f93ad"                                                1088
INK = "#23262b"                                                        1089
MUTED = "#6b7280"                                                      1090
LINE = "#dfe3e8"                                                       1091
# 灰度梯度：黑白打印时相邻扇区 / 条柱仍可区分                         1092
GRAYS = ["#1f3a5f", "#456188", "#6d86a8", "#96a9c4",                   1093
    "#bcc9da", "#dbe3ec"]                                              1094
def _fmt(v, digits=4):                                                 1095
    try:                                                               1096
        return ("%%.%df" % digits) % float(v)                          1097
    except (TypeError, ValueError):                                    1098
        return str(v)                                                  1099
def hbars(items, width=620, bar_h=22, gap=12, label_w=150,             1100
```

```
    value_w=76):                                                    1101
    """水平条形图。items: [(标签, 数值, 显示文本)]"""                    1102
    if not items:                                                   1103
        return '<p class="muted">暂无数据</p>'                       1104
    n = len(items)                                                  1105
    height = n * (bar_h + gap) + 10                                 1106
    max_v = max((abs(float(v)) for _, v, _ in items),               1107
    default=1) or 1                                                 1108
    body_w = width - label_w - value_w                              1109
    parts = ['<svg viewBox="0 0 %d %d" width="100%%"                1110
    height="%d" '                                                   1111
              'role="img" aria-label="条形图" '                      1112
              'style="max-width:%dpx;font-family:inherit;">'        1113
    % (width, height, height, width)]                               1114
    for i, (label, value, disp) in enumerate(items):                1115
        y = i * (bar_h + gap) + 5                                   1116
        frac = (abs(float(value)) / max_v) if max_v else 0          1117
        w = max(2.0, body_w * frac)                                 1118
        fill = GRAYS[i % len(GRAYS)]                                1119
        parts.append(                                               1120
            '<text x="%d" y="%d" text-anchor="end"                  1121
    font-size="12" fill="%s">%s</text>'                             1122
            % (label_w - 10, y + bar_h * 0.72, INK,                 1123
    escape(str(label))))                                            1124
        parts.append(                                               1125
            '<rect x="%d" y="%d" width="%.1f" height="%d"           1126
    rx="3" fill="%s"/>'                                             1127
            % (label_w, y, w, bar_h, fill))                         1128
        parts.append(                                               1129
            '<text x="%d" y="%d" font-size="12" fill="%s" '         1130
            'style="font-variant-numeric:tabular-nums;">%s</        1131
    text>'                                                          1132
            % (label_w + w + 8, y + bar_h * 0.72, MUTED,            1133
    escape(str(disp))))                                             1134
    parts.append('</svg>')                                          1135
    return "".join(parts)                                           1136
def donut(items, size=210, thickness=34):                           1137
    """环形图（权重分布）。items: [(标签, 数值, 显示文本)]              1138
    图例中给出数值与占比，黑白打印时靠图例与灰度深浅区分。                1139
    """                                                             1140
    vals = [(l, float(v), d) for l, v, d in items if                1141
    float(v) > 0]                                                   1142
    if not vals:                                                    1143
        return '<p class="muted">暂无数据</p>'                       1144
    total = sum(v for _, v, _ in vals) or 1                         1145
    r = (size - thickness) / 2                                      1146
    cx = cy = size / 2                                              1147
    import math                                                     1148
    parts = ['<svg viewBox="0 0 %d %d" width="%d"                   1149
    height="%d" role="img" '                                        1150
```

```
            'aria-label="权重分布环形图">' % (size, size, size,
    size)]
    angle = -math.pi / 2
    for i, (label, v, disp) in enumerate(vals):
        sweep = 2 * math.pi * (v / total)
        a0, a1 = angle, angle + sweep
        x0, y0 = cx + r * math.cos(a0), cy + r *
    math.sin(a0)
        x1, y1 = cx + r * math.cos(a1), cy + r *
    math.sin(a1)
        large = 1 if sweep > math.pi else 0
        parts.append(
            '<path d="M %.2f %.2f A %.2f %.2f 0 %d 1 %.2f
    %.2f" '
            'fill="none" stroke="%s" stroke-width="%d"/>'
           % (x0, y0, r, r, large, x1, y1, GRAYS[i %
    len(GRAYS)], thickness))
        angle = a1
    parts.append('<text x="%d" y="%d" text-anchor="middle"
    font-size="12" '
                 'fill="%s">合计</text>' % (cx, cy - 2,
    MUTED))
    parts.append('<text x="%d" y="%d" text-anchor="middle"
    font-size="15" '
                 'fill="%s" style="font-variant-numeric:tabu
    lar-nums;">%s</text>'
                 % (cx, cy + 18, INK, _fmt(total, 4)))
    parts.append('</svg>')
    legend = ['<table style="width:auto;margin-top:10px;"><t
    head><tr>'
              '<th scope="col">项目</th><th
    scope="col">数值</th>'
              '<th scope="col">占比</th></tr></thead><tbody>']
    for i, (label, v, disp) in enumerate(vals):
        legend.append(
            '<tr><td style="text-align:left;">'
            '<span style="display:inline-block;width:10px;he
    ight:10px;'
            'border-radius:2px;background:%s;margin-right:6p
    x;"></span>%s</td>'
            '<td class="mono">%s</td><td
    class="mono">%.1f%%</td></tr>'
           % (GRAYS[i % len(GRAYS)], escape(str(label)),
    escape(str(disp)),
              100.0 * v / total))
    legend.append('</tbody></table>')
    return ('<div style="display:flex;flex-wrap:wrap;gap:20p
    x;align-items:center;">'
            + "".join(parts) + "".join(legend) + '</div>')
# ==== app/utils/browser.py ====
```

```
# -*- coding: utf-8 -*-
"""HTML 转 PDF（报告导出）。
复用本机已安装的 Chrome/Edge 无头模式打印，避免引入 reportlab、
weasyprint 等额外依赖。生成过程使用系统临时目录中的 ASCII 文件名，
避免中文路径在 file:// URL 下出问题。
"""
import os
import shutil
import subprocess
import tempfile
import uuid
CANDIDATES = [
    r"C:\Program Files\Google\Chrome\Application\chrome.exe",
    r"C:\Program Files (x86)\Google\Chrome\Application\chrome.exe",
    r"C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe",
    r"C:\Program Files\Microsoft\Edge\Application\msedge.exe",
]
# 允许通过环境变量覆盖，便于在其他机器上指定浏览器
ENV_KEY = "DSH_BROWSER_PATH"
def browser_path():
    """定位可用的浏览器可执行文件，找不到返回 None。"""
    override = os.environ.get(ENV_KEY)
    if override and os.path.isfile(override):
        return override
    for p in CANDIDATES:
        if os.path.isfile(p):
            return p
    for name in ("chrome", "msedge", "chromium"):
        found = shutil.which(name)
        if found:
            return found
    return None
def html_to_pdf(html, timeout=90):
    """把 HTML 字符串渲染为 PDF，返回字节；失败返回 (None, 原因)。"""
    exe = browser_path()
    if exe is None:
        return None, ("未找到可用的浏览器（Chrome/Edge），无法生成 PDF。"
                      "可设置环境变量 %s 指向浏览器可执行文件。" % ENV_KEY)
    tmpdir = tempfile.mkdtemp(prefix="dshpdf_")
    tag = uuid.uuid4().hex[:8]
    html_path = os.path.join(tmpdir, "r_%s.html" % tag)
    pdf_path = os.path.join(tmpdir, "r_%s.pdf" % tag)
    try:
        with open(html_path, "w", encoding="utf-8") as f:
            f.write(html)
        cmd = [
```

```
            exe, "--headless=new", "--disable-gpu",
    "--no-sandbox",
            "--no-first-run", "--no-default-browser-check",
            "--no-pdf-header-footer",
            "--virtual-time-budget=4000",
            "--print-to-pdf=%s" % pdf_path,
            "file:///" + html_path.replace("\\", "/"),
        ]
        p = subprocess.run(cmd, capture_output=True,
    text=True,
                           timeout=timeout,
    encoding="utf-8", errors="replace")
        if not os.path.isfile(pdf_path) or
    os.path.getsize(pdf_path) < 800:
            return None, "浏览器未能生成 PDF（%s）" % (p.stderr or
    "").strip()[:200]
        with open(pdf_path, "rb") as f:
            return f.read(), None
    except subprocess.TimeoutExpired:
        return None, "生成 PDF 超时，请稍后重试"
    except Exception as e:
        return None, "生成 PDF 失败：%s" % e
    finally:
        shutil.rmtree(tmpdir, ignore_errors=True)
# ==== app/services/ahp_service.py ====
# -*- coding: utf-8 -*-
"""AHP 权重引擎。
严格按论文 4.2.1 节实现：
- 式(2)(3)(4)：特征根法求权重（行元素乘积 → n 次方根 → 归一化）
- 式(5)(6)(7)：一致性检验（λmax、CI、CR）
- 论文要求 CR < 0.1；RI 按阶数查表
"""
import numpy as np
from app.utils.constants import RI_TABLE, CR_THRESHOLD
def matrix_from_upper(upper):
    """由判断矩阵上三角构造完整互反矩阵。
    参数
    ----
    upper : list[list[float]] 或 np.ndarray，上三角元素（不含对角线）。
        例：3 阶时传入 [[a12, a13], [a23]]。
        对角线为 1，下三角取倒数（a_ji = 1/a_ij）。
    """
    upper = np.asarray(upper, dtype=float)
    k = upper.shape[0]
    n = int(round((1 + (1 + 8 * k) ** 0.5) / 2))
    A = np.ones((n, n), dtype=float)
    idx = 0
    for i in range(n):
        for j in range(i + 1, n):
            A[i, j] = upper[idx]
```

```
                A[j, i] = 1.0 / upper[idx]
                idx += 1
    return A
def weights_and_consistency(A):
    """由判断矩阵 A 计算权重与一致性指标。
    返回
    ----
    dict：{weights: np.ndarray, lambda_max, ci, cr, n,
    consistent: bool}
    """
    A = np.asarray(A, dtype=float)
    n = A.shape[0]
    # 式(2) 行元素乘积
    row_prod = np.prod(A, axis=1)
    # 式(3) 开 n 次方根
    root = row_prod ** (1.0 / n)
    # 式(4) 归一化
    weights = root / root.sum()
    # 式(5) 最大特征值 λmax = (1/n) Σ (A·W)_i / W_i
    aw = A.dot(weights)
    lambda_max = float(np.sum(aw / weights) / n)
    # 式(6) 一致性指标 CI
    ci = (lambda_max - n) / (n - 1) if n > 1 else 0.0
    # 式(7) 一致性比例 CR = CI / RI
    ri = RI_TABLE.get(n, 1.49)
    cr = ci / ri if ri else 0.0
    return {
        "weights": weights,
        "lambda_max": lambda_max,
        "ci": ci,
        "cr": cr,
        "n": n,
        "consistent": cr < CR_THRESHOLD,
    }
def find_most_inconsistent(A):
    """定位判断矩阵中最不一致的一对比较。
    依据：完全一致矩阵应满足 a_ij = w_i / w_j。
    计算每个上三角元素与其理想值的相对偏差，返回偏差最大者。
    返回
    ----
    dict：{i, j, value, expected, deviation}，偏差最大的 (i, j)
    及对应取值。
    """
    A = np.asarray(A, dtype=float)
    r = weights_and_consistency(A)
    w = r["weights"]
    n = A.shape[0]
    best = None
    for i in range(n):
        for j in range(i + 1, n):
```

```
                expected = w[i] / w[j]                                  1351
                if expected == 0:                                       1352
                    continue                                            1353
                deviation = abs(A[i, j] - expected) / expected          1354
                if best is None or deviation >                          1355
    best["deviation"]:                                                  1356
                    best = {"i": i, "j": j, "value": A[i, j],           1357
                            "expected": expected, "deviation":          1358
    deviation}                                                          1359
    return best                                                         1360
def aggregate_matrices(matrices):                                       1361
    """多专家判断矩阵聚合：几何平均（保持互反性）。                      1362
    参数                                                                1363
    ----                                                                1364
    matrices : list[np.ndarray]，多个同阶判断矩阵。                     1365
    """                                                                 1366
    stack = np.stack([np.asarray(m, dtype=float) for m in               1367
    matrices])                                                          1368
    return np.prod(stack, axis=0) ** (1.0 / len(stack))                 1369
def judge_from_weights(weights):                                        1370
    """由权重向量构造完全一致判断矩阵并检验。                           1371
    用途：验证给定权重向量能否作为某个完全一致判断矩阵的主特征向量      1372
    （A_ij = w_i / w_j 时 λmax = n、CR = 0）。                          1373
    """                                                                 1374
    w = np.asarray(weights, dtype=float)                                1375
    n = w.shape[0]                                                      1376
    A = np.outer(w, 1.0 / w)                                            1377
    r = weights_and_consistency(A)                                      1378
    r["matrix"] = A                                                     1379
    return r                                                            1380
# ==== app/services/standardization.py ====                             1381
# -*- coding: utf-8 -*-                                                 1382
"""指标标准化（的一部分）。                                             1383
按论文 4.2.2 节式(8) 实现：                                             1384
    x' = (x - x_min) / (x_max - x_min)                                  1385
按指标性质分为四类：越大越优 / 越小越优 / 适中 / 离散赋值。             1386
所有理想极值取自 app.utils.constants，可配置。                          1387
"""                                                                     1388
from app.utils.constants import (                                       1389
    IDEAL_SOIL_RATIO, IDEAL_THICKNESS, IDEAL_PH_RANGE,                  1390
    IDEAL_ORGANIC,                                                      1391
    IDEAL_COVERAGE, IDEAL_SHANNON, HEAVY_METAL_LIMIT,                   1392
    HEAVY_METAL_EQUAL_WEIGHT, SLOPE_BREAK, ASPECT_SCORES,               1393
    PH_TOL_LOW, PH_TOL_HIGH,                                            1394
)                                                                       1395
def std_benefit(value, ideal):                                          1396
    """越大越优指标标准化：以理想值为上限，按比例给分，饱和于 [0,1]。"""  1397
    if value is None:                                                   1398
        return None                                                     1399
    return max(0.0, min(1.0, value / ideal))                            1400
```

```
def std_cost(value, extreme):
    """越小越优指标标准化：以极值为参考，线性折减至 [0,1]。"""
    if value is None:
        return None
    return max(0.0, min(1.0, 1.0 - value / extreme))
def std_mid_ph(value, lo, hi, tol_lo=PH_TOL_LOW,
    tol_hi=PH_TOL_HIGH):
    """适中型指标标准化（pH）：理想区间内取 1，区间外线性衰减。"""
    if value is None:
        return None
    if lo <= value <= hi:
        return 1.0
    if value > hi:
        return max(0.0, (tol_hi - value) / (tol_hi - hi))
    return max(0.0, (value - tol_lo) / (lo - tol_lo))
def std_slope(value, break_point=SLOPE_BREAK, upper=90.0):
    """坡度标准化。
    论文 4.2.2 节：以 25° 为陡坡重要分界。25° 及以下记满分 1，
    超过 25° 线性折减，90° 记 0。
    """
    if value is None:
        return None
    if value <= break_point:
        return 1.0
    return max(0.0, (upper - value) / (upper - break_point))
def aspect_score(aspect):
    """坡向离散赋值：阳坡 0.2 / 阴阳坡 0.6 / 阴坡 1.0（论文 4.2.2 节）。"""
    if aspect is None:
        return None
    return ASPECT_SCORES.get(aspect)
def pollution_score(heavy_metals, limits=None):
    """污染指标：8 种重金属等权（各 0.125）合成。
    论文注：未检测 Cr、Zn 的，从矿山类型看未受污染，按无污染(=1)处理。
    heavy_metals : dict，键为金属名，值为含量 mg/kg；缺失键视为未检测。
    limits : 可选，金属 → 极值；不传则用代码常量（界面改过就传库里的值）。
    """
    table = limits or HEAVY_METAL_LIMIT
    total = 0.0
    for metal, extreme in table.items():
        value = heavy_metals.get(metal)
        total += std_cost(value, extreme) if value is not
    None else 1.0
    return total / len(table)
def standardize_unit(unit_type, data, weights=None,
    settings=None):
    """对单个微地质单元的全部指标做标准化。
    参数
    ----
    unit_type : str，岩质边坡 / 碎石边坡 / 平台
    data : dict，实测数据，字段见接口契约：
```

```
    soil_ratio, thickness, ph, organic_matter,
heavy_metals(dict),
    coverage, shannon, slope, aspect
weights : dict，该单元类型的权重（app.utils.constants.DEFAULT_WEIGHTS 结构），
            未提供时用默认值。
settings : app.services.unit_config.UnitSettings，图斑级的基准值与理想取值。
            不传则全部按代码常量 —— 历史数据与验证算例走这条路径，结果不变。
返回
----
dict：{ 一级指标： { 二级指标： （实测值， 标准化值， 权重， 贡献） } }
标准化值可能为 None，表示该指标不适用或方法待定。
"""
from app.utils.constants import DEFAULT_WEIGHTS
from app.services import unit_config as uc
if weights is None:
    weights = DEFAULT_WEIGHTS[unit_type]
if settings is None:
    settings = uc.UnitSettings()
# 理想取值：库里有数字就用库里的，否则用常量
def ideal(code, fallback):
    return settings.ideal(code, fallback)
l1_enabled = weights["l1"].keys()
l2w = weights["l2"]
result = {}
# 一级指标是否启用：库里配过就按库里的（界面上的「启用单元」），没配按常量
def group_on(l1_name, l1_code):
    return settings.group_enabled(unit_type, l1_name,
l1_code) is not False
# ---- 土壤指标 ----
if "土壤指标" in l1_enabled and group_on("土壤指标", "soil"):
    result["土壤指标"] = {
        "土石比": (data.get("soil_ratio"),
                   std_benefit(data.get("soil_ratio"),
                               ideal("soil_ratio",
IDEAL_SOIL_RATIO)),
                   l2w["土壤指标"]["土石比"]),
        "土层厚度": (data.get("thickness"),
                     std_benefit(data.get("thickness"),
                                 ideal("thickness",
IDEAL_THICKNESS)),
                     l2w["土壤指标"]["土层厚度"]),
        "pH": (data.get("ph"),
               std_mid_ph(data.get("ph"),
*IDEAL_PH_RANGE),
               l2w["土壤指标"]["pH"]),
        "有机质": (data.get("organic_matter"),
                   std_benefit(data.get("organic_matter")
,
```

```
                                                ideal("organic_matter",
    IDEAL_ORGANIC)),
                            l2w["土壤指标"]["有机质"]),
            "污染指标": (data.get("heavy_metals") or {},
                         pollution_score(data.get("heavy_meta
    ls") or {},

    uc.metal_limits(settings)),
                         l2w["土壤指标"]["污染指标"]),
        }
    # ---- 植被指标 ----
    if "植被指标" in l1_enabled and group_on("植被指标", "veg"):
        result["植被指标"] = {
            "植被覆盖度": (data.get("coverage"),
                           std_benefit(data.get("coverage"),
                                       ideal("coverage",
    IDEAL_COVERAGE)),
                           l2w["植被指标"]["植被覆盖度"]),
            "生物多样性": (data.get("shannon"),
                           std_benefit(data.get("shannon"),
                                       ideal("shannon",
    IDEAL_SHANNON)),
                           l2w["植被指标"]["生物多样性"]),
        }
    # ---- 地形指标 ----
    if "地形指标" in l1_enabled and group_on("地形指标", "terrain"):
        result["地形指标"] = {
            "坡度": (data.get("slope"),
                     std_slope(data.get("slope")),
                     l2w["地形指标"]["坡度"]),
            "坡向": (data.get("aspect"),
                     aspect_score(data.get("aspect")),
                     l2w["地形指标"]["坡向"]),
        }
    return result
# ==== app/services/weight_service.py ====
# -*- coding: utf-8 -*-
"""权重方案的存取与生效权重解析。
系统有两套权重来源：
1. 论文表 3 给出的默认权重（app.utils.constants.DEFAULT_WEIGHTS），
   是**权威取值来源**，案例复算与报告结论以此为准；
2. 通过层次分析法在本系统中实际算出的权重，保存在
   weight_schemes / weight_items 两张表中，可标记为「生效」。
本模块负责把这两者合成为计算引擎直接可用的嵌套权重结构：
以论文表 3 为底，用生效方案中已保存的条目覆盖。
没有任何生效方案时，即完全使用论文表 3 的权重。
这样既保证默认行为与论文一致，又使 AHP 计算结果真正参与评价，
而不是算完即弃。
"""
import copy
```

```
from app.models import db, WeightScheme, WeightItem
from app.utils.constants import DEFAULT_WEIGHTS
from app.services.indicator_service import TREE
#  方案名称：每个微地质单元类型固定一个可生效的方案
ACTIVE_NAME = "自定义权重方案（AHP 计算结果）"
def _code_maps():
    """由指标树导出 编码 → 名称 的映射，避免名称在两处各写一遍。"""
    l1, l2 = {}, {}
    for row in TREE:
        code, name, level, parent = row[0], row[1], row[2],
    row[3]
        if level == 1:
            l1[code] = name
        elif level == 2:
            l2[code] = (l1.get(parent, parent), name)
    return l1, l2
def effective_weights(unit_type):
    """返回该单元类型当前生效的权重。
    返回 (weights, source_label, scheme)
        weights       嵌套权重结构，可直接交给标准化与评价引擎
        source_label  权重来源说明，用于界面与报告标注
        scheme        生效的权重方案对象，无自定义方案时为 None
    """
    base = DEFAULT_WEIGHTS.get(unit_type)
    if base is None:
        return None, "未定义该微地质单元类型的权重", None
    scheme = WeightScheme.query.filter_by(unit_type=unit_typ
    e,

    name=ACTIVE_NAME).first()
    if scheme is None:
        return copy.deepcopy(base), "论文表 3 默认权重", None
    merged = copy.deepcopy(base)
    l1_map, l2_map = _code_maps()
    applied = 0
    for item in WeightItem.query.filter_by(scheme_id=scheme.
    id).all():
        code, w = item.indicator_code, item.weight
        if w is None or w <= 0:
            continue
        if code in l1_map and l1_map[code] in merged["l1"]:
            merged["l1"][l1_map[code]] = w
            applied += 1
        elif code in l2_map:
            l1_name, l2_name = l2_map[code]
            if l1_name in merged["l2"] and l2_name in
    merged["l2"][l1_name]:
                merged["l2"][l1_name][l2_name] = w
                applied += 1
    if applied == 0:
```

```
        return copy.deepcopy(base), "论文表 3 默认权重", None
    return merged, "自定义方案（%d 项覆盖论文表 3）" % applied, scheme
def save_weights(unit_type, code_weight_pairs, note=None):
    """把一组「指标编码 → 权重」写入生效方案（已存在则更新对应条目）。
    仅覆盖本次传入的条目，其余条目保留，便于分层次（一级 / 二级）
    分次录入而无需一次填完。
    """
    scheme = WeightScheme.query.filter_by(unit_type=unit_type,
                                          name=ACTIVE_NAME).first()
    if scheme is None:
        scheme = WeightScheme(unit_type=unit_type, name=ACTIVE_NAME,
                              level=1, is_default=True)
        db.session.add(scheme)
        db.session.flush()
    if note:
        scheme.level = scheme.level or 1
    if not scheme.is_default:
        scheme.is_default = True
    written = 0
    for code, w in code_weight_pairs:
        if w is None or w <= 0:
            continue
        item = WeightItem.query.filter_by(scheme_id=scheme.id,
                                          indicator_code=code).first()
        if item is None:
            item = WeightItem(scheme_id=scheme.id, indicator_code=code)
            db.session.add(item)
        item.weight = float(w)
        written += 1
    db.session.commit()
    return scheme, written
def reset_weights(unit_type):
    """删除该单元类型的自定义方案，恢复为论文表 3 的默认权重。"""
    schemes = WeightScheme.query.filter_by(unit_type=unit_type,
                                           name=ACTIVE_NAME).all()
    n = 0
    for s in schemes:
        WeightItem.query.filter_by(scheme_id=s.id).delete(
            synchronize_session=False)
        db.session.delete(s)
        n += 1
    db.session.commit()
```

```
    return n
def scheme_items(unit_type):
    """返回该单元类型已保存的自定义权重条目 {指标编码： 权重}。"""
    scheme = WeightScheme.query.filter_by(unit_type=unit_type,
                                          name=ACTIVE_NAME).first()
    if scheme is None:
        return {}, None
    items = {i.indicator_code: i.weight
             for i in WeightItem.query.filter_by(scheme_id=scheme.id).all()}
    return items, scheme
def summary():
    """各单元类型的权重来源概览，供权重首页展示。"""
    rows = []
    for unit_type in DEFAULT_WEIGHTS:
        weights, label, scheme = effective_weights(unit_type)
        rows.append({
            "unit_type": unit_type,
            "label": label,
            "custom": scheme is not None,
        })
    return rows
# ==== app/services/evaluation_service.py ====
# -*- coding: utf-8 -*-
"""适宜性评价引擎。
按论文式（9） 实现：
- 单元级：F' = Σ（二级指标权重 × 该二级指标下三级指标标准化值）
- 矿山级：F = Σ（Si × F'i），Si 为各微地质单元面积占比
- 等级判定按论文表 4。
"""
from app.utils.constants import DEFAULT_WEIGHTS, grade_of
from app.services import standardization as std
def score_unit(unit_type, data, weights=None,
               settings=None):
    """计算单个微地质单元的评价因子 F'。
    参数
    ----
    weights  ： 该单元类型的权重；不传用论文默认权重
    settings ： 图斑级标准化参数（基准值与理想取值）；不传全部按代码常量
    返回
    ----
    dict：{ l1_scores： {一级指标： 得分}, unit_score： F', details：
    [...] }
    若某一级指标下存在"不适用"（标准化值为 None）的二级指标，
    则按其余项权重归一化后计算，保证单元得分始终有定义。
    """
    if weights is None:
```

```
        weights = DEFAULT_WEIGHTS[unit_type]
    std_result = std.standardize_unit(unit_type, data,
    weights, settings)
    l1_weights = weights["l1"]
    l1_scores = {}
    details = []
    for l1_name, items in std_result.items():
        wsum = 0.0
        acc = 0.0
        for l2_name, (raw, std_value, w2) in items.items():
            if std_value is None:
                # 不适用项：不计入
                details.append({
                    "l1": l1_name, "l2": l2_name, "raw":
    raw,
                    "std_value": None, "weight": w2,
    "contribution": None,
                })
                continue
            contribution = std_value * w2
            acc += contribution
            wsum += w2
            details.append({
                "l1": l1_name, "l2": l2_name, "raw": raw,
                "std_value": std_value, "weight": w2,
                "contribution": contribution,
            })
        l1_scores[l1_name] = acc / wsum if wsum > 0 else 0.0
    # 一级指标权重按实际参与计算的指标归一化。
    # 论文表 3 的权重本身已归一（和为 1），此处为恒等变换；
    # 若某层指标缺失或自定义权重不完整，可避免单元得分被整体压低。
    lw_sum = sum(l1_weights[k] for k in l1_scores if k in
    l1_weights)
    if lw_sum > 0:
        unit_score = sum(l1_weights[k] * l1_scores[k]
                         for k in l1_scores if k in
    l1_weights) / lw_sum
    else:
        unit_score = 0.0
    return {"l1_scores": l1_scores, "unit_score":
    unit_score, "details": details}
def score_mine(units, weights_map=None, settings=None):
    """计算整个矿山的评价因子 F 与适宜性等级。
    参数
    ----
    units : list[dict]，每个元素：
        {unit_type, ratio, data}
        ratio 为该单元面积占比（0~1），data 为实测数据 dict。
    weights_map : dict，{单元类型： 权重结构}。未提供的单元类型
        使用论文表 3 的默认权重。权重可来自 AHP 计算并经
```

```
        weight_service 保存生效，从而真正参与评价。
    settings : 图斑级标准化参数（参照生态系统基准、指标理想取值），
        由 app.services.unit_config.for_patch() 生成；不传按代码常量。
    返回
    ----
    dict:{ final_score, grade, grade_info, unit_results }
    """
    weights_map = weights_map or {}
    total = 0.0
    unit_results = []
    for u in units:
        r = score_unit(u["unit_type"], u["data"],
                       weights_map.get(u["unit_type"]),
    settings)
        weighted = u["ratio"] * r["unit_score"]
        total += weighted
        unit_results.append({
            "unit_type": u["unit_type"],
            "ratio": u["ratio"],
            "unit_score": r["unit_score"],
            "weighted": weighted,
            "l1_scores": r["l1_scores"],
            "details": r["details"],
        })
    grade_info = grade_of(total)
    return {
        "final_score": total,
        "grade": grade_info["grade"],
        "grade_info": grade_info,
        "unit_results": unit_results,
    }
# ==== app/services/report_service.py ====
# -*- coding: utf-8 -*-
"""报告生成：生成 Word 评价报告。
依赖 python-docx。报告结构：
标题 → 图斑信息 → 评价结论 → 各微地质单元得分 → 完整得分明细 → 生成时间。
"""
import io
import json
from docx import Document
from docx.shared import Pt, Cm
from docx.oxml.ns import qn
from docx.enum.text import WD_ALIGN_PARAGRAPH
from app.utils.timefmt import to_local, now_utc
from app.services import workflow as wf
def _set_run(run, text=None, size=12, bold=False):
    if text is not None:
        run.text = text
    run.font.name = "Times New Roman"
    run.font.size = Pt(size)
```

```
    run.font.bold = bold
    # 中文字体
    run._element.rPr.rFonts.set(qn("w:eastAsia"), "宋体")
    return run
def _para(doc, text, size=12, bold=False, align=None,
    space_after=6):
    p = doc.add_paragraph()
    if align is not None:
        p.alignment = align
    p.paragraph_format.space_after = Pt(space_after)
    _set_run(p.add_run(), text, size, bold)
    return p
def _heading(doc, text, level=1):
    h = doc.add_heading(text, level=level)
    for run in h.runs:
        run.font.name = "Times New Roman"
        run._element.rPr.rFonts.set(qn("w:eastAsia"), "黑体")
    return h
def _table(doc, headers, rows, widths=None):
    t = doc.add_table(rows=1, cols=len(headers))
    t.style = "Table Grid"
    hdr = t.rows[0].cells
    for i, htext in enumerate(headers):
        _set_run(hdr[i].paragraphs[0].add_run(), htext,
    10.5, bold=True)
    for row in rows:
        cells = t.add_row().cells
        for i, val in enumerate(row):
            _set_run(cells[i].paragraphs[0].add_run(),
    str(val), 10.5)
    return t
def build_report(patch, mine_result, record=None,
    weak=None, operator=None):
    """生成评价报告 Word 文档，返回 BytesIO。
    参数
    ----
    patch : MinePatch，图斑对象
    mine_result : evaluation_service.score_mine 的返回 dict
    record : EvaluationResult，用于标注计算人与参数快照来源
    weak : [(单元, 指标, 实测值, 标准化值, 提升空间)]，短板提示
    operator : str，计算人显示名
    """
    doc = Document()
    # 页边距
    for section in doc.sections:
        section.top_margin = Cm(2.5)
        section.bottom_margin = Cm(2.5)
        section.left_margin = Cm(2.5)
        section.right_margin = Cm(2.5)
    _heading(doc, "历史遗留废弃矿山自然恢复适宜性评价报告", 0)
```

```
_para(doc, "软件：历史遗留废弃矿山自然恢复适宜性评价系统 V1.0",
        size=10.5, align=WD_ALIGN_PARAGRAPH.CENTER)
_para(doc, "生成时间：%s" % to_local(now_utc(), "%Y-%m-%d
%H:%M:%S"),
        size=10.5, align=WD_ALIGN_PARAGRAPH.CENTER,
space_after=12)
#  计算人与参数来源：报告要能说明「谁算的、用的什么参数」
if record is not None:
    from app.views.evaluate import param_source_summary
    try:
        params = json.loads(record.params_json) if
record.params_json else {}
    except ValueError:
        params = {}
    meta = [
        ("计算人", operator or "—"),
        ("计算时间", to_local(record.created_at, "%Y-%m-%d
%H:%M:%S")),
        ("评价记录编号", record.id),
        ("计算参数", param_source_summary(params)),
    ]
    _table(doc, ["项目", "内容"], meta)
#  图斑信息
_heading(doc, "一、图斑基本信息", 1)
info = [
    ("图斑名称", patch.name),
    ("所在行政区", patch.admin_region or "—"),
    ("图斑面积（公顷）", str(patch.area_ha) if patch.area_ha is
not None else "—"),
    ("开采矿种", patch.mineral_type or "—"),
    ("开采方式", patch.mining_method or "—"),
]
_table(doc, ["项目", "内容"], info)
#  评价结论
_heading(doc, "二、评价结论", 1)
gi = mine_result["grade_info"]
concl = [
    ("最终评价因子 F", "%.4f" % mine_result["final_score"]),
    ("适宜性等级", mine_result["grade"]),
    ("判定依据", gi["meaning"]),
    ("推荐措施", gi["measure"]),
]
_table(doc, ["项目", "结果"], concl)
#  各单元得分
_heading(doc, "三、各微地质单元得分", 1)
rows = []
for u in mine_result["unit_results"]:
    rows.append((u["unit_type"], "%.2f%%" % (u["ratio"]
* 100),
                 "%.4f" % u["unit_score"], "%.4f" %
```

```
u["weighted"]))
_table(doc, ["微地质单元", "面积占比", "单元评价因子 F'", "加权贡献"],
rows)
# 完整得分明细
_heading(doc, "四、得分明细", 1)
for u in mine_result["unit_results"]:
    _para(doc, "%s（占比 %.2f%%）" % (u["unit_type"],
u["ratio"] * 100),
          size=11, bold=True, space_after=4)
    detail_rows = []
    for d in u["details"]:
        raw = "—" if d["raw"] is None else d["raw"]
        std = "—" if d["std_value"] is None else "%.4f"
% d["std_value"]
        w = "%.4f" % d["weight"]
        c = "—" if d["contribution"] is None else
"%.4f" % d["contribution"]
        detail_rows.append((d["l1"], d["l2"], raw, std,
w, c))
    _table(doc, ["一级指标", "二级指标", "实测值", "标准化值", "权重",
"贡献"],
           detail_rows)
    # 一级指标得分
    l1_row = []
    for k, v in u["l1_scores"].items():
        l1_row.append("%s=%.4f" % (k, v))
    _para(doc, "一级指标得分：" + ",".join(l1_row), size=10.5,
space_after=10)
# 数据完整度：未检测项必须写明，否则读报告的人会以为这些项是实测达标
undetected = wf.undetected_metals(patch.id)
if undetected:
    _heading(doc, "五、数据完整度说明", 1)
    parts = ["%s：%s" % (t, "、".join(m)) for t, m in
undetected]
    _para(doc, "以下重金属项未检测，按论文注记以「未受污染」计入污染指标"
               "（并非实测达标）：%s。" % "；".join(parts),
size=10.5)
# 短板提示：告诉使用方先改哪几项最有效
if weak:
    _heading(doc, "六、短板提示", 1)
    _para(doc, "按「权重 ×（1 − 标准化值）」排序，提升下列指标对总分的拉动最大。",
          size=10.5, space_after=4)
    _table(doc, ["微地质单元", "指标", "实测值", "标准化值", "提升空间"],
           [(u, l2, "—" if raw is None else raw, "%.4f"
% std,
             "%.4f" % gain) for u, l2, raw, std, gain
in weak])
# 签字栏：报告是评价结论的书面载体，留出复核与签发的位置（与 PDF 版一致）
_para(doc, "", size=10.5, space_after=18)
_para(doc, "评价人（签字）：_____ ")
```

```
                    "日期：_____________", size=10.5)
    buf = io.BytesIO()
    doc.save(buf)
    buf.seek(0)
    return buf
# ==== app/services/indicator_service.py ====
# -*- coding: utf-8 -*-
"""指标定义维护。
把论文表 2/表 3 的指标体系写入 indicators 表，使指标名称、单位、
理想值与取值依据可在界面上查看维护，而不再只存在于代码常量中。
代码常量（app.utils.constants）仍是计算时的**权威取值来源**；
本表用于展示、核对与记录取值依据。二者由 seed_indicators() 保持同步。
"""
from app.models import db, Indicator
from app.utils.constants import (
    UNIT_TYPES, UNIT_ENABLED_INDICATORS, IDEAL_SOIL_RATIO,
    IDEAL_THICKNESS,
    IDEAL_PH_RANGE, IDEAL_ORGANIC, IDEAL_COVERAGE,
    IDEAL_SHANNON,
    HEAVY_METAL_LIMIT, SLOPE_BREAK, ASPECT_SCORES,
)
ALL_UNITS = ",".join(UNIT_TYPES)
# (code, name, level, parent, unit, ideal_value,
    ideal_type, enabled_units, desc, is_constraint)
TREE = [
    # ---- 一级指标 ----
    ("soil", "土壤指标", 1, None, None, None, None, "碎石边坡,平台",
    "论文表 2 评价指标之一级指标", False),
    ("veg", "植被指标", 1, None, None, None, None, ALL_UNITS,
    "论文表 2 评价指标之一级指标", False),
    ("terrain", "地形指标", 1, None, None, None, None,
    "岩质边坡,碎石边坡", "论文表 2 评价指标之一级指标", False),
    # ---- 二级指标：土壤 ----
    ("soil_ratio", "土石比", 2, "soil", "—",
    str(IDEAL_SOIL_RATIO), "benefit", "碎石边坡,平台",
     "论文 4.2.2 节：理想值 0.8，根据矿山修复实际情况确定", False),
    ("thickness", "土层厚度", 2, "soil", "cm",
    str(IDEAL_THICKNESS), "benefit", "碎石边坡,平台",
     "论文 4.2.2 节：理想值 20 cm", False),
    ("ph", "pH", 2, "soil", "—", "%g~%g" % IDEAL_PH_RANGE,
    "mid", "碎石边坡,平台",
     "论文 4.2.2 节：pH 用适中指标的标准化方法，理想区间 %g~%g" %
    IDEAL_PH_RANGE, False),
    ("organic_matter", "有机质", 2, "soil", "g/kg",
    str(IDEAL_ORGANIC), "benefit", "碎石边坡,平台",
     "《全国第二次土壤普查养分等级划分标准》二等（较丰富）最低值 30 g/kg", False),
    ("pollution", "污染指标", 2, "soil", "mg/kg", "见三级指标",
    "cost", "碎石边坡,平台",
     "同时作为限制性指标与评价指标；8 种重金属等权合成（各 0.125）", True),
    # ---- 二级指标：植被 ----
```

```
        ("coverage", "植被覆盖度", 2, "veg", "%",                                    2001
        str(IDEAL_COVERAGE), "benefit", ALL_UNITS,                                   2002
          "论文 4.2.2 节：参照生态系统取值 75%", False),                               2003
        ("shannon", "生物多样性", 2, "veg", "—", str(IDEAL_SHANNON),                 2004
        "benefit", ALL_UNITS,                                                        2005
          "Shannon-Wiener 多样性指数，参照生态系统取值 0.7", False),                  2006
        # ---- 二级指标：地形 ----                                                   2007
        ("slope", "坡度", 2, "terrain", "°", "%g° 分界" %                            2008
        SLOPE_BREAK, "cost", "岩质边坡,碎石边坡",                                    2009
          "论文 4.2.2 节引《新一轮退耕还林还草总体方案》发改西部〔2014〕1772 号：25°  2010
        为陡坡重要分界", False),                                                     2011
        ("aspect", "坡向", 2, "terrain", "—",                                        2012
        "阳坡0.2/阴阳坡0.6/阴坡1.0", "discrete", "岩质边坡,碎石边坡",               2013
          "论文 4.2.2 节：阴坡及半阴坡水分条件更优，故赋值高于阳坡", False),        2014
]                                                                                    2015
# 污染指标下的三级指标（8 种重金属）                                                2016
for _m, _limit in HEAVY_METAL_LIMIT.items():                                         2017
        TREE.append((                                                                2018
                "hm_" + _m, _m, 3, "pollution", "mg/kg",                             2019
        str(_limit), "cost",                                                         2020
                "碎石边坡,平台",                                                     2021
                "《绿化种植土壤》CJ/T 340—2016 土壤重金属含量 IV 级最高值 %g mg/kg" % 2022
        _limit,                                                                      2023
                False,                                                               2024
        ))                                                                           2025
def seed_indicators():                                                               2026
        """写入/更新指标定义（幂等）。返回写入条数。"""                              2027
        code_map = {                                                                 2028
                "benefit": "越大越优", "cost": "越小越优",                           2029
                "mid": "适中型", "discrete": "离散赋值",                             2030
        }                                                                            2031
        n = 0                                                                        2032
        for (code, name, level, parent, unit, ideal, itype,                          2033
        units, desc, is_con) in TREE:                                                2034
                ind = Indicator.query.filter_by(code=code).first()                   2035
                if ind is None:                                                      2036
                        ind = Indicator(code=code)                                   2037
                        db.session.add(ind)                                          2038
                ind.name = name                                                      2039
                ind.level = level                                                    2040
                ind.parent_code = parent                                             2041
                ind.unit = unit                                                      2042
                ind.ideal_value = ideal                                              2043
                ind.ideal_type = itype                                               2044
                ind.enabled_units = units                                            2045
                ind.description = desc                                               2046
                ind.is_constraint = is_con                                           2047
                ind.is_evaluative = (level in (2, 3))                                2048
                n += 1                                                               2049
        db.session.commit()                                                          2050
```

```
    return n
def ideal_type_label(code):
    return {"benefit": "越大越优", "cost": "越小越优",
            "mid": "适中型", "discrete": "离散赋值"}.get(code,
    code or "—")
# ==== app/services/import_service.py ====
# -*- coding: utf-8 -*-
"""指标数据批量导入。
支持 CSV 与 Excel（.xlsx/.xls）。一行对应一个微地质单元，
按「单元类型」匹配图斑内已有的微地质单元后写入指标数据。
表头约定（缺失列按不适用处理）：
    单元类型， 土石比， 土层厚度， pH， 有机质，
    Hg, As, Ni, Cu, Cd, Pb, Cr, Zn,
    植被覆盖度， Shannon， 坡度， 坡向
"""
import csv
import io
import json
from app.utils.constants import HEAVY_METALS, ASPECT_TYPES,
    UNIT_TYPES
# 表头（顺序即导出模板顺序）
HEADERS = (["单元类型", "土石比", "土层厚度", "pH", "有机质"]
           + list(HEAVY_METALS)
           + ["植被覆盖度", "Shannon", "坡度", "坡向"])
FIELD_ALIASES = {
    "单元类型": "unit_type",
    "土石比": "soil_ratio",
    "土层厚度": "thickness",
    "pH": "ph", "PH": "ph", "ph": "ph",
    "有机质": "organic_matter",
    "植被覆盖度": "coverage",
    "Shannon": "shannon", "shannon": "shannon", "生物多样性":
    "shannon",
    "坡度": "slope",
    "坡向": "aspect",
}
for _m in HEAVY_METALS:
    FIELD_ALIASES[_m] = "hm_" + _m
def build_template_csv():
    """生成导入模板（含表头与示例行）。"""
    buf = io.StringIO()
    w = csv.writer(buf)
    w.writerow(HEADERS)
    # 示例：碎石边坡一行，说明每列含义
    row = {h: "" for h in HEADERS}
    row.update({"单元类型": "碎石边坡", "土石比": "0.34", "土层厚度": "10",
                "pH": "9.24", "有机质": "4.89", "Hg": "0.05",
    "As": "5.17",
                "Ni": "12.60", "Cu": "11.70", "Cd": "0.04",
    "Pb": "6.50",
```

```
                    "植被覆盖度": "87.92", "Shannon": "0.60",
                    "坡度": "28.75", "坡向": "阳坡"})
    w.writerow([row[h] for h in HEADERS])
    return buf.getvalue()
def _read_rows(filename, stream):
    """读取上传文件为「表头 -> 值」的字典列表。"""
    ext = filename.rsplit(".", 1)[-1].lower()
    if ext == "csv":
        text = stream.read().decode("utf-8-sig",
errors="replace")
        reader = csv.DictReader(io.StringIO(text))
        return [dict(r) for r in reader]
    if ext in ("xlsx", "xls"):
        from openpyxl import load_workbook
        wb = load_workbook(io.BytesIO(stream.read()),
data_only=True)
        ws = wb.active
        rows = list(ws.iter_rows(values_only=True))
        if not rows:
            return []
        header = [str(c).strip() if c is not None else ""
for c in rows[0]]
        out = []
        for r in rows[1:]:
            if all(c is None or str(c).strip() == "" for c
in r):
                continue
            out.append({header[i]: (r[i] if i < len(r) else
None)
                        for i in range(len(header)) if
header[i]})
        return out
    raise ValueError("不支持的文件类型：.%s" % ext)
def _num(value):
    if value is None:
        return None, None
    s = str(value).strip()
    if s == "":
        return None, None
    try:
        return float(s), None
    except ValueError:
        return None, "「%s」不是有效数值" % s
def parse_upload(filename, stream, unit_type_map):
    """解析上传文件。
    参数
    ----
    unit_type_map : dict, {单元类型: MicroUnit 对象}
    返回
    ----
```

```
    (parsed, errors)
       parsed : list[dict]，每项 {unit_type, fields{...},
    heavy_metals{}}
       errors : list[str]，逐行问题说明
    """
    rows = _read_rows(filename, stream)
    parsed, errors = [], []
    if not rows:
        return [], ["文件为空或没有数据行"]
    # 表头校验
    unknown = [h for h in (rows[0].keys() if rows else [])
    if h not in FIELD_ALIASES]
    if unknown:
        errors.append("存在无法识别的列，将被忽略：%s" %
    "、".join(map(str, unknown)))
    for idx, raw in enumerate(rows, start=2):       # 第 1 行是表头
        rec = {}
        for k, v in raw.items():
            key = FIELD_ALIASES.get(str(k).strip())
            if key:
                rec[key] = v
        unit_type = str(rec.get("unit_type") or "").strip()
        if not unit_type:
            errors.append("第 %d 行：缺少「单元类型」" % idx)
            continue
        if unit_type not in unit_type_map:
            errors.append("第 %d 行：图斑内不存在单元类型「%s」（现有：%s）"
                          % (idx, unit_type,
    "、".join(unit_type_map) or "无"))
            continue
        if unit_type not in UNIT_TYPES:
            errors.append("第 %d 行：单元类型「%s」不是系统支持的三种之一"
                          % (idx, unit_type))
            continue
        fields, heavy = {}, {}
        ok = True
        for key, val in rec.items():
            if key in ("unit_type",):
                continue
            if key.startswith("hm_"):
                n, err = _num(val)
                if err:
                    errors.append("第 %d 行：%s %s" % (idx,
    key[3:], err))
                    ok = False
                elif n is not None:
                    heavy[key[3:]] = n
                continue
            if key == "aspect":
                s = str(val).strip() if val is not None
```

```
    else ""                                                          2201
                if s == "":                                          2202
                    fields["aspect"] = None                          2203
                elif s in ASPECT_TYPES:                              2204
                    fields["aspect"] = s                             2205
                else:                                                2206
                    errors.append("第 %d 行：坡向「%s」无效（应为 %s）"  2207
                                  % (idx, s,                         2208
    "、".join(ASPECT_TYPES)))                                        2209
                    ok = False                                       2210
                continue                                             2211
            n, err = _num(val)                                       2212
            if err:                                                  2213
                errors.append("第 %d 行：%s %s" % (idx, key,         2214
    err))                                                            2215
                ok = False                                           2216
            else:                                                    2217
                fields[key] = n                                      2218
        if ok:                                                       2219
            parsed.append({"unit_type": unit_type,                   2220
    "fields": fields,                                                2221
                           "heavy_metals": heavy})                   2222
    return parsed, errors                                            2223
# ==== app/services/workflow.py ====                                2224
# -*- coding: utf-8 -*-                                            2225
"""评价流程前置条件（定量评价与报告输出的流程门禁）。                  2226
论文 4.1 节确定的评价流程存在严格的先后关系：                          2227
    建立图斑 → 限制性条件判定（通过） → 划分微地质单元并计算面积占比    2228
    → 逐单元录入指标数据 → 计算评价因子并判定等级 → 输出报告           2229
论文原文：「先进行政策、位置约束地质灾害判断……对于满足限制性条件的      2230
废弃矿山，按照地形地貌划分，将其划分为平台、碎石边坡和岩质边坡三种      2231
地质微单元类型，并计算各种地形面积占比。」限制性条件判定（一票否决）    2232
在前，微地质单元划分在后，顺序不可颠倒。                              2233
本模块把该顺序固化为可检查的状态。后续模块在入口处调用它，              2234
前置步骤未完成时直接拦截并给出「缺什么、去哪补」的提示，                2235
避免在条件不具备时产出看似合理的评价结论。                            2236
"""                                                                  2237
from app.models import (db, MicroUnit, IndicatorData,                2238
    ConstraintCheck,                                                 2239
                        EvaluationResult)                            2240
from app.services import unit_config                                 2241
# 指标数据非空字段（用于判断某单元是否已录入数据）                    2242
DATA_FIELDS = ("soil_ratio", "thickness", "ph",                      2243
    "organic_matter",                                                2244
               "coverage", "shannon", "slope", "aspect")             2245
def _has_data(ind):                                                  2246
    if ind is None:                                                  2247
        return False                                                 2248
    if any(getattr(ind, f, None) is not None for f in                2249
    DATA_FIELDS):                                                    2250
```

```
            return True                                                    2251
        return bool(ind.heavy_metals_dict())                               2252
def _data_of(ind):                                                         2253
    """把 IndicatorData 行转成标准化接口要的 dict。"""                      2254
    if ind is None:                                                        2255
        return {}                                                          2256
    data = {f: getattr(ind, f, None) for f in DATA_FIELDS}                 2257
    data["heavy_metals"] = ind.heavy_metals_dict()                         2258
    return data                                                            2259
def patch_progress(patch_id):                                              2260
    """检查某图斑的评价流程完成情况。                                      2261
    返回 dict：                                                            2262
        units         {done, count, ratio_sum, detail}                     2263
        constraint    {done, passed, detail}                               2264
        data          {done, ready, missing, detail}                       2265
        blocking      未满足的前置步骤列表（按流程顺序）                   2266
        ready         是否可进入定量评价                                   2267
    """                                                                    2268
    units = MicroUnit.query.filter_by(patch_id=patch_id).all               2269
    ()                                                                     2270
    ratio_sum = sum(u.ratio or 0.0 for u in units)                         2271
    # ---- 步骤一：微地质单元 ----                                         2272
    units_step = {                                                         2273
        "done": bool(units),                                               2274
        "count": len(units),                                               2275
        "ratio_sum": ratio_sum,                                            2276
        "detail": ("已划分 %d 个微地质单元，面积占比合计 %.2f%%"           2277
                   % (len(units), ratio_sum * 100))                        2278
                  if units else "尚未划分微地质单元",                      2279
    }                                                                      2280
    # ---- 步骤二：限制性条件判定 ----                                     2281
    cc = ConstraintCheck.query.filter_by(patch_id=patch_id)                2282
    \                                                                      2283
        .order_by(ConstraintCheck.id.desc()).first()                       2284
    if cc is None:                                                         2285
        constraint_step = {"done": False, "passed": False,                 2286
                           "detail": "尚未执行限制性条件判定"}             2287
    elif cc.passed:                                                        2288
        constraint_step = {"done": True, "passed": True,                   2289
                           "detail": "限制性条件判定已通过"}               2290
    else:                                                                  2291
        constraint_step = {"done": True, "passed": False,                  2292
                           "detail": "限制性条件判定未通过（%s）" %        2293
    (cc.reason or "")}                                                     2294
    # ---- 步骤三：指标数据 ----                                          2295
    # 判断标准是「该单元类型启用的必填项是否齐全」，而不是「有没有填过任意一个字段」。  2296
    # 早先只要 8 个字段里有一个非空就算齐，于是碎石边坡只填一个坡度也能跑出            2297
    # F=0.69「基本适宜」；而 score_unit 又会把缺的项剔掉、把权重摊给其余指标，        2298
    # 结论方向还偏乐观。污染指标按论文注记允许「未检测」，不算缺。                    2299
    settings = unit_config.for_patch(patch_id)                             2300
```

```
    missing = []                    #  完全没录数据的单元类型                                  2301
    incomplete = []                 #  录了但不齐的单元类型  →  [(单元类型，  [缺的字段])]  2302
    for u in units:                                                                          2303
        ind = IndicatorData.query.filter_by(unit_id=u.id).fi                                 2304
    rst()                                                                                    2305
        data = _data_of(ind)                                                                 2306
        gaps = settings.missing_fields(u.unit_type, data)                                    2307
        if not _has_data(ind):                                                               2308
            missing.append(u.unit_type)                                                      2309
        elif gaps:                                                                           2310
            incomplete.append((u.unit_type, gaps))                                           2311
    if not units:                                                                            2312
        data_step = {"done": False, "ready": False,                                          2313
    "missing": [],                                                                           2314
                     "detail": "尚无可录入数据的微地质单元"}                                 2315
    elif missing:                                                                            2316
        data_step = {"done": False, "ready": False,                                          2317
    "missing": missing,                                                                      2318
                     "detail": "以下微地质单元尚未录入指标数据：%s"                          2319
                               % "、".join(missing)}                                         2320
    elif incomplete:                                                                         2321
        data_step = {"done": False, "ready": False,                                          2322
                     "missing": [t for t, _ in incomplete],                                  2323
                     "detail": "以下微地质单元的必填指标未填齐：%s"                          2324
                               % "；".join("%s缺 %s" % (t,                                   2325
    "、".join(g))                                                                            2326
                                           for t, g in                                       2327
    incomplete)}                                                                             2328
    else:                                                                                    2329
        data_step = {"done": True, "ready": True,                                            2330
    "missing": [],                                                                           2331
                     "detail": "全部 %d 个微地质单元的指标数据已录入" %                      2332
    len(units)}                                                                              2333
    #  占比合计必须落在 100% 附近才算通过。只挡「偏小」是不够的：                           2334
    #  库里若出现合计 1.5 的记录（手工改库、导入异常），F = Σ(Si×F'i)                       2335
    #  会被放大到 1.5 倍，所以上下都要挡。                                                  2336
    ratio_ok = 0.999 <= ratio_sum <= 1.001 if units else                                     2337
    False                                                                                    2338
    if units and not ratio_ok:                                                               2339
        units_step["detail"] += "（占比合计不等于 100%，请检查面积录入）"                    2340
    #  ---- 汇总未满足的前置条件（按论文 4.1 节的流程顺序：限制性判定优先） ----           2341
    #  reason 只写「缺什么」这一个事实，不写为什么重要的说明 ——                             2342
    #  缺少什么由 label 指明，怎么补由 fix_label 给出，页面不必再解释一遍。                 2343
    blocking = []                                                                            2344
    if not constraint_step["done"]:                                                          2345
        blocking.append({                                                                    2346
            "step": "constraint", "label": "限制性条件判定未执行",                          2347
            "reason": "五项限制性条件尚未逐项判定。",                                       2348
            "fix_url": "/constraint/%d" % patch_id,                                          2349
    "fix_label": "去执行判定",                                                               2350
```

```
            })
        elif not constraint_step["passed"]:
            blocking.append({
                "step": "constraint", "label": "限制性条件判定未通过",
                "reason": cc.reason or "",
                "fix_url": "/constraint/%d" % patch_id,
        "fix_label": "去复核判定结果",
            })
        if not units_step["done"]:
            blocking.append({
                "step": "units", "label": "未划分微地质单元",
                "reason": "该图斑下没有任何微地质单元。",
                "fix_url": "/unit/%d" % patch_id, "fix_label":
        "去划分",
            })
        elif not ratio_ok:
            blocking.append({
                "step": "units", "label": "面积占比合计不为 100%",
                "reason": "当前合计 %.2f%%。" % (ratio_sum * 100),
                "fix_url": "/unit/%d" % patch_id, "fix_label":
        "去核对面积",
            })
        if units_step["done"] and not data_step["ready"]:
            blocking.append({
                "step": "data", "label": "指标数据未录入完整",
                "reason": data_step["detail"],
                "fix_url": "/indicator/", "fix_label": "去录入",
            })
        return {
            "units": units_step,
            "constraint": constraint_step,
            "data": data_step,
            "blocking": blocking,
            "ready": not blocking,
        }
def undetected_metals(patch_id):
    """列出该图斑各单元「未检测」的重金属，供结果页与报告如实标注。
    论文注记允许未检测的重金属按未受污染计入，但报告里必须写明是哪些 ——
    否则读报告的人会以为这些项是实测达标。
    只列启用了土壤指标的单元：岩质边坡本来就不评价土壤，列 8 项未检测是噪音。
    """
    settings = unit_config.for_patch(patch_id)
    out = []
    for u in MicroUnit.query.filter_by(patch_id=patch_id).al
    l():
        if settings.group_enabled(u.unit_type, "土壤指标",
    "soil") is False:
            continue
        ind = IndicatorData.query.filter_by(unit_id=u.id).fi
    rst()
```

```
            if ind is None:
                continue
            metals = settings.undetected_metals(_data_of(ind))
            if metals:
                out.append((u.unit_type, metals))
    return out
def latest_result(patch_id):
    """取该图斑最近一次的评价记录。"""
    return EvaluationResult.query.filter_by(patch_id=patch_id) \
        .order_by(EvaluationResult.id.desc()).first()
def report_progress(patch_id):
    """报告输出的前置条件：评价流程齐备，且已有评价记录。
    报告本质上是评价结论的呈现，因此必须先真实执行过定量评价，
    导出时读取已保存的评价结果，而不是在导出环节重新计算。
    否则评价完成后再修改指标数据，报告会与系统记录的评价结论不一致，
    同一份评价出现两个数字，结论无法追溯。
    返回 dict：{ready, blocking, result, flow}
        result 为最近一次评价记录（EvaluationResult），无记录时为空。
    """
    flow = patch_progress(patch_id)
    blocking = list(flow["blocking"])
    record = latest_result(patch_id)
    if not blocking and record is None:
        blocking.append({
            "step": "evaluate", "label": "尚未执行适宜性评价",
            "reason": "该图斑还没有评价记录。",
            "fix_url": "/evaluate/%d" % patch_id,
            "fix_label": "去执行评价",
        })
    return {
        "ready": not blocking,
        "blocking": blocking,
        "result": record,
        "flow": flow,
    }
# ==== app/services/dashboard.py ====
# -*- coding: utf-8 -*-
"""工作台数据：统计数字、评价流程步骤状态、侧边栏待办角标。
首页（工作台）需要把「现在有哪些数据」「流程走到哪一步」「哪一步还差活」
一次算清楚，供模板直接渲染。集中在此，避免在视图与模板里各算一遍。
"""
from flask import g
from app.models import (db, Project, MinePatch, MicroUnit,
    IndicatorData,
                        ConstraintCheck, EvaluationResult,
    Standard)
from app.services import workflow as wf
#  评价流程的八个步骤（顺序即论文 4.1 节的作业顺序）
#     key            步骤标识
```

```
#     no            步骤编号（01~08）
#     title         名称
#     icon          图标名（见 templates/_macros.html 的 ICONS）
#     desc          一句话说明
#     href          入口地址
STEPS = [
    dict(key="project", no="01", title="项目与矿山档案",
    icon="folder",
          href="/project/",
          desc="建立评价项目、录入矿山图斑、挂接遥感影像、设置参照生态系统"),
    dict(key="standard", no="02", title="政策与标准知识库",
    icon="book",
          href="/standard/",
          desc="收录地方标准与政策文件，结构化判定规则，支持省际口径比对"),
    dict(key="constraint", no="03", title="限制性指标判定",
    icon="shield",
          href="/constraint/",
          desc="政策、区位约束、土壤污染、地质灾害、耕地属性五项一票否决"),
    dict(key="unit", no="04", title="微地质单元", icon="layers",
          href="/unit/",
          desc="划分岩质边坡、碎石边坡、平台三类单元，自动计算面积占比"),
    dict(key="ahp", no="05", title="指标体系与权重",
    icon="sliders",
          href="/ahp/",
          desc="指标定义维护，判断矩阵录入、权重计算与一致性检验、多专家汇总"),
    dict(key="indicator", no="06", title="指标数据采集",
    icon="database",
          href="/indicator/",
          desc="按微地质单元录入土壤、植被、地形指标，支持 CSV / Excel 批量导入"),
    dict(key="evaluate", no="07", title="适宜性定量评价",
    icon="target",
          href="/evaluate/",
          desc="指标标准化、分单元赋分、面积加权合成，判定适宜性等级"),
    dict(key="report", no="08", title="报告与结果输出",
    icon="file",
          href="/report/",
          desc="结果图表与逐层得分明细，导出 PDF 与 Word 评价报告"),
]
def _has_data(ind):
    return wf._has_data(ind)
#  系统的九个功能模块（登录页介绍用，含不在评价流程内的账号与权限管理）
MODULES = [
    ("01", "账号与权限管理",  "自助注册与登录注销、管理员与操作员两种职务、写操作自动留痕"),
    ("02", "项目与矿山档案",  "建立项目、录入图斑、挂接遥感影像、设置参照生态系统"),
    ("03", "政策与标准知识库",  "标准判定条件结构化为规则，支持省际口径比对"),
    ("04", "限制性指标判定",  "政策、区位、污染、地质灾害、耕地五项一票否决"),
    ("05", "微地质单元管理",  "划分岩质边坡、碎石边坡、平台，自动算面积占比"),
    ("06", "指标体系与权重",  "指标定义维护、AHP 判断矩阵与一致性检验、多专家汇总"),
    ("07", "指标数据采集",  "按单元类型录入土壤、植被、地形指标，支持批量导入"),
    ("08", "适宜性定量评价",  "四类标准化、分单元赋分、面积加权、等级判定"),
```

```
    ("09", "报告与结果输出", "结果图表与逐层得分明细，导出 PDF 与 Word 报告"),
]
def snapshot():
    """一次算出工作台需要的全部数据（同一请求内只算一次）。
    返回 dict：
        stats      {projects, patches, unit_types, standards}
        steps      八个步骤，附 available / pending / note
        badges     {key: 待办数量}，仅含有待办的项
        progress   {patches, units, data_units, ready,
    evaluated}
    """
    cached = getattr(g, "_dash_snapshot", None)
    if cached is None:
        cached = _compute()
        g._dash_snapshot = cached
    return cached
def _compute():
    projects = Project.query.count()
    patches = MinePatch.query.all()
    patch_ids = [p.id for p in patches]
    units = MicroUnit.query.filter(
        MicroUnit.patch_id.in_(patch_ids)).all() if
    patch_ids else []
    unit_by_patch = {}
    for u in units:
        unit_by_patch.setdefault(u.patch_id, []).append(u)
    unit_ids = [u.id for u in units]
    data_unit_ids = set()
    if unit_ids:
        for ind in IndicatorData.query.filter(
                IndicatorData.unit_id.in_(unit_ids)).all():
            if _has_data(ind):
                data_unit_ids.add(ind.unit_id)
    passed_constraint = set()
    if patch_ids:
        for c in ConstraintCheck.query.filter(
                ConstraintCheck.patch_id.in_(patch_ids)).all
    ():
            if c.passed:
                passed_constraint.add(c.patch_id)
    evaluated = set()
    if patch_ids:
        for r in EvaluationResult.query.filter(
                EvaluationResult.patch_id.in_(patch_ids)).al
    l():
            evaluated.add(r.patch_id)
    # ---- 各步骤的待办数量 ----
    pending = {
        "constraint": sum(1 for p in patches if p.id not in
    passed_constraint),
```

```
        "unit": sum(1 for p in patches if not
    unit_by_patch.get(p.id)),
        "indicator": 0,
        "evaluate": 0,
    }
    ready_patches = 0
    for p in patches:
        us = unit_by_patch.get(p.id, [])
        if us and any(u.id not in data_unit_ids for u in
    us):
            pending["indicator"] += 1
        pr = wf.patch_progress(p.id)
        if pr["ready"]:
            ready_patches += 1
            if p.id not in evaluated:
                pending["evaluate"] += 1
    # ---- 步骤可用性：前置数据不具备时置灰 ----
    avail = {
        "project": True,
        "standard": True,
        "constraint": bool(patches),
        "unit": bool(patches),
        "ahp": True,
        "indicator": bool(units),
        "evaluate": ready_patches > 0,
        "report": bool(evaluated),
    }
    notes = {
        "constraint": "需先建立矿山图斑",
        "unit": "需先建立矿山图斑",
        "indicator": "需先划分微地质单元",
        "evaluate": "需前置步骤齐备的图斑",
        "report": "需先执行适宜性评价",
    }
    steps = []
    for s in STEPS:
        steps.append(dict(
            s,
            available=avail[s["key"]],
            pending=pending.get(s["key"], 0),
            note=None if avail[s["key"]] else
    notes.get(s["key"], ""),
        ))
    unit_types = len({u.unit_type for u in units})
    stats = {
        "projects": projects,
        "patches": len(patches),
        "unit_types": unit_types,
        "standards": Standard.query.count(),
    }
```

```
    return {                                                                    2601
        "stats": stats,                                                         2602
        "steps": steps,                                                         2603
        "badges": {k: v for k, v in pending.items() if v},                      2604
        "progress": {                                                           2605
            "patches": len(patches),                                            2606
            "units": len(units),                                                2607
            "data_units": len(data_unit_ids),                                   2608
            "ready": ready_patches,                                             2609
            "evaluated": len(evaluated),                                        2610
        },                                                                      2611
    }                                                                           2612
def nav_badges():                                                               2613
    """侧边栏待办角标：{入口地址前缀： 数量}，只含有待办的项。"""                                2614
    snap = snapshot()                                                           2615
    return {                                                                    2616
        "/constraint/": snap["badges"].get("constraint", 0),                   2617
        "/unit/": snap["badges"].get("unit", 0),                                2618
        "/indicator/": snap["badges"].get("indicator", 0),                      2619
        "/evaluate/": snap["badges"].get("evaluate", 0),                        2620
    }                                                                           2621
# ==== app/services/audit.py ====                                               2622
# -*- coding: utf-8 -*-                                                         2623
"""操作审计。                                                                    2624
在应用工厂中以 after_request 钩子统一埋点：凡是成功的写操作（POST/PUT/DELETE）且已登录，即写入 operation_logs 表，无需在每个视图里手工调用。  2625
                                                                                2626
动作名称由「端点 → 中文动作」映射表给出，未收录的端点回退为方法 + 路径。                    2627
"""                                                                             2628
from flask import request                                                       2629
from flask_login import current_user                                            2630
from app.models import db, OperationLog                                         2631
# 端点 → 可读动作名                                                              2632
ACTION_NAMES = {                                                                2633
    "auth.login": "登录系统",                                                    2634
    "auth.register": "注册账号",                                                 2635
    "auth.logout": "退出登录",                                                   2636
    "auth.add_user": "新增账号",                                                 2637
    "auth.edit_user": "编辑账号",                                                2638
    "auth.delete_user": "删除账号",                                              2639
    "auth.reset_password": "重置用户密码",                                       2640
    "auth.profile": "修改个人资料",                                              2641
    "auth.upload_avatar": "更新头像",                                            2642
    "auth.delete_avatar": "移除头像",                                            2643
    "project.add": "新建项目",                                                   2644
    "project.edit": "编辑项目",                                                  2645
    "project.delete": "删除项目",                                                2646
    "project.patch_add": "新建矿山图斑",                                         2647
    "project.patch_edit": "编辑矿山图斑",                                        2648
    "project.patch_delete": "删除矿山图斑",                                      2649
    "project.upload_image": "上传遥感影像",                                      2650
```

```
    "unit.add": "添加微地质单元",
    "unit.edit": "编辑微地质单元",
    "unit.delete": "删除微地质单元",
    "standard.add": "添加标准/政策文件",
    "standard.edit": "编辑标准/政策文件",
    "standard.delete": "删除标准/政策文件",
    "standard.rules": "维护标准规则",
    "standard.rule_delete": "删除标准规则",
    "constraint.check": "执行限制性条件判定",
    "ahp.save_expert": "录入专家判断矩阵",
    "ahp.delete_expert": "删除专家判断矩阵",
    "ahp.aggregate": "汇总多专家矩阵",
    "indicator.edit": "保存指标数据",
    "indicator.import_data": "批量导入指标数据",
    "indicator.update_meta": "维护指标定义",
    "evaluate.run": "执行适宜性评价",
}
# 无需记录的操作（查看类或纯导航）
SKIP_ENDPOINTS = {"static", "auth.index"}
def action_name(endpoint):
    # 个人设置页的两个表单共用一个端点，按提交的动作区分
    if endpoint == "auth.profile":
        return "修改密码" if request.form.get("action") ==
    "password" \
            else "修改个人资料"
    if endpoint in ACTION_NAMES:
        return ACTION_NAMES[endpoint]
    return "写操作 %s %s" % (request.method, request.path)
def extract_target_id():
    """从视图参数中取出被操作对象的主键，取不到则返回 None。"""
    args = request.view_args or {}
    for key in ("pid", "patch_id", "unit_id", "sid", "rid",
    "result_id", "user_id"):
        if key in args:
            try:
                return int(args[key])
            except (TypeError, ValueError):
                return None
    return None
def record(endpoint, status_code):
    """记录一次写操作。失败（4xx/5xx）也记录，便于追溯。"""
    if endpoint in SKIP_ENDPOINTS:
        return
    try:
        detail = "路径 %s" % request.path
        if status_code >= 400:
            detail += "；结果 HTTP %d" % status_code
        db.session.add(OperationLog(
            user_id=current_user.id if
    current_user.is_authenticated else None,
```

```
                action=action_name(endpoint),                          2701
                target_type=(endpoint or "").split(".")[0] or          2702
    None,                                                              2703
                target_id=extract_target_id(),                         2704
                detail=detail,                                         2705
            ))                                                         2706
            db.session.commit()                                        2707
    except Exception:                                                  2708
            # 审计失败不应影响业务响应                                 2709
            db.session.rollback()                                      2710
# ==== app/views/auth.py ====                                        2711
# -*- coding: utf-8 -*-                                              2712
"""账号与权限 —— 登录、注册、个人资料、用户维护、操作日志。          2713
页面组织：                                                           2714
  /login          登录页（含「点我注册」入口）                       2715
  /register       自助注册（默认「操作员」）                         2716
  /profile        个人设置，分类切换：info / security / users / logs 2717
                  后两个分类仅管理员可见，由本模块统一管控。         2718
"""                                                                  2719
from flask import (Blueprint, render_template, redirect,             2720
    url_for, request,                                                2721
                  flash, abort, current_app)                         2722
from flask_login import login_user, logout_user,                     2723
    login_required, current_user                                     2724
from werkzeug.security import check_password_hash,                   2725
    generate_password_hash                                           2726
from app.forms import (LoginForm, EmptyForm, ProfileForm,            2727
    PasswordForm,                                                    2728
                       RegisterForm, email_ok)                       2729
from app.models import db, User, OperationLog                        2730
from app.utils import uploads as up                                  2731
from app.utils.constants import ROLE_LABELS, DEFAULT_ROLE,           2732
    GENDERS                                                          2733
bp = Blueprint("auth", __name__)                                     2734
# 「个人设置」的四个分类；后两个是管理员专属                         2735
PROFILE_TABS = ("info", "security", "users", "logs")                 2736
ADMIN_TABS = ("users", "logs")                                       2737
# 口令最短长度：自助注册与管理员建号保持一致，免得出现「a」这种口令 2738
MIN_PASSWORD_LEN = 6                                                 2739
def require_admin():                                                 2740
    """非管理员直接 403，避免误操作基础数据。"""                    2741
    if current_user.role != "admin":                                 2742
        abort(403)                                                   2743
def _intro_data():                                                   2744
    """登录页下方的系统介绍所用的静态内容。"""                      2745
    from app.services import dashboard as dash                       2746
    return {"modules": dash.MODULES}                                 2747
@bp.route("/")                                                       2748
def index():                                                         2749
    if current_user.is_authenticated:                                2750
```

```
        from app.services import dashboard as dash
        return render_template("auth/home.html",
    dash=dash.snapshot())
    return redirect(url_for("auth.login"))
@bp.route("/login", methods=["GET", "POST"])
def login():
    if current_user.is_authenticated:
        return redirect(url_for("auth.index"))
    form = LoginForm()
    if form.validate_on_submit():
        user = User.query.filter_by(username=form.username.d
    ata).first()
        if user and check_password_hash(user.password_hash,
    form.password.data):
            if not user.active:
                flash("该账号已停用，请联系管理员", "toast-error")
            else:
                login_user(user, remember=form.remember.data
    )
                # 用 toast 类别：渲染为页面中上方的浮层，淡入停留后淡出。
                # 登录成功只是告知性反馈，不需要用户停下来阅读。
                flash("登录成功", "toast")
                return redirect(url_for("auth.index"))
        else:
            flash("账户名称或密码错误", "toast-error")
    # 字段为空之类的校验失败，错误留在表单里逐项显示，不走浮层：
    # 那类消息要对着输入框看，一闪而过反而让人以为「点了没反应」。
    return render_template("auth/login.html", form=form,
    **_intro_data())
@bp.route("/register", methods=["GET", "POST"])
def register():
    """自助注册。新账号一律为「操作员」，权限只能由管理员事后调整。"""
    if current_user.is_authenticated:
        return redirect(url_for("auth.index"))
    form = RegisterForm()
    if form.validate_on_submit():
        username = form.username.data.strip()
        if User.query.filter_by(username=username).first():
            form.username.errors.append("该账户名称已被使用")
        else:
            user = User(username=username,
                        role=DEFAULT_ROLE,
                        password_hash=generate_password_hash
    (form.password.data),
                        display_name=(form.display_name.data
    or "").strip() or None,
                        gender=(form.gender.data or
    "").strip() or None,
                        email=(form.email.data or
    "").strip() or None,
```

```
                              phone=(form.phone.data or                2801
    "").strip() or None)                                               2802
            rel, err = _save_avatar(request.files.get("avata           2803
    r"))                                                               2804
            if err:                                                    2805
                flash(err, "error")                                    2806
            else:                                                      2807
                user.avatar = rel                                      2808
                db.session.add(user)                                   2809
                db.session.commit()                                    2810
                login_user(user)                                       2811
                flash("注册成功，欢迎使用", "toast")                    2812
                return redirect(url_for("auth.index"))                 2813
    return render_template("auth/register.html", form=form)            2814
@bp.route("/logout")                                                   2815
@login_required                                                        2816
def logout():                                                          2817
    logout_user()                                                      2818
    flash("已退出登录", "ok")                                          2819
    return redirect(url_for("auth.login"))                             2820
# ---------------------------- 个人设置 ----------------------------   2821
    ----                                                               2822
# 任何登录用户在这里维护自己；管理员额外能管理账号与查看日志。                2823
# 「用户与操作日志」不再是独立菜单，而是本页的管理员分类。                    2824
@bp.route("/profile", methods=["GET", "POST"])                         2825
@login_required                                                        2826
def profile():                                                         2827
    is_admin = current_user.role == "admin"                            2828
    tab = request.values.get("tab") or "info"                          2829
    if tab not in PROFILE_TABS or (tab in ADMIN_TABS and               2830
    not is_admin):                                                     2831
        tab = "info"                                                   2832
    # 表单放在同一页，用字段前缀区分，避免 submit / csrf_token 重名          2833
    info_form = ProfileForm(prefix="info", obj=current_user)           2834
    pwd_form = PasswordForm(prefix="pwd")                              2835
    if request.method == "POST":                                       2836
        action = request.form.get("action")                            2837
        # 校验失败要重新渲染本页，此时必须回到提交表单所在的那个分类，              2838
        # 否则错误提示会出现在另一个分类里，用户根本看不到。                      2839
        if action == "password":                                       2840
            tab = "security"                                           2841
        elif action == "info":                                         2842
            tab = "info"                                               2843
        if action == "info" and info_form.validate_on_submit           2844
    ():                                                                2845
            current_user.display_name =                                2846
    (info_form.display_name.data or "").strip() or None                2847
            current_user.gender = (info_form.gender.data or            2848
    "").strip() or None                                                2849
            current_user.email = (info_form.email.data or              2850
```

```
        "").strip() or None                                                2851
                current_user.phone = (info_form.phone.data or              2852
        "").strip() or None                                                2853
                db.session.commit()                                        2854
                flash("个人资料已保存", "ok")                                2855
                return redirect(url_for("auth.profile",                    2856
        tab="info"))                                                       2857
            if action == "password" and pwd_form.validate_on_sub           2858
        mit():                                                             2859
                if not check_password_hash(current_user.password           2860
        _hash,                                                             2861
                                                                           2862
        pwd_form.old_password.data):                                       2863
                    pwd_form.old_password.errors.append("当前密码不正确      2864
        ")                                                                 2865
                else:                                                      2866
                    current_user.password_hash =                           2867
        generate_password_hash(                                            2868
                        pwd_form.new_password.data)                        2869
                    db.session.commit()                                    2870
                    flash("密码已修改", "ok")                                2871
                    return redirect(url_for("auth.profile",                2872
        tab="security"))                                                   2873
        ctx = dict(info_form=info_form, pwd_form=pwd_form,                 2874
        avatar_form=EmptyForm(),                                           2875
                    tab=tab, role_labels=ROLE_LABELS,                      2876
        genders=GENDERS)                                                   2877
        if tab == "users":                                                 2878
            ctx.update(_users_context())                                   2879
        elif tab == "logs":                                                2880
            ctx.update(_logs_context())                                    2881
        return render_template("auth/profile.html", **ctx)                 2882
def _users_context():                                                      2883
        """管理员分类「账号」所需的数据。"""                                    2884
        return {"users": User.query.order_by(User.id).all(),               2885
                "form": EmptyForm()}                                       2886
def _logs_context():                                                       2887
        """管理员分类「操作日志」所需的数据。"""                                2888
        per_page = 50                                                      2889
        q = OperationLog.query.order_by(OperationLog.id.desc())            2890
        total = q.count()                                                  2891
        pages = max(1, (total + per_page - 1) // per_page)                 2892
        # 页码先夹到 [1, pages] 再算 offset：手工把 page 写成天文数字时，          2893
        # (page-1)*per_page 会超出 SQLite 能绑定的整数范围，直接 500。           2894
        try:                                                               2895
            page = int(request.args.get("page", 1))                        2896
        except (TypeError, ValueError):                                    2897
            page = 1                                                       2898
        page = max(1, min(page, pages))                                    2899
        records = q.offset((page - 1) * per_page).limit(per_page           2900
```

```
    ).all()                                                          2901
    users_map = {u.id: u.username for u in User.query.all()}         2902
    return {"records": records, "page": page, "pages":               2903
    pages,                                                           2904
            "total": total, "users_map": users_map}                  2905
# ---------------------------- 头像 ----------------------------    2906
    --                                                               2907
def _avatar_length_ok(file_storage):                                 2908
    """按字节检查头像大小。超限时 werkzeug 读到内存里的内容也能量出来。"""          2909
    limit = current_app.config.get("MAX_AVATAR_LENGTH", 4 *          2910
    1024 * 1024)                                                     2911
    stream = file_storage.stream                                     2912
    pos = stream.tell()                                              2913
    stream.seek(0, 2)                                                2914
    size = stream.tell()                                             2915
    stream.seek(pos)                                                 2916
    return size <= limit, size                                       2917
def _save_avatar(file):                                              2918
    """保存头像文件，返回 (相对路径, 错误信息)。没有文件时返回 (None, None)。"""      2919
    if file is None or not file.filename:                            2920
        return None, None                                            2921
    ok, size = _avatar_length_ok(file)                               2922
    if not ok:                                                       2923
        return None, ("头像不能超过 %.0f MB（当前 %.1f MB）"              2924
                      % (current_app.config["MAX_AVATAR_LENG         2925
    TH"] / 1024 / 1024,                                              2926
                         size / 1024 / 1024))                        2927
    return up.save_upload(file, "avatars",                           2928
                          current_app.config["ALLOWED_AVATAR         2929
    _EXT"],                                                          2930
                          current_app.config["UPLOAD_FOLDER"         2931
    ])                                                               2932
@bp.route("/profile/avatar", methods=["POST"])                       2933
@login_required                                                      2934
def upload_avatar():                                                 2935
    form = EmptyForm()                                               2936
    back = url_for("auth.profile", tab=request.form.get("tab         2937
    ") or "info")                                                    2938
    if not form.validate_on_submit():                                2939
        flash("表单校验失败，请重试", "error")                              2940
        return redirect(back)                                        2941
    rel, err = _save_avatar(request.files.get("avatar"))             2942
    if err:                                                          2943
        flash(err, "error")                                          2944
        return redirect(back)                                        2945
    if rel is None:                                                  2946
        flash("请选择头像图片", "error")                                   2947
        return redirect(back)                                        2948
    # 换新头像时删掉旧文件，避免残留                                          2949
    up.remove_upload(current_user.avatar,                            2950
```

```
    current_app.config["UPLOAD_FOLDER"])                                2951
    current_user.avatar = rel                                           2952
    db.session.commit()                                                 2953
    flash("头像已更新", "ok")                                            2954
    return redirect(back)                                               2955
@bp.route("/profile/avatar/delete", methods=["POST"])                   2956
@login_required                                                         2957
def delete_avatar():                                                    2958
    form = EmptyForm()                                                  2959
    back = url_for("auth.profile", tab=request.form.get("tab2960
    ") or "info")                                                       2961
    if form.validate_on_submit():                                       2962
        up.remove_upload(current_user.avatar,                           2963
    current_app.config["UPLOAD_FOLDER"])                                2964
        current_user.avatar = None                                      2965
        db.session.commit()                                             2966
        flash("头像已移除", "ok")                                        2967
    return redirect(back)                                               2968
# ---------------------------- 账号维护（仅管理员）                      2969
    ------------------------------                                      2970
@bp.route("/users")                                                     2971
@login_required                                                         2972
def users():                                                            2973
    """旧地址保留，统一并入「个人设置 · 账号」。"""                          2974
    return redirect(url_for("auth.profile", tab="users"))               2975
@bp.route("/users/add", methods=["GET", "POST"])                        2976
@login_required                                                         2977
def add_user():                                                         2978
    require_admin()                                                     2979
    if request.method == "POST":                                        2980
        username = request.form.get("username", "").strip()             2981
        password = request.form.get("password", "")                     2982
        role = request.form.get("role", DEFAULT_ROLE)                   2983
        if role not in ROLE_LABELS:                                     2984
            role = DEFAULT_ROLE                                         2985
        if not username or not password:                                2986
            flash("账户名称和密码不能为空", "error")                     2987
            return redirect(url_for("auth.add_user"))                   2988
        if len(username) > 64:                                          2989
            flash("账户名称不能超过 64 个字符", "error")                 2990
            return redirect(url_for("auth.add_user"))                   2991
        if len(password) < MIN_PASSWORD_LEN:                            2992
            flash("密码至少 %d 位（与自助注册保持一致）" % MIN_PASSWORD_LEN,2993
                  "error")                                              2994
            return redirect(url_for("auth.add_user"))                   2995
        if User.query.filter_by(username=username).first():             2996
            flash("账户名称已存在", "error")                            2997
            return redirect(url_for("auth.add_user"))                   2998
        db.session.add(User(                                            2999
            username=username, role=role,                               3000
```

```
            password_hash=generate_password_hash(password),
            display_name=request.form.get("display_name",
"").strip() or None,
            gender=request.form.get("gender", "").strip()
or None,
            email=request.form.get("email", "").strip() or
None,
            phone=request.form.get("phone", "").strip() or
None))
        db.session.commit()
        flash("账号已添加", "ok")
        return redirect(url_for("auth.profile",
tab="users"))
    return render_template("auth/add_user.html",
role_labels=ROLE_LABELS,
                           genders=GENDERS,
default_role=DEFAULT_ROLE,
                           form=EmptyForm())
@bp.route("/users/<int:user_id>/edit", methods=["GET",
"POST"])
@login_required
def edit_user(user_id):
    require_admin()
    user = db.get_or_404(User, user_id)
    if request.method == "POST":
        role = request.form.get("role", user.role)
        if role not in ROLE_LABELS:
            role = user.role
        active = request.form.get("active") == "1"
        # 保护：不能把最后一个管理员降级或停用，否则无人能管理系统
        if user.role == "admin" and (role != "admin" or not
active):
            others = User.query.filter(User.role == "admin",
                                       User.id != user.id,

User.active.is_(True)).count()
            if others == 0:
                flash("系统至少需保留一名启用状态的管理员", "error")
                return redirect(url_for("auth.edit_user",
user_id=user.id))
        # 个人资料字段（用户自己也能在「个人设置」里改，管理员在此代改）
        email = request.form.get("email", "").strip()
        if not email_ok(email):
            flash("邮箱格式不正确", "error")
            return redirect(url_for("auth.edit_user",
user_id=user.id))
        user.role = role
        user.active = active
        user.display_name = request.form.get("display_name",
"").strip() or None
```

```
        user.gender = request.form.get("gender",                    3051
    "").strip() or None                                             3052
        user.email = email or None                                  3053
        user.phone = request.form.get("phone", "").strip()          3054
    or None                                                         3055
        new_password = request.form.get("password",                 3056
    "").strip()                                                     3057
        if new_password:                                            3058
            if len(new_password) < MIN_PASSWORD_LEN:                3059
                flash("密码至少 %d 位" % MIN_PASSWORD_LEN,           3060
    "error")                                                        3061
                return redirect(url_for("auth.edit_user",           3062
    user_id=user.id))                                               3063
            user.password_hash = generate_password_hash(new_3064
    password)                                                       3065
        db.session.commit()                                         3066
        flash("账号信息已保存", "ok")                                3067
        return redirect(url_for("auth.profile",                     3068
    tab="users"))                                                   3069
    return render_template("auth/edit_user.html", user=user,3070
                           role_labels=ROLE_LABELS,                 3071
    genders=GENDERS,                                                3072
                           form=EmptyForm())                        3073
@bp.route("/users/<int:user_id>/delete", methods=["POST"])          3074
@login_required                                                     3075
def delete_user(user_id):                                           3076
    require_admin()                                                 3077
    user = db.get_or_404(User, user_id)                             3078
    if user.id == current_user.id:                                  3079
        flash("不能删除当前登录的账号", "error")                     3080
        return redirect(url_for("auth.profile",                     3081
    tab="users"))                                                   3082
    if user.role == "admin":                                        3083
        others = User.query.filter(User.role == "admin",            3084
                                   User.id != user.id,              3085
                                   User.active.is_(True)).co3086
    unt()                                                           3087
        if others == 0:                                             3088
            flash("系统至少需保留一名启用状态的管理员", "error")     3089
            return redirect(url_for("auth.profile",                 3090
    tab="users"))                                                   3091
    db.session.delete(user)                                         3092
    db.session.commit()                                             3093
    flash("账号「%s」已删除" % user.username, "ok")                  3094
    return redirect(url_for("auth.profile", tab="users"))           3095
# ---------------------------- 操作日志（仅管理员）                  3096
    ----------------------------                                    3097
@bp.route("/logs")                                                  3098
@login_required                                                     3099
def logs():                                                         3100
```

```
    """旧地址保留，统一并入「个人设置 · 操作日志」。"""
    return redirect(url_for("auth.profile", tab="logs"))
# ==== app/views/project.py ====
# -*- coding: utf-8 -*-
"""项目与矿山档案 —— 项目、图斑、参照生态系统、遥感影像挂接。"""
from flask import (Blueprint, render_template, request,
    redirect, url_for,
                   flash, current_app)
from flask_login import login_required, current_user
from app.forms import EmptyForm
from app.models import db, Project, MinePatch,
    ReferenceEcosystem
from app.services import cleanup
from app.utils.parsing import FieldError, parse_float
from app.utils.timefmt import utcnow
bp = Blueprint("project", __name__, url_prefix="/project")
@bp.route("/")
@login_required
def index():
    projects = Project.query.order_by(Project.id.desc()).all
    ()
    return render_template("project/index.html",
    projects=projects,
                           form=EmptyForm())
@bp.route("/add", methods=["GET", "POST"])
@login_required
def add():
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        p = Project(
            name=request.form.get("name", "").strip(),
            code=request.form.get("code", "").strip() or
    None,
            description=request.form.get("description",
    "").strip() or None,
            created_by=current_user.id,
        )
        if not p.name:
            flash("项目名称不能为空", "error")
        else:
            db.session.add(p)
            db.session.commit()
            flash("项目已创建", "ok")
            return redirect(url_for("project.index"))
    return render_template("project/edit.html", form=form,
    project=None)
@bp.route("/<int:pid>/edit", methods=["GET", "POST"])
@login_required
def edit(pid):
```

```
    p = db.get_or_404(Project, pid)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit():
        p.name = request.form.get("name", "").strip()
        p.code = request.form.get("code", "").strip() or None
        p.description = request.form.get("description", "").strip() or None
        p.status = request.form.get("status", "进行中")
        db.session.commit()
        flash("项目已保存", "ok")
        return redirect(url_for("project.index"))
    return render_template("project/edit.html", form=form, project=p)
@bp.route("/<int:pid>/delete", methods=["POST"])
@login_required
def delete(pid):
    p = db.get_or_404(Project, pid)
    # 项目下的图斑由 ORM 级联删除，但图斑下面的指标数据、限制性判定、
    # 评价结果、得分明细、参照生态系统不会跟着走，必须先逐个图斑清理，
    # 否则每删一个项目都在库里留下一批孤儿数据（见 app/services/cleanup.py）。
    cleanup.delete_project(p)
    flash("项目已删除", "ok")
    return redirect(url_for("project.index"))
# ---------------------------- 矿山图斑 ----------------------------
@bp.route("/<int:pid>/patches")
@login_required
def patches(pid):
    p = db.get_or_404(Project, pid)
    return render_template("project/patches.html",
    project=p,
                           form=EmptyForm())
@bp.route("/patch/<int:patch_id>/delete", methods=["POST"])
@login_required
def patch_delete(patch_id):
    """删除矿山图斑，及其微地质单元、指标数据与评价记录。"""
    patch = db.get_or_404(MinePatch, patch_id)
    pid = patch.project_id
    # 清理逻辑统一放在 app/services/cleanup.py：项目删除走的是同一个函数，
    # 免得两条删除路径各写一份、漏掉某张子表（曾经就漏过）。
    deleted, _counts = cleanup.delete_patch(patch.id)
    flash("图斑「%s」及其数据已删除" % (deleted.name if deleted else patch.name),
          "ok")
    return redirect(url_for("project.patches", pid=pid))
@bp.route("/<int:pid>/patch/add", methods=["GET", "POST"])
@login_required
def patch_add(pid):
```

```
    p = db.get_or_404(Project, pid)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit():
        def fv(name, cast=str):
            v = request.form.get(name, "").strip()
            return cast(v) if v != "" else None
        try:
            area_ha = parse_float(request.form.get("area_ha"), "图斑面积（公顷）",
                                  minimum=0)
            stop_years = parse_float(request.form.get("stop_years"), "停产年限",
                                     minimum=0)
        except FieldError as e:
            flash(str(e), "error")
            return redirect(url_for("project.patch_add", pid=p.id))
        patch = MinePatch(
            project_id=p.id,
            name=request.form.get("name", "").strip(),
            admin_region=fv("admin_region"),
            area_ha=area_ha,
            mineral_type=fv("mineral_type"),
            mining_method=fv("mining_method"),
            stop_years=stop_years,
            responsible_entity_missing=bool(request.form.get("responsible_entity_missing")),
        )
        if not patch.name:
            flash("图斑名称不能为空", "error")
        else:
            db.session.add(patch)
            db.session.flush()
            # 参照生态系统默认取论文值
            db.session.add(ReferenceEcosystem(
                patch_id=patch.id, coverage_benchmark=75.0, shannon_benchmark=0.7))
            db.session.commit()
            flash("图斑已创建", "ok")
            return redirect(url_for("project.patches", pid=p.id))
    return render_template("project/patch_edit.html", form=form, project=p, patch=None)

@bp.route("/patch/<int:patch_id>/edit", methods=["GET", "POST"])
@login_required
def patch_edit(patch_id):
    patch = db.get_or_404(MinePatch, patch_id)
    form = EmptyForm()
```

```
    if request.method == "POST" and form.validate_on_submit(
):
        def fv(name, cast=str):
            v = request.form.get(name, "").strip()
            return cast(v) if v != "" else None
        # 数值字段统一走 parse_float：填了非数字给中文提示，而不是 500
        try:
            area_ha = parse_float(request.form.get("area_ha"
), "图斑面积（公顷）",
                                  minimum=0)
            stop_years = parse_float(request.form.get("stop_
years"), "停产年限",
                                     minimum=0)
            cov = parse_float(request.form.get("coverage_ben
chmark"),
                              "植被覆盖度基准（%）", minimum=0,
maximum=100)
            sha = parse_float(request.form.get("shannon_benc
hmark"),
                              "Shannon 基准", minimum=0)
        except FieldError as e:
            flash(str(e), "error")
            return redirect(url_for("project.patch_edit",
patch_id=patch.id))
        patch.name = request.form.get("name", "").strip()
        patch.admin_region = fv("admin_region")
        patch.area_ha = area_ha
        patch.mineral_type = fv("mineral_type")
        patch.mining_method = fv("mining_method")
        patch.stop_years = stop_years
        patch.responsible_entity_missing =
bool(request.form.get("responsible_entity_missing"))
        # 参照生态系统
        ref = ReferenceEcosystem.query.filter_by(patch_id=pa
tch.id).first()
        if ref is None:
            ref = ReferenceEcosystem(patch_id=patch.id)
            db.session.add(ref)
        # 基准值：用户填 0 是合法输入（不能写成 `or 75.0`，那样 0 会被悄悄换成默认值）
        ref.coverage_benchmark = 75.0 if cov is None else
cov
        ref.shannon_benchmark = 0.7 if sha is None else sha
        ref.dominant_species = fv("dominant_species")
        patch.image_note = fv("image_note")
        db.session.commit()
        flash("图斑已保存", "ok")
        return redirect(url_for("project.patches",
pid=patch.project_id))
    return render_template("project/patch_edit.html",
form=form,
```

```
                                project=patch.project,                3301
    patch=patch,                                                      3302
                                reference=ReferenceEcosystem.quer     3303
    y.filter_by(                                                      3304
                                    patch_id=patch.id).first())       3305
# ---------------------------- 遥感影像挂接                           3306
    ----------------------------                                      3307
@bp.route("/patch/<int:patch_id>/image", methods=["POST"])            3308
@login_required                                                       3309
def upload_image(patch_id):                                           3310
    """上传遥感底图 / 无人机正射影像并挂接到图斑。"""                  3311
    from app.utils import uploads as up                               3312
    patch = db.get_or_404(MinePatch, patch_id)                        3313
    form = EmptyForm()                                                3314
    if not form.validate_on_submit():                                 3315
        flash("表单校验失败，请重试", "error")                        3316
        return redirect(url_for("project.patch_edit",                 3317
    patch_id=patch.id))                                               3318
    rel, err = up.save_upload(request.files.get("image"),             3319
    "patches",                                                        3320
                              current_app.config["ALLOWED_IM3321
    AGE_EXT"],                                                        3322
                              current_app.config["UPLOAD_FOL3323
    DER"])                                                            3324
    if err:                                                           3325
        flash(err, "error")                                           3326
        return redirect(url_for("project.patch_edit",                 3327
    patch_id=patch.id))                                               3328
    # 替换旧影像时清理原文件，避免残留                                3329
    if patch.image_path:                                              3330
        up.remove_upload(patch.image_path,                            3331
    current_app.config["UPLOAD_FOLDER"])                              3332
    patch.image_path = rel                                            3333
    patch.image_uploaded_at = utcnow()                                3334
    note = request.form.get("image_note", "").strip()                 3335
    if note:                                                          3336
        patch.image_note = note                                       3337
    db.session.commit()                                               3338
    flash("遥感影像已挂接", "ok")                                     3339
    return redirect(url_for("project.patch_edit",                     3340
    patch_id=patch.id))                                               3341
@bp.route("/patch/<int:patch_id>/image/delete",                       3342
    methods=["POST"])                                                 3343
@login_required                                                       3344
def delete_image(patch_id):                                           3345
    from app.utils import uploads as up                               3346
    patch = db.get_or_404(MinePatch, patch_id)                        3347
    form = EmptyForm()                                                3348
    if form.validate_on_submit():                                     3349
        up.remove_upload(patch.image_path,                            3350
```

```
    current_app.config["UPLOAD_FOLDER"])
        patch.image_path = None
        patch.image_uploaded_at = None
        db.session.commit()
        flash("遥感影像已移除", "ok")
    return redirect(url_for("project.patch_edit",
    patch_id=patch.id))
# ==== app/views/unit.py ====
# -*- coding: utf-8 -*-
"""微地质单元管理 —— 岩质边坡 / 碎石边坡 / 平台的划分与面积占比。"""
from flask import Blueprint, render_template, request,
    redirect, url_for, flash
from flask_login import login_required
from app.forms import EmptyForm
from app.models import db, MinePatch, MicroUnit,
    IndicatorData
from app.utils.constants import UNIT_TYPES
from app.utils.parsing import FieldError, parse_float
bp = Blueprint("unit", __name__, url_prefix="/unit")
def recalc_ratio(patch_id):
    """按面积重算该图斑各单元面积占比。"""
    units = MicroUnit.query.filter_by(patch_id=patch_id).all
    ()
    total = sum(u.area or 0.0 for u in units)
    if total > 0:
        for u in units:
            u.ratio = round((u.area or 0.0) / total, 6)
    return total
@bp.route("/")
@login_required
def list_patches():
    """微地质单元入口：列出各图斑及其单元划分情况。"""
    patches = MinePatch.query.order_by(MinePatch.id.desc()).
    all()
    rows = []
    for p in patches:
        units = MicroUnit.query.filter_by(patch_id=p.id).all
    ()
        rows.append({
            "patch": p,
            "count": len(units),
            "ratio_sum": sum(u.ratio or 0.0 for u in units),
        })
    return render_template("unit/list.html", rows=rows)
@bp.route("/<int:patch_id>")
@login_required
def index(patch_id):
    patch = db.get_or_404(MinePatch, patch_id)
    units = MicroUnit.query.filter_by(patch_id=patch_id).all
    ()
```

```
    total = sum(u.area or 0.0 for u in units)
    ratio_sum = sum(u.ratio or 0.0 for u in units)
    return render_template("unit/index.html", patch=patch,
    units=units,
                           total_area=total,
    ratio_sum=ratio_sum,
                           unit_types=UNIT_TYPES,
    form=EmptyForm())
@bp.route("/<int:patch_id>/add", methods=["POST"])
@login_required
def add(patch_id):
    patch = db.get_or_404(MinePatch, patch_id)
    form = EmptyForm()
    if form.validate_on_submit():
        unit_type = request.form.get("unit_type")
        if unit_type not in UNIT_TYPES:
            flash("微地质单元类型不正确", "error")
            return redirect(url_for("unit.index",
    patch_id=patch_id))
        # 论文 4.1 节将图斑分为平台、碎石边坡、岩质边坡三类，每类至多一个。
        # 允许重复会让面积占比与按单元类型组织的指标数据对不上号。
        if MicroUnit.query.filter_by(patch_id=patch.id,
                                     unit_type=unit_type).fi
    rst():
            flash("该图斑已存在「%s」，同一类型只能划分一个" % unit_type,
    "error")
            return redirect(url_for("unit.index",
    patch_id=patch_id))
        area = request.form.get("area", "").strip()
        try:
            area_value = parse_float(area, "面积（公顷）",
    minimum=0)
        except FieldError as e:
            flash(str(e), "error")
            return redirect(url_for("unit.index",
    patch_id=patch_id))
        u = MicroUnit(patch_id=patch.id,
    unit_type=unit_type,
                      area=area_value,
                      description=request.form.get("descript
    ion", "").strip() or None)
        db.session.add(u)
        db.session.flush()
        recalc_ratio(patch.id)
        db.session.commit()
        flash("微地质单元已添加", "ok")
    return redirect(url_for("unit.index",
    patch_id=patch_id))
@bp.route("/edit/<int:unit_id>", methods=["GET", "POST"])
@login_required
```

```
def edit(unit_id):
    """单个微地质单元的编辑页。"""
    u = db.get_or_404(MicroUnit, unit_id)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit():
        new_type = request.form.get("unit_type", u.unit_type)
        if new_type not in UNIT_TYPES:
            flash("微地质单元类型不正确", "error")
            return redirect(url_for("unit.edit", unit_id=u.id))
        # 改类型时同样不允许与本图斑已有的其他单元重复
        dup = MicroUnit.query.filter_by(patch_id=u.patch_id,
                                        unit_type=new_type).first()
        if dup is not None and dup.id != u.id:
            flash("该图斑已存在「%s」，同一类型只能划分一个" % new_type,
                  "error")
            return redirect(url_for("unit.edit", unit_id=u.id))
        u.unit_type = new_type
        area = request.form.get("area", "").strip()
        try:
            u.area = parse_float(area, "面积（公顷）", minimum=0)
        except FieldError as e:
            flash(str(e), "error")
            return redirect(url_for("unit.edit", unit_id=u.id))
        u.description = request.form.get("description", "").strip() or None
        recalc_ratio(u.patch_id)
        db.session.commit()
        flash("微地质单元已保存", "ok")
        return redirect(url_for("unit.index", patch_id=u.patch_id))
    return render_template("unit/edit.html", unit=u, patch=u.patch,
                           unit_types=UNIT_TYPES, form=form)
@bp.route("/delete/<int:unit_id>", methods=["POST"])
@login_required
def delete(unit_id):
    u = db.get_or_404(MicroUnit, unit_id)
    pid = u.patch_id
    name = u.unit_type
    # 一并删除该单元的指标数据，避免残留
    IndicatorData.query.filter_by(unit_id=u.id).delete(synchronize_session=False)
    db.session.delete(u)
    db.session.flush()
```

```
    recalc_ratio(pid)
    db.session.commit()
    flash("微地质单元「%s」已删除" % name, "ok")
    return redirect(url_for("unit.index", patch_id=pid))
# ==== app/views/standard.py ====
# -*- coding: utf-8 -*-
"""政策与标准知识库 —— 标准 / 政策文件的结构化规则与省际口径比对。"""
from flask import Blueprint, render_template, request,
    redirect, url_for, flash
from flask_login import login_required
from app.forms import EmptyForm
from app.models import db, Standard, StandardRule
bp = Blueprint("standard", __name__, url_prefix="/standard")
@bp.route("/")
@login_required
def index():
    standards = Standard.query.order_by(Standard.category,
    Standard.province).all()
    local = [s for s in standards if s.category == "地方标准"]
    policy = [s for s in standards if s.category != "地方标准"]
    return render_template("standard/index.html",
    local=local, policy=policy,
                           form=EmptyForm())
@bp.route("/add", methods=["GET", "POST"])
@login_required
def add():
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        s = Standard(
            province=request.form.get("province",
    "").strip(),
            name=request.form.get("name", "").strip(),
            doc_number=request.form.get("doc_number",
    "").strip() or None,
            publish_date=request.form.get("publish_date",
    "").strip() or None,
            implement_date=request.form.get("implement_date"
    , "").strip() or None,
            category=request.form.get("category", "地方标准"),
            summary=request.form.get("summary", "").strip()
    or None,
        )
        if not s.province or not s.name:
            flash("省份与名称不能为空", "error")
        else:
            db.session.add(s)
            db.session.commit()
            flash("标准已添加", "ok")
            return redirect(url_for("standard.index"))
```

```
    return render_template("standard/edit.html", form=form,
    standard=None)
@bp.route("/<int:sid>/edit", methods=["GET", "POST"])
@login_required
def edit(sid):
    s = db.get_or_404(Standard, sid)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        s.province = request.form.get("province",
    "").strip()
        s.name = request.form.get("name", "").strip()
        s.doc_number = request.form.get("doc_number",
    "").strip() or None
        s.publish_date = request.form.get("publish_date",
    "").strip() or None
        s.implement_date = request.form.get("implement_date"
    , "").strip() or None
        s.category = request.form.get("category", "地方标准")
        s.summary = request.form.get("summary", "").strip()
    or None
        db.session.commit()
        flash("已保存", "ok")
        return redirect(url_for("standard.rules", sid=s.id))
    return render_template("standard/edit.html", form=form,
    standard=s)
@bp.route("/<int:sid>/delete", methods=["POST"])
@login_required
def delete(sid):
    s = db.get_or_404(Standard, sid)
    StandardRule.query.filter_by(standard_id=sid).delete()
    db.session.delete(s)
    db.session.commit()
    flash("已删除", "ok")
    return redirect(url_for("standard.index"))
@bp.route("/<int:sid>/rules", methods=["GET", "POST"])
@login_required
def rules(sid):
    s = db.get_or_404(Standard, sid)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        r = StandardRule(
            standard_id=s.id,
            item=request.form.get("item", "").strip(),
            operator=request.form.get("operator", ">="),
            threshold=request.form.get("threshold",
    "").strip() or None,
            condition_type=request.form.get("condition_type"
    , "必要条件"),
```

```
            region=request.form.get("region", "").strip()
    or None,
            note=request.form.get("note", "").strip() or
    None,
        )
        if not r.item:
            flash("判定项不能为空", "error")
        else:
            db.session.add(r)
            db.session.commit()
            flash("规则已添加", "ok")
        return redirect(url_for("standard.rules", sid=s.id))
    rs = StandardRule.query.filter_by(standard_id=s.id).all(
    )
    return render_template("standard/rules.html",
    form=form, standard=s, rules=rs)
@bp.route("/rule/<int:rid>/delete", methods=["POST"])
@login_required
def rule_delete(rid):
    r = db.get_or_404(StandardRule, rid)
    sid = r.standard_id
    db.session.delete(r)
    db.session.commit()
    flash("规则已删除", "ok")
    return redirect(url_for("standard.rules", sid=sid))
@bp.route("/compare")
@login_required
def compare():
    """省际口径比对：同一指标在不同省份的阈值横向对比。"""
    rules = StandardRule.query.join(Standard).all()
    groups = {}
    for r in rules:
        groups.setdefault(r.item, []).append(r)
    return render_template("standard/compare.html",
    groups=groups)
# ==== app/views/constraint.py ====
# -*- coding: utf-8 -*-
"""限制性指标判定 —— 一票否决判定。
按论文表 2 与 4.1 节流程，共 5 项：政策、区位约束、土壤污染、地质灾害、耕地属性。
任一项不通过即判定为"人工辅助或生态重建"，不再进入定量评价。
"""
from flask import Blueprint, render_template, request,
    redirect, url_for, flash
from flask_login import login_required
from app.forms import EmptyForm
from app.models import db, MinePatch, ConstraintCheck
from app.utils.constants import CONSTRAINT_ITEMS
bp = Blueprint("constraint", __name__,
    url_prefix="/constraint")
# 字段名 -> 中文名
```

```
FIELDS = [                                                              3651
    ("policy_pass", "政策约束"),                                        3652
    ("location_pass", "区位约束"),                                      3653
    ("pollution_pass", "土壤污染"),                                     3654
    ("geo_pass", "地质灾害"),                                           3655
    ("farmland_pass", "耕地属性"),                                      3656
]                                                                       3657
@bp.route("/")                                                          3658
@login_required                                                         3659
def index():                                                            3660
    patches = MinePatch.query.order_by(MinePatch.id.desc()).3661
    all()                                                               3662
    latest = {}                                                         3663
    for p in patches:                                                   3664
        latest[p.id] = ConstraintCheck.query.filter_by(patch3665
    _id=p.id) \                                                         3666
            .order_by(ConstraintCheck.id.desc()).first()                3667
    return render_template("constraint/index.html",                     3668
    patches=patches, latest=latest)                                     3669
@bp.route("/<int:patch_id>", methods=["GET", "POST"])                   3670
@login_required                                                         3671
def check(patch_id):                                                    3672
    patch = db.get_or_404(MinePatch, patch_id)                          3673
    form = EmptyForm()                                                  3674
    record = ConstraintCheck.query.filter_by(patch_id=patch_3675
    id) \                                                               3676
        .order_by(ConstraintCheck.id.desc()).first()                    3677
    if request.method == "POST" and form.validate_on_submit(3678
    ):                                                                  3679
        if record is None:                                              3680
            record = ConstraintCheck(patch_id=patch.id)                 3681
            db.session.add(record)                                      3682
        for field, _ in FIELDS:                                         3683
            setattr(record, field, request.form.get(field)   3684
    == "1")                                                             3685
        record.passed = all(getattr(record, f) for f, _ in   3686
    FIELDS)                                                             3687
        failed = [name for f, name in FIELDS if not                     3688
    getattr(record, f)]                                                 3689
        record.reason = ("全部通过" if record.passed                    3690
                         else "未通过：" + "、".join(failed))           3691
        db.session.commit()                                             3692
        flash("判定结果已保存", "ok")                                   3693
        return redirect(url_for("constraint.check",                     3694
    patch_id=patch.id))                                                 3695
    # 预处理各条件的当前取值，供模板直接使用（Jinja 无 getattr）       3696
    field_states = [(f, name, None if record is None else    3697
    getattr(record, f))                                                 3698
                    for f, name in FIELDS]                              3699
    return render_template("constraint/check.html",                     3700
```

```
    patch=patch, form=form,                                          3701
                                  fields=FIELDS,                     3702
    field_states=field_states,                                       3703
                                  record=record)                     3704
# ==== app/views/ahp.py ====                                        3705
# -*- coding: utf-8 -*-                                            3706
"""指标体系与 AHP 权重 —— 路由与页面逻辑。"""                                3707
import json                                                          3708
import math                                                          3709
from flask import Blueprint, render_template, request,               3710
    redirect, url_for, flash                                         3711
from app.forms import AHPSelectForm, EmptyForm                       3712
from app.models import db, Indicator, ExpertMatrix,                  3713
    WeightScheme                                                     3714
from app.utils.constants import DEFAULT_WEIGHTS, UNIT_TYPES          3715
from app.services import ahp_service as ahp                          3716
from app.services import indicator_service as ind_svc                3717
from app.services import weight_service as ws                        3718
bp = Blueprint("ahp", __name__, url_prefix="/ahp")                   3719
# 一次计算的权重结果对应的指标编码：名称 → 编码                                3720
_L1_CODE = {"土壤指标": "soil", "植被指标": "veg", "地形指标":                 3721
    "terrain"}                                                       3722
_L2_CODE = {                                                         3723
    ("土壤指标", "土石比"): "soil_ratio",                                  3724
    ("土壤指标", "土层厚度"): "thickness",                                 3725
    ("土壤指标", "pH"): "ph",                                          3726
    ("土壤指标", "有机质"): "organic_matter",                              3727
    ("土壤指标", "污染指标"): "pollution",                                 3728
    ("植被指标", "植被覆盖度"): "coverage",                                3729
    ("植被指标", "生物多样性"): "shannon",                                 3730
    ("地形指标", "坡度"): "slope",                                       3731
    ("地形指标", "坡向"): "aspect",                                      3732
}                                                                    3733
def to_code_pairs(unit_type, level, l1, names, weights):             3734
    """把一次 AHP 计算的权重结果转成「指标编码 → 权重」，便于落库。"""                3735
    pairs = []                                                       3736
    for name, w in zip(names, weights):                              3737
        if level == 1:                                               3738
            code = _L1_CODE.get(name)                                3739
        else:                                                        3740
            code = _L2_CODE.get((l1, name))                          3741
        if code:                                                     3742
            pairs.append((code, w))                                  3743
    return pairs                                                     3744
def normalize_l1(unit_type, level, l1):                              3745
    """把 l1 参数规整为该单元类型下确实存在的一级指标名。                          3746
    level=2 的页面（判断矩阵、多专家矩阵）都按 l1 取二级指标。                    3747
    直接带 ?level=2 访问而没给 l1 时，原先会 KeyError: None 直接 500；       3748
    这里退回该单元类型的第一个一级指标，页面照常可用。                             3749
    """                                                              3750
```

```
    if level != 2:
        return None
    valid = list(DEFAULT_WEIGHTS.get(unit_type,
    {}).get("l2", {}))
    if not valid:
        return None
    return l1 if l1 in valid else valid[0]
def indicator_list(unit_type, level, l1=None):
    """返回 （指标名称列表， 默认权重列表）。
    level=1 时返回一级指标；level=2 时返回指定一级指标下的二级指标。
    l1 缺失或不属于该单元类型时，退回第一个一级指标，不抛异常。
    """
    weights = DEFAULT_WEIGHTS[unit_type]
    if level == 1:
        names = list(weights["l1"].keys())
        vec = [weights["l1"][n] for n in names]
        return names, vec
    l2_all = weights["l2"]
    if l1 not in l2_all:
        l1 = next(iter(l2_all))
    l2 = l2_all[l1]
    names = list(l2.keys())
    vec = [l2[n] for n in names]
    return names, vec
def l1_choices(unit_type):
    """某单元类型的一级指标名称列表（用于二级指标录入时选择父指标）。"""
    return list(DEFAULT_WEIGHTS[unit_type]["l1"].keys())
@bp.route("/", methods=["GET", "POST"])
def index():
    form = AHPSelectForm()
    # 先根据请求中的单元类型填充 l1 选项，再进行表单校验
    unit = request.form.get("unit_type") or
    form.unit_type.choices[0][0]
    form.l1.choices = [(n, n) for n in l1_choices(unit)]
    if form.validate_on_submit():
        unit_type = form.unit_type.data
        level = int(form.level.data)
        if level == 1:
            return redirect(url_for("ahp.matrix",
    unit_type=unit_type, level=1))
        if not form.l1.data:
            flash("选择「二级指标」时，请同时选择所属的一级指标", "error")
            return render_template("ahp/index.html",
    form=form,
                                   weight_rows=ws.summary())
        return redirect(url_for("ahp.matrix",
    unit_type=unit_type,
                                level=2, l1=form.l1.data))
    return render_template("ahp/index.html", form=form,
                           weight_rows=ws.summary())
```

```
@bp.route("/matrix", methods=["GET", "POST"])
def matrix():
    unit_type = request.values.get("unit_type")
    # level 是手改地址栏就能改的参数，写成 abc 不该把页面打成 500
    try:
        level = int(request.values.get("level", 1))
    except (TypeError, ValueError):
        level = 1
    if level not in (1, 2):
        level = 1
    l1 = normalize_l1(unit_type, level,
    request.values.get("l1"))
    if unit_type not in DEFAULT_WEIGHTS:
        flash("无效的微地质单元类型", "error")
        return redirect(url_for("ahp.index"))
    names, vec = indicator_list(unit_type, level, l1)
    n = len(names)
    form = EmptyForm()
    # 预填默认值：论文表 3 权重对应的完全一致矩阵 a_ij = w_i / w_j
    default = [[round(vec[i] / vec[j], 4) for j in
    range(n)] for i in range(n)]
    def compute(upper):
        A = ahp.matrix_from_upper(upper)
        r = ahp.weights_and_consistency(A)
        bad = ahp.find_most_inconsistent(A)
        return {
            "weights": [round(v, 4) for v in r["weights"]],
            "lambda_max": round(r["lambda_max"], 4),
            "ci": r["ci"],
            "cr": r["cr"],
            "consistent": r["consistent"],
            "matrix": [[round(A[i, j], 4) for j in
    range(n)] for i in range(n)],
            "most_inconsistent": bad,
        }
    result = None
    if request.method == "POST" and form.validate_on_submit(
    ):
        action = request.form.get("action", "calc")
        try:
            upper = [float(request.form.get("m_%d_%d" % (i,
    j), "1"))
                     for i in range(n) for j in range(i +
    1, n)]
            # 0 / 负数 / inf / nan 能过 float()，但会让 n 次方根算出 inf
    或 nan，
            # 保存时直接撞上 NOT NULL 约束（weight 为 nan）→ 500
            if any((not math.isfinite(x)) or x <= 0 for x
    in upper):
                raise ValueError("矩阵元素必须是正数")
```

```
                result = compute(upper)                                    3851
            except (ValueError, ZeroDivisionError):                        3852
                flash("矩阵输入有误：请填入正数（对角线自动为 1，下三角自动取倒数）",      3853
    "error")                                                               3854
                return redirect(url_for("ahp.matrix",                      3855
    unit_type=unit_type,                                                   3856
                                            level=level, l1=l1))           3857
            if action == "save":                                           3858
                if not result["consistent"]:                               3859
                    flash("一致性比例 CR=%.4f ≥ 0.1，判断矩阵不满足一致性要求，"      3860
                          "请调整后再保存" % result["cr"], "error")            3861
                else:                                                      3862
                    pairs = to_code_pairs(unit_type, level, l1,            3863
    names,                                                                 3864
                                          result["weights"])               3865
                    scheme, cnt = ws.save_weights(unit_type,               3866
    pairs)                                                                 3867
                    flash("已保存并为「%s」生效 %d 项权重（CR=%.4f）。"                3868
                          "此后该单元类型的评价将以此权重计算。"                        3869
                          % (unit_type, cnt, result["cr"]),                3870
    "ok")                                                                  3871
                    return redirect(url_for("ahp.matrix",                  3872
    unit_type=unit_type,                                                   3873
                                            level=level, l1=l1))           3874
    elif request.values.get("calc") == "1":                                3875
            # 按默认（论文表 3）矩阵直接试算，便于一键查看权重与一致性                    3876
            result = compute([default[i][j] for i in range(n)              3877
    for j in range(i + 1, n)])                                             3878
    _, source_label, custom_scheme = ws.effective_weights(un               3879
    it_type)                                                               3880
    return render_template(                                                3881
            "ahp/matrix.html",                                             3882
            form=form, unit_type=unit_type, level=level, l1=l1,            3883
            names=names, vec=vec, default=default,                         3884
    result=result,                                                         3885
            source_label=source_label, custom_scheme=custom_sche           3886
    me,                                                                    3887
    )                                                                      3888
@bp.route("/weights/reset", methods=["POST"])                              3889
def weights_reset():                                                       3890
    """删除自定义权重方案，恢复为该单元类型的论文表 3 默认权重。"""                          3891
    unit_type = request.form.get("unit_type")                              3892
    form = EmptyForm()                                                     3893
    if form.validate_on_submit():                                          3894
        if unit_type not in DEFAULT_WEIGHTS:                               3895
            flash("无效的微地质单元类型", "error")                                3896
        else:                                                              3897
            n = ws.reset_weights(unit_type)                                3898
            if n:                                                          3899
                flash("已恢复「%s」为论文表 3 默认权重" % unit_type,                 3900
```

```
        "ok")                                                                3901
                else:                                                        3902
                    flash("「%s」本就是论文表 3 默认权重，无需恢复" % unit_type,  3903
        "ok")                                                                3904
        return redirect(url_for("ahp.index"))                                3905
# ---------------------------- 指标维护 ----------------------------3906
        ----                                                                 3907
@bp.route("/indicators")                                                     3908
def indicators():                                                            3909
        """指标定义维护：查看指标树、单位、理想值与取值依据。"""                  3910
        if Indicator.query.count() == 0:                                     3911
            ind_svc.seed_indicators()                                        3912
        all_ind = Indicator.query.order_by(Indicator.level,                  3913
        Indicator.id).all()                                                  3914
        by_parent = {}                                                       3915
        for i in all_ind:                                                    3916
            by_parent.setdefault(i.parent_code, []).append(i)                3917
        return render_template("ahp/indicators.html",                        3918
        by_parent=by_parent,                                                 3919
                                    all_ind=all_ind,                         3920
        unit_types=UNIT_TYPES,                                               3921
                                    type_label=ind_svc.ideal_type_lab3922
        el,                                                                  3923
                                    form=EmptyForm())                        3924
@bp.route("/indicators/<code>/edit", methods=["GET",                         3925
        "POST"])                                                             3926
def indicator_edit(code):                                                    3927
        ind = Indicator.query.filter_by(code=code).first_or_404(             3928
        )                                                                    3929
        form = EmptyForm()                                                   3930
        if request.method == "POST" and form.validate_on_submit(             3931
        ):                                                                   3932
            name = request.form.get("name", "").strip()                      3933
            if not name:                                                     3934
                flash("指标名称不能为空", "error")                             3935
                return redirect(url_for("ahp.indicator_edit",                3936
        code=code))                                                          3937
            ind.name = name                                                  3938
            ind.unit = request.form.get("unit", "").strip() or               3939
        None                                                                 3940
            ind.ideal_value = request.form.get("ideal_value",                3941
        "").strip() or None                                                  3942
            ind.ideal_type = request.form.get("ideal_type") or               3943
        ind.ideal_type                                                       3944
            units = request.form.getlist("enabled_units")                    3945
            ind.enabled_units = ",".join(units) if units else                3946
        None                                                                 3947
            ind.is_constraint = request.form.get("is_constraint"3948
        ) == "1"                                                             3949
            ind.description = request.form.get("description",                3950
```

```
    "").strip() or None                                                3951
        db.session.commit()                                            3952
        flash("指标「%s」已保存" % ind.name, "ok")                        3953
        return redirect(url_for("ahp.indicators"))                     3954
    return render_template("ahp/indicator_edit.html",                  3955
    ind=ind,                                                           3956
                                unit_types=UNIT_TYPES,                 3957
                                type_label=ind_svc.ideal_type_lab      3958
    el, form=form)                                                     3959
@bp.route("/indicators/reset", methods=["POST"])                       3960
def indicators_reset():                                                3961
    """按论文取值重置指标定义。"""                                          3962
    form = EmptyForm()                                                 3963
    if form.validate_on_submit():                                      3964
        n = ind_svc.seed_indicators()                                  3965
        flash("已按论文取值重置 %d 项指标定义" % n, "ok")                    3966
    return redirect(url_for("ahp.indicators"))                         3967
# ---------------------------- 多专家矩阵 ------------------------------3968
    -----                                                              3969
def expert_scope():                                                    3970
    """从请求中取（并规整）单元类型、层级与所属一级指标。"""                    3971
    unit_type = request.values.get("unit_type") or                     3972
    UNIT_TYPES[0]                                                      3973
    if unit_type not in DEFAULT_WEIGHTS:                               3974
        unit_type = UNIT_TYPES[0]                                      3975
    try:                                                               3976
        level = int(request.values.get("level", 1))                   3977
    except (TypeError, ValueError):                                    3978
        level = 1                                                      3979
    if level not in (1, 2):                                            3980
        level = 1                                                      3981
    l1 = normalize_l1(unit_type, level,                                3982
    request.values.get("l1"))                                          3983
    return unit_type, level, l1                                        3984
def expert_query(unit_type, level, l1):                                3985
    q = ExpertMatrix.query.filter_by(unit_type=unit_type,              3986
    level=level)                                                       3987
    if level == 2:                                                     3988
        q = q.filter_by(l1_name=l1)                                    3989
    else:                                                              3990
        q = q.filter(ExpertMatrix.l1_name.is_(None))                   3991
    return q.order_by(ExpertMatrix.id)                                 3992
def default_scheme(unit_type, level, l1=None):                         3993
    """取（或建）该范围的权重方案，专家矩阵统一挂到它下面。"""                  3994
    name = "%s · %s%s" % (unit_type,                                   3995
                          "一级指标" if level == 1 else "二级指标",      3996
                          "(%s)" % l1 if l1 else "")                   3997
    scheme = WeightScheme.query.filter_by(unit_type=unit_typ           3998
    e,                                                                 3999
                                          level=level,                 4000
```

```
    name=name).first()                                              4001
    if scheme is None:                                              4002
        scheme = WeightScheme(unit_type=unit_type,                  4003
level=level, name=name,                                             4004
                              is_default=False)                     4005
        db.session.add(scheme)                                      4006
        db.session.flush()                                          4007
    return scheme                                                   4008
@bp.route("/experts", methods=["GET", "POST"])                      4009
def experts():                                                      4010
    """多专家判断矩阵录入与汇总。"""                                4011
    unit_type, level, l1 = expert_scope()                           4012
    if unit_type not in DEFAULT_WEIGHTS:                            4013
        flash("无效的微地质单元类型", "error")                      4014
        return redirect(url_for("ahp.index"))                       4015
    names, vec = indicator_list(unit_type, level, l1)               4016
    n = len(names)                                                  4017
    form = EmptyForm()                                              4018
    if request.method == "POST" and form.validate_on_submit(        4019
    ):                                                              4020
        action = request.form.get("action")                         4021
        if action == "add":                                         4022
            expert_name = request.form.get("expert_name",           4023
"").strip() or "未具名专家"                                         4024
            try:                                                    4025
                upper = [float(request.form.get("m_%d_%d" %         4026
(i, j), "1"))                                                       4027
                         for i in range(n) for j in range(i         4028
+ 1, n)]                                                           4029
            except ValueError:                                      4030
                flash("矩阵输入有误，请填入正数", "error")          4031
                return redirect(url_for("ahp.experts",              4032
unit_type=unit_type,                                                4033
                                        level=level, l1=l1))        4034
            # 0 / 负数 / inf / nan 都能过 float()，但会让 n 次方根算出      4035
inf 或 nan,                                                         4036
            # 于是库里存下 [[1,0],[Infinity,1]] 这种矩阵、CR 还显示        4037
0.0000。                                                            4038
            if any((not math.isfinite(x)) or x <= 0 for x           4039
in upper):                                                          4040
                flash("判断矩阵只能填正数（大于 0 的有限数字）", "error")   4041
                return redirect(url_for("ahp.experts",              4042
unit_type=unit_type,                                                4043
                                        level=level, l1=l1))        4044
            A = ahp.matrix_from_upper(upper)                        4045
            r = ahp.weights_and_consistency(A)                      4046
            scheme = default_scheme(unit_type, level, l1)           4047
            db.session.add(ExpertMatrix(                            4048
                scheme_id=scheme.id,                                4049
                unit_type=unit_type, l1_name=(l1 if level           4050
```

```
    == 2 else None),                                                    4051
                expert_name=expert_name, level=level,                   4052
                matrix_json=json.dumps([list(map(float,                 4053
row)) for row in A]),                                                   4054
                cr=round(r["cr"], 6)))                                  4055
            db.session.commit()                                         4056
            flash("已录入专家「%s」的判断矩阵（CR=%.4f）" % (expert_name,  4057
r["cr"]), "ok")                                                         4058
        elif action == "save_agg":                                      4059
            # 把汇总权重落库并生效，使多专家结果真正参与评价计算          4060
            weights = _aggregate(unit_type, level, l1)                  4061
            if weights is None:                                         4062
                flash("暂无可汇总的专家矩阵", "error")                   4063
            elif not weights["consistent"]:                             4064
                flash("汇总矩阵一致性比例 CR=%.4f ≥ 0.1，不满足要求，"   4065
                      "请先检查或调整专家矩阵" % weights["cr"],          4066
"error")                                                                4067
            else:                                                       4068
                names_, _ = indicator_list(unit_type,                   4069
level, l1)                                                              4070
                pairs = to_code_pairs(unit_type, level, l1,             4071
names_,                                                                 4072
                                      weights["weights"])               4073
                _, cnt = ws.save_weights(unit_type, pairs)              4074
                flash("已保存并为「%s」生效 %d 项汇总权重（CR=%.4f）"     4075
                      % (unit_type, cnt, weights["cr"]),                4076
"ok")                                                                   4077
        else:                                                           4078
            # action 缺失或值不认识时以前是静默跳回，用户只觉得「点了没反应」  4079
            flash("未识别的操作，请从页面上的按钮重新提交", "error")      4080
        return redirect(url_for("ahp.experts",                          4081
unit_type=unit_type,                                                    4082
                                level=level, l1=l1))                    4083
    rows = expert_query(unit_type, level, l1).all()                     4084
    default = [[round(vec[i] / vec[j], 4) for j in                      4085
range(n)] for i in range(n)]                                            4086
    return render_template("ahp/experts.html",                          4087
    unit_type=unit_type, level=level,                                   4088
                                l1=l1, names=names, rows=rows,          4089
    default=default,                                                    4090
                                n=n, form=form)                         4091
@bp.route("/experts/<int:mid>/delete", methods=["POST"])                4092
def expert_delete(mid):                                                 4093
    row = db.get_or_404(ExpertMatrix, mid)                              4094
    unit_type, level, l1 = row.unit_type, row.level,                    4095
    row.l1_name                                                         4096
    form = EmptyForm()                                                  4097
    if form.validate_on_submit():                                       4098
        db.session.delete(row)                                          4099
        db.session.commit()                                             4100
```

```
        flash("已删除专家矩阵", "ok")                                    4101
    return redirect(url_for("ahp.experts",                              4102
    unit_type=unit_type,                                                4103
                                level=level, l1=l1))                    4104
def _aggregate(unit_type, level, l1):                                   4105
    """按几何平均汇总多专家矩阵并做一致性检验。无矩阵时返回 None。"""                4106
    rows = expert_query(unit_type, level, l1).all()                     4107
    mats = []                                                           4108
    for r in rows:                                                      4109
        try:                                                            4110
            mats.append([list(map(float, line))                         4111
                         for line in json.loads(r.matrix_jso            4112
    n)])                                                                4113
        except Exception:                                               4114
            continue                                                    4115
    if not mats:                                                        4116
        return None                                                     4117
    agg = ahp.aggregate_matrices(mats)                                  4118
    res = ahp.weights_and_consistency(agg)                              4119
    return {                                                            4120
        "count": len(mats),                                             4121
        "weights": res["weights"],                                      4122
        "lambda_max": res["lambda_max"],                                4123
        "ci": res["ci"], "cr": res["cr"],                               4124
        "consistent": res["consistent"],                                4125
        "most_inconsistent": ahp.find_most_inconsistent(agg)            4126
    ,                                                                   4127
        "expert_crs": [r.cr for r in rows if r.cr is not                4128
    None],                                                              4129
    }                                                                   4130
@bp.route("/experts/aggregate")                                         4131
def experts_aggregate():                                                4132
    """按几何平均汇总多专家矩阵，并做一致性检验。"""                                4133
    unit_type, level, l1 = expert_scope()                               4134
    if unit_type not in DEFAULT_WEIGHTS:                                4135
        flash("无效的微地质单元类型", "error")                                4136
        return redirect(url_for("ahp.index"))                           4137
    names, vec = indicator_list(unit_type, level, l1)                   4138
    rows = expert_query(unit_type, level, l1).all()                     4139
    result = _aggregate(unit_type, level, l1)                           4140
    if result is not None:                                              4141
        result["weights"] = [round(v, 4) for v in                       4142
    result["weights"]]                                                  4143
        result["lambda_max"] = round(result["lambda_max"],              4144
    4)                                                                  4145
    return render_template("ahp/experts_aggregate.html",                4146
    unit_type=unit_type,                                                4147
                               level=level, l1=l1, names=names,         4148
    rows=rows,                                                          4149
                               result=result, form=EmptyForm())         4150
```

```
# ==== app/views/indicator.py ====
# -*- coding: utf-8 -*-
"""指标数据采集与管理 —— 路由与页面逻辑。
按微地质单元组织录入；系统按单元类型自动决定应录入哪些指标
（岩质边坡无土壤指标、平台无地形指标）。
"""
import io
import json
from flask import (Blueprint, render_template, request,
    redirect, url_for,
                   flash, send_file)
from app.forms import EmptyForm
from app.models import db, MinePatch, MicroUnit,
    IndicatorData
from app.utils.constants import HEAVY_METALS, ASPECT_TYPES,
    UNIT_ENABLED_INDICATORS
from app.utils.parsing import FieldError, parse_float
from app.services import import_service as imp
bp = Blueprint("indicator", __name__,
    url_prefix="/indicator")
# 指标字段定义：字段名 -> (中文名, 单位, 是否可选)
SOIL_FIELDS = [
    ("soil_ratio", "土石比", ""),
    ("thickness", "土层厚度", "cm"),
    ("ph", "pH", ""),
    ("organic_matter", "有机质", "g/kg"),
]
VEG_FIELDS = [
    ("coverage", "植被覆盖度", "%"),
    ("shannon", "Shannon-Wiener 多样性指数", ""),
]
TERRAIN_FIELDS = [
    ("slope", "坡度", "°"),
    ("aspect", "坡向", ""),
]
def unit_field_config(unit_type):
    """根据单元类型返回需录入的字段分组。"""
    enabled = UNIT_ENABLED_INDICATORS.get(unit_type, ())
    groups = []
    if "土壤指标" in enabled:
        groups.append(("土壤指标", SOIL_FIELDS, True))    # True
    = 含重金属
    if "植被指标" in enabled:
        groups.append(("植被指标", VEG_FIELDS, False))
    if "地形指标" in enabled:
        groups.append(("地形指标", TERRAIN_FIELDS, False))
    return groups
@bp.route("/")
def index():
    patches = MinePatch.query.order_by(MinePatch.id.desc()).
```

```
    all()
    return render_template("indicator/index.html",
    patches=patches)
@bp.route("/<int:unit_id>", methods=["GET", "POST"])
def edit(unit_id):
    unit = db.get_or_404(MicroUnit, unit_id)
    ind = IndicatorData.query.filter_by(unit_id=unit.id).fir
    st()
    form = EmptyForm()
    groups = unit_field_config(unit.unit_type)
    if request.method == "POST" and form.validate_on_submit(
    ):
        # 各字段的合理范围（覆盖度是百分数、pH 0–14、坡度 0–90，其余非负）。
        # 以前这里直接 float(v)，用户填个「约 3.5」看到的就是英文 500 页。
        LIMITS = {
            "soil_ratio": ("土石比", 0, None),
            "thickness": ("土层厚度(cm)", 0, None),
            "ph": ("pH", 0, 14),
            "organic_matter": ("有机质(g/kg)", 0, None),
            "coverage": ("植被覆盖度(%)", 0, 100),
            "shannon": ("Shannon 指数", 0, None),
            "slope": ("坡度(°)", 0, 90),
        }
        try:
            values = {}
            for name, (label, lo, hi) in LIMITS.items():
                values[name] = parse_float(request.form.get(
    name), label,
                                           minimum=lo,
    maximum=hi)
            heavy = {}
            for m in HEAVY_METALS:
                v = parse_float(request.form.get("hm_" + m),
                                "重金属 %s(mg/kg)" % m,
    minimum=0)
                if v is not None:
                    heavy[m] = v
        except FieldError as e:
            flash(str(e), "error")
            return redirect(url_for("indicator.edit",
    unit_id=unit.id))
        if ind is None:
            ind = IndicatorData(unit_id=unit.id)
            db.session.add(ind)
        for name, value in values.items():
            setattr(ind, name, value)
        ind.heavy_metals = json.dumps(heavy,
    ensure_ascii=False) if heavy else None
        ind.aspect = request.form.get("aspect", "").strip()
    or None
```

```
        db.session.commit()                                          4251
        flash("指标数据已保存", "ok")                                  4252
        return redirect(url_for("indicator.index"))                  4253
    # 组装当前值供模板回显                                             4254
    current = {}                                                     4255
    if ind:                                                          4256
        current = {                                                  4257
            "soil_ratio": ind.soil_ratio, "thickness":               4258
    ind.thickness,                                                   4259
            "ph": ind.ph, "organic_matter":                          4260
    ind.organic_matter,                                              4261
            "coverage": ind.coverage, "shannon":                     4262
    ind.shannon,                                                     4263
            "slope": ind.slope, "aspect": ind.aspect,                4264
        }                                                            4265
        current["heavy"] = ind.heavy_metals_dict()                   4266
    return render_template("indicator/edit.html",                    4267
                           unit=unit, form=form,                     4268
    groups=groups,                                                   4269
                           heavy_metals=HEAVY_METALS,                4270
                           aspect_types=ASPECT_TYPES,                4271
    current=current)                                                 4272
# ---------------------------- 批量导入 ----------------------------4273
    ----                                                             4274
@bp.route("/import", methods=["GET", "POST"])                        4275
def import_data():                                                   4276
    """批量导入指标数据（CSV / Excel）。"""                             4277
    patches = MinePatch.query.order_by(MinePatch.id.desc()).4278
    all()                                                            4279
    form = EmptyForm()                                               4280
    report = None                                                    4281
    if request.method == "POST" and form.validate_on_submit(4282
    ):                                                               4283
        patch_id = request.form.get("patch_id", type=int)            4284
        patch = db.session.get(MinePatch, patch_id) if               4285
    patch_id else None                                               4286
        if patch is None:                                            4287
            flash("请选择矿山图斑", "error")                           4288
            return redirect(url_for("indicator.import_data")4289
    )                                                                4290
        file = request.files.get("datafile")                         4291
        if file is None or not file.filename:                        4292
            flash("请选择要导入的文件", "error")                       4293
            return redirect(url_for("indicator.import_data")4294
    )                                                                4295
        ext = file.filename.rsplit(".", 1)[-1].lower()               4296
        if ext not in ("csv", "xlsx", "xls"):                        4297
            flash("仅支持 CSV 或 Excel（.xlsx/.xls）文件", "error")   4298
            return redirect(url_for("indicator.import_data")4299
    )                                                                4300
```

```
    units = MicroUnit.query.filter_by(patch_id=patch.id)
.all()
    unit_map = {}
    for u in units:
        unit_map.setdefault(u.unit_type, u)
    if not unit_map:
        flash("该图斑尚未划分微地质单元，请先划分后再导入", "error")
        return redirect(url_for("indicator.import_data")
)
    try:
        parsed, errors = imp.parse_upload(file.filename,
file.stream, unit_map)
    except Exception as e:
        flash("文件解析失败：%s" % e, "error")
        return redirect(url_for("indicator.import_data")
)
    saved = 0
    for rec in parsed:
        unit = unit_map[rec["unit_type"]]
        ind = IndicatorData.query.filter_by(unit_id=unit
.id).first()
        if ind is None:
            ind = IndicatorData(unit_id=unit.id)
            db.session.add(ind)
        f = rec["fields"]
        for key in ("soil_ratio", "thickness", "ph",
"organic_matter",
                    "coverage", "shannon", "slope",
"aspect"):
            if key in f:
                setattr(ind, key, f[key])
        if rec["heavy_metals"]:
            ind.heavy_metals = json.dumps(rec["heavy_met
als"],

ensure_ascii=False)
        saved += 1
    db.session.commit()
    report = {"patch": patch.name, "saved": saved,
"errors": errors,
              "total": len(parsed) + len(
                  [e for e in errors if e.startswith("第
")])}
    if saved:
        flash("已导入 %d 个微地质单元的指标数据" % saved, "ok")
    else:
        flash("没有可导入的数据行，请检查文件内容", "error")
return render_template("indicator/import.html",
patches=patches,
                       form=form, report=report,
```

```
    headers=imp.HEADERS)                                                    4351
@bp.route("/import/template")                                               4352
def import_template():                                                      4353
    """下载导入模板（CSV，含一行示例）。"""                                  4354
    csv_text = imp.build_template_csv()                                     4355
    data = csv_text.encode("utf-8-sig")       # 带 BOM，便于               4356
    Excel 正确识别中文                                                      4357
    return send_file(io.BytesIO(data), as_attachment=True,                  4358
                     download_name="指标数据导入模板.csv",                  4359
                     mimetype="text/csv")                                   4360
# ==== app/views/evaluate.py ====                                           4361
# -*- coding: utf-8 -*-                                                     4362
"""适宜性定量评价引擎 —— 路由与页面逻辑。                                   4363
从数据库加载某图斑的微地质单元与指标数据，调用评价引擎计算，               4364
展示各单元得分、最终评价因子 F、适宜性等级与推荐措施。                     4365
入口处按论文 4.1 节的流程顺序检查前置条件（见 app/services/workflow.py），4366
前置步骤未完成时不予计算，避免在条件不具备时产出评价结论。                4367
"""                                                                         4368
import json                                                                 4369
from flask import Blueprint, render_template, request,                      4370
    redirect, url_for, flash                                                4371
from flask_login import current_user                                        4372
from app.forms import EmptyForm                                             4373
from app.models import (db, MinePatch, MicroUnit,                           4374
    IndicatorData,                                                          4375
                        EvaluationResult, ScoreDetail)                      4376
from app.services import evaluation_service as ev                           4377
from app.services import workflow as wf                                     4378
from app.services import unit_config                                        4379
bp = Blueprint("evaluate", __name__, url_prefix="/evaluate")                4380
def load_patch_input(patch_id):                                             4381
    """从数据库组装评价引擎输入。"""                                        4382
    units = MicroUnit.query.filter_by(patch_id=patch_id).all                4383
    ()                                                                      4384
    result = []                                                             4385
    for u in units:                                                         4386
        ind = IndicatorData.query.filter_by(unit_id=u.id).fi                4387
    rst()                                                                   4388
        data = {                                                            4389
            "soil_ratio": ind.soil_ratio if ind else None,                  4390
            "thickness": ind.thickness if ind else None,                    4391
            "ph": ind.ph if ind else None,                                  4392
            "organic_matter": ind.organic_matter if ind                     4393
    else None,                                                              4394
            "heavy_metals": ind.heavy_metals_dict() if ind                  4395
    else {},                                                                4396
            "coverage": ind.coverage if ind else None,                      4397
            "shannon": ind.shannon if ind else None,                        4398
            "slope": ind.slope if ind else None,                            4399
            "aspect": ind.aspect if ind else None,                          4400
```

```
            }                                                                    4401
            result.append({                                                      4402
                "unit_id": u.id,                                                 4403
                "unit_type": u.unit_type,                                        4404
                "ratio": u.ratio if u.ratio is not None else                     4405
        0.0,                                                                     4406
                "data": data,                                                    4407
            })                                                                   4408
        return result                                                            4409
    def _round(v, n=6):                                                          4410
        return None if v is None else round(v, n)                                4411
    def load_weights(units):                                                     4412
        """解析各单元类型当前生效的权重与来源说明。                                  4413
        权重可来自 AHP 计算并保存的生效方案；未配置时用论文表 3 的默认值。           4414
        来源一并记录，便于报告标注与事后复核。                                      4415
        """                                                                      4416
        from app.services import weight_service as ws                            4417
        weights, sources = {}, {}                                                4418
        for u in units:                                                          4419
            ut = u["unit_type"]                                                  4420
            if ut not in weights:                                                4421
                w, label, _ = ws.effective_weights(ut)                           4422
                weights[ut] = w                                                  4423
                sources[ut] = label                                              4424
        return weights, sources                                                  4425
    def pack_units(unit_results, sources=None):                                  4426
        """把评价结果整理为可持久化的 JSON。                                        4427
        完整保留各单元得分、一级指标得分与逐条二级指标明细，                        4428
        使结果页与报告能够复现「已记录的那一次评价」，                              4429
        而不是每次打开时重新推算。                                                4430
        """                                                                      4431
        sources = sources or {}                                                  4432
        out = []                                                                 4433
        for u in unit_results:                                                   4434
            out.append({                                                         4435
                "unit_type": u["unit_type"],                                     4436
                "ratio": _round(u["ratio"]),                                     4437
                "unit_score": _round(u["unit_score"]),                           4438
                "weighted": _round(u["weighted"]),                               4439
                "weight_source": sources.get(u["unit_type"]),                    4440
                "l1_scores": {k: _round(v) for k, v in                           4441
        u["l1_scores"].items()},                                                 4442
                "details": [{                                                    4443
                    "l1": d["l1"], "l2": d["l2"],                                4444
                    "raw": None if d["raw"] is None else                         4445
        str(d["raw"]),                                                           4446
                    "std_value": _round(d["std_value"]),                         4447
                    "weight": _round(d["weight"]),                               4448
                    "contribution": _round(d["contribution"]),                   4449
                } for d in u["details"]],                                        4450
```

```
        })                                                                  4451
    return out                                                              4452
def pack_params(settings, sources=None):                                    4453
    """把本次计算用到的参数整理成可持久化的快照。                                  4454
    参数（指标理想取值、图斑参照生态系统基准、权重方案）都能在界面上改，                        4455
    所以结果记录必须连「当时用的什么参数」一起留下，否则事后无法复现。                          4456
    """                                                                     4457
    from app.utils.constants import (IDEAL_SOIL_RATIO,                      4458
    IDEAL_THICKNESS,                                                        4459
                                     IDEAL_ORGANIC,                         4460
    IDEAL_COVERAGE,                                                         4461
                                     IDEAL_SHANNON,                         4462
    HEAVY_METAL_LIMIT)                                                      4463
    defaults = {                                                            4464
        "soil_ratio": IDEAL_SOIL_RATIO, "thickness":                        4465
    IDEAL_THICKNESS,                                                        4466
        "organic_matter": IDEAL_ORGANIC, "coverage":                        4467
    IDEAL_COVERAGE,                                                         4468
        "shannon": IDEAL_SHANNON,                                           4469
    }                                                                       4470
    for metal, limit in HEAVY_METAL_LIMIT.items():                          4471
        defaults["hm_" + metal] = limit                                     4472
    used = {}                                                               4473
    for code, fallback in sorted(defaults.items()):                         4474
        value = settings.params.get(code) if settings else                  4475
    None                                                                    4476
        if value is None:                                                   4477
            used[code] = {"value": fallback, "default":                     4478
    fallback,                                                               4479
                          "source": "论文给定取值"}                             4480
        elif abs(float(value) - float(fallback)) < 1e-9:                    4481
            #  库里存着这一项，但取值与论文一致 —— 不能标成「界面配置」，                  4482
            #  否则每份报告都会声称用了自定义参数，反而看不出真正的偏离。                  4483
            used[code] = {"value": value, "default":                        4484
    fallback,                                                               4485
                          "source": "论文给定取值"}                             4486
        else:                                                               4487
            used[code] = {"value": value, "default":                        4488
    fallback,                                                               4489
                          "source": "界面配置（论文给定值 %.4g）" %             4490
    fallback}                                                               4491
    return {"ideals": used, "weight_sources": dict(sources                  4492
    or {})}                                                                 4493
def save_evaluation(patch_id, units):                                       4494
    """执行评价并把结果与逐层明细落库，返回评价记录。                                 4495
    权重取各单元类型当前生效的方案（ＡＨＰ 计算结果），未配置时用                        4496
    论文表 3 的默认权重；来源一并记录。结果 ＪＳＯＮ 保留各单元得分、                    4497
    一级指标得分与逐条二级指标明细，结果页与报告据此复现                              4498
    记录下来的那次评价，不再重新推算。                                            4499
    """                                                                     4500
```

```
    weights, sources = load_weights(units)
    #  标准化参数取自该图斑的参照生态系统与指标定义（界面上可改），
    #  没配过的项自动回落到代码常量，历史结果与验证算例不受影响。
    settings = unit_config.for_patch(patch_id)
    res = ev.score_mine(units, weights, settings)
    saved = EvaluationResult(
        patch_id=patch_id,
        final_score=round(res["final_score"], 4),
        grade=res["grade"],
        measures=res["grade_info"]["measure"],
        unit_scores=json.dumps(pack_units(res["unit_results"
], sources),
                               ensure_ascii=False),
        #  参数快照 + 计算人：参数现在可以在界面上改，不留快照就复现不了这次结果
        params_json=json.dumps(pack_params(settings,
sources),
                               ensure_ascii=False),
        created_by=current_user.id if getattr(current_user,
"is_authenticated", False)
        else None,
    )
    db.session.add(saved)
    db.session.flush()        #  先取得 result.id，再写明细
    unit_ids = {u["unit_type"]: u["unit_id"] for u in units}
    for u in res["unit_results"]:
        for d in u["details"]:
            db.session.add(ScoreDetail(
                result_id=saved.id,
                unit_id=unit_ids.get(u["unit_type"]),
                l1_name=d["l1"], l2_name=d["l2"],
                raw_value=None if d["raw"] is None else
str(d["raw"]),
                std_value=d["std_value"],
weight=d["weight"],
                contribution=d["contribution"]))
    db.session.commit()
    return saved
def weak_points(unit_scores, top=3):
    """从已保存的结果里挑出「提升空间最大」的指标。
    提升空间 = 权重 ×（1 − 标准化值）：标准化值越低、权重越大，抬它越划算。
    只读已保存的记录，不重新计算，保证与报告口径一致。
    """
    rows = []
    for u in unit_scores or []:
        for d in u.get("details") or []:
            std = d.get("std_value")
            weight = d.get("weight")
            if std is None or weight is None:
                continue              #  不适用项（未参与计算）不进短板
            rows.append({
```

```
                    "unit_type": u.get("unit_type"),
                    "l2": d.get("l2"),
                    "raw": d.get("raw"),
                    "std_value": std,
                    "gain": weight * (1.0 - std),
                })
    rows.sort(key=lambda x: x["gain"], reverse=True)
    return [x for x in rows if x["gain"] > 1e-9][:top]
def user_label(user_id):
    """把计算人 id 换成「昵称（账户名称）」。"""
    from app.models import User
    if not user_id:
        return None
    u = db.session.get(User, user_id)
    if u is None:
        return "已删除的账号"
    return u.shown_name
def param_source_summary(params):
    """一句话说清参数从哪来：有没有偏离论文给定值。
    多数情况下全部等于论文取值，报告应直接说明「未偏离」；
    只有真被改过时才点名，便于审阅者一眼看到差异。
    """
    ideals = (params or {}).get("ideals") or {}
    custom = sorted(k for k, v in ideals.items()
                    if v.get("source",
    "").startswith("界面配置"))
    if not custom:
        return "全部采用论文给定取值，未被改动"
    return "以下 %d 项被改为界面配置值：%s" % (len(custom),
    "、".join(custom))
def block_reasons(progress):
    """把未满足的前置条件转成可读的拦截原因。"""
    return ["%s：%s" % (b["label"], b["reason"]) for b in
    progress["blocking"]]
@bp.route("/")
def index():
    patches = MinePatch.query.order_by(MinePatch.id.desc()).
    all()
    latest = {}
    progress = {}
    for p in patches:
        latest[p.id] = EvaluationResult.query.filter_by(patc
    h_id=p.id) \
            .order_by(EvaluationResult.id.desc()).first()
        progress[p.id] = wf.patch_progress(p.id)
    return render_template("evaluate/index.html",
    patches=patches,
                           latest=latest, progress=progress)
@bp.route("/<int:patch_id>", methods=["GET", "POST"])
def run(patch_id):
```

```
    patch = db.get_or_404(MinePatch, patch_id)                    4601
    form = EmptyForm()                                            4602
    progress = wf.patch_progress(patch_id)                        4603
    if request.method == "POST" and form.validate_on_submit(      4604
    ):                                                            4605
        if not progress["ready"]:                                 4606
            for reason in block_reasons(progress):                4607
                flash("无法执行评价 —— " + reason, "error")          4608
            return redirect(url_for("evaluate.run",               4609
    patch_id=patch_id))                                           4610
        units = load_patch_input(patch_id)                        4611
        saved = save_evaluation(patch.id, units)                  4612
        return redirect(url_for("evaluate.result",                4613
    result_id=saved.id))                                          4614
    # 展示将采用的权重来源，便于在执行前确认                              4615
    units_now = load_patch_input(patch_id)                        4616
    _, sources = load_weights(units_now) if units_now else        4617
    ({}, {})                                                      4618
    return render_template("evaluate/run.html",                   4619
    patch=patch, form=form,                                       4620
                                progress=progress,                4621
    weight_sources=sources)                                       4622
@bp.route("/result/<int:result_id>")                              4623
def result(result_id):                                            4624
    r = db.get_or_404(EvaluationResult, result_id)                4625
    patch = db.session.get(MinePatch, r.patch_id)                 4626
    unit_scores = json.loads(r.unit_scores) if                    4627
    r.unit_scores else []                                         4628
    params = json.loads(r.params_json) if r.params_json           4629
    else {}                                                       4630
    return render_template("evaluate/result.html", r=r,           4631
    patch=patch,                                                  4632
                               unit_scores=unit_scores,           4633
                               undetected=wf.undetected_metals(p  4634
    atch.id)                                                      4635
                               if patch else [],                  4636
                               weak=weak_points(unit_scores),     4637
                               params=params.get("ideals") or     4638
    {},                                                           4639
                               param_sources=param_source_summar  4640
    y(params),                                                    4641
                               operator=user_label(r.created_by)  4642
    )                                                             4643
# ==== app/views/report.py ====                                   4644
# -*- coding: utf-8 -*-                                         4645
"""报告与结果输出 —— 得分明细、图表、Word / PDF 导出。"""              4646
import io                                                         4647
import json                                                       4648
from datetime import datetime                                     4649
from flask import Blueprint, render_template, send_file,          4650
```

```
    flash, redirect, url_for
from app.models import db, MinePatch, ReferenceEcosystem
from app.services import report_service as rs
from app.services import workflow as wf
from app.utils import charts
from app.utils import browser
from app.utils.constants import grade_of
from app.utils.timefmt import now_utc
bp = Blueprint("report", __name__, url_prefix="/report")
def _prepare(patch_id):
    """加载图斑并取出已记录的评价结果，供明细页与导出复用。
    报告是评价结论的呈现，因此这里读取最近一次保存的评价记录，
    而不是重新计算：评价完成后再改动指标数据，也不会让报告与
    系统记录的评价结论对不上。没有任何评价记录时，返回流程检查
    结果，由调用方拦截并提示先去执行评价。
    """
    patch = db.get_or_404(MinePatch, patch_id)
    prog = wf.report_progress(patch_id)
    if not prog["ready"]:
        return patch, None, None, None, prog
    rec = prog["result"]
    result = {
        "final_score": rec.final_score,
        "grade": rec.grade,
        "grade_info": grade_of(rec.final_score),
        "unit_results": json.loads(rec.unit_scores) if
    rec.unit_scores else [],
    }
    reference = ReferenceEcosystem.query.filter_by(patch_id=
    patch_id).first()
    return patch, reference, result, rec, prog
def _deny(progress, patch_id, endpoint="report.detail"):
    """前置条件未满足时的统一处理：逐条提示并退回报告列表。"""
    for b in progress["blocking"]:
        flash("暂不能生成报告 —— %s：%s" % (b["label"],
    b["reason"]), "error")
    if progress["blocking"]:
        flash("请先完成上述步骤后再生成报告。", "error")
    return redirect(url_for("report.index"))
def _weak_points(result, top=3):
    """提升空间最大的指标：权重 ×（1 − 标准化值）。
    与结果页用同一套口径，便于向使用方解释「先改哪几项」。
    """
    rows = []
    for u in result.get("unit_results") or []:
        for d in u.get("details") or []:
            std, w = d.get("std_value"), d.get("weight")
            if std is None or w is None:
                continue
            rows.append((u["unit_type"], d["l2"],
```

```
    d.get("raw"), std,                                              4701
                              w * (1.0 - std)))                     4702
    rows.sort(key=lambda x: x[4], reverse=True)                     4703
    return [r for r in rows if r[4] > 1e-9][:top]                   4704
def _operator(record):                                              4705
    """报告里的计算人：昵称（账户名称）。"""                                4706
    from app.models import User                                     4707
    uid = getattr(record, "created_by", None)                       4708
    if not uid:                                                     4709
        return None                                                 4710
    u = db.session.get(User, uid)                                   4711
    return u.shown_name if u else "已删除的账号"                        4712
def _charts(result):                                                4713
    """生成两张图表：单元得分对比 + 一级指标平均得分分布。"""                  4714
    units = [(u["unit_type"], u["unit_score"],                      4715
              "%.4f" % u["unit_score"]) for u in                    4716
    result["unit_results"]]                                         4717
    chart_units = charts.hbars(units)                               4718
    # 各一级指标在各单元上的平均得分，反映其对评价结果的实际影响                  4719
    acc, cnt = {}, {}                                               4720
    for u in result["unit_results"]:                                4721
        for k, v in (u.get("l1_scores") or {}).items():             4722
            acc[k] = acc.get(k, 0.0) + v                            4723
            cnt[k] = cnt.get(k, 0) + 1                              4724
    items = [(k, acc[k] / cnt[k], "%.4f" % (acc[k] /                4725
    cnt[k])) for k in acc]                                          4726
    items.sort(key=lambda x: -x[1])                                 4727
    chart_weight = charts.donut(items)                              4728
    return chart_units, chart_weight                                4729
@bp.route("/")                                                      4730
def index():                                                        4731
    patches = MinePatch.query.order_by(MinePatch.id.desc()).        4732
    all()                                                           4733
    progress = {p.id: wf.report_progress(p.id) for p in             4734
    patches}                                                        4735
    return render_template("report/index.html",                     4736
    patches=patches,                                                4737
                                progress=progress)                  4738
@bp.route("/<int:patch_id>")                                        4739
def detail(patch_id):                                               4740
    patch, reference, result, record, prog =                        4741
    _prepare(patch_id)                                              4742
    if result is None:                                              4743
        return _deny(prog, patch_id)                                4744
    chart_units, chart_weight = _charts(result)                     4745
    return render_template("report/detail.html",                    4746
    patch=patch, result=result,                                     4747
                                record=record,                      4748
    reference=reference,                                            4749
                                chart_units=chart_units,            4750
```

```
    chart_weight=chart_weight,                                          4751
                                undetected=wf.undetected_metals(p       4752
    atch.id))                                                           4753
@bp.route("/<int:patch_id>/word")                                       4754
def word(patch_id):                                                     4755
    patch, reference, result, record, prog =                            4756
    _prepare(patch_id)                                                  4757
    if result is None:                                                  4758
        return _deny(prog, patch_id)                                    4759
    buf = rs.build_report(patch, result, record=record,                 4760
                          weak=_weak_points(result),                    4761
                          operator=_operator(record))                   4762
    filename = "自然恢复适宜性评价报告_%s.docx" % patch.name             4763
    return send_file(buf, as_attachment=True,                           4764
    download_name=filename,                                             4765
                     mimetype="application/vnd.openxmlformat            4766
    s-officedocument."                                                  4767
                                "wordprocessingml.document")            4768
@bp.route("/<int:patch_id>/pdf")                                        4769
def pdf(patch_id):                                                      4770
    """导出 PDF 报告：服务端渲染打印模板，再交浏览器无头打印。"""          4771
    patch, reference, result, record, prog =                            4772
    _prepare(patch_id)                                                  4773
    if result is None:                                                  4774
        return _deny(prog, patch_id)                                    4775
    chart_units, chart_weight = _charts(result)                         4776
    html = render_template(                                             4777
        "report/print.html", patch=patch,                               4778
    reference=reference, result=result,                                 4779
        chart_units=chart_units, chart_weight=chart_weight,             4780
        generated_at=now_utc())                                         4781
    data, err = browser.html_to_pdf(html)                               4782
    if err:                                                             4783
        flash("PDF 导出失败：%s" % err, "error")                         4784
        return redirect(url_for("report.detail",                        4785
    patch_id=patch_id))                                                 4786
    return send_file(io.BytesIO(data), as_attachment=True,              4787
                     download_name="自然恢复适宜性评价报告_%s.pdf" %      4788
    patch.name,                                                         4789
                     mimetype="application/pdf")                        4790
# ==== app/views/search.py ====                                         4791
# -*- coding: utf-8 -*-                                                 4792
"""全局搜索：在评价项目、矿山图斑、标准与政策文件中查找关键词。          4793
顶栏的搜索框指向本模块。只做关键词包含匹配，不做分词，                  4794
但覆盖名称、编号、行政区、矿种、文号、摘要等常用检索字段。              4795
"""                                                                     4796
from sqlalchemy import or_                                              4797
from flask import Blueprint, render_template, request                   4798
from app.models import db, Project, MinePatch, Standard                 4799
bp = Blueprint("search", __name__)                                      4800
```

```
LIMIT = 50              #  每类结果上限，避免一次渲染过多                                4801
@bp.route("/search")                                                                    4802
def index():                                                                            4803
    q = (request.args.get("q") or "").strip()                                           4804
    projects, patches, standards = [], [], []                                           4805
    if q:                                                                               4806
        like = "%" + q + "%"                                                            4807
        projects = Project.query.filter(or_(                                            4808
            Project.name.like(like),                                                    4809
            Project.code.like(like),                                                    4810
            Project.description.like(like),                                             4811
        )).order_by(Project.id.desc()).limit(LIMIT).all()                               4812
        patches = MinePatch.query.filter(or_(                                           4813
            MinePatch.name.like(like),                                                  4814
            MinePatch.admin_region.like(like),                                          4815
            MinePatch.mineral_type.like(like),                                          4816
        )).order_by(MinePatch.id.desc()).limit(LIMIT).all()                             4817
        standards = Standard.query.filter(or_(                                          4818
            Standard.name.like(like),                                                   4819
            Standard.doc_number.like(like),                                             4820
            Standard.province.like(like),                                               4821
            Standard.summary.like(like),                                                4822
        )).order_by(Standard.id).limit(LIMIT).all()                                     4823
    total = len(projects) + len(patches) + len(standards)                               4824
    return render_template("search.html", q=q,                                          4825
    projects=projects,                                                                  4826
                                  patches=patches,                                      4827
    standards=standards, total=total)                                                   4828
# ==== app/templates/base.html ====                                                     4829
{% import "_macros.html" as ui %}<!DOCTYPE html>                                        4830
<html lang="zh-CN">                                                                     4831
<head>                                                                                  4832
  <meta charset="UTF-8">                                                                4833
  <meta name="viewport" content="width=device-width,                                    4834
    initial-scale=1.0">                                                                 4835
  <link rel="icon" href="{{ url_for('static',                                           4836
    filename='favicon.ico') }}">                                                        4837
  <title>{% block title %}历史遗留废弃矿山自然恢复适宜性评价系统{% endblock            4838
    %}</title>                                                                          4839
  <style>                                                                               4840
    /* ---------- 设计变量：绿色主色 + 中性灰阶 ---------- */                          4841
    :root {                                                                             4842
      --accent: #2c6b5a;                  /* 主色：青绿，千里江山石绿偏青 */          4843
      --accent-hover: #1e5548;                                                          4844
      --accent-soft: #e6f0ec;                                                           4845
      --side-1: #1a4a4a;                  /* 侧边栏：深青墨 */                        4846
      --side-2: #0f3338;                                                                4847
      --danger: #a0392a;                  /* 朱砂红 */                                4848
      --danger-soft: #fbeae6;                                                           4849
      --ok: #2c6b5a;                                                                    4850
```

```
    --ok-soft: #e6f0ec;                                                    4851
    --todo: #c08a3e;                    /* 赭石色待办 */                   4852
    --ink: #2a2520;                     /* 深褐墨色 */                     4853
    --ink-soft: #6b5d4f;                /* 褐灰 */                         4854
    --line: #e0d8c8;                    /* 绢本边线 */                     4855
    --surface: #fdfaf2;                 /* 暖白绢本 */                     4856
    --canvas: #f5efe0;                  /* 绢本米黄背景 */                 4857
    --radius: 10px;                                                        4858
    --shadow: 0 2px 8px rgba(42, 37, 32, .06);                             4859
    --header-h: 60px;                                                      4860
    --nav-w: 226px;                                                        4861
}                                                                          4862
* { box-sizing: border-box; }                                              4863
html, body { margin: 0; padding: 0; }                                      4864
body {                                                                     4865
    font-family: "Microsoft YaHei", "PingFang SC", "Noto                   4866
Serif SC", "SimSun", serif;                                                4867
    font-size: 16.5px;                                                     4868
    line-height: 1.6;                                                      4869
    color: var(--ink);                                                     4870
    background: var(--canvas);                                             4871
}                                                                          4872
/* ---------- 跳到主要内容（键盘可达性） ---------- */                      4873
.skip-link {                                                               4874
    position: absolute;                                                    4875
    left: 12px;                                                            4876
    top: -60px;                                                            4877
    z-index: 30;                                                           4878
    padding: 8px 14px;                                                     4879
    background: var(--accent);                                             4880
    color: #fff;                                                           4881
    border-radius: var(--radius);                                          4882
    text-decoration: none;                                                 4883
    transition: top .15s ease-out;                                         4884
}                                                                          4885
.skip-link:focus { top: 12px; }                                            4886
/* ---------- 整体骨架：左侧深绿导航 + 右侧内容 ----------                 4887
   固定为一屏高，页面滚动交给右侧内容区，                                  4888
   这样左侧导航栏（含底部用户区）不会随长页面一起被拉到下面去。 */         4889
.app-shell {                                                               4890
    display: flex;                                                         4891
    align-items: stretch;                                                  4892
    height: 100vh;                                                         4893
    height: 100dvh;                                                        4894
    overflow: hidden;                                                      4895
}                                                                          4896
/* ---------- 侧边栏 ---------- */                                         4897
.app-side {                                                                4898
    flex: 0 0 var(--nav-w);                                                4899
    width: var(--nav-w);                                                   4900
```

```
  display: flex;                                                    4901
  flex-direction: column;                                           4902
  background: linear-gradient(180deg, var(--side-1) 0%,            4903
var(--side-2) 100%);                                                4904
  color: #fff;                                                      4905
}                                                                   4906
.app-brand {                                                        4907
  display: flex;                                                    4908
  align-items: center;                                              4909
  gap: 11px;                                                        4910
  padding: 17px 16px 15px;                                          4911
  border-bottom: 1px solid rgba(255, 255, 255, .10);                4912
}                                                                   4913
.app-brand__mark {                                                  4914
  flex: none;                                                       4915
  display: grid;                                                    4916
  place-items: center;                                              4917
  width: 36px;                                                      4918
  height: 36px;                                                     4919
  border-radius: 10px;                                              4920
  background: rgba(255, 255, 255, .15);                             4921
}                                                                   4922
.app-brand__name {                                                  4923
  font-size: 16px;                                                  4924
  font-weight: 600;                                                 4925
  line-height: 1.35;                                                4926
  letter-spacing: .2px;                                             4927
}                                                                   4928
.app-brand__sub {                                                   4929
  font-size: 11.5px;                                                4930
  line-height: 1.35;                                                4931
  color: rgba(255, 255, 255, .58);                                  4932
}                                                                   4933
.app-nav { flex: 1; padding: 6px 12px 14px; overflow-y:             4934
auto; }                                                             4935
.app-nav__group {                                                   4936
  margin: 12px 0 5px;                                               4937
  padding: 0 10px;                                                  4938
  font-size: 11.5px;                                                4939
  letter-spacing: .05em;                                            4940
  color: rgba(255, 255, 255, .48);                                  4941
}                                                                   4942
.app-nav ul { margin: 0; padding: 0; list-style: none; }            4943
.app-nav a {                                                        4944
  display: flex;                                                    4945
  align-items: center;                                              4946
  gap: 10px;                                                        4947
  padding: 9px 11px;                                                4948
  margin-bottom: 2px;                                               4949
  border-radius: 8px;                                               4950
```

```
  color: rgba(255, 255, 255, .80);
  text-decoration: none;
  transition: background-color .16s ease, color .16s
ease;
}
.app-nav a:hover { background: rgba(255, 255, 255,
.10); color: #fff; }
.app-nav a:focus-visible {
  outline: 2px solid rgba(255, 255, 255, .60);
  outline-offset: 1px;
}
.app-nav a[aria-current="page"] {
  background: rgba(255, 255, 255, .17);
  color: #fff;
  font-weight: 600;
}
.app-nav .ico { flex: none; opacity: .92; }
.app-nav__badge {
  margin-left: auto;
  min-width: 20px;
  padding: 0 6px;
  font-size: 11px;
  font-weight: 600;
  line-height: 18px;
  text-align: center;
  border-radius: 9px;
  background: var(--todo);
  color: #fff;
  font-variant-numeric: tabular-nums;
}
.app-user {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 10px 14px;
  border-top: 1px solid rgba(255, 255, 255, .10);
}
.app-user__me {
  display: flex;
  flex: 1;
  min-width: 0;
  align-items: center;
  gap: 10px;
  padding: 4px 6px;
  border-radius: 8px;
  color: inherit;
  text-decoration: none;
  transition: background-color .16s ease;
}
.app-user__me:hover { background: rgba(255, 255, 255,
```

```
.10); }
.app-user__me:focus-visible {
  outline: 2px solid rgba(255, 255, 255, .60);
  outline-offset: 1px;
}
.app-user__avatar {
  flex: none;
  display: grid;
  place-items: center;
  width: 32px;
  height: 32px;
  border-radius: 50%;
  background: rgba(255, 255, 255, .16);
  font-size: 15.5px;
  font-weight: 600;
  object-fit: cover;
}
.app-user__name {
  display: block;
  font-size: 15.5px;
  line-height: 1.3;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
.app-user__state {
  display: flex;
  align-items: center;
  gap: 5px;
  font-size: 11.5px;
  line-height: 1.3;
  color: rgba(255, 255, 255, .60);
}
.app-user__dot {
  width: 6px;
  height: 6px;
  border-radius: 50%;
  background: #6ed49a;
}
.app-user__out {
  flex: none;
  padding: 6px;
  border-radius: 6px;
  color: rgba(255, 255, 255, .68);
  text-decoration: none;
}
.app-user__out:hover { color: #fff; background:
rgba(255, 255, 255, .12); }
/* ---------- 右侧内容区 ----------
  整个骨架固定一屏高，滚动只发生在内容区内部：
```

```
     左侧导航栏（含底部用户区）始终留在原位，右侧页面自己滚。  */
.app-content {
  flex: 1;
  min-width: 0;
  display: flex;
  flex-direction: column;
  overflow-y: auto;
  overflow-x: hidden;
  /* 滚动条槽位始终占位。滚动条是出现在这个滚动容器内部的，
     内容高的页面（工作台、标准库、操作日志）会因此把顶栏挤窄约 10px，
     内容刚好放得下的页面（限制性判定、项目档案…）则不会 ——
     结果就是「顶栏在不同页面不一样宽」。预留槽位后各页一致。 */
  scrollbar-gutter: stable;
  /* 细滚动条：默认半透明，悬停时加深，不抢视线 */
  scrollbar-width: thin;
  scrollbar-color: rgba(31, 107, 69, .28) transparent;
}
.app-content::-webkit-scrollbar { width: 12px; }
.app-content::-webkit-scrollbar-track { background:
transparent; }
.app-content::-webkit-scrollbar-thumb {
  background: rgba(31, 107, 69, .26);
  border: 3px solid transparent;
  background-clip: content-box;
  border-radius: 999px;
}
.app-content::-webkit-scrollbar-thumb:hover {
  background: rgba(31, 107, 69, .46);
  background-clip: content-box;
}
/* ---------- 顶栏：面包屑 + 搜索 ----------
   粘在内容区顶部，滚动时面包屑与搜索框始终可用 */
.app-header {
  position: sticky;
  top: 0;
  z-index: 20;
  display: flex;
  align-items: center;
  gap: 14px;
  height: var(--header-h);
  padding: 0 22px;
  background: var(--surface);
  border-bottom: 1px solid var(--line);
}
.crumb {
  display: flex;
  align-items: center;
  gap: 7px;
  font-size: 15.5px;
  color: var(--ink-soft);
```

```
  white-space: nowrap;
}
.crumb a { color: var(--ink-soft); text-decoration:
none; }
.crumb a:hover { color: var(--accent); }
.crumb strong { color: var(--ink); font-weight: 600; }
.crumb .ico { color: #c3c9d0; }
.app-header__spacer { flex: 1; }
.search-box {
  display: flex;
  align-items: center;
  gap: 8px;
  width: 268px;
  padding: 7px 12px;
  background: var(--canvas);
  border: 1px solid var(--line);
  border-radius: 9px;
  color: var(--ink-soft);
  transition: border-color .16s ease, background-color
.16s ease;
}
.search-box:focus-within { border-color: var(--accent);
background: #fff; }
.search-box input {
  flex: 1;
  min-width: 0;
  border: none;
  outline: none;
  background: transparent;
  font-family: inherit;
  font-size: 15.5px;
  color: var(--ink);
}
.search-box button {
  display: grid;
  place-items: center;
  padding: 0;
  border: none;
  background: transparent;
  color: inherit;
  cursor: pointer;
}
.icon-btn {
  display: grid;
  place-items: center;
  width: 34px;
  height: 34px;
  border-radius: 9px;
  border: 1px solid transparent;
  color: var(--ink-soft);
```

```
    text-decoration: none;
    transition: background-color .16s ease, color .16s ease,
                border-color .16s ease;
}
.icon-btn:hover {
    color: var(--accent);
    background: var(--accent-soft);
    border-color: rgba(31, 107, 69, .18);
}
.app-main {
    flex: 1;
    min-width: 0;
    padding: 22px 26px 26px;
}
.container { max-width: none; width: 100%; }
/* ---------- 页脚 ---------- */
.app-foot {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 14px;
    padding: 0 26px 18px;
    font-size: 14.5px;
    color: var(--ink-soft);
}
/* ---------- 未登录时的裸页面（登录页） ---------- */
.login-shell {
    display: flex;
    flex-direction: column;
    min-height: 100dvh;
}
/* ---------- 卡片 ---------- */
.card {
    background: var(--surface);
    border: 1px solid var(--line);
    border-radius: var(--radius);
    box-shadow: var(--shadow);
    padding: 22px 24px;
    margin-bottom: 18px;
}
.card > h2 {
    margin: 0 0 14px;
    font-size: 18.5px;
    font-weight: 600;
    color: var(--accent);
    text-wrap: balance;
}
.card > h3 {
    margin: 20px 0 10px;
```

```
    font-size: 16.5px;
    font-weight: 600;
    color: var(--accent);
}
/* ---------- 表格 ---------- */
table {
  width: 100%;
  border-collapse: collapse;
  margin: 10px 0 16px;
  font-variant-numeric: tabular-nums;
}
caption {
  text-align: left;
  color: var(--ink-soft);
  font-size: 14.5px;
  padding-bottom: 6px;
}
th, td {
  border: 1px solid var(--line);
  padding: 8px 12px;
  text-align: center;
  vertical-align: middle;
}
th {
  background: var(--accent-soft);
  font-weight: 600;
  white-space: nowrap;
}
tbody tr:hover { background: #fafbfc; }
/* ---------- 表单控件 ---------- */
label { display: inline-block; }
.field { margin-bottom: 12px; }
.field__label {
  display: block;
  margin-bottom: 4px;
  color: var(--ink-soft);
  font-size: 15.5px;
}
input[type="text"], input[type="number"],
input[type="password"],
select, textarea {
  padding: 6px 9px;
  font-family: inherit;
  font-size: 16.5px;
  color: var(--ink);
  background: #fff;
  border: 1px solid #c6ccd4;
  border-radius: 4px;
  max-width: 100%;
}
```

```
input[type="number"] {
  width: 104px;
  text-align: center;
  font-variant-numeric: tabular-nums;
}
textarea { width: 100%; min-height: 64px; resize:
vertical; }
input:focus-visible, select:focus-visible,
textarea:focus-visible {
  outline: 2px solid var(--accent);
  outline-offset: 1px;
  border-color: var(--accent);
}
.checkbox {
  display: inline-flex;
  align-items: center;
  gap: 7px;
  cursor: pointer;
  user-select: none;
}
.checkbox input { width: 16px; height: 16px; margin: 0;
cursor: pointer; }
/* ---------- 按钮 ----------
   统一为圆角矩形：默认半透明底色，鼠标悬停时填充为实色。 */
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 6px;
  padding: 8px 18px;
  font-family: inherit;
  font-size: 16.5px;
  line-height: 1.4;
  color: #fff;
  background: rgba(31, 107, 69, .80);
  border: 1px solid rgba(31, 107, 69, .30);
  border-radius: 8px;
  cursor: pointer;
  text-decoration: none;
  transition: background-color .16s ease, border-color
.16s ease,
              color .16s ease, box-shadow .16s ease,
transform .16s ease;
}
.btn:hover {
  background: var(--accent);
  border-color: var(--accent);
  box-shadow: 0 3px 12px rgba(31, 107, 69, .22);
}
.btn:active { transform: translateY(1px); box-shadow:
```

```
none; }
.btn:focus-visible { outline: 2px solid var(--accent);
outline-offset: 2px; }
.btn--ghost {
  color: var(--accent);
  background: rgba(31, 107, 69, .06);
  border-color: rgba(31, 107, 69, .22);
}
.btn--ghost:hover {
  color: #fff;
  background: var(--accent);
  border-color: var(--accent);
}
.btn--danger {
  color: var(--danger);
  background: rgba(179, 38, 30, .07);
  border-color: rgba(179, 38, 30, .26);
}
.btn--danger:hover {
  color: #fff;
  background: var(--danger);
  border-color: var(--danger);
  box-shadow: 0 3px 12px rgba(179, 38, 30, .20);
}
.btn--sm { padding: 5px 13px; font-size: 15.5px;
border-radius: 7px; }
.btn:disabled { opacity: .45; cursor: not-allowed; }
.btn:disabled:hover {
  background: rgba(31, 107, 69, .80);
  border-color: rgba(31, 107, 69, .30);
  box-shadow: none;
}
.btn--ghost:disabled:hover { background: rgba(31, 107,
69, .06); color: var(--accent); }
/* ---------- 表格操作列的行内小按钮 ----------
   取代原来「文字 | 文字 | 文字」的写法：
   圆角矩形 + 半透明底色，悬停时变实色。 */
.row-actions {
  display: inline-flex;
  flex-wrap: nowrap;          /* 按钮不折行；宁可让旁边长名称换行 */
  align-items: center;
  gap: 6px;
  white-space: nowrap;
}
.row-actions form { display: inline-flex; margin: 0; }
.link-btn {
  display: inline-flex;
  align-items: center;
  padding: 4px 11px;
  font-family: inherit;
```

```
    font-size: 15.5px;
    line-height: 1.5;
    color: var(--accent);
    background: rgba(31, 107, 69, .07);
    border: 1px solid rgba(31, 107, 69, .20);
    border-radius: 7px;
    cursor: pointer;
    text-decoration: none;
    white-space: nowrap;
    transition: background-color .16s ease, border-color .16s ease,
                color .16s ease, box-shadow .16s ease, transform .16s ease;
}
.link-btn:hover {
    color: #fff;
    background: var(--accent);
    border-color: var(--accent);
    box-shadow: 0 2px 8px rgba(31, 107, 69, .20);
}
.link-btn:active { transform: translateY(1px); box-shadow: none; }
.link-btn:focus-visible { outline: 2px solid var(--accent); outline-offset: 2px; }
.link-btn--danger {
    color: var(--danger);
    background: rgba(179, 38, 30, .07);
    border-color: rgba(179, 38, 30, .24);
}
.link-btn--danger:hover {
    color: #fff;
    background: var(--danger);
    border-color: var(--danger);
    box-shadow: 0 2px 8px rgba(179, 38, 30, .22);
}
.actions { display: flex; flex-wrap: wrap; align-items: center; gap: 8px; }
.actions form { display: inline; }
/* ---------- 提示条 ---------- */
.flash {
    padding: 10px 14px;
    margin-bottom: 14px;
    border-radius: var(--radius);
    border-left: 4px solid;
}
.flash.error {
    color: var(--danger);
    background: var(--danger-soft);
    border-left-color: var(--danger);
}
```

```
.flash.ok {
  color: var(--ok);
  background: var(--ok-soft);
  border-left-color: var(--ok);
}
/* ---------- 浮层提示（登录成功、各类操作完成、登录失败） ---------- */
   外层 toast-stack 固定在页面中上方，内层每条各自淡入 → 停留 → 淡出，
   多条提示会依次排开而不重叠。
   底色是浅绿半透明：既有一层底色把文字托住，又能透出后面的内容，
   不用高饱和色块，提示不该比页面内容更抢眼。
   pointer-events: none 让它不拦截鼠标，出现期间也能正常操作。 */
.toast-stack {
  position: fixed;
  top: 16px;
  left: 50%;
  z-index: 100;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 6px;
  transform: translateX(-50%);
  pointer-events: none;
}
.toast-center {
  max-width: min(620px, 82vw);
  padding: 8px 22px;
  border-radius: 999px;
  /* 浅绿、半透明：不遮住背后的内容，也比纯透明更托得住文字 */
  background: rgba(223, 241, 231, .76);
  border: 1px solid rgba(31, 107, 69, .14);
  color: #2f4a3c;
  font-size: 18.5px;
  font-weight: 500;
  line-height: 1.65;
  letter-spacing: .03em;
  text-align: center;
  text-wrap: pretty;
  backdrop-filter: blur(6px) saturate(1.1);
  -webkit-backdrop-filter: blur(6px) saturate(1.1);
  box-shadow: 0 4px 16px rgba(23, 32, 27, .07);
  animation: toast-fade 2.6s ease-in-out forwards;
}
/* 失败类提示：同一套观感，换成浅红底，一眼能区分成败 */
.toast-center--error {
  background: rgba(252, 233, 230, .78);
  border-color: rgba(163, 43, 34, .16);
  color: #9c2b23;
}
@keyframes toast-fade {
  0%         { opacity: 0; transform: translateY(-6px); }
```

```
    14%, 74% { opacity: 1; transform: none; }
    100%     { opacity: 0; transform: none; }
}
/* 系统设置里要求「减少动态效果」时，去掉位移，只做淡入淡出 */
@media (prefers-reduced-motion: reduce) {
    .toast-center { animation: toast-fade-plain 2.6s
linear forwards; }
    @keyframes toast-fade-plain {
      0%         { opacity: 0; }
      14%, 74%   { opacity: 1; }
      100%       { opacity: 0; }
    }
}
/* ---------- 文字工具类 ---------- */
.muted { color: var(--ink-soft); font-size: 14px; }
.mono { font-family: Consolas, "Courier New",
monospace; font-variant-numeric: tabular-nums; }
.num { font-variant-numeric: tabular-nums; }
.warn { color: var(--danger); font-weight: 600; }
.ok { color: var(--ok); font-weight: 600; }
.nowrap { white-space: nowrap; }
/* 表格操作列里常把删除按钮放在 <form> 中。form 默认是块级元素，
   会把「删除」挤到下一行，导致操作列参差不齐，这里让它留在行内。 */
.nowrap form { display: inline; margin: 0; }
.stack { display: flex; flex-direction: column; gap:
14px; }
/* 状态标签：不依赖颜色单独表意 */
.tag {
    display: inline-block;
    padding: 1px 9px;
    font-size: 14px;
    border-radius: 10px;
    border: 1px solid;
    white-space: nowrap;
}
.tag--ok { color: var(--ok); background:
var(--ok-soft); border-color: #b8ddc6; }
.tag--warn { color: var(--danger); background:
var(--danger-soft); border-color: #eec4c0; }
.tag--muted { color: var(--ink-soft); background:
#f1f3f6; border-color: #dde1e6; }
/* ---------- 登录页 ---------- */
/* 仿 Apple 账户页：接近纯白的底，仅在顶部有一层极淡的绿色晕染，
   让彩色徽标成为整页唯一的视觉重心。排版见 auth/login.html。 */
.body--login {
    background:
      radial-gradient(1100px 640px at 50% 8%,
                      #edf6f1 0%, rgba(237, 246, 241, 0)
72%),
      linear-gradient(180deg, #ffffff 0%, #fbfdfc 100%);
```

```
}
.body--login .app-main { padding: 0; }
.body--login .container { max-width: none; width: 100%;
}
/* 仅供读屏软件的隐藏文本 */
.visually-hidden {
  position: absolute !important;
  width: 1px;
  height: 1px;
  margin: -1px;
  padding: 0;
  overflow: hidden;
  clip: rect(0 0 0 0);
  white-space: nowrap;
  border: 0;
}
/* ---------- 工作台：页首横幅 ---------- */
.hero {
  position: relative;
  overflow: hidden;
  min-height: 176px;
  padding: 26px 28px;
  margin-bottom: 16px;
  border-radius: 12px;
  /* 左侧压深保证文字可读，右侧透出远山插画；
     背景按底部对齐，避免远山被居中裁掉 */
  background:
    linear-gradient(100deg, rgba(15, 40, 45, .90) 0%,
                            rgba(20, 55, 58, .55) 45%,
                            rgba(20, 55, 58, .12) 100%),
    url("{{ url_for('static', filename='hero_landscape.jpg') }}")
    center center / cover no-repeat;
  color: #fff;
}
.hero__body { max-width: 720px; }
.hero__title {
  margin: 0 0 10px;
  font-size: 23.5px;
  font-weight: 700;
  line-height: 1.4;
  letter-spacing: .3px;
}
.hero__desc {
  margin: 0;
  max-width: 640px;
  font-size: 14px;
  line-height: 1.85;
  color: rgba(255, 255, 255, .86);
}
```

```
.hero__actions {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  margin-top: 18px;
}
.hero-btn {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  padding: 7px 15px;
  font-size: 15.5px;
  border-radius: 8px;
  text-decoration: none;
  transition: background-color .16s ease, border-color .16s ease,
              color .16s ease;
}
.hero-btn--solid {
  color: #14432f;
  background: rgba(255, 255, 255, .92);
  border: 1px solid rgba(255, 255, 255, .92);
}
.hero-btn--solid:hover { background: #fff; }
.hero-btn--ghost {
  color: #fff;
  background: rgba(255, 255, 255, .14);
  border: 1px solid rgba(255, 255, 255, .45);
}
.hero-btn--ghost:hover { background: rgba(255, 255, 255, .26); }
.hero-btn:focus-visible { outline: 2px solid #fff; outline-offset: 2px; }
/* ---------- 工作台：数据概览 ---------- */
.stat-cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(190px, 1fr));
  gap: 12px;
  margin-bottom: 22px;
}
.stat-card {
  display: block;
  padding: 15px 17px;
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  color: var(--ink);
  text-decoration: none;
```

```
    transition: border-color .16s ease, box-shadow .16s ease;
}
.stat-card:hover {
    border-color: rgba(31, 107, 69, .45);
    box-shadow: 0 3px 12px rgba(19, 60, 43, .08);
}
.stat-card__label {
    display: block;
    font-size: 14px;
    color: var(--ink-soft);
}
.stat-card__value {
    display: block;
    margin: 3px 0 2px;
    font-size: 28.5px;
    font-weight: 700;
    line-height: 1.25;
    color: var(--accent);
    font-variant-numeric: tabular-nums;
}
.stat-card__unit {
    margin-left: 3px;
    font-size: 15.5px;
    font-weight: 500;
    color: var(--ink-soft);
}
.stat-card__note {
    display: block;
    font-size: 14.5px;
    color: var(--ink-soft);
}
/* ---------- 工作台：分区标题 ---------- */
.section { margin-bottom: 22px; }
.section__head {
    display: flex;
    align-items: baseline;
    flex-wrap: wrap;
    gap: 6px 14px;
    margin-bottom: 12px;
}
.section__title { margin: 0; font-size: 18.5px; }
.section__desc { margin: 0; font-size: 14px; color: var(--ink-soft); }
/* ---------- 工作台：流程步骤卡 ---------- */
.step-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(238px, 1fr));
    gap: 12px;
```

```
}
.step-card {
  position: relative;
  display: block;
  padding: 16px 16px 15px;
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  color: var(--ink);
  text-decoration: none;
  transition: border-color .16s ease, box-shadow .16s
ease,
              transform .16s ease;
}
.step-card:hover {
  border-color: rgba(31, 107, 69, .55);
  box-shadow: 0 5px 16px rgba(19, 60, 43, .10);
  transform: translateY(-1px);
}
.step-card:focus-visible { outline: 2px solid
var(--accent); outline-offset: 2px; }
.step-card__no {
  position: absolute;
  top: 13px;
  right: 15px;
  font-size: 19.5px;
  font-weight: 700;
  color: #d7e3db;
  font-variant-numeric: tabular-nums;
}
.step-card__icon {
  display: grid;
  place-items: center;
  width: 32px;
  height: 32px;
  margin-bottom: 10px;
  border-radius: 9px;
  background: var(--accent-soft);
  color: var(--accent);
}
.step-card__title {
  display: block;
  margin-bottom: 5px;
  font-size: 16.5px;
  font-weight: 600;
  color: var(--accent);
}
.step-card__desc {
  display: block;
```

```
    font-size: 17px;
    line-height: 1.65;
    color: var(--ink-soft);
    text-wrap: pretty;
}
.step-card__badge {
    display: inline-block;
    margin-top: 10px;
    padding: 1px 9px;
    font-size: 11.5px;
    border-radius: 9px;
    background: #fdf1e2;
    color: #9a5a12;
    border: 1px solid #f0d3ab;
}
/* 前置数据不具备时置灰，并说明缺什么 */
.step-card--locked {
    background: #fafbfa;
    border-color: #eceff0;
    box-shadow: none;
    cursor: default;
}
.step-card--locked .step-card__no { color: #e6e9e8; }
.step-card--locked .step-card__icon {
    background: #f1f3f2;
    color: #a9b2ae;
}
.step-card--locked .step-card__title { color: #9aa3a0; }
.step-card--locked .step-card__desc { color: #b3bab7; }
.step-card__lock {
    display: inline-block;
    margin-top: 10px;
    padding: 1px 9px;
    font-size: 11.5px;
    border-radius: 9px;
    background: #f1f3f2;
    color: #8b948f;
}
/* ---------- 结果摘要 ---------- */
.stat-row {
    display: flex;
    flex-wrap: wrap;
    gap: 28px;
    margin: 4px 0 6px;
}
.stat__label { font-size: 14px; color: var(--ink-soft);
}
.stat__value { font-size: 22.5px; font-weight: 700;
font-variant-numeric: tabular-nums; }
/* ---------- 流程步骤条 ---------- */
```

```
.steps { display: flex; flex-wrap: wrap; gap: 8px;
margin: 4px 0 18px; }
.step {
  display: flex; align-items: center; gap: 7px;
  padding: 8px 12px; font-size: 15.5px;
  background: #fff; border: 1px solid var(--line);
  border-radius: var(--radius);
}
.step__no {
  display: inline-flex; align-items: center;
justify-content: center;
  width: 20px; height: 20px; border-radius: 50%;
  background: #eef1f5; color: var(--ink-soft);
  font-size: 14.5px; font-variant-numeric: tabular-nums;
}
.step__name { font-weight: 600; }
.step__state { font-size: 14px; color: var(--ink-soft);
}
.step--done { border-color: #b8ddc6; background:
var(--ok-soft); }
.step--done .step__no { background: var(--ok); color:
#fff; }
.step--done .step__state { color: var(--ok); }
.step--fail { border-color: #eec4c0; background:
var(--danger-soft); }
.step--fail .step__no { background: var(--danger);
color: #fff; }
.step--fail .step__state { color: var(--danger);
font-weight: 600; }
.step--next { border-color: var(--accent); }
.step--next .step__no { background: var(--accent);
color: #fff; }
/* ---------- 提示块（中性底色，不用整块警示色压页面） ---------- */
.notice {
  padding: 14px 16px; margin: 0 0 16px;
  background: var(--surface);
  border: 1px solid var(--line);
  border-left: 3px solid var(--todo);
  border-radius: var(--radius);
}
.notice__title { margin: 0 0 8px; font-weight: 600; }
/* ---------- 前置条件表（与页面主表同款，只把待处理的行标成浅红） ---------- */
.prereq-table { margin: 0 0 18px; }
.prereq-table > caption { padding-bottom: 7px; }
.prereq-row > td { background: var(--danger-soft); }
.prereq-row:hover > td { background: #f6dcd7; }
.prereq-flag { color: var(--danger); font-weight: 600; }
.prereq-row .muted { color: #9a6b62; }
/* ---------- 图斑面板（按图斑分块展示，如指标数据采集） ---------- */
.patch-panel {
```

```
    border: 1px solid var(--line);
    border-radius: var(--radius);
    background: var(--surface);
    padding: 14px 16px;
    margin: 0 0 14px;
  }
  .patch-panel:last-child { margin-bottom: 0; }
  .patch-panel__head {
    display: flex;
    align-items: baseline;
    flex-wrap: wrap;
    gap: 6px 14px;
    margin: 0 0 12px;
    padding-bottom: 9px;
    border-bottom: 1px solid var(--line);
  }
  .patch-panel__name {
    margin: 0;
    font-size: 17.5px;
    font-weight: 600;
  }
  .patch-panel__meta {
    font-size: 14px;
    color: var(--ink-soft);
  }
  .patch-panel__meta a { color: var(--accent); }
  .patch-panel__tail { margin-left: auto; }
  .patch-panel .notice { margin: 0; }
  .patch-panel table { margin: 0; }
  /* ---------- 窄屏适配 ---------- */
  @media (max-width: 960px) {
    /* 上下布局时取消「一屏高 + 内容区内部滚动」，
       否则导航会占掉半屏、内容被挤在剩下的小格子里滚动 */
    .app-shell {
      flex-direction: column;
      height: auto;
      min-height: 100vh;
      min-height: 100dvh;
      overflow: visible;
    }
    .app-content { overflow: visible; }
    .app-header { position: static; }
    .app-side { width: auto; flex: none; }
    .app-brand { padding: 12px 14px; }
    .app-nav { padding: 4px 10px 10px; }
    .app-nav ul { display: flex; flex-wrap: wrap; gap:
  4px; }
    .app-nav a { margin-bottom: 0; }
    .app-nav__group { margin: 8px 0 4px; }
    .app-user { padding: 10px 14px; }
```

```
        .app-header { padding: 0 14px; gap: 10px; }
        .search-box { width: 180px; }
        .app-main { padding: 16px 14px 20px; }
        .app-foot { padding: 0 14px 14px; }
      }
      @media (max-width: 640px) {
        .crumb { display: none; }
        .search-box { width: auto; flex: 1; }
      }
    </style>
</head>
<body class="{% if not current_user.is_authenticated %}body--login{% endif %}">
  <a class="skip-link" href="#main">跳到主要内容</a>
  {# 取一次即可（同一请求内重复调用返回相同内容）。
     类别为 toast / ok / toast-error 的渲染成中上方浮层，出现后自动淡出；
     其余仍是页面顶部的提示条 ——
     需要对照着改表单的错误，留在页面上比一闪而过更有用。 #}
  {% set flash_msgs = get_flashed_messages(with_categories=true) %}
  <div class="toast-stack">
    {% for category, message in flash_msgs if category in ("toast", "ok", "toast-error") %}
    <div class="toast-center{% if category == 'toast-error' %} toast-center--error{% endif %}"
         role="status" aria-live="polite">{{ message }}</div>
    {% endfor %}
  </div>
  {% if current_user.is_authenticated %}
  <div class="app-shell">
    <aside class="app-side">
      <div class="app-brand">
        <span class="app-brand__mark">{{ ui.icon("mountain", 21) }}</span>
        <span>
          <span class="app-brand__name">矿山自然恢复评价</span><br>
          <span class="app-brand__sub">适宜性评价系统 V1.0</span>
        </span>
      </div>
      <nav class="app-nav" aria-label="功能导航">
        <ul>
          <li>
            <a href="/"{% if home_active %} aria-current="page"{% endif %}>
              {{ ui.icon("home") }}<span>工作台</span>
            </a>
          </li>
        </ul>
        {% for g in nav_groups %}
```

```
      <p class="app-nav__group">{{ g.title }}</p>
      <ul>
        {% for item in g.links %}
        <li>
          <a href="{{ item.href }}"{% if item.active %}
aria-current="page"{% endif %}>
            {{ ui.icon(item.icon) }}<span>{{ item.label
}}</span>
            {% if item.badge %}<span
class="app-nav__badge">{{ item.badge }}</span>{% endif
%}
          </a>
        </li>
        {% endfor %}
      </ul>
      {% endfor %}
    </nav>
    <div class="app-user">
      <a class="app-user__me" href="/profile"
title="个人设置">
        {% if current_user.avatar %}
        <img class="app-user__avatar"
             src="{{ url_for('static',
filename=current_user.avatar) }}" alt="">
        {% else %}
        <span class="app-user__avatar"
aria-hidden="true">{{ current_user.initial }}</span>
        {% endif %}
        <span style="min-width:0;">
          <span class="app-user__name">{{
current_user.shown_name }}</span>
          <span class="app-user__state">
            <span class="app-user__dot"
aria-hidden="true"></span>
            {{ role_labels.get(current_user.role,
current_user.role) }}
          </span>
        </span>
      </a>
      <a class="app-user__out" href="/logout" title="退出登录"
         aria-label="退出登录">{{ ui.icon("logout", 17) }}</a>
    </div>
  </aside>
  <div class="app-content">
    <header class="app-header">
      <nav class="crumb" aria-label="当前位置">
        <a href="/">首页</a>
        {{ ui.icon("chevron", 13) }}
        <strong>{{ page_title }}</strong>
      </nav>
```

```
          <span class="app-header__spacer"></span>
          <form class="search-box" method="get"
    action="/search" role="search">
            {{ ui.icon("search", 16) }}
            <label for="q" class="visually-hidden">搜索项目、图斑、标准</
    label>
            <input id="q" type="search" name="q"
    autocomplete="off"
                   placeholder="搜索项目、图斑、标准" value="{{
    search_q }}">
            <button type="submit" aria-label="搜索">{{
    ui.icon("chevron", 15) }}</button>
          </form>
          {% if current_user.is_admin %}
          <a class="icon-btn" href="/profile?tab=logs"
    title="操作日志"
             aria-label="操作日志">{{ ui.icon("bell", 18) }}</a>
          {% endif %}
        </header>
    {% else %}
    <div class="login-shell">
    {% endif %}
      <main id="main" class="app-main">
        <div class="container">
          {% for category, message in flash_msgs if category
    not in ("toast", "ok", "toast-error") %}
          <p class="flash {{ category }}" role="status">{{
    message }}</p>
          {% endfor %}
          {% block content %}{% endblock %}
        </div>
      </main>
    {% if current_user.is_authenticated %}
        <footer class="app-foot">
          <span>历史遗留废弃矿山自然恢复适宜性评价系统 Ｖ１．０</span>
          <span>数据截止 {{ today }}</span>
        </footer>
      </div>
    </div>
    {% else %}
    </div>
    {% endif %}
    {% block scripts %}{% endblock %}
</body>
</html>
# ==== app/templates/_macros.html ====
{# 共用宏：图标、流程状态面板 #}
{# ============================================================
   ================
    图标：内联 SVG 线性图标（２４×２４ 视图框，随文字颜色走）
```

```
    不引图标字体或外部图片，避免额外请求与资源依赖；
    stroke 用 currentColor，在深色侧边栏与浅色内容区都能自适应。
    ================================================================
      ================ #}
{% set ICONS = {
  "dot": '<circle cx="12" cy="12" r="3"/>',
  "home": '<path d="M3 9.6 12 3l9 6.6V20a2 2 0 0 1-2 2H5a2
    2 0 0 1-2-2z"/><path d="M9.5 22v-7h5v7"/>',
  "folder": '<path d="M3 7.5A2 2 0 0 1 5 5.5h3.6l2 2.2H19a2
    2 0 0 1 2 2v7.8a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2z"/>',
  "book": '<path d="M4 4.8A2.8 2.8 0 0 1 6.8
    2H20v17.5H6.8A2.8 2.8 0 0 0 4 22z"/><path d="M4
    19.5A2.8 2.8 0 0 1 6.8 16.7H20"/>',
  "shield": '<path d="M12 22s8-4 8-10V5.2L12 2 4 5.2V12c0 6
    8 10 8 10z"/><path d="m9 12 2.2 2.2L15.5 10"/>',
  "layers": '<path d="m12 2.8 9 4.7-9 4.7-9-4.7z"/><path
    d="m3 12.5 9 4.7 9-4.7"/><path d="m3 17 9 4.7 9-4.7"/>',
  "sliders": '<path d="M4 21v-6M4 11V3M12 21v-9M12 8V3M20
    21v-4M20 13V3"/><path d="M1.5 15h5M9.5 12h5M17.5
    17h5"/>',
  "database": '<ellipse cx="12" cy="5.5" rx="8"
    ry="3.2"/><path d="M4 5.5v13c0 1.8 3.6 3.2 8 3.2s8-1.4
    8-3.2v-13"/><path d="M4 12c0 1.8 3.6 3.2 8 3.2s8-1.4
    8-3.2"/>',
  "target": '<circle cx="12" cy="12" r="9"/><circle cx="12"
    cy="12" r="4.6"/><circle cx="12" cy="12" r="1"/>',
  "file": '<path d="M14 2.8H7a2 2 0 0 0-2 2v14.4a2 2 0 0 0
    2 2h10a2 2 0 0 0 2-2V7.8z"/><path d="M14
    2.8V7.8h5"/><path d="M8.8 13h6.4M8.8 17h6.4"/>',
  "users": '<circle cx="9.5" cy="8" r="3.6"/><path d="M3.5
    20.5v-1.4A3.6 3.6 0 0 1 7.1 15.5h4.8a3.6 3.6 0 0 1 3.6
    3.6v1.4"/><path d="M17 4.9a3.6 3.6 0 0 1 0 6.9"/><path
    d="M19 15.8a3.6 3.6 0 0 1 2.4 3.4v1.3"/>',
  "user": '<circle cx="12" cy="8" r="4"/><path d="M4.5
    21v-1.6A4.4 4.4 0 0 1 8.9 15h6.2a4.4 4.4 0 0 1 4.4
    4.4V21"/>',
  "mail": '<rect x="3" y="5" width="18" height="14"
    rx="2.5"/><path d="m3.8 7 8.2 5.8L20.2 7"/>',
  "phone": '<path d="M6.4 3h2.8l1.5 4.4-2 1.4a12.4 12.4 0 0
    0 6.5 6.5l1.4-2L21 14.8v2.8a2.4 2.4 0 0 1-2.6 2.4A16.6
    16.6 0 0 1 3.5 5.6 2.4 2.4 0 0 1 5.9 3z"/>',
  "lock": '<rect x="4" y="10.4" width="16" height="10.6"
    rx="2.6"/><path d="M8 10.4V7.6a4 4 0 0 1 8 0v2.8"/>',
  "briefcase": '<rect x="3" y="7.4" width="18"
    height="13.1" rx="2.6"/><path d="M8.4 7.4V6A2 2 0 0 1
    10.4 4h3.2a2 2 0 0 1 2 2v1.4"/><path d="M3 12.6h18"/>',
  "search": '<circle cx="11" cy="11" r="7"/><path d="m20.5
    20.5-4-4"/>',
  "bell": '<path d="M18 9.5a6 6 0 1 0-12 0c0 5-2.2 6.2-2.2
    6.2h16.4S18 14.5 18 9.5"/><path d="M13.8 19.5a2 2 0 0
```

```
      1-3.6 0"/>',
   "plus": '<path d="M12 5v14M5 12h14"/>',
   "upload": '<path d="M12 16V4"/><path d="m7.5 8.5 4.5-4.5
      4.5 4.5"/><path d="M4 16.5v2A2.5 2.5 0 0 0 6.5
      21h11a2.5 2.5 0 0 0 2.5-2.5v-2"/>',
   "logout": '<path d="M15 4h3.5A2.5 2.5 0 0 1 21 6.5v11A2.5
      2.5 0 0 1 18.5 20H15"/><path d="M10 16.5 5.5 12 10
      7.5"/><path d="M5.5 12H15"/>',
   "chevron": '<path d="m9.5 5.5 6.5 6.5-6.5 6.5"/>',
   "warn": '<path d="M12 3.5 21.5 20H2.5z"/><path d="M12
      10v4.2"/><path d="M12 17.4h.01"/>',
   "mountain": '<path d="m3 20 6.5-11L14 17l2.5-4L21
      20z"/><circle cx="17" cy="6" r="2.2"/>',
} %}
{% macro icon(name, size=18) %}
<svg class="ico" width="{{ size }}" height="{{ size }}"
     viewBox="0 0 24 24"
      fill="none" stroke="currentColor" stroke-width="1.7"
      stroke-linecap="round" stroke-linejoin="round"
      aria-hidden="true" focusable="false">{{
     ICONS.get(name, ICONS.dot)|safe }}</svg>
{% endmacro %}
{# 评价流程步骤条：按论文 4.1 节顺序显示各前置步骤的完成情况
   顺序为：限制性条件判定 → 划分微地质单元 → 录入指标数据 → 执行评价。
   论文原文：「先进行政策、位置约束地质灾害判断……对于满足限制性条件的
   废弃矿山，按照地形地貌划分……三种地质微单元类型」，故判定在前。 #}
{% macro workflow_steps(progress) %}
<div class="steps">
  <div class="step {% if progress.constraint.passed
    %}step--done{% elif progress.constraint.done
    %}step--fail{% else %}step--todo{% endif %}">
    <span class="step__no">1</span>
    <span class="step__name">限制性条件判定</span>
    <span class="step__state">
      {% if progress.constraint.passed %}已通过
      {% elif progress.constraint.done %}未通过
      {% else %}待完成{% endif %}
    </span>
  </div>
  <div class="step {% if progress.units.done %}step--done{%
    else %}step--todo{% endif %}">
    <span class="step__no">2</span>
    <span class="step__name">划分微地质单元</span>
    <span class="step__state">
      {% if progress.units.done %}已完成{% else %}待完成{% endif
    %}
    </span>
  </div>
  <div class="step {% if progress.data.ready %}step--done{%
    else %}step--todo{% endif %}">
```

```
      <span class="step__no">3</span>                                          6101
      <span class="step__name">录入指标数据</span>                              6102
      <span class="step__state">                                               6103
        {% if progress.data.ready %}已完成{% else %}待完成{% endif             6104
      %}                                                                       6105
      </span>                                                                  6106
    </div>                                                                     6107
    <div class="step step--next">                                              6108
      <span class="step__no">4</span>                                          6109
      <span class="step__name">执行适宜性评价</span>                            6110
      <span class="step__state">当前步骤</span>                                6111
    </div>                                                                     6112
</div>                                                                         6113
{% endmacro %}                                                                 6114
{# 前置条件未满足时的提示：做成与页面主表同款的表格，只把待处理的那几行标成浅红，          6115
   说明写在 caption 里，缺什么、去哪补都在行内。 #}                             6116
{% macro workflow_blocking(progress) %}                                        6117
{% if progress.blocking %}                                                     6118
<table class="prereq-table">                                                   6119
  <caption>本步骤的前置条件尚未满足，按论文 4.1 节的流程顺序先补上：</caption>   6120
  <thead>                                                                      6121
    <tr>                                                                       6122
      <th scope="col">前置步骤</th>                                            6123
      <th scope="col">说明</th>                                                6124
      <th scope="col">操作</th>                                                6125
    </tr>                                                                      6126
  </thead>                                                                     6127
  <tbody>                                                                      6128
    {% for b in progress.blocking %}                                           6129
    <tr class="prereq-row">                                                    6130
      <td>{{ b.label }}</td>                                                   6131
      <td class="muted">{{ b.reason }}</td>                                    6132
      <td><a class="btn btn--sm" href="{{ b.fix_url }}">{{                     6133
    b.fix_label }}</a></td>                                                    6134
    </tr>                                                                      6135
    {% endfor %}                                                               6136
  </tbody>                                                                     6137
</table>                                                                       6138
{% endif %}                                                                    6139
{% endmacro %}                                                                 6140
{# 一行式状态摘要，用于列表页 #}                                               6141
{% macro workflow_badge(progress) %}                                           6142
{% if progress.ready %}                                                        6143
<span class="tag tag--ok">可执行评价</span>                                    6144
{% elif progress.blocking %}                                                   6145
<span class="tag tag--warn">{{ progress.blocking[0].label                      6146
  }}</span>                                                                    6147
<span class="muted">{{ progress.blocking[0].reason }}</span>                   6148
{% endif %}                                                                    6149
{% endmacro %}                                                                 6150
```

```
{# 层级引导：本系统是「项目 → 图斑」两级结构，评价针对的是图斑而非项目。
   只在流程里**第一个**按图斑操作的页面（限制性指标判定）列一次，后面各页
   不再重复；建好图斑才会出现在下面的表里，但离「能进入本页流程」还差别的步骤，
   所以措辞只说到「出现在这里」为止。 #}
{% macro patch_guide(projects_without_patch) %}
{% if projects_without_patch %}
<table class="prereq-table">
  <caption>以下项目下尚未建立矿山图斑，本页按图斑组织数据，建好图斑后才会出现在这里：</caption>
  <thead>
    <tr>
      <th scope="col">所属项目</th>
      <th scope="col">矿山图斑</th>
      <th scope="col">状态</th>
      <th scope="col">操作</th>
    </tr>
  </thead>
  <tbody>
    {% for p in projects_without_patch %}
    <tr class="prereq-row">
      <td>项目「{{ p.name }}」</td>
      <td class="muted">未建立</td>
      <td><span class="prereq-flag">缺少矿山图斑</span></td>
      <td>
        <a class="btn btn--sm" href="/project/{{ p.id }}/patch/add">
          {{ icon("plus", 14) }}去新建图斑
        </a>
      </td>
    </tr>
    {% endfor %}
  </tbody>
</table>
{% endif %}
{% endmacro %}
# ==== app/templates/_login_badge.svg ====
<svg class="lp-badge__svg" xmlns="http://www.w3.org/2000/svg
    " viewBox="0 0 240 240" role="img"
    aria-label="历史遗留废弃矿山自然恢复适宜性评价系统徽标">
  <title>系统徽标</title>
  <g class="lp-ring" style="--k: 2.7">
    <circle cx="162.00" cy="120.00" r="2.60"
    fill="#c4f5ab"/>
    <circle cx="157.84" cy="138.22" r="2.97"
    fill="#b8f5ab"/>
    <circle cx="146.19" cy="152.84" r="3.06"
    fill="#acf5ab"/>
    <circle cx="129.35" cy="160.95" r="2.80"
    fill="#abf5b7"/>
    <circle cx="110.65" cy="160.95" r="2.40"
    fill="#abf5c2"/>
```

```
    <circle cx="93.81" cy="152.84" r="2.14" fill="#abf5ce"/>
    <circle cx="82.16" cy="138.22" r="2.23" fill="#abf5da"/>
    <circle cx="78.00" cy="120.00" r="2.60" fill="#abf5e6"/>
    <circle cx="82.16" cy="101.78" r="2.97" fill="#abf5da"/>
    <circle cx="93.81" cy="87.16" r="3.06" fill="#abf5ce"/>
    <circle cx="110.65" cy="79.05" r="2.80" fill="#abf5c2"/>
    <circle cx="129.35" cy="79.05" r="2.40" fill="#abf5b7"/>
    <circle cx="146.19" cy="87.16" r="2.14" fill="#acf5ab"/>
    <circle cx="157.84" cy="101.78" r="2.23"
    fill="#b8f5ab"/>
  </g>
  <g class="lp-ring" style="--k: 2.15">
    <circle cx="177.29" cy="129.07" r="3.92"
    fill="#a4e28d"/>
    <circle cx="171.68" cy="146.33" r="4.01"
    fill="#9be28d"/>
    <circle cx="161.01" cy="161.01" r="3.87"
    fill="#91e28d"/>
    <circle cx="146.33" cy="171.68" r="3.55"
    fill="#8de292"/>
    <circle cx="129.07" cy="177.29" r="3.17"
    fill="#8de29c"/>
    <circle cx="110.93" cy="177.29" r="2.88"
    fill="#8de2a6"/>
    <circle cx="93.67" cy="171.68" r="2.79" fill="#8de2af"/>
    <circle cx="78.99" cy="161.01" r="2.93" fill="#8de2b9"/>
    <circle cx="68.32" cy="146.33" r="3.25" fill="#8de2c2"/>
    <circle cx="62.71" cy="129.07" r="3.63" fill="#8de2cc"/>
    <circle cx="62.71" cy="110.93" r="3.92" fill="#8de2cc"/>
    <circle cx="68.32" cy="93.67" r="4.01" fill="#8de2c2"/>
    <circle cx="78.99" cy="78.99" r="3.87" fill="#8de2b9"/>
    <circle cx="93.67" cy="68.32" r="3.55" fill="#8de2af"/>
    <circle cx="110.93" cy="62.71" r="3.17" fill="#8de2a6"/>
    <circle cx="129.07" cy="62.71" r="2.88" fill="#8de29c"/>
    <circle cx="146.33" cy="68.32" r="2.79" fill="#8de292"/>
    <circle cx="161.01" cy="78.99" r="2.93" fill="#91e28d"/>
    <circle cx="171.68" cy="93.67" r="3.25" fill="#9be28d"/>
    <circle cx="177.29" cy="110.93" r="3.63"
    fill="#a4e28d"/>
  </g>
  <g class="lp-ring" style="--k: 1.85">
    <circle cx="194.00" cy="120.00" r="4.94"
    fill="#91cf72"/>
    <circle cx="191.85" cy="137.71" r="4.92"
    fill="#88cf72"/>
    <circle cx="185.52" cy="154.39" r="4.73"
    fill="#80cf72"/>
    <circle cx="175.39" cy="169.07" r="4.42"
    fill="#78cf72"/>
    <circle cx="162.04" cy="180.90" r="4.06"
```

```
    fill="#72cf73"/>                                                    6251
    <circle cx="146.24" cy="189.19" r="3.73"                            6252
    fill="#72cf7b"/>                                                    6253
    <circle cx="128.92" cy="193.46" r="3.51"                            6254
    fill="#72cf83"/>                                                    6255
    <circle cx="111.08" cy="193.46" r="3.45"                            6256
    fill="#72cf8b"/>                                                    6257
    <circle cx="93.76" cy="189.19" r="3.56" fill="#72cf93"/>            6258
    <circle cx="77.96" cy="180.90" r="3.81" fill="#72cf9c"/>            6259
    <circle cx="64.61" cy="169.07" r="4.16" fill="#72cfa4"/>            6260
    <circle cx="54.48" cy="154.39" r="4.51" fill="#72cfac"/>            6261
    <circle cx="48.15" cy="137.71" r="4.80" fill="#72cfb4"/>            6262
    <circle cx="46.00" cy="120.00" r="4.94" fill="#72cfbc"/>            6263
    <circle cx="48.15" cy="102.29" r="4.92" fill="#72cfb4"/>            6264
    <circle cx="54.48" cy="85.61" r="4.73" fill="#72cfac"/>             6265
    <circle cx="64.61" cy="70.93" r="4.42" fill="#72cfa4"/>             6266
    <circle cx="77.96" cy="59.10" r="4.06" fill="#72cf9c"/>             6267
    <circle cx="93.76" cy="50.81" r="3.73" fill="#72cf93"/>             6268
    <circle cx="111.08" cy="46.54" r="3.51" fill="#72cf8b"/>            6269
    <circle cx="128.92" cy="46.54" r="3.45" fill="#72cf83"/>            6270
    <circle cx="146.24" cy="50.81" r="3.56" fill="#72cf7b"/>            6271
    <circle cx="162.04" cy="59.10" r="3.81" fill="#72cf73"/>            6272
    <circle cx="175.39" cy="70.93" r="4.16" fill="#78cf72"/>            6273
    <circle cx="185.52" cy="85.61" r="4.51" fill="#80cf72"/>            6274
    <circle cx="191.85" cy="102.29" r="4.80"                            6275
    fill="#88cf72"/>                                                    6276
  </g>                                                                  6277
  <g class="lp-ring" style="--k: 1.62">                                 6278
    <circle cx="209.57" cy="128.82" r="5.68"                            6279
    fill="#76bb59"/>                                                    6280
    <circle cx="206.12" cy="146.13" r="5.40"                            6281
    fill="#6fbb59"/>                                                    6282
    <circle cx="199.37" cy="162.43" r="5.06"                            6283
    fill="#68bb59"/>                                                    6284
    <circle cx="189.57" cy="177.10" r="4.71"                            6285
    fill="#61bb59"/>                                                    6286
    <circle cx="177.10" cy="189.57" r="4.41"                            6287
    fill="#5abb59"/>                                                    6288
    <circle cx="162.43" cy="199.37" r="4.19"                            6289
    fill="#59bb5f"/>                                                    6290
    <circle cx="146.13" cy="206.12" r="4.10"                            6291
    fill="#59bb66"/>                                                    6292
    <circle cx="128.82" cy="209.57" r="4.15"                            6293
    fill="#59bb6d"/>                                                    6294
    <circle cx="111.18" cy="209.57" r="4.32"                            6295
    fill="#59bb73"/>                                                    6296
    <circle cx="93.87" cy="206.12" r="4.60" fill="#59bb7a"/>            6297
    <circle cx="77.57" cy="199.37" r="4.94" fill="#59bb81"/>            6298
    <circle cx="62.90" cy="189.57" r="5.29" fill="#59bb88"/>            6299
    <circle cx="50.43" cy="177.10" r="5.59" fill="#59bb8f"/>            6300
```

```
    <circle cx="40.63" cy="162.43" r="5.81" fill="#59bb96" /> 6301
    <circle cx="33.88" cy="146.13" r="5.90" fill="#59bb9d" /> 6302
    <circle cx="30.43" cy="128.82" r="5.85" fill="#59bba4" /> 6303
    <circle cx="30.43" cy="111.18" r="5.68" fill="#59bba4" /> 6304
    <circle cx="33.88" cy="93.87" r="5.40" fill="#59bb9d" /> 6305
    <circle cx="40.63" cy="77.57" r="5.06" fill="#59bb96" /> 6306
    <circle cx="50.43" cy="62.90" r="4.71" fill="#59bb8f" /> 6307
    <circle cx="62.90" cy="50.43" r="4.41" fill="#59bb88" /> 6308
    <circle cx="77.57" cy="40.63" r="4.19" fill="#59bb81" /> 6309
    <circle cx="93.87" cy="33.88" r="4.10" fill="#59bb7a" /> 6310
    <circle cx="111.18" cy="30.43" r="4.15" fill="#59bb73" /> 6311
    <circle cx="128.82" cy="30.43" r="4.32" fill="#59bb6d" /> 6312
    <circle cx="146.13" cy="33.88" r="4.60" fill="#59bb66" /> 6313
    <circle cx="162.43" cy="40.63" r="4.94" fill="#59bb5f" /> 6314
    <circle cx="177.10" cy="50.43" r="5.29" fill="#5abb59" /> 6315
    <circle cx="189.57" cy="62.90" r="5.59" fill="#61bb59" /> 6316
    <circle cx="199.37" cy="77.57" r="5.81" fill="#68bb59" /> 6317
    <circle cx="206.12" cy="93.87" r="5.90" fill="#6fbb59" /> 6318
    <circle cx="209.57" cy="111.18" r="5.85"                 6319
    fill="#76bb59" />                                        6320
</g>                                                         6321
<g class="lp-ring" style="--k: 1.46">                        6322
    <circle cx="226.00" cy="120.00" r="5.94"                 6323
    fill="#65a843" />                                        6324
    <circle cx="224.55" cy="137.45" r="5.62"                 6325
    fill="#5fa843" />                                        6326
    <circle cx="220.26" cy="154.42" r="5.29"                 6327
    fill="#59a843" />                                        6328
    <circle cx="213.22" cy="170.45" r="5.00"                 6329
    fill="#53a843" />                                        6330
    <circle cx="203.65" cy="185.11" r="4.77"                 6331
    fill="#4da843" />                                        6332
    <circle cx="191.79" cy="197.99" r="4.63"                 6333
    fill="#47a843" />                                        6334
    <circle cx="177.98" cy="208.74" r="4.59"                 6335
    fill="#43a846" />                                        6336
    <circle cx="162.58" cy="217.07" r="4.67"                 6337
    fill="#43a84c" />                                        6338
    <circle cx="146.02" cy="222.76" r="4.85"                 6339
    fill="#43a852" />                                        6340
    <circle cx="128.75" cy="225.64" r="5.10"                 6341
    fill="#43a858" />                                        6342
    <circle cx="111.25" cy="225.64" r="5.42"                 6343
    fill="#43a85e" />                                        6344
    <circle cx="93.98" cy="222.76" r="5.75" fill="#43a864" /> 6345
    <circle cx="77.42" cy="217.07" r="6.06" fill="#43a86a" /> 6346
    <circle cx="62.02" cy="208.74" r="6.33" fill="#43a870" /> 6347
    <circle cx="48.21" cy="197.99" r="6.52" fill="#43a876" /> 6348
    <circle cx="36.35" cy="185.11" r="6.60" fill="#43a87c" /> 6349
    <circle cx="26.78" cy="170.45" r="6.58" fill="#43a882" /> 6350
```

```
      <circle cx="19.74" cy="154.42" r="6.45" fill="#43a888"/>
      <circle cx="15.45" cy="137.45" r="6.23" fill="#43a88e"/>
      <circle cx="14.00" cy="120.00" r="5.94" fill="#43a894"/>
      <circle cx="15.45" cy="102.55" r="5.62" fill="#43a88e"/>
      <circle cx="19.74" cy="85.58" r="5.29" fill="#43a888"/>
      <circle cx="26.78" cy="69.55" r="5.00" fill="#43a882"/>
      <circle cx="36.35" cy="54.89" r="4.77" fill="#43a87c"/>
      <circle cx="48.21" cy="42.01" r="4.63" fill="#43a876"/>
      <circle cx="62.02" cy="31.26" r="4.59" fill="#43a870"/>
      <circle cx="77.42" cy="22.93" r="4.67" fill="#43a86a"/>
      <circle cx="93.98" cy="17.24" r="4.85" fill="#43a864"/>
      <circle cx="111.25" cy="14.36" r="5.10" fill="#43a85e"/>
      <circle cx="128.75" cy="14.36" r="5.42" fill="#43a858"/>
      <circle cx="146.02" cy="17.24" r="5.75" fill="#43a852"/>
      <circle cx="162.58" cy="22.93" r="6.06" fill="#43a84c"/>
      <circle cx="177.98" cy="31.26" r="6.33" fill="#43a846"/>
      <circle cx="191.79" cy="42.01" r="6.52" fill="#47a843"/>
      <circle cx="203.65" cy="54.89" r="6.60" fill="#4da843"/>
      <circle cx="213.22" cy="69.55" r="6.58" fill="#53a843"/>
      <circle cx="220.26" cy="85.58" r="6.45" fill="#59a843"/>
      <circle cx="224.55" cy="102.55" r="6.23"
      fill="#5fa843"/>
   </g>
   <g class="lp-badge__mark">
      <circle cx="120" cy="120" r="27" fill="#14432f"/>
      <g transform="translate(120 120) scale(1.5)
      translate(-12 -12)" fill="none" stroke="#ffffff"
      stroke-width="1.9" stroke-linecap="round"
      stroke-linejoin="round">
         <path d="m3 20 6.5-11L14 17l2.5-4L21 20z"/>
         <circle cx="17" cy="6" r="2.2"/>
      </g>
   </g>
</svg>
# ==== app/templates/error.html ====
{% extends "base.html" %}
{% block title %}{{ code }} · {{ message }}{% endblock %}
{% block content %}
<div class="card" style="max-width:560px;">
   <h2>{{ code }} · {{ message }}</h2>
   <p class="muted">
      请返回上一页重试；若确认该操作应当可用，请检查当前账号的角色权限。
   </p>
   <p class="actions" style="margin-top:16px;">
      <a class="btn" href="/">返回首页</a>
   </p>
</div>
{% endblock %}
# ==== app/templates/auth/login.html ====
{% extends "base.html" %}
```

```
{% from "_macros.html" import icon %}
{% block title %}登录 · 历史遗留废弃矿山自然恢复适宜性评价系统{% endblock %}
{% block content %}
<style>
  /* ================== 登录页（仿 Apple 账户页） ================== */
    纯白底、居中、大量留白；色彩只用在徽标与按钮上。
    首屏放徽标、标题、一句话说明与登录按钮；系统介绍在下方，
    随滚动逐条弹入、往回滚时弹回。 */
  .lp {
    color: #1d1d1f;
    background:
      linear-gradient(180deg, rgba(245, 239, 224, .88) 0%,
                              rgba(245, 239, 224, .92)
    100%),
      url("../static/hero_qinglv.jpg")
      center top / cover no-repeat fixed;
  }
  /* ---------- 首屏 ---------- */
  .lp-hero {
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    text-align: center;
    min-height: 100dvh;
    padding: 56px 20px 72px;
    position: relative;
    background:
      linear-gradient(180deg, rgba(15, 40, 45, .72) 0%,
                              rgba(20, 55, 58, .50) 50%,
                              rgba(15, 40, 45, .76) 100%),
      url("../static/hero_qinglv.jpg")
      center center / cover no-repeat;
  }
  /* 徽标：滚动时各圈向四周散开并淡出。
     --p 为 0~1 的滚动进度，由页面脚本写入；
     每圈自带 --k（散开倍数，由内向外递减），使各圈逐渐拉开距离 */
  .lp-badge {
    width: 212px;
    height: 212px;
    margin-bottom: 36px;
    --p: 0;
    animation: lp-in .7s ease both;
  }
  .lp-badge__svg {
    display: block;
    width: 100%;
    height: 100%;
    overflow: visible;                    /* 散开时圆点不被画布裁掉 */
    opacity: calc(1 - var(--p) * .94);
```

```
    filter: blur(calc(var(--p) * 2.6px));
  }
  .lp-ring {
    transform-box: fill-box;
    transform-origin: center;
    transform: scale(calc(1 + (var(--k) - 1) * var(--p)));
  }
  .lp-badge__mark {
    transform-box: fill-box;
    transform-origin: center;
    transform: scale(calc(1 + .18 * var(--p)));
  }
  .lp-title {
    margin: 0 0 18px;
    font-size: clamp(28px, 4.2vw, 42px);
    font-weight: 700;
    line-height: 1.3;
    letter-spacing: -.4px;
    color: #fff;
    text-shadow: 0 2px 12px rgba(0, 0, 0, .3);
    animation: lp-in .7s ease .06s both;
  }
  .lp-cta {
    padding: 13px 52px;
    font-family: inherit;
    font-size: 17px;
    font-weight: 500;
    color: #fff;
    background: var(--accent);
    border: none;
    border-radius: 980px;
    cursor: pointer;
    animation: lp-in .7s ease .18s both;
    transition: background-color .2s ease, box-shadow .2s
    ease,
                transform .2s ease;
  }
  .lp-cta:hover {
    background: var(--accent-hover);
    box-shadow: 0 10px 26px rgba(44, 107, 90, .4);
  }
  .lp-cta:active { transform: translateY(1px); box-shadow:
    none; }
  .lp-cta:focus-visible { outline: 3px solid rgba(31, 107,
    69, .40); outline-offset: 3px; }
  .lp-more {
    margin-top: 38px;
    font-size: 13px;
    color: rgba(255, 255, 255, .7);
    text-decoration: none;
```

```
    animation: lp-in .7s ease .24s both;
  }
  .lp-more:hover { color: #fff; }
  @keyframes lp-in {
    from { opacity: 0; transform: translateY(12px); }
    to   { opacity: 1; transform: none; }
  }
  /* ---------- 滚动进场 ---------- 
     进入视口时从屏外飞入，离开视口时飞回，可反复。
     交错延迟只加在「进入」的规则上，退出时不延迟，否则会显得拖沓。 */
  .lp-reveal {
    opacity: 0;
    transition: opacity .7s ease,
                transform 1.05s cubic-bezier(.18, 1.04, .3, 1),
                border-color .62s ease, background-color .62s ease;
    transition-delay: 0s;
    will-change: opacity, transform;
  }
  /* 起始位置整体在屏外：横向按视口宽度取位移，纵向按视口高度取位移，
     再带一点缩小，落位时按贝塞尔曲线的过冲轻微回弹。 */
  .lp-reveal[data-dir="left"]  { transform: translate3d(-56vw, 5vh, 0) scale(.9); }
  .lp-reveal[data-dir="right"] { transform: translate3d(56vw, 5vh, 0) scale(.9); }
  .lp-reveal[data-dir="up"]    { transform: translate3d(0, 24vh, 0) scale(.94); }
  .lp-reveal.is-in {
    opacity: 1;
    transform: none;
    transition-delay: calc(var(--i, 0) * 110ms);
  }
  /* 标题颜色由淡到深，强化「浮现」的层次感 */
  .lp-reveal h2,
  .lp-reveal h3,
  .lp-reveal .lp-mod__name { color: #c2d2c8; transition: color .7s ease; }
  .lp-reveal.is-in h3,
  .lp-reveal.is-in .lp-mod__name { color: var(--accent); }
  .lp-reveal.is-in h2 { color: #1d1d1f; }
  /* ---------- 系统介绍 ---------- */
  .lp-intro {
    max-width: none;
    margin: 0;
    padding: 56px 24px 72px;
    background: transparent;
    /* 卡片从屏外飞入，横向超出的部分裁掉，免得出现横向滚动条；
       用 clip 而不是 hidden，纵向滚动仍由页面本身负责 */
    overflow-x: clip;
```

```
}
.lp-intro > * { max-width: 1040px; margin-left: auto;
  margin-right: auto; }
.lp-intro__title {
  margin: 0 auto 26px;
  font-size: clamp(22px, 2.6vw, 30px);
  text-align: center;
  letter-spacing: -.3px;
}
/* 三张特点卡与九个模块卡都按三等分排版，并整体居中 */
.lp-points {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 16px;
  margin-bottom: 46px;
}
.lp-point {
  padding: 30px 26px;
  background: rgba(253, 250, 242, .82);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  border: 1px solid rgba(224, 216, 200, .6);
  border-radius: 22px;
  box-shadow: 0 24px 60px rgba(42, 37, 32, .10), 0 8px
  20px rgba(42, 37, 32, .05);
  transition: transform .25s ease, box-shadow .25s ease;
}
.lp-point:hover {
  transform: translateY(-4px);
  box-shadow: 0 32px 72px rgba(42, 37, 32, .14), 0 12px
  28px rgba(42, 37, 32, .07);
}
.lp-point__icon {
  display: grid;
  place-items: center;
  width: 40px;
  height: 40px;
  margin-bottom: 16px;
  border-radius: 12px;
  background: rgba(44, 107, 90, .1);
  color: var(--accent);
}
.lp-point h3 { margin: 0 0 8px; font-size: 16px; }
.lp-point p {
  margin: 0;
  font-size: 13.5px;
  line-height: 1.85;
  color: #6e6e73;
  text-wrap: pretty;
}
```

```
.lp-sub-title {
  margin: 0 auto 20px;
  font-size: 17px;
  text-align: center;
}
.lp-mods {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 12px;
  margin-bottom: 46px;
  align-items: start;
}
/* 模块卡：默认只显示编号与名称，鼠标悬停或键盘聚焦时才展开职责说明 */
.lp-mod {
  position: relative;
  padding: 17px 18px 17px 48px;
  background: rgba(253, 250, 242, .75);
  backdrop-filter: blur(8px);
  -webkit-backdrop-filter: blur(8px);
  border: 1px solid rgba(224, 216, 200, .6);
  border-radius: 14px;
  outline: none;
  transition: border-color .3s ease, box-shadow .3s ease;
}
.lp-mod:hover,
.lp-mod:focus-within,
.lp-mod:focus {
  border-color: rgba(44, 107, 90, .5);
  box-shadow: 0 8px 22px rgba(42, 37, 32, .08);
}
.lp-mod__no {
  position: absolute;
  left: 17px;
  top: 17px;
  font-size: 13px;
  font-weight: 700;
  color: #c8d6cd;
  font-variant-numeric: tabular-nums;
}
.lp-mod__name {
  display: block;
  font-size: 14px;
  font-weight: 600;
}
/* 用 grid-template-rows 0fr→1fr 展开，高度过渡自然 */
.lp-mod__body {
  display: grid;
  grid-template-rows: 0fr;
  transition: grid-template-rows .34s ease, margin-top
  .34s ease;
```

```
}
.lp-mod:hover .lp-mod__body,
.lp-mod:focus-within .lp-mod__body {
  grid-template-rows: 1fr;
  margin-top: 7px;
}
.lp-mod__desc {
  overflow: hidden;
  margin: 0;
  font-size: 12.5px;
  line-height: 1.7;
  color: #6e6e73;
  opacity: 0;
  transition: opacity .3s ease;
  text-wrap: pretty;
}
.lp-mod:hover .lp-mod__desc,
.lp-mod:focus-within .lp-mod__desc { opacity: 1; }
/* 触摸设备没有悬停，直接把说明摊开 */
@media (hover: none) {
  .lp-mod__body { grid-template-rows: 1fr; margin-top:
  7px; }
  .lp-mod__desc { opacity: 1; }
}
.lp-foot {
  /* margin 简写会把 .lp-intro > * 里的左右 auto 一起清掉，块就贴到左边
     （文字自己居中，看着却整体偏左）—— 与标题那两处是同一个坑 */
  margin: 0 auto;
  padding-top: 26px;
  border-top: 1px solid #e6e6e8;
  text-align: center;
  font-size: 12.5px;
  color: #86868b;
}
/* ---------- 登录面板：点击后渐变浮现 ---------- */
.lp-auth {
  position: fixed;
  inset: 0;
  z-index: 60;
  display: grid;
  place-items: center;
  padding: 24px;
  opacity: 0;
  visibility: hidden;
  transition: opacity .42s ease, visibility 0s linear
  .42s;
}
.lp-auth.is-open {
  opacity: 1;
  visibility: visible;
```

```
    transition: opacity .42s ease, visibility 0s;
}
/* 遮罩用绿色径向渐变，形成「渐变渗出来」的观感 */
.lp-auth__scrim {
    position: absolute;
    inset: 0;
    background:
      radial-gradient(1000px 640px at 50% 0%,
                      rgba(31, 107, 69, .22), rgba(31, 107,
    69, 0) 70%),
      linear-gradient(180deg, rgba(255, 255, 255, .84),
                                  rgba(246, 250, 247, .95));
    backdrop-filter: blur(16px) saturate(1.25);
    -webkit-backdrop-filter: blur(16px) saturate(1.25);
}
.lp-auth__panel {
    position: relative;
    width: 100%;
    max-width: 396px;
    padding: 36px 36px 26px;
    background: #fff;
    border-radius: 20px;
    box-shadow: 0 28px 80px rgba(14, 46, 32, .18);
    opacity: 0;
    transform: translateY(18px) scale(.975);
    transition: transform .55s cubic-bezier(.16, .84, .34,
    1) .05s,
                opacity .55s ease .05s;
}
.lp-auth.is-open .lp-auth__panel { opacity: 1; transform:
    none; }
.lp-auth__close {
    position: absolute;
    top: 14px;
    right: 14px;
    display: grid;
    place-items: center;
    width: 30px;
    height: 30px;
    color: #86868b;
    background: transparent;
    border: none;
    border-radius: 50%;
    cursor: pointer;
    transition: background-color .18s ease, color .18s ease;
}
.lp-auth__close:hover { background: #f5f5f7; color:
    #1d1d1f; }
.lp-auth__title {
    margin: 0 0 6px;
```

```
    text-align: center;
    font-size: 24px;
    font-weight: 700;
    letter-spacing: -.2px;
  }
  .lp-auth__sub {
    margin: 0 0 26px;
    text-align: center;
    font-size: 13px;
    color: #86868b;
  }
  /* 字段校验错误：贴着输入框显示，不做成会消失的浮层 —— 这类消息要对着输入框看，一闪而过会让人以为「点了没反应」 */
  .lp-field__error {
    margin: 7px 0 0;
    font-size: 12.5px;
    color: #9c2b23;
  }
  .lp-field input[aria-invalid="true"] {
    background: #fdeeec;
    border-color: rgba(156, 43, 35, .35);
  }
  .lp-field { margin-bottom: 15px; }
  .lp-field label {
    display: block;
    margin-bottom: 7px;
    font-size: 13px;
    color: #6e6e73;
  }
  .lp-field input {
    width: 100%;
    padding: 13px 16px;
    font-family: inherit;
    font-size: 16px;              /* 不小于 16px，避免手机端聚焦时自动放大 */
    color: #1d1d1f;
    background: #f5f5f7;
    border: 1px solid transparent;
    border-radius: 12px;
    outline: none;
    transition: background-color .18s ease, border-color .18s ease,
                box-shadow .18s ease;
  }
  .lp-field input::placeholder { color: #a1a1a6; }
  .lp-field input:focus {
    background: #fff;
    border-color: rgba(31, 107, 69, .55);
    box-shadow: 0 0 0 3px rgba(31, 107, 69, .14);
  }
  .lp-remember {
```

```
    display: flex;                                                    6801
    align-items: center;                                              6802
    gap: 8px;                                                         6803
    margin: 4px 0 20px;                                               6804
    font-size: 13px;                                                  6805
    color: #6e6e73;                                                   6806
    cursor: pointer;                                                  6807
}                                                                     6808
.lp-remember input {                                                  6809
    width: 16px;                                                      6810
    height: 16px;                                                     6811
    margin: 0;                                                        6812
    accent-color: var(--accent);                                      6813
    cursor: pointer;                                                  6814
}                                                                     6815
.lp-submit {                                                          6816
    width: 100%;                                                      6817
    padding: 13px 20px;                                               6818
    font-family: inherit;                                             6819
    font-size: 16px;                                                  6820
    font-weight: 500;                                                 6821
    color: #fff;                                                      6822
    background: var(--accent);                                        6823
    border: none;                                                     6824
    border-radius: 980px;                                             6825
    cursor: pointer;                                                  6826
    transition: background-color .2s ease, box-shadow .2s             6827
    ease,                                                             6828
                transform .2s ease;                                   6829
}                                                                     6830
.lp-submit:hover {                                                    6831
    background: var(--accent-hover);                                  6832
    box-shadow: 0 10px 24px rgba(31, 107, 69, .26);                   6833
}                                                                     6834
.lp-submit:active { transform: translateY(1px);                       6835
    box-shadow: none; }                                               6836
.lp-submit:focus-visible { outline: 3px solid rgba(31,                6837
    107, 69, .40); outline-offset: 3px; }                             6838
.lp-auth__hint {                                                      6839
    margin: 18px 0 0;                                                 6840
    text-align: center;                                               6841
    font-size: 12.5px;                                                6842
    color: #a1a1a6;                                                   6843
}                                                                     6844
.lp-auth__reg {                                                       6845
    color: var(--accent);                                             6846
    font-weight: 500;                                                 6847
    text-decoration: none;                                            6848
    border-bottom: 1px solid rgba(31, 107, 69, .35);                  6849
    transition: border-color .18s ease;                               6850
```

```
  }
  .lp-auth__reg:hover { border-bottom-color: var(--accent); }
  /* 系统设置里要求减少动态效果时，去掉位移、缩放与滚动特效 */
  @media (prefers-reduced-motion: reduce) {
    .lp-badge, .lp-title, .lp-cta, .lp-more { animation: none; }
    .lp-badge__svg { opacity: 1; filter: none; }
    .lp-ring, .lp-badge__mark { transform: none; }
    .lp-reveal { opacity: 1; transform: none !important; }
    .lp-reveal h2, .lp-reveal h3, .lp-reveal .lp-mod__name { color: inherit; }
    .lp-reveal.is-in h2 { color: #1d1d1f; }
    .lp-reveal.is-in h3, .lp-reveal.is-in .lp-mod__name { color: var(--accent); }
    .lp-auth, .lp-auth__panel { transition: opacity .2s linear; }
    .lp-auth__panel { transform: none; }
  }
  /* 窄屏退成两列、一列，避免卡片被挤变形 */
  @media (max-width: 900px) {
    .lp-points, .lp-mods { grid-template-columns: repeat(2, minmax(0, 1fr)); }
  }
  @media (max-width: 620px) {
    .lp-points, .lp-mods { grid-template-columns: minmax(0, 1fr); }
  }
  @media (max-width: 560px) {
    .lp-hero { padding: 36px 18px 48px; min-height: auto; }
    .lp-badge { width: 160px; height: 160px; margin-bottom: 26px; }
    .lp-auth__panel { padding: 30px 24px 22px; }
  }
</style>
<div class="lp">
  {# ---------------- 首屏：徽标 + 标题 + 说明 + 登录入口 ---------------- #}
  <section class="lp-hero">
    <div class="lp-badge" id="lp-badge" aria-hidden="true">
      {% include "_login_badge.svg" %}
    </div>
    <h1 class="lp-title">历史遗留废弃矿山<br>自然恢复适宜性评价系统</h1>
    <button type="button" class="lp-cta" id="lp-open"
            aria-expanded="false" aria-controls="lp-auth">登录</button>
    <a class="lp-more" href="#intro">
      向下滑动，了解系统 <span aria-hidden="true">↓</span>
    </a>
  </section>
```

```
{# ---------------- 系统介绍：随滚动逐条弹入 ---------------- #}
<section class="lp-intro" id="intro">
  <h2 class="lp-intro__title lp-reveal"
  data-dir="up">一套把评价流程固定下来的工具</h2>
  <div class="lp-points">
    {% set pdirs = ['left', 'up', 'right'] %}
    {% set points = [
      ('sliders', '方法可溯',
       '权重由层次分析法算出并做一致性检验，指标标准化、分单元赋分与面积加权合成的每一步都留有记录，可逐层回看。'),
      ('shield', '流程可控',
       '政策、区位约束、土壤污染、地质灾害、耕地属性五项限制性条件一票否决；未通过即终止评价，直接给出生态重建或人工辅助的结论。'),
      ('file', '结果可复算',
       '评价结果与逐层得分明细一并留存，报告取自已保存的评价记录，事后修改数据不会让报告与结论对不上。')
    ] %}
    {% for ic, title, text in points %}
    <div class="lp-point lp-reveal" data-dir="{{
  pdirs[loop.index0 % 3] }}"
         style="--i: {{ loop.index0 }}">
      <span class="lp-point__icon">{{ icon(ic, 20)
  }}</span>
      <h3>{{ title }}</h3>
      <p>{{ text }}</p>
    </div>
    {% endfor %}
  </div>
  <h3 class="lp-sub-title lp-reveal"
  data-dir="up">九个功能模块</h3>
  <div class="lp-mods">
    {% set mdirs = ['left', 'up', 'right'] %}
    {% for no, name, desc in modules %}
    <div class="lp-mod lp-reveal" data-dir="{{
  mdirs[loop.index0 % 3] }}"
         style="--i: {{ loop.index0 % 3 }}" tabindex="0">
      <span class="lp-mod__no">{{ no }}</span>
      <span class="lp-mod__name">{{ name }}</span>
      <div class="lp-mod__body"><p class="lp-mod__desc">{{
  desc }}</p></div>
    </div>
    {% endfor %}
  </div>
  <p class="lp-foot lp-reveal" data-dir="up">
    历史遗留废弃矿山自然恢复适宜性评价系统 Ｖ１．０ ·
    运行于浏览器，数据保存在本机 ＳＱＬｉｔｅ 数据库
  </p>
</section>
{# ---------------- 登录面板：点击「登录」后渐变浮现 ---------------- #}
<div class="lp-auth" id="lp-auth" aria-hidden="true">
  <div class="lp-auth__scrim" data-lp-close></div>
```

```
<div class="lp-auth__panel" role="dialog"
aria-modal="true"
     aria-labelledby="lp-auth-title">
  <button type="button" class="lp-auth__close"
data-lp-close
          aria-label="关闭登录框">
    <svg width="16" height="16" viewBox="0 0 24 24"
fill="none"
         stroke="currentColor" stroke-width="2"
stroke-linecap="round"
         aria-hidden="true"><path d="M6 6l12 12M18 6L6
18"/></svg>
  </button>
  <h2 class="lp-auth__title" id="lp-auth-title">登录</h2>
  <p class="lp-auth__sub">使用系统账号登录</p>
  <form method="post" novalidate>
    {{ form.hidden_tag() }}
    <div class="lp-field">
      <label for="{{ form.username.id }}">账户名称</label>
      {{ form.username(id=form.username.id,
autocomplete="username",
                       placeholder="请输入账户名称",
                       **({'aria-invalid': 'true'} if
form.username.errors else {})) }}
      {% for e in form.username.errors %}
      <p class="lp-field__error">{{ e }}</p>
      {% endfor %}
    </div>
    <div class="lp-field">
      <label for="{{ form.password.id }}">密码</label>
      {{ form.password(id=form.password.id,
autocomplete="current-password",
                       placeholder="请输入密码",
                       **({'aria-invalid': 'true'} if
form.password.errors else {})) }}
      {% for e in form.password.errors %}
      <p class="lp-field__error">{{ e }}</p>
      {% endfor %}
    </div>
    <label class="lp-remember" for="{{ form.remember.id
}}">
      {{ form.remember(id=form.remember.id) }}
      <span>记住我（下次自动登录，一个月内有效）</span>
    </label>
    {{ form.submit(class_="lp-submit") }}
  </form>
  <p class="lp-auth__hint">
    还没有账号？<a class="lp-auth__reg"
                   href="{{ url_for('auth.register')
}}">点我注册</a>
```

```
        </p>
        <p class="lp-auth__hint" style="margin-top:10px;">
          演示账号：admin 密码：admin123
        </p>
      </div>
    </div>
</div>
{% endblock %}
{% block scripts %}
<script>
(function () {
  var reduce = window.matchMedia &&
    window.matchMedia('(prefers-reduced-motion:
    reduce)').matches;
  /* ---------- 登录面板 ---------- */
  var btn = document.getElementById('lp-open');
  var auth = document.getElementById('lp-auth');
  var lastFocus = null;
  /* 清掉上一次提交留下的字段错误（红框 + 输入框下方的提示）。
     登录失败时页面是直接渲染的，错误留在 DOM 里；如果只是把面板关掉再打开，
     那些红框还在，看起来就像「刚点开就报错」。 */
  function clearFieldErrors(scope) {
    var root = scope || auth;
    Array.prototype.forEach.call(
      root.querySelectorAll('.lp-field__error'),
      function (el) { el.remove(); });
    Array.prototype.forEach.call(
      root.querySelectorAll('[aria-invalid="true"]'),
      function (el) { el.removeAttribute('aria-invalid'); });
    });
  }
  function openPanel(keepErrors) {
    // keepErrors：由「登录失败自动展开」触发时保留错误提示；
    // 用户自己点「登录」重新打开时则清掉，给一个干净的开始。
    if (!keepErrors) clearFieldErrors();
    lastFocus = document.activeElement;
    auth.classList.add('is-open');
    auth.setAttribute('aria-hidden', 'false');
    btn.setAttribute('aria-expanded', 'true');
    document.body.style.overflow = 'hidden';
    // 记到地址栏，直接访问 /login#login 也能展开登录框
    if (location.hash !== '#login') {
      history.replaceState(null, '', location.pathname +
    '#login');
    }
    var first = auth.querySelector('input');
    if (first) window.setTimeout(function () {
    first.focus(); }, 300);
  }
  function closePanel() {
```

```
    auth.classList.remove('is-open');
    auth.setAttribute('aria-hidden', 'true');
    btn.setAttribute('aria-expanded', 'false');
    document.body.style.overflow = '';
    if (location.hash === '#login') {
      history.replaceState(null, '', location.pathname);
    }
    if (lastFocus && lastFocus.focus) lastFocus.focus();
  }
  if (btn && auth) {
    // 注意别把事件对象当成参数传进去（那样会被当成 keepErrors=true）
    btn.addEventListener('click', function () {
    openPanel(false); });
    Array.prototype.forEach.call(
      auth.querySelectorAll('[data-lp-close]'),
      function (el) { el.addEventListener('click',
    closePanel); }
    );
    document.addEventListener('keydown', function (e) {
      if (e.key === 'Escape' && auth.classList.contains('is-
    open')) closePanel();
    });
    // 登录失败回到本页、或直接带 #login 访问时，直接展开登录框。
    // 失败可能落在两处：字段校验错误在表单里，凭据错误在浮层里。
    // 这条路径保留错误提示（keepErrors），用户才知道刚才哪儿没填。
    if (location.hash === '#login'
        || document.querySelector('.lp-field__error')
        || document.querySelector('.toast-center')) {
      openPanel(true);
    }
    // 一开始输入就把这一项的红色状态撤掉：错误已经不被犯了，红框留着只会让人困惑
    Array.prototype.forEach.call(
      auth.querySelectorAll('input'),
      function (input) {
        input.addEventListener('input', function () {
          var field = input.closest('.lp-field');
          if (field) clearFieldErrors(field);
        });
      });
  }
  /* ---------- 内容进场 + 徽标散开：同一套滚动计算 ---------- */
     原先用 IntersectionObserver，实测在滚动到位置后回调并未生效，
     内容始终停在 opacity：0。改为直接按元素与视口的相对位置判断，
     与徽标用同一次滚动回调，行为可预期、也便于排查。
     判断「进没进视野」用的是**布局位置**，不是 getBoundingClientRect()：
     进场动画本身就是位移（左右两列横向 56vw、中列自下方 24vh 升起），
     用视觉位置判断时，中列的盒子一直比自己的位置低一截，
     于是它总要再往下滚一段才触发，看起来比左右两列慢半拍。
     transform 不影响 offsetTop，所以这里沿 offsetParent 把布局位置累加出来，
     只在加载时算一次；滚动过程中不再测量，也就没有读写交替带来的重排。 */
```

```
var items = document.querySelectorAll('.lp-reveal');
var badge = document.getElementById('lp-badge');
var ticking = false;
var slots = [];
function layoutTop(el) {
  var top = 0;
  var node = el;
  while (node) {
    top += node.offsetTop || 0;
    node = node.offsetParent;
  }
  return top;
}
if (!reduce) {
  for (var k = 0; k < items.length; k++) {
    slots.push({
      el: items[k],
      top: layoutTop(items[k]),
      height: items[k].offsetHeight
    });
  }
}
function applyScroll() {
  ticking = false;
  var vh = window.innerHeight || 800;
  var y = window.scrollY || window.pageYOffset || 0;
  /* 徽标：滚动前约三分之一屏内完成散开淡出 */
  if (badge) {
    var range = vh * 0.34;
    var p = range > 0 ? y / range : 0;
    badge.style.setProperty('--p', Math.min(1,
  Math.max(0, p)).toFixed(3));
  }
  if (reduce || !slots.length) return;
  for (var j = 0; j < slots.length; j++) {
    var s = slots[j];
    var viewTop = s.top - y;
    // 上沿进入视口下部、下沿尚未越出视口上部 → 视为在视野内
    var shown = viewTop < vh * 0.88 && viewTop + s.height
  > vh * 0.06;
    if (shown) s.el.classList.add('is-in');
    else s.el.classList.remove('is-in');
  }
}
function onScroll() {
  if (!ticking) {
    ticking = true;
    window.requestAnimationFrame(applyScroll);
  }
}
```

```
  //  减少动态效果时不做滚动特效，内容直接显示
  if (reduce) {
    Array.prototype.forEach.call(items, function (el) {
      el.classList.add('is-in');
    });
    if (badge) badge.style.setProperty('--p', '0');
  } else {
    window.addEventListener('scroll', onScroll, { passive:
    true });
    window.addEventListener('resize', onScroll);
    applyScroll();
  }
})();
</script>
{% endblock %}
# ==== app/templates/auth/register.html ====
{% extends "base.html" %}
{% block title %}注册账号 · 历史遗留废弃矿山自然恢复适宜性评价系统{% endblock %}
{% block content %}
<style>
  /* ================== 注册页 ==================
     与登录页同一套观感：浅绿渐变底 + 白色卡片。
     必填只有账户名称与密码，其余收在「选填」分隔线以下，
     这样首次注册不会被一堆输入框挡住。 */
  .rg {
    min-height: 100dvh;
    display: grid;
    place-items: center;
    padding: 40px 20px 56px;
    background:
      radial-gradient(900px 520px at 50% 0%,
                      rgba(31, 107, 69, .14), rgba(31, 107,
    69, 0) 72%),
      #f6faf7;
  }
  .rg-card {
    width: 100%;
    max-width: 470px;
    padding: 38px 40px 30px;
    background: #fff;
    border-radius: 20px;
    box-shadow: 0 26px 70px rgba(14, 46, 32, .13);
    animation: rg-in .5s cubic-bezier(.16, .84, .34, 1)
    both;
  }
  @keyframes rg-in {
    from { opacity: 0; transform: translateY(16px)
    scale(.98); }
    to   { opacity: 1; transform: none; }
  }
```

```
.rg-title {
  margin: 0 0 6px;
  text-align: center;
  font-size: 25px;
  font-weight: 700;
  letter-spacing: -.2px;
}
.rg-sub {
  margin: 0 0 28px;
  text-align: center;
  font-size: 13px;
  color: #86868b;
}
.rg-field { margin-bottom: 15px; }
.rg-field > label {
  display: block;
  margin-bottom: 7px;
  font-size: 13px;
  color: #6e6e73;
}
.rg-field > label .rg-opt {
  margin-left: 5px;
  font-size: 12px;
  color: #a1a1a6;
}
.rg-field input[type="text"],
.rg-field input[type="password"] {
  width: 100%;
  padding: 12px 15px;
  font-family: inherit;
  font-size: 16px;              /* 不小于 16px，避免手机端聚焦时自动放大 */
  color: #1d1d1f;
  background: #f5f5f7;
  border: 1px solid transparent;
  border-radius: 12px;
  outline: none;
  transition: background-color .18s ease, border-color
  .18s ease,
              box-shadow .18s ease;
}
.rg-field input::placeholder { color: #a1a1a6; }
.rg-field input:focus {
  background: #fff;
  border-color: rgba(31, 107, 69, .55);
  box-shadow: 0 0 0 3px rgba(31, 107, 69, .14);
}
.rg-field input[aria-invalid="true"] {
  border-color: rgba(180, 60, 50, .55);
  background: #fff;
}
```

```
.rg-field select {
  width: 100%;
  padding: 12px 15px;
  font-family: inherit;
  font-size: 15px;
  color: #1d1d1f;
  background: #f5f5f7;
  border: 1px solid transparent;
  border-radius: 12px;
  outline: none;
}
.rg-field select:focus {
  background: #fff;
  border-color: rgba(31, 107, 69, .55);
  box-shadow: 0 0 0 3px rgba(31, 107, 69, .14);
}
.rg-err {
  margin: 6px 0 0;
  font-size: 12.5px;
  color: var(--danger);
}
/* 「选填」分隔线 */
.rg-split {
  display: flex;
  align-items: center;
  gap: 12px;
  margin: 24px 0 18px;
  font-size: 12.5px;
  color: #a1a1a6;
}
.rg-split::before,
.rg-split::after {
  content: "";
  flex: 1;
  height: 1px;
  background: #ececee;
}
.rg-pair {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 0 14px;
}
@media (max-width: 480px) { .rg-pair {
  grid-template-columns: 1fr; } }
/* 头像：圆形预览 + 选择文件 */
.rg-avatar {
  display: flex;
  align-items: center;
  gap: 14px;
  margin-bottom: 15px;
```

```
}
.rg-avatar__face {
  display: grid;
  place-items: center;
  flex: none;
  width: 54px;
  height: 54px;
  border-radius: 50%;
  background: #eef2f0;
  color: #9aa8a1;
  font-size: 12px;
  overflow: hidden;
  background-size: cover;
  background-position: center;
}
.rg-avatar__pick { min-width: 0; }
.rg-avatar__pick input[type="file"] {
  max-width: 100%;
  font-family: inherit;
  font-size: 12.5px;
  color: #6e6e73;
}
.rg-avatar__tip {
  margin: 4px 0 0;
  font-size: 12px;
  color: #a1a1a6;
}
.rg-submit {
  width: 100%;
  margin-top: 24px;
  padding: 13px 20px;
  font-family: inherit;
  font-size: 16px;
  font-weight: 500;
  color: #fff;
  background: var(--accent);
  border: none;
  border-radius: 980px;
  cursor: pointer;
  transition: background-color .2s ease, box-shadow .2s
  ease,
              transform .2s ease;
}
.rg-submit:hover {
  background: var(--accent-hover);
  box-shadow: 0 10px 24px rgba(31, 107, 69, .26);
}
.rg-submit:active { transform: translateY(1px);
  box-shadow: none; }
.rg-submit:focus-visible { outline: 3px solid rgba(31,
```

```
      107, 69, .40); outline-offset: 3px; }
   .rg-foot {
      margin: 20px 0 0;
      text-align: center;
      font-size: 13px;
      color: #86868b;
   }
   .rg-foot a { color: var(--accent); text-decoration: none;
      }
   .rg-foot a:hover { text-decoration: underline; }
   @media (prefers-reduced-motion: reduce) {
      .rg-card { animation: none; }
   }
</style>
<div class="rg">
   <div class="rg-card">
      <h1 class="rg-title">创建账号</h1>
      <p class="rg-sub">新账号为「操作员」，职务由管理员调整</p>
      <form method="post" enctype="multipart/form-data"
      novalidate>
         {{ form.hidden_tag() }}
         <div class="rg-field">
            <label for="{{ form.username.id }}">账户名称</label>
            {{ form.username(id=form.username.id,
      autocomplete="username",
                              placeholder="2–64 个字符",
                              **({'aria-invalid': 'true'} if
      form.username.errors else {})) }}
            {% for e in form.username.errors %}<p
      class="rg-err">{{ e }}</p>{% endfor %}
         </div>
         <div class="rg-field">
            <label for="{{ form.password.id }}">密码</label>
            {{ form.password(id=form.password.id,
      autocomplete="new-password",
                              placeholder="至少 6 位",
                              **({'aria-invalid': 'true'} if
      form.password.errors else {})) }}
            {% for e in form.password.errors %}<p
      class="rg-err">{{ e }}</p>{% endfor %}
         </div>
         <div class="rg-field">
            <label for="{{ form.confirm.id }}">确认密码</label>
            {{ form.confirm(id=form.confirm.id,
      autocomplete="new-password",
                             placeholder="再次输入密码",
                              **({'aria-invalid': 'true'} if
      form.confirm.errors else {})) }}
            {% for e in form.confirm.errors %}<p
      class="rg-err">{{ e }}</p>{% endfor %}
```

```
        </div>                                                                  7401
        <p class="rg-split">以下均为选填</p>                                    7402
        <div class="rg-avatar">                                                 7403
          <span class="rg-avatar__face" id="rg-face"                            7404
    aria-hidden="true">头像</span>                                              7405
          <div class="rg-avatar__pick">                                         7406
            <label class="visually-hidden"                                      7407
    for="rg-avatar">选择头像图片</label>                                        7408
            <input id="rg-avatar" type="file" name="avatar"                     7409
                   accept=".png,.jpg,.jpeg,.gif,.webp">                         7410
            <p class="rg-avatar__tip">PNG / JPG / GIF /                         7411
    WebP，不超过 4 MB</p>                                                       7412
          </div>                                                                7413
        </div>                                                                  7414
        <div class="rg-pair">                                                   7415
          <div class="rg-field">                                                7416
            <label for="{{ form.display_name.id }}">昵称</label>                7417
            {{ form.display_name(id=form.display_name.id,                       7418
                                 placeholder="留空则显示账户名称") }}           7419
          </div>                                                                7420
          <div class="rg-field">                                                7421
            <label for="{{ form.gender.id }}">性别</label>                      7422
            {{ form.gender(id=form.gender.id) }}                                7423
          </div>                                                                7424
        </div>                                                                  7425
        <div class="rg-field">                                                  7426
          <label for="{{ form.email.id }}">邮箱</label>                         7427
          {{ form.email(id=form.email.id,                                       7428
    autocomplete="email",                                                       7429
                        placeholder="用于联系，可留空",                         7430
                        **({'aria-invalid': 'true'} if                          7431
    form.email.errors else {})) }}                                              7432
          {% for e in form.email.errors %}<p                                    7433
    class="rg-err">{{ e }}</p>{% endfor %}                                      7434
        </div>                                                                  7435
        <div class="rg-field" style="margin-bottom:0;">                         7436
          <label for="{{ form.phone.id }}">手机号</label>                       7437
          {{ form.phone(id=form.phone.id, autocomplete="tel",                   7438
    placeholder="可留空") }}                                                    7439
        </div>                                                                  7440
        {{ form.submit_reg(class_="rg-submit") }}                               7441
    </form>                                                                     7442
    <p class="rg-foot">已有账号？<a href="{{                                    7443
    url_for('auth.login') }}">返回登录</a></p>                                  7444
  </div>                                                                        7445
</div>                                                                          7446
{% endblock %}                                                                  7447
{% block scripts %}                                                             7448
<script>                                                                        7449
// 选好头像后立刻在左侧圆框里预览，省得传上去才知道选错了                      7450
```

```
(function () {
  var input = document.getElementById('rg-avatar');
  var face = document.getElementById('rg-face');
  if (!input || !face || !window.FileReader) return;
  input.addEventListener('change', function () {
    var file = input.files && input.files[0];
    if (!file) return;
    var reader = new FileReader();
    reader.onload = function (e) {
      face.textContent = '';
      face.style.backgroundImage = 'url("' +
    e.target.result + '")';
    };
    reader.readAsDataURL(file);
  });
})();
</script>
{% endblock %}
# ==== app/templates/auth/home.html ====
{% extends "base.html" %}
{% from "_macros.html" import icon %}
{% block title %}工作台 · 历史遗留废弃矿山自然恢复适宜性评价系统{% endblock %}
{% block content %}
{# ---------- 页首横幅 ---------- #}
<section class="hero">
  <div class="hero__body">
    <h1 class="hero__title">历史遗留废弃矿山自然恢复适宜性评价系统</h1>
    <p class="hero__desc">
      以论文《历史遗留废弃矿山自然恢复适宜性评价指标体系建设及典型案例》
      建立的评价方法为算法内核。下方按论文 4.1 节的评价流程顺序排列，
      系统会在每一步检查前置条件，前置步骤未完成时不会放行后续计算。
    </p>
    <div class="hero__actions">
      <a class="hero-btn hero-btn--ghost"
    href="/indicator/import">
        {{ icon("upload", 16) }}批量导入
      </a>
      <a class="hero-btn hero-btn--solid"
    href="/project/add">
        {{ icon("plus", 16) }}新建评价项目
      </a>
    </div>
  </div>
</section>
{# ---------- 数据概览 ---------- #}
<section class="stat-cards">
  <a class="stat-card" href="/project/">
    <span class="stat-card__label">评价项目</span>
    <span class="stat-card__value">{{ dash.stats.projects
    }}<span class="stat-card__unit">个</span></span>
```

```
      <span class="stat-card__note">
        {%- if projects_without_patch -%}
          其中 {{ projects_without_patch|length }} 个项目尚未建立图斑
        {%- elif dash.progress.patches -%}
          已建立 {{ dash.progress.patches }} 个矿山图斑
        {%- else -%}
          尚未建立矿山图斑
        {%- endif -%}
      </span>
    </a>
    <a class="stat-card" href="/project/">
      <span class="stat-card__label">矿山图斑</span>
      <span class="stat-card__value">{{ dash.stats.patches
      }}<span class="stat-card__unit">个</span></span>
      <span class="stat-card__note">
        {% if dash.progress.ready %}其中 {{ dash.progress.ready
      }} 个具备评价条件{% else %}暂无具备评价条件的图斑{% endif %}
      </span>
    </a>
    <a class="stat-card" href="/unit/">
      <span class="stat-card__label">微地质单元</span>
      <span class="stat-card__value">{{ dash.stats.unit_types
      }}<span class="stat-card__unit">类</span></span>
      <span class="stat-card__note">
        {% if dash.progress.units %}共划分 {{
      dash.progress.units }} 个单元{% else %}尚未划分单元{% endif %}
      </span>
    </a>
    <a class="stat-card" href="/standard/">
      <span class="stat-card__label">标准与政策</span>
      <span class="stat-card__value">{{ dash.stats.standards
      }}<span class="stat-card__unit">项</span></span>
      <span class="stat-card__note">地方标准与政策文件</span>
    </a>
</section>
{# ---------- 评价流程 ---------- #}
<section class="section">
    <div class="section__head">
      <h2 class="section__title">评价流程</h2>
      <p class="section__desc">
        按论文 4.1 节顺序排列，前置步骤未完成时不会执行后续计算
      </p>
    </div>
    <div class="step-grid">
      {% for s in dash.steps %}
      {% if s.available %}
      <a class="step-card" href="{{ s.href }}">
        <span class="step-card__no">{{ s.no }}</span>
        <span class="step-card__icon">{{ icon(s.icon, 18)
      }}</span>
```

```
        <span class="step-card__title">{{ s.title }}</span>
        <span class="step-card__desc">{{ s.desc }}</span>
        {% if s.pending %}
        <span class="step-card__badge">待处理 {{ s.pending
    }}</span>
        {% endif %}
    </a>
    {% else %}
    <div class="step-card step-card--locked"
    aria-disabled="true">
        <span class="step-card__no">{{ s.no }}</span>
        <span class="step-card__icon">{{ icon(s.icon, 18)
    }}</span>
        <span class="step-card__title">{{ s.title }}</span>
        <span class="step-card__desc">{{ s.desc }}</span>
        <span class="step-card__lock">{{ s.note }}</span>
    </div>
    {% endif %}
    {% endfor %}
  </div>
</section>
{# ---------- 系统管理 ---------- #}
{# 个人设置、账号与操作日志的入口都在左侧导航栏，工作台不再重复列出 #}
{% endblock %}
# ==== app/templates/auth/add_user.html ====
{% extends "base.html" %}
{% block title %}新增账号{% endblock %}
{% block content %}
<div class="card" style="max-width:560px;">
  <h2>新增账号</h2>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="username">账户名称</label
    >
      <input id="username" type="text" name="username"
    required
           maxlength="64" autocomplete="off"
    style="width:280px;">
    </div>
    <div class="field">
      <label class="field__label" for="password">密码</label>
      <input id="password" type="password" name="password"
    required
           autocomplete="new-password"
    style="width:280px;">
    </div>
    <div class="field">
      <label class="field__label" for="role">职务</label>
      <select id="role" name="role">
```

```
          {% for code, label in role_labels.items() %}                    7601
          <option value="{{ code }}" {% if code ==                        7602
      default_role %}selected{% endif %}>{{ label }}</option>             7603
          {% endfor %}                                                    7604
        </select>                                                         7605
        <p class="muted" style="margin:6px 0 0;">                         7606
          管理员可管理系统、维护基础数据并查看操作日志；操作员可使用全部业务功能。  7607
        </p>                                                              7608
      </div>                                                              7609
      <hr style="margin:22px 0;border:none;border-top:1px                 7610
      solid var(--line);">                                                7611
      <h3 style="margin:0 0 12px;font-size:15px;">个人资料（选填）</h3>   7612
      >                                                                   7613
      <div class="field">                                                 7614
        <label class="field__label" for="display_name">昵称</lab          7615
      el>                                                                 7616
        <input id="display_name" type="text"                              7617
      name="display_name" maxlength="64"                                  7618
               placeholder="留空则显示账户名称">                          7619
      </div>                                                              7620
      <div class="field">                                                 7621
        <label class="field__label" for="gender">性别</label>             7622
        <select id="gender" name="gender">                                7623
          <option value="">不填</option>                                  7624
          {% for g in genders %}<option value="{{ g }}">{{ g              7625
      }}</option>{% endfor %}                                             7626
        </select>                                                         7627
      </div>                                                              7628
      <div class="field">                                                 7629
        <label class="field__label" for="email">邮箱</label>              7630
        <input id="email" type="text" name="email"                        7631
      maxlength="128">                                                    7632
      </div>                                                              7633
      <div class="field">                                                 7634
        <label class="field__label" for="phone">手机号</label>            7635
        <input id="phone" type="text" name="phone"                        7636
      maxlength="32">                                                     7637
      </div>                                                              7638
      <p class="actions" style="margin-top:18px;">                        7639
        <button type="submit" class="btn">保存</button>                   7640
        <a class="btn btn--ghost" href="{{                                7641
      url_for('auth.profile', tab='users') }}">返回</a>                   7642
      </p>                                                                7643
    </form>                                                               7644
</div>                                                                    7645
{% endblock %}                                                            7646
# ==== app/templates/auth/edit_user.html ====                             7647
{% extends "base.html" %}                                                 7648
{% block title %}编辑账号 · {{ user.username }}{% endblock %}             7649
{% block content %}                                                       7650
```

```
<div class="card" style="max-width:560px;">
  <h2>编辑账号 · {{ user.username }}</h2>
  <h3 style="margin:0 0 12px;font-size:15px;">个人资料</h3>
  <p class="muted" style="margin:0 0 14px;font-size:12.5px;">
    以下内容用户也可在「个人设置」中自行修改，此处为管理员代改。
  </p>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="display_name">昵称</label>
      <input id="display_name" type="text" name="display_name" maxlength="64"
             value="{{ user.display_name or '' }}"
             placeholder="留空则显示账户名称">
    </div>
    <div class="field">
      <label class="field__label" for="gender">性别</label>
      <select id="gender" name="gender">
        <option value="">不填</option>
        {% for g in genders %}
        <option value="{{ g }}" {% if user.gender == g %}selected{% endif %}>{{ g }}</option>
        {% endfor %}
      </select>
    </div>
    <div class="field">
      <label class="field__label" for="email">邮箱</label>
      <input id="email" type="text" name="email" maxlength="128"
             value="{{ user.email or '' }}">
    </div>
    <div class="field">
      <label class="field__label" for="phone">手机号</label>
      <input id="phone" type="text" name="phone" maxlength="32"
             value="{{ user.phone or '' }}">
    </div>
    <hr style="margin:22px 0;border:none;border-top:1px solid var(--line);">
    <h3 style="margin:0 0 12px;font-size:15px;">职务与权限</h3>
    <p class="muted" style="margin:0 0 14px;font-size:12.5px;">
      职务只能由管理员设置：管理员可管理系统、维护基础数据并查看操作日志；
      操作员可使用全部业务功能。
    </p>
    <div class="field">
      <label class="field__label" for="role">职务</label>
      <select id="role" name="role">
```

```
          {% for code, label in role_labels.items() %}
          <option value="{{ code }}" {% if user.role == code
      %}selected{% endif %}>{{ label }}</option>
          {% endfor %}
        </select>
      </div>
      <div class="field">
        <label class="checkbox" for="active">
          <input id="active" type="checkbox" name="active"
      value="1"
                 {% if user.active %}checked{% endif %}>
          <span>启用该账号</span>
        </label>
        <p class="muted" style="margin:6px 0 0;">
          停用后该账号无法登录，但其历史操作记录会保留。
          系统至少需保留一名启用状态的管理员。
        </p>
      </div>
      <div class="field">
        <label class="field__label" for="password">重置密码（留空则不修改
      ）</label>
        <input id="password" type="password" name="password"
               autocomplete="new-password"
      style="width:280px;"
               placeholder="留空表示保持原密码">
      </div>
      <p class="actions" style="margin-top:18px;">
        <button type="submit" class="btn">保存</button>
        <a class="btn btn--ghost" href="{{
      url_for('auth.profile', tab='users') }}">返回</a>
      </p>
    </form>
</div>
{% endblock %}
# ==== app/templates/auth/profile.html ====
{% extends "base.html" %}
{% from "_macros.html" import icon %}
{% block title %}个人设置 · 历史遗留废弃矿山自然恢复适宜性评价系统{% endblock %}
{% block content %}
<style>
  /* ================== 个人设置（仿 Apple 账户页） ==================
     左栏是身份与分类，右栏是该分类下的功能卡。
     每张卡默认只显示当前值，点开才展开对应的修改表单 ——
     不点就不会误改，页面也不会一上来堆满输入框。 */
  .acct {
    display: grid;
    grid-template-columns: 244px 1fr;
    gap: 40px;
    align-items: start;
  }
```

```
@media (max-width: 900px) {
  .acct { grid-template-columns: 1fr; gap: 24px; }
}
/* ---------- 左栏 ---------- */
.acct-rail__avatar {
  display: grid;
  place-items: center;
  width: 88px;
  height: 88px;
  border-radius: 50%;
  background: #e9edeb;
  color: #97a09b;
  font-size: 32px;
  font-weight: 500;
  object-fit: cover;
}
.acct-rail__name {
  margin: 16px 0 2px;
  font-size: 16px;
  font-weight: 600;
}
.acct-rail__sub {
  margin: 0 0 24px;
  font-size: 13px;
  color: #86868b;
  word-break: break-all;
}
.acct-tabs {
  display: flex;
  flex-direction: column;
  gap: 2px;
  margin-bottom: 24px;
}
.acct-tabs a {
  padding: 7px 12px;
  border: 1px solid transparent;
  border-radius: 7px;
  font-size: 15px;
  color: #1d1d1f;
  text-decoration: none;
  transition: background-color .16s ease, border-color
  .16s ease;
}
.acct-tabs a:hover { background: #f5f5f7; }
.acct-tabs a.is-active {
  font-weight: 600;
  border-color: #bcc9c1;
  background: #fff;
}
/* 更换头像：折叠在左栏，点开才出现文件选择 */
```

```
.acct-avatar-edit > summary {
  display: inline-block;
  padding: 5px 12px;
  font-size: 13px;
  color: var(--accent);
  background: var(--accent-soft);
  border-radius: 7px;
  cursor: pointer;
  list-style: none;
}
.acct-avatar-edit > summary::-webkit-details-marker {
  display: none; }
.acct-avatar-edit > summary:hover { background: #dcebe2; }
.acct-avatar-edit__body { margin-top: 12px; }
.acct-avatar-edit__body .btn { width: 100%;
  margin-bottom: 8px; }
.acct-avatar-edit__file {
  margin: 0 0 10px;
  font-size: 12px;
  color: var(--ink-soft);
  word-break: break-all;
}
/* ---------- 右栏 ---------- */
.acct-title {
  margin: 0 0 8px;
  font-size: 26px;
  font-weight: 700;
  letter-spacing: -.3px;
}
.acct-lead {
  margin: 0 0 26px;
  font-size: 14px;
  color: #6e6e73;
}
/* 固定两列，与 Apple 账户页的 2×2 一致；
   卡片也宽一些，邮箱之类的长值不至于被拆成两行 */
.acct-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 16px;
  align-items: start;
}
@media (max-width: 760px) {
  .acct-grid { grid-template-columns: 1fr; }
}
/* 功能卡：details/summary 实现，点开才展开表单 */
.acct-card {
  background: #fff;
  border: 1px solid #e6e6e8;
  border-radius: 14px;
```

```
    transition: border-color .2s ease, box-shadow .2s ease;
  }
  .acct-card[open] {
    border-color: rgba(31, 107, 69, .45);
    box-shadow: 0 8px 22px rgba(19, 60, 43, .08);
  }
  .acct-card > summary {
    position: relative;
    display: block;
    padding: 18px 56px 18px 20px;
    border-radius: 14px;
    cursor: pointer;
    list-style: none;
  }
  .acct-card > summary::-webkit-details-marker { display:
    none; }
  .acct-card > summary:hover { background: #fafbfa; }
  .acct-card > summary:focus-visible {
    outline: 2px solid var(--accent);
    outline-offset: 2px;
  }
  .acct-card__title {
    display: block;
    font-size: 15px;
    font-weight: 600;
  }
  .acct-card__value {
    display: block;
    margin-top: 3px;
    font-size: 14px;
    color: #6e6e73;
    overflow-wrap: anywhere;
  }
  .acct-card__icon {
    position: absolute;
    top: 17px;
    right: 17px;
    display: grid;
    place-items: center;
    width: 27px;
    height: 27px;
    border-radius: 8px;
    background: var(--accent-soft);
    color: var(--accent);
  }
  .acct-card__body {
    padding: 4px 20px 20px;
    border-top: 1px solid #f0f1f2;
    margin-top: 2px;
  }
```

```
  .acct-card__body .field:first-child { margin-top: 16px; }
  .acct-card__body .btn { width: 100%; }
  /* 只读信息列表 */
  .acct-kv {
    display: grid;
    grid-template-columns: 88px 1fr;
    gap: 7px 12px;
    margin: 16px 0 0;
    font-size: 13.5px;
  }
  .acct-kv dt { color: var(--ink-soft); }
  .acct-kv dd { margin: 0; }
  .field__error {
    margin: 5px 0 0;
    font-size: 12.5px;
    color: var(--danger);
  }
  /* 管理员分类（账号 / 操作日志）里的表格面板 */
  .acct-panel {
    background: #fff;
    border: 1px solid #e6e6e8;
    border-radius: 14px;
    overflow: hidden;
  }
  .acct-panel__head {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 12px;
    padding: 13px 20px;
    border-bottom: 1px solid #f0f1f2;
  }
  .acct-panel__count {
    margin: 0;
    font-size: 13.5px;
    color: #6e6e73;
  }
  .acct-panel table { margin: 0; }
  .acct-panel th,
  .acct-panel td { border: 0; border-bottom: 1px solid
    #f0f1f2; }
  .acct-panel thead th { background: #f7f9f8; }
  .acct-panel tbody tr:last-child td { border-bottom: 0; }
  .acct-panel tbody tr:hover { background: #fafbfa; }
</style>
<div class="acct">
  {# ---------------- 左栏：身份 + 分类 ---------------- #}
  <aside class="acct-rail">
    {% if current_user.avatar %}
    <img class="acct-rail__avatar"
```

```
        src="{{ url_for('static', filename=current_user.ava
tar) }}"
        alt="{{ current_user.shown_name }} 的头像">
{% else %}
<span class="acct-rail__avatar" aria-hidden="true">{{
current_user.initial }}</span>
{% endif %}
<p class="acct-rail__name">{{ current_user.shown_name
}}</p>
<p class="acct-rail__sub">
  {{ role_labels.get(current_user.role,
current_user.role) }}
  {% if current_user.email %} · {{ current_user.email
}}{% endif %}
</p>
<nav class="acct-tabs" aria-label="个人设置分类">
  <a href="{{ url_for('auth.profile', tab='info') }}"
     {% if tab == 'info' %}class="is-active"
aria-current="page"{% endif %}>个人信息</a>
  <a href="{{ url_for('auth.profile', tab='security')
}}"
     {% if tab == 'security' %}class="is-active"
aria-current="page"{% endif %}>登录与安全性</a>
  {% if current_user.is_admin %}
  <a href="{{ url_for('auth.profile', tab='users') }}"
     {% if tab == 'users' %}class="is-active"
aria-current="page"{% endif %}>账号</a>
  <a href="{{ url_for('auth.profile', tab='logs') }}"
     {% if tab == 'logs' %}class="is-active"
aria-current="page"{% endif %}>操作日志</a>
  {% endif %}
</nav>
<details class="acct-avatar-edit">
  <summary>更换头像</summary>
  <div class="acct-avatar-edit__body">
    <form method="post" action="/profile/avatar"
          enctype="multipart/form-data">
      {{ avatar_form.hidden_tag() }}
      <input type="hidden" name="tab" value="{{ tab }}">
      <label class="visually-hidden"
for="avatar-file">选择头像图片</label>
      <input id="avatar-file" type="file" name="avatar"
             accept=".png,.jpg,.jpeg,.gif,.webp">
      <p class="acct-avatar-edit__file">PNG / JPG / GIF
/ WebP，不超过 4 MB</p>
      <button type="submit" class="btn
btn--sm">上传头像</button>
    </form>
    {% if current_user.avatar %}
    <form method="post" action="/profile/avatar/delete"
```

```
                onsubmit="return confirm('确定移除当前头像吗？');">
            {{ avatar_form.hidden_tag() }}
            <input type="hidden" name="tab" value="{{ tab }}">
            <button type="submit" class="btn btn--sm
    btn--ghost">移除头像</button>
        </form>
        {% endif %}
      </div>
    </details>
  </aside>
  {# ---------------- 右栏：功能卡 ---------------- #}
  <div class="acct-main">
    {% if tab == 'security' %}
    <h1 class="acct-title">登录与安全性</h1>
    <p class="acct-lead">管理与登录账号、账号安全有关的设置。</p>
    <div class="acct-grid">
      <details class="acct-card"
                {% if pwd_form.old_password.errors or
    pwd_form.new_password.errors or pwd_form.confirm.errors
    %}open{% endif %}>
        <summary>
          <span class="acct-card__title">密码</span>
          <span class="acct-card__value">修改登录密码</span>
          <span class="acct-card__icon">{{ icon("lock", 15)
    }}</span>
        </summary>
        <div class="acct-card__body">
          <form method="post">
            {{ pwd_form.hidden_tag() }}
            <input type="hidden" name="action"
    value="password">
            <input type="hidden" name="tab"
    value="security">
            <div class="field">
              <label class="field__label" for="{{
    pwd_form.old_password.id }}">当前密码</label>
              {{ pwd_form.old_password(id=pwd_form.old_passw
    ord.id,


    autocomplete="current-password") }}
              {% for e in pwd_form.old_password.errors %}
              <p class="field__error">{{ e }}</p>
              {% endfor %}
            </div>
            <div class="field">
              <label class="field__label" for="{{
    pwd_form.new_password.id }}">新密码</label>
              {{ pwd_form.new_password(id=pwd_form.new_passw
    ord.id,
```

```
autocomplete="new-password") }}                                    8051
            {% for e in pwd_form.new_password.errors %}            8052
            <p class="field__error">{{ e }}</p>                    8053
            {% endfor %}                                           8054
          </div>                                                   8055
          <div class="field">                                      8056
            <label class="field__label" for="{{                    8057
pwd_form.confirm.id }}">确认新密码</label>                          8058
            {{ pwd_form.confirm(id=pwd_form.confirm.id,            8059
                                autocomplete="new-password8060
") }}                                                              8061
            {% for e in pwd_form.confirm.errors %}                 8062
            <p class="field__error">{{ e }}</p>                    8063
            {% endfor %}                                           8064
          </div>                                                   8065
          <p class="actions">{{ pwd_form.submit_pwd(class_8066
="btn") }}</p>                                                     8067
        </form>                                                    8068
      </div>                                                       8069
    </details>                                                     8070
    <details class="acct-card">                                    8071
      <summary>                                                    8072
        <span class="acct-card__title">登录账号</span>             8073
        <span class="acct-card__value">{{                          8074
current_user.username }}</span>                                    8075
        <span class="acct-card__icon">{{ icon("user", 15)          8076
}}</span>                                                          8077
      </summary>                                                   8078
      <div class="acct-card__body">                                8079
        <dl class="acct-kv">                                       8080
          <dt>登录账号</dt><dd class="mono">{{                     8081
current_user.username }}</dd>                                      8082
          <dt>职务</dt>                                            8083
          <dd>{{ role_labels.get(current_user.role,                8084
current_user.role) }}</dd>                                         8085
          <dt>状态</dt>                                            8086
          <dd>                                                     8087
            {% if current_user.active %}                           8088
            <span class="tag tag--ok">启用</span>                  8089
            {% else %}                                             8090
            <span class="tag tag--warn">停用</span>                8091
            {% endif %}                                            8092
          </dd>                                                    8093
          <dt>创建时间</dt>                                        8094
          <dd class="mono">{{ current_user.created_at |            8095
localdt }}</dd>                                                    8096
        </dl>                                                      8097
      </div>                                                       8098
    </details>                                                     8099
</div>                                                             8100
```

```
{% elif tab == 'users' %}
<h1 class="acct-title">账号</h1>
<p class="acct-lead">管理本系统的账号、职务与启用状态。新注册的账号默认为「操作员」。</p>
{% include "auth/_users.html" %}
{% elif tab == 'logs' %}
<h1 class="acct-title">操作日志</h1>
<p class="acct-lead">记录各账号在本系统中的关键操作。</p>
{% include "auth/_logs.html" %}
{% else %}
<h1 class="acct-title">个人信息</h1>
<p class="acct-lead">管理你的昵称、性别以及可以联系到你的邮箱与手机号。</p>
<form method="post">
  {{ info_form.hidden_tag() }}
  <input type="hidden" name="action" value="info">
  <input type="hidden" name="tab" value="info">
  <div class="acct-grid">
    <details class="acct-card" {% if info_form.display_name.errors %}open{% endif %}>
      <summary>
        <span class="acct-card__title">昵称</span>
        <span class="acct-card__value">
          {{ current_user.display_name or "未设置" }}
        </span>
        <span class="acct-card__icon">{{ icon("user", 15) }}</span>
      </summary>
      <div class="acct-card__body">
        <div class="field">
          <label class="field__label" for="{{ info_form.display_name.id }}">
            昵称
          </label>
          {{ info_form.display_name(id=info_form.display_name.id,
                                    placeholder="留空则显示账户名称") }}
          {% for e in info_form.display_name.errors %}
          <p class="field__error">{{ e }}</p>
          {% endfor %}
        </div>
        <p class="actions">{{ info_form.submit_info(class_="btn") }}</p>
      </div>
    </details>
    <details class="acct-card" {% if info_form.gender.errors %}open{% endif %}>
      <summary>
        <span class="acct-card__title">性别</span>
        <span class="acct-card__value">{{
```

```
current_user.gender or "未设置" }}</span>                          8151
          <span class="acct-card__icon">{{ icon("user",          8152
15) }}</span>                                                     8153
        </summary>                                                8154
        <div class="acct-card__body">                             8155
          <div class="field">                                     8156
            <label class="field__label" for="{{                   8157
info_form.gender.id }}">性别</label>                              8158
            {{ info_form.gender(id=info_form.gender.id) }}        8159
            {% for e in info_form.gender.errors %}                8160
            <p class="field__error">{{ e }}</p>                   8161
            {% endfor %}                                          8162
          </div>                                                  8163
          <p class="actions">{{ info_form.submit_info(clas        8164
s_="btn") }}</p>                                                  8165
        </div>                                                    8166
      </details>                                                  8167
      <details class="acct-card" {% if                            8168
info_form.email.errors %}open{% endif %}>                         8169
        <summary>                                                 8170
          <span class="acct-card__title">邮箱</span>              8171
          <span class="acct-card__value">{{                       8172
current_user.email or "未设置" }}</span>                          8173
          <span class="acct-card__icon">{{ icon("mail",           8174
15) }}</span>                                                     8175
        </summary>                                                8176
        <div class="acct-card__body">                             8177
          <div class="field">                                     8178
            <label class="field__label" for="{{                   8179
info_form.email.id }}">邮箱</label>                               8180
            {{ info_form.email(id=info_form.email.id) }}          8181
            {% for e in info_form.email.errors %}                 8182
            <p class="field__error">{{ e }}</p>                   8183
            {% endfor %}                                          8184
          </div>                                                  8185
          <p class="actions">{{ info_form.submit_info(clas        8186
s_="btn") }}</p>                                                  8187
        </div>                                                    8188
      </details>                                                  8189
      <details class="acct-card" {% if                            8190
info_form.phone.errors %}open{% endif %}>                         8191
        <summary>                                                 8192
          <span class="acct-card__title">手机号</span>            8193
          <span class="acct-card__value">{{                       8194
current_user.phone or "未设置" }}</span>                          8195
          <span class="acct-card__icon">{{ icon("phone",          8196
15) }}</span>                                                     8197
        </summary>                                                8198
        <div class="acct-card__body">                             8199
          <div class="field">                                     8200
```

```
                    <label class="field__label" for="{{                8201
      info_form.phone.id }}">手机号</label>                            8202
                    {{ info_form.phone(id=info_form.phone.id) }}       8203
                    {% for e in info_form.phone.errors %}              8204
                    <p class="field__error">{{ e }}</p>                8205
                    {% endfor %}                                       8206
                  </div>                                               8207
                  <p class="actions">{{ info_form.submit_info(clas     8208
      s_="btn") }}</p>                                                 8209
              </div>                                                   8210
            </details>                                                 8211
          </div>                                                       8212
      </form>                                                          8213
      {% endif %}                                                      8214
    </div>                                                             8215
</div>                                                                 8216
{% endblock %}                                                         8217
{% block scripts %}                                                    8218
<script>                                                               8219
// 同时只展开一张卡，避免点开多张后页面被撑得很长                      8220
(function () {                                                         8221
  var cards = document.querySelectorAll('.acct-card');                 8222
  Array.prototype.forEach.call(cards, function (card) {                8223
    card.addEventListener('toggle', function () {                      8224
      if (!card.open) return;                                          8225
      Array.prototype.forEach.call(cards, function (other) {           8226
        if (other !== card && other.open) other.open =                 8227
    false;                                                             8228
      });                                                              8229
    });                                                                8230
  });                                                                  8231
})();                                                                  8232
</script>                                                              8233
{% endblock %}                                                         8234
# ==== app/templates/auth/_users.html ====                             8235
{# 管理员的「账号」分类。由 auth/profile.html 内联引入，不单独成页。 #}  8236
<div class="acct-panel">                                               8237
  <div class="acct-panel__head">                                       8238
    <p class="acct-panel__count">共 {{ users|length }}                 8239
    个账号</p>                                                         8240
    <a class="btn btn--sm" href="{{ url_for('auth.add_user')           8241
    }}">新增账号</a>                                                  8242
  </div>                                                               8243
  <table>                                                              8244
    <thead>                                                            8245
      <tr>                                                             8246
        <th scope="col">ID</th>                                        8247
        <th scope="col">账户名称</th>                                  8248
        <th scope="col">昵称</th>                                      8249
        <th scope="col">职务</th>                                      8250
```

```
      <th scope="col">状态</th>                                          8251
      <th scope="col">操作</th>                                          8252
    </tr>                                                                8253
</thead>                                                                 8254
<tbody>                                                                  8255
    {% for u in users %}                                                 8256
    <tr>                                                                 8257
      <td class="num">{{ u.id }}</td>                                    8258
      <td class="mono">                                                  8259
        {{ u.username }}                                                 8260
        {% if u.id == current_user.id %}<span                            8261
class="muted">（当前登录）</span>{% endif %}                               8262
      </td>                                                              8263
      <td style="text-align:left;">                                      8264
        {% if u.display_name %}{{ u.display_name }}                      8265
        {% else %}<span class="muted">—</span>{% endif %}                8266
      </td>                                                              8267
      <td>                                                               8268
        <span class="tag {{ 'tag--ok' if u.role ==                       8269
'admin' else 'tag--muted' }}">                                           8270
          {{ role_labels.get(u.role, u.role) }}                          8271
        </span>                                                          8272
      </td>                                                              8273
      <td>                                                               8274
        {% if u.active %}<span class="tag                                8275
tag--ok">启用</span>                                                     8276
        {% else %}<span class="tag tag--muted">停用</span>{%             8277
endif %}                                                                 8278
      </td>                                                              8279
      <td class="nowrap">                                                8280
        <span class="row-actions">                                       8281
          <a class="link-btn" href="{{                                   8282
url_for('auth.edit_user', user_id=u.id) }}">编辑</a>                     8283
          <form method="post"                                            8284
                action="{{ url_for('auth.delete_user',                   8285
user_id=u.id) }}"                                                        8286
                onsubmit="return confirm('确定删除账号「{{               8287
u.username }}」吗？\n该操作无法恢复。');">                               8288
            {{ form.hidden_tag() }}                                      8289
            <button type="submit" class="link-btn                        8290
link-btn--danger"                                                        8291
                    {% if u.id == current_user.id                        8292
%}disabled                                                               8293
                    title="不能删除当前登录的账号"{% endif               8294
%}>删除</button>                                                         8295
          </form>                                                        8296
        </span>                                                          8297
      </td>                                                              8298
    </tr>                                                                8299
    {% endfor %}                                                         8300
```

```
      </tbody>
    </table>
</div>
# ==== app/templates/auth/_logs.html ====
{# 管理员的「操作日志」分类。由 auth/profile.html 内联引入，不单独成页。 #}
<div class="acct-panel">
  <div class="acct-panel__head">
    <p class="acct-panel__count">
      共 {{ total }} 条记录，当前第 {{ page }} / {{ pages }} 页
    </p>
    {% if pages > 1 %}
    <nav class="row-actions" aria-label="日志分页">
      {% if page > 1 %}
      <a class="link-btn"
         href="{{ url_for('auth.profile', tab='logs',
    page=page - 1) }}">上一页</a>
      {% endif %}
      {% if page < pages %}
      <a class="link-btn"
         href="{{ url_for('auth.profile', tab='logs',
    page=page + 1) }}">下一页</a>
      {% endif %}
    </nav>
    {% endif %}
  </div>
  {% if not records %}
  <p class="muted" style="padding:18px 20px;">暂无操作记录。</p>
  {% else %}
  <table>
    <thead>
      <tr>
        <th scope="col">时间</th>
        <th scope="col">操作人</th>
        <th scope="col">操作</th>
        <th scope="col">对象</th>
        <th scope="col">说明</th>
      </tr>
    </thead>
    <tbody>
      {% for r in records %}
      <tr>
        <td class="mono nowrap">{{ r.created_at |
    localdt("%Y-%m-%d %H:%M:%S") }}</td>
        <td>{{ users_map.get(r.user_id, "—") }}</td>
        <td style="text-align:left;">{{ r.action }}</td>
        <td>
          {% if r.target_type %}{{ r.target_type }}{% if
    r.target_id %}#{{ r.target_id }}{% endif %}
          {% else %}—{% endif %}
        </td>
```

```
            <td style="text-align:left;">{{ r.detail or "—"
        }}</td>
          </tr>
          {% endfor %}
        </tbody>
      </table>
      {% endif %}
</div>
# ==== app/templates/project/index.html ====
{% extends "base.html" %}
{% block title %}项目与矿山档案{% endblock %}
{% block content %}
<div class="card">
  <h2>项目与矿山档案</h2>
  <p class="actions" style="margin-bottom:12px;">
    <a class="btn btn--sm" href="/project/add">新建项目</a>
  </p>
  {% if not projects %}
  <p class="muted">
    暂无项目。点击上方「新建项目」开始建立评价项目，
    或运行 <span class="mono">python seed_demo.py</span>
    载入论文案例。
  </p>
  {% else %}
  <table>
    <caption class="muted">共 {{ projects|length }}
    个项目</caption>
    <thead>
      <tr>
        <th scope="col">项目名称</th>
        <th scope="col">编号</th>
        <th scope="col">状态</th>
        <th scope="col">图斑数</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for p in projects %}
      <tr>
        <td style="text-align:left;">{{ p.name }}</td>
        <td class="mono">{{ p.code or "—" }}</td>
        <td><span class="tag tag--muted">{{ p.status
    }}</span></td>
        <td class="num">
          {% if p.patches.count() %}
            {{ p.patches.count() }}
          {% else %}
            <span class="tag tag--warn">0</span>
          {% endif %}
        </td>
```

```
            <td class="nowrap">
              <span class="row-actions">
                <a class="link-btn" href="/project/{{ p.id
}}/patches">{% if p.patches.count() %}矿山图斑{% else %}+
新建图斑{% endif %}</a>
                <a class="link-btn" href="/project/{{ p.id
}}/edit">编辑</a>
                <form method="post" action="/project/{{ p.id
}}/delete"
                      onsubmit="return confirm('确定删除项目「{{
p.name }}」吗？\n该项目下的图斑、微地质单元与评价数据将一并删除，且无法恢复。');">
                  {{ form.hidden_tag() }}
                  <button type="submit" class="link-btn
link-btn--danger">删除</button>
                </form>
              </span>
            </td>
          </tr>
          {% endfor %}
        </tbody>
      </table>
      {% endif %}
</div>
{% endblock %}
# ==== app/templates/project/edit.html ====
{% extends "base.html" %}
{% block title %}{{ "编辑项目" if project else "新建项目" }}{%
    endblock %}
{% block content %}
<div class="card" style="max-width:640px;">
  <h2>{{ "编辑项目" if project else "新建项目" }}</h2>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="name">项目名称</label>
      <input id="name" type="text" name="name"
    style="width:100%;" required
             value="{{ project.name if project else '' }}">
    </div>
    <div class="field">
      <label class="field__label" for="code">项目编号</label>
      <input id="code" type="text" name="code"
    style="width:240px;"
             value="{{ project.code or '' if project else
    '' }}">
    </div>
    {% if project %}
    <div class="field">
      <label class="field__label" for="status">状态</label>
      <select id="status" name="status">
```

```
          {% for s in ["进行中", "已完成", "已归档"] %}                          8451
          <option value="{{ s }}" {% if project.status == s                    8452
      %}selected{% endif %}>{{ s }}</option>                                   8453
          {% endfor %}                                                         8454
        </select>                                                              8455
      </div>                                                                   8456
      {% endif %}                                                              8457
      <div class="field">                                                      8458
        <label class="field__label" for="description">项目说明</la             8459
      bel>                                                                     8460
        <textarea id="description" name="description">{{                     8461
      project.description or '' if project else ''                             8462
      }}</textarea>                                                            8463
      </div>                                                                   8464
      <p class="actions" style="margin-top:18px;">                             8465
        <button type="submit" class="btn">保存</button>                        8466
        <a class="btn btn--ghost" href="/project/">返回</a>                    8467
      </p>                                                                     8468
    </form>                                                                    8469
</div>                                                                         8470
{% endblock %}                                                                 8471
# ==== app/templates/project/patches.html ====                                 8472
{% extends "base.html" %}                                                      8473
{% block title %}矿山图斑 · {{ project.name }}{% endblock %}                   8474
{% block content %}                                                            8475
<div class="card">                                                             8476
  <h2>矿山图斑 · {{ project.name }}</h2>                                       8477
  <p class="actions" style="margin-bottom:12px;">                              8478
    <a class="btn btn--sm" href="/project/{{ project.id                        8479
    }}/patch/add">新建图斑</a>                                                 8480
    <a class="btn btn--sm btn--ghost"                                          8481
    href="/project/">返回项目列表</a>                                          8482
  </p>                                                                         8483
  {% if not project.patches.count() %}                                         8484
  <p class="muted">该项目下暂无图斑。点击上方「新建图斑」建立第一个矿山图斑。</p>  8485
  {% else %}                                                                   8486
  <table>                                                                      8487
    <caption class="muted">共 {{ project.patches.count() }}                    8488
    个图斑</caption>                                                           8489
    <thead>                                                                    8490
      <tr>                                                                     8491
        <th scope="col">图斑名称</th>                                          8492
        <th scope="col">行政区</th>                                            8493
        <th scope="col">面积（公顷）</th>                                      8494
        <th scope="col">开采矿种</th>                                          8495
        <th scope="col">微地质单元</th>                                        8496
        <th scope="col">操作</th>                                              8497
      </tr>                                                                    8498
    </thead>                                                                   8499
    <tbody>                                                                    8500
```

```
        {% for p in project.patches %}                                    8501
        <tr>                                                              8502
          <td style="text-align:left;">{{ p.name }}</td>                  8503
          <td>{{ p.admin_region or "—" }}</td>                            8504
          <td class="num">{{ p.area_ha if p.area_ha is not                8505
      none else "—" }}</td>                                               8506
          <td>{{ p.mineral_type or "—" }}</td>                            8507
          <td class="num">{{ p.units.count() }}</td>                       8508
          <td class="nowrap">                                             8509
            <span class="row-actions">                                    8510
              <a class="link-btn" href="/unit/{{ p.id                     8511
      }}">微地质单元</a>                                                  8512
              <a class="link-btn" href="/constraint/{{ p.id               8513
      }}">限制性判定</a>                                                  8514
              <a class="link-btn" href="/project/patch/{{                 8515
      p.id }}/edit">编辑</a>                                              8516
              <form method="post" action="/project/patch/{{               8517
      p.id }}/delete"                                                     8518
                    onsubmit="return confirm('确定删除图斑「{{            8519
      p.name }}」吗？\n该图斑的微地质单元、指标数据与评价结果将一并删除，且无法恢复。');">  8520
                {{ form.hidden_tag() }}                                   8521
                <button type="submit" class="link-btn                     8522
      link-btn--danger">删除</button>                                     8523
              </form>                                                     8524
            </span>                                                       8525
          </td>                                                           8526
        </tr>                                                             8527
        {% endfor %}                                                      8528
      </tbody>                                                            8529
    </table>                                                              8530
    {% endif %}                                                           8531
</div>                                                                    8532
{% endblock %}                                                            8533
# ==== app/templates/project/patch_edit.html ====                         8534
{% extends "base.html" %}                                                 8535
{% block title %}{{ "编辑图斑" if patch else "新建图斑" }}{%              8536
     endblock %}                                                          8537
{% block content %}                                                       8538
<div class="card" style="max-width:720px;">                               8539
    <h2>{{ "编辑图斑" if patch else "新建图斑" }} · {{ project.name       8540
      }}</h2>                                                             8541
    <form method="post">                                                  8542
      {{ form.hidden_tag() }}                                             8543
      <div class="field">                                                 8544
        <label class="field__label" for="patch-name">图斑名称</lab        8545
      el>                                                                 8546
        <input id="patch-name" type="text" name="name"                    8547
      style="width:100%;" required                                        8548
               value="{{ patch.name if patch else '' }}">                 8549
      </div>                                                              8550
```

```
<div class="field">
  <label class="field__label" for="admin-region">所在行政区</label>
  <input id="admin-region" type="text" name="admin_region" style="width:100%;"
         value="{{ patch.admin_region or '' if patch else '' }}">
</div>
<div class="field">
  <label class="field__label" for="area-ha">图斑面积（公顷）</label>
  <input id="area-ha" type="number" step="any" name="area_ha"
         value="{{ patch.area_ha if patch and patch.area_ha is not none else '' }}">
</div>
<div class="field">
  <label class="field__label" for="mineral-type">开采矿种</label>
  <input id="mineral-type" type="text" name="mineral_type" style="width:240px;"
         value="{{ patch.mineral_type or '' if patch else '' }}">
</div>
<div class="field">
  <label class="field__label" for="mining-method">开采方式</label>
  <input id="mining-method" type="text" name="mining_method" style="width:240px;"
         value="{{ patch.mining_method or '' if patch else '' }}">
</div>
<div class="field">
  <label class="field__label" for="stop-years">停产年限</label>
  <input id="stop-years" type="number" step="any" name="stop_years"
         value="{{ patch.stop_years if patch and patch.stop_years is not none else '' }}">
</div>
<div class="field">
  <label class="checkbox" for="responsible-missing">
    <input id="responsible-missing" type="checkbox" name="responsible_entity_missing" value="1"
           {% if not patch or patch.responsible_entity_missing %}checked{% endif %}>
    <span>责任主体缺失</span>
  </label>
</div>
<h3>参照生态系统（指标标准化基准）</h3>
```

```
      <p class="muted">                                                    8601
        论文 4.2.2 节取值：植被覆盖度 75%、Shannon-Wiener 多样性指数 0.7。  8602
        该基准与图斑绑定管理，可按不同参照生态系统分别设定。                8603
      </p>                                                                 8604
      <div class="field">                                                  8605
        <label class="field__label" for="coverage-benchmark">植            8606
      被覆盖度基准（%）</label>                                            8607
        <input id="coverage-benchmark" type="number"                       8608
      step="any" name="coverage_benchmark"                                 8609
               value="{{ reference.coverage_benchmark if                   8610
      reference else 75 }}">                                               8611
      </div>                                                               8612
      <div class="field">                                                  8613
        <label class="field__label" for="shannon-benchmark">Sh             8614
      annon-Wiener 多样性指数基准</label>                                  8615
        <input id="shannon-benchmark" type="number"                        8616
      step="any" name="shannon_benchmark"                                  8617
               value="{{ reference.shannon_benchmark if                    8618
      reference else 0.7 }}">                                              8619
      </div>                                                               8620
      <div class="field">                                                  8621
        <label class="field__label" for="dominant-species">建群种          8622
      </label>                                                             8623
        <input id="dominant-species" type="text"                           8624
      name="dominant_species" style="width:100%;"                          8625
               value="{{ reference.dominant_species or '' if               8626
      reference else '' }}">                                               8627
      </div>                                                               8628
      <p class="actions" style="margin-top:18px;">                         8629
        <button type="submit" class="btn">保存</button>                    8630
        <a class="btn btn--ghost" href="/project/{{                        8631
      project.id }}/patches">返回</a>                                      8632
      </p>                                                                 8633
    </form>                                                                8634
</div>                                                                     8635
{% if patch %}                                                             8636
<div class="card" style="max-width:720px;">                                8637
    <h2>遥感影像挂接</h2>                                                  8638
    <p class="muted">                                                      8639
      挂接遥感底图或无人机正射影像，用于辅助微地质单元划分与植被覆盖度判读。  8640
      支持 PNG、JPG、GIF、WebP、BMP、TIFF，单个文件不超过 32 MB。          8641
    </p>                                                                   8642
    {% if patch.image_path %}                                              8643
    <figure style="margin:0 0 16px;">                                      8644
      <img src="{{ url_for('static', filename=patch.image_path             8645
      ) }}"                                                                8646
           alt="图斑「{{ patch.name }}」已挂接的遥感影像"                  8647
           style="max-width:100%;border:1px solid                          8648
      var(--line);border-radius:6px;">                                     8649
      <figcaption class="muted" style="margin-top:6px;">                   8650
```

```
        {{ patch.image_note or "遥感影像" }}                                8651
        {% if patch.image_uploaded_at %}                                    8652
        · 上传于 {{ patch.image_uploaded_at | localdt }}                    8653
        {% endif %}                                                         8654
      </figcaption>                                                         8655
    </figure>                                                               8656
    <form method="post" action="/project/patch/{{ patch.id                  8657
      }}/image/delete"                                                      8658
          onsubmit="return confirm('确定移除该遥感影像吗？');">             8659
      {{ form.hidden_tag() }}                                               8660
      <button type="submit" class="btn btn--sm                              8661
      btn--danger">移除影像</button>                                        8662
    </form>                                                                 8663
    <hr style="margin:16px 0;border:none;border-top:1px solid               8664
      var(--line);">                                                        8665
    {% endif %}                                                             8666
    <form method="post" action="/project/patch/{{ patch.id                  8667
      }}/image"                                                             8668
          enctype="multipart/form-data">                                    8669
      {{ form.hidden_tag() }}                                               8670
      <div class="field">                                                   8671
        <label class="field__label" for="image-file">                       8672
          {{ "替换影像文件" if patch.image_path else "选择影像文件" }}      8673
        </label>                                                            8674
        <input id="image-file" type="file" name="image"                     8675
               accept=".png,.jpg,.jpeg,.gif,.webp,.bmp,.tif,.t              8676
      iff" required>                                                        8677
      </div>                                                                8678
      <div class="field">                                                   8679
        <label class="field__label" for="image-note">影像说明（如来源、     8680
      时相）</label>                                                        8681
        <input id="image-note" type="text" name="image_note"                8682
      style="width:100%;"                                                   8683
               placeholder="如：2025 年 6 月无人机正射影像"                 8684
               value="{{ patch.image_note or '' }}">                        8685
      </div>                                                                8686
      <p class="actions">                                                   8687
        <button type="submit" class="btn">{{ "替换影像" if                 8688
      patch.image_path else "上传并挂接" }}</button>                        8689
      </p>                                                                  8690
    </form>                                                                 8691
</div>                                                                      8692
{% endif %}                                                                 8693
{% endblock %}                                                              8694
# ==== app/templates/unit/list.html ====                                    8695
{% extends "base.html" %}                                                   8696
{% block title %}微地质单元{% endblock %}                                   8697
{% block content %}                                                         8698
<div class="card">                                                          8699
  <h2>微地质单元</h2>                                                       8700
```

```
  {% if not rows %}
  <p class="muted">
    暂无矿山图斑。请先到「项目与矿山档案」建立图斑。
  </p>
  {% else %}
  <table>
    <thead>
      <tr>
        <th scope="col">所属项目</th>
        <th scope="col">矿山图斑</th>
        <th scope="col">已划分单元</th>
        <th scope="col">占比合计</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for r in rows %}
      <tr>
        <td>{{ r.patch.project.name if r.patch.project else
    "—" }}</td>
        <td style="text-align:left;">{{ r.patch.name }}</td>
        <td class="num">{{ r.count }} 个</td>
        <td class="num">
          {% if r.count %}
            {{ "%.2f%%"|format(r.ratio_sum * 100) }}
            {% if r.ratio_sum < 0.999 or r.ratio_sum >
    1.001 %}
              <span class="warn">（不等于 100%）</span>
            {% endif %}
          {% else %}—{% endif %}
        </td>
        <td class="nowrap">
          <span class="row-actions">
            <a class="link-btn" href="/unit/{{ r.patch.id
    }}">划分与维护</a>
            <a class="link-btn" href="/indicator/">录入指标数据</a
    >
          </span>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/unit/index.html ====
{% extends "base.html" %}
{% block title %}微地质单元 · {{ patch.name }}{% endblock %}
{% block content %}
```

```
<div class="card">
  <h2>微地质单元 · {{ patch.name }}</h2>
  {% if units %}
  <table>
    <caption class="muted">
      共 {{ units|length }} 个单元；
      面积合计 <span class="num">{{ "%.4f"|format(total_area)
    }}</span>；
      占比合计 <span class="num">{{ "%.2f%%"|format(ratio_sum *
    100) }}</span>
      {% if ratio_sum and (ratio_sum < 0.999 or ratio_sum >
    1.001) %}
      <span class="warn">（占比合计不等于 100%，请检查面积录入）</span>
      {% endif %}
    </caption>
    <thead>
      <tr>
        <th scope="col">微地质单元</th>
        <th scope="col">面积</th>
        <th scope="col">面积占比</th>
        <th scope="col">说明</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for u in units %}
      <tr>
        <td>{{ u.unit_type }}</td>
        <td class="num">{{ u.area if u.area is not none
    else "—" }}</td>
        <td class="num">
          {% if u.ratio is not none %}{{
    "%.2f%%"|format(u.ratio * 100) }}{% else %}—{% endif %}
        </td>
        <td style="text-align:left;">{{ u.description or
    "—" }}</td>
        <td class="nowrap">
          <span class="row-actions">
            <a class="link-btn" href="/unit/edit/{{ u.id
    }}">编辑</a>
            <form method="post" action="/unit/delete/{{
    u.id }}"
                  onsubmit="return confirm('确定删除微地质单元「{{
    u.unit_type }}」吗？\n该单元已录入的指标数据将一并删除，且无法恢复。');">
              {{ form.hidden_tag() }}
              <button type="submit" class="link-btn
    link-btn--danger">删除</button>
            </form>
          </span>
        </td>
```

```
        </tr>                                                                    8801
        {% endfor %}                                                             8802
      </tbody>                                                                   8803
    </table>                                                                     8804
    {% else %}                                                                   8805
    <p class="muted">尚未划分微地质单元。请在下方添加。</p>                                  8806
    {% endif %}                                                                  8807
    <h3>添加微地质单元</h3>                                                          8808
    <form method="post" action="/unit/{{ patch.id }}/add">                       8809
      {{ form.hidden_tag() }}                                                    8810
      <div class="actions">                                                      8811
        <label class="field__label" for="add-unit-type"                          8812
      style="margin:0;">类型</label>                                               8813
        <select id="add-unit-type" name="unit_type">                             8814
          {% for t in unit_types %}<option value="{{ t }}">{{                    8815
      t }}</option>{% endfor %}                                                  8816
        </select>                                                                8817
        <label class="field__label" for="add-unit-area"                          8818
      style="margin:0;">面积</label>                                               8819
        <input id="add-unit-area" type="number" step="any"                       8820
      name="area">                                                               8821
        <label class="field__label" for="add-unit-desc"                          8822
      style="margin:0;">说明</label>                                               8823
        <input id="add-unit-desc" type="text"                                    8824
      name="description" style="width:220px;">                                   8825
        <button type="submit" class="btn btn--sm">添加</button>                  8826
      </div>                                                                     8827
    </form>                                                                      8828
    <p class="actions" style="margin-top:20px;">                                 8829
      <a class="btn btn--sm btn--ghost" href="/project/{{                        8830
      patch.project_id }}/patches">返回图斑列表</a>                                  8831
      <a class="btn btn--sm btn--ghost"                                          8832
      href="/indicator/">去录入指标数据</a>                                          8833
    </p>                                                                         8834
</div>                                                                           8835
{% endblock %}                                                                   8836
# ==== app/templates/unit/edit.html ====                                         8837
{% extends "base.html" %}                                                        8838
{% block title %}编辑微地质单元{% endblock %}                                        8839
{% block content %}                                                              8840
<div class="card">                                                               8841
    <h2>编辑微地质单元 · {{ patch.name }}</h2>                                       8842
    <p class="muted">修改面积后，系统会自动重算本图斑各单元的面积占比。</p>                    8843
    <form method="post">                                                         8844
      {{ form.hidden_tag() }}                                                    8845
      <div class="field">                                                        8846
        <label class="field__label" for="unit-type">微地质单元类型</l 8847
      abel>                                                                      8848
        <select id="unit-type" name="unit_type">                                 8849
          {% for t in unit_types %}                                              8850
```

```
          <option value="{{ t }}" {% if unit.unit_type == t     8851
      %}selected{% endif %}>{{ t }}</option>                    8852
          {% endfor %}                                          8853
        </select>                                               8854
      </div>                                                    8855
      <div class="field">                                       8856
        <label class="field__label" for="unit-area">面积</label>8857
        <input id="unit-area" type="number" step="any"          8858
      name="area"                                               8859
               value="{{ unit.area if unit.area is not none     8860
      else '' }}">                                              8861
      </div>                                                    8862
      <div class="field">                                       8863
        <label class="field__label" for="unit-desc">说明</label>8864
        <input id="unit-desc" type="text" name="description"    8865
      style="width:420px;"                                      8866
               value="{{ unit.description or '' }}">           8867
      </div>                                                    8868
      <p class="actions" style="margin-top:16px;">              8869
        <button type="submit" class="btn">保存</button>         8870
        <a class="btn btn--ghost" href="/unit/{{                8871
      unit.patch_id }}">返回</a>                                8872
      </p>                                                      8873
    </form>                                                     8874
</div>                                                          8875
{% endblock %}                                                  8876
#  ==== app/templates/standard/index.html ====                  8877
{% extends "base.html" %}                                       8878
{% block title %}政策与标准知识库{% endblock %}                  8879
{% block content %}                                             8880
<div class="card">                                              8881
    <h2>政策与标准知识库</h2>  <p class="actions"               8882
      style="margin-bottom:14px;">                              8883
      <a class="btn btn--sm" href="/standard/add">添加标准</a>   8884
      <a class="btn btn--sm btn--ghost"                         8885
      href="/standard/compare">省际口径比对</a>                 8886
    </p>                                                        8887
    <h3>地方标准</h3>                                           8888
    {% if local %}                                              8889
    <table>                                                     8890
      <thead>                                                   8891
        <tr>                                                    8892
          <th scope="col">省份</th>                             8893
          <th scope="col">名称</th>                             8894
          <th scope="col">编号</th>                             8895
          <th scope="col">规则数</th>                           8896
          <th scope="col">操作</th>                             8897
        </tr>                                                   8898
      </thead>                                                  8899
      <tbody>                                                   8900
```

```
        {% for s in local %}                                                    8901
        <tr>                                                                    8902
          <td>{{ s.province }}</td>                                             8903
          <td style="text-align:left;">{{ s.name }}</td>                        8904
          <td class="mono">{{ s.doc_number or "—" }}</td>                       8905
          <td class="num">{{ s.rules|length }}</td>                             8906
          <td class="nowrap">                                                   8907
            <span class="row-actions">                                          8908
              <a class="link-btn" href="/standard/{{ s.id                       8909
    }}/rules">规则</a>                                                        8910
              <a class="link-btn" href="/standard/{{ s.id                       8911
    }}/edit">编辑</a>                                                         8912
              <form method="post" action="/standard/{{ s.id                     8913
    }}/delete"                                                                  8914
                    onsubmit="return confirm('确定删除「{{                      8915
    s.province }} {{ s.name }}」吗？\n该标准下的 {{ s.rules|length              8916
    }} 条规则将一并删除，且无法恢复。');">                                      8917
                {{ form.hidden_tag() }}                                         8918
                <button type="submit" class="link-btn                           8919
    link-btn--danger">删除</button>                                           8920
              </form>                                                           8921
            </span>                                                             8922
          </td>                                                                 8923
        </tr>                                                                   8924
        {% endfor %}                                                            8925
      </tbody>                                                                  8926
    </table>                                                                    8927
    {% else %}                                                                  8928
    <p class="muted">暂无地方标准。运行 <span class="mono">python              8929
      seed_demo.py</span> 可载入论文表 1 的 11 项标准与政策文件。</p>          8930
    {% endif %}                                                                 8931
    <h3>政策文件</h3>                                                           8932
    {% if policy %}                                                             8933
    <table>                                                                     8934
      <thead>                                                                   8935
        <tr>                                                                    8936
          <th scope="col">省份</th>                                             8937
          <th scope="col">名称</th>                                             8938
          <th scope="col">文号</th>                                             8939
          <th scope="col">规则数</th>                                           8940
          <th scope="col">操作</th>                                             8941
        </tr>                                                                   8942
      </thead>                                                                  8943
      <tbody>                                                                   8944
        {% for s in policy %}                                                   8945
        <tr>                                                                    8946
          <td>{{ s.province }}</td>                                             8947
          <td style="text-align:left;">{{ s.name }}</td>                        8948
          <td class="mono">{{ s.doc_number or "—" }}</td>                       8949
          <td class="num">{{ s.rules|length }}</td>                             8950
```

```
            <td class="nowrap">
              <span class="row-actions">
                <a class="link-btn" href="/standard/{{ s.id
}}/rules">规则</a>
                <a class="link-btn" href="/standard/{{ s.id
}}/edit">编辑</a>
                <form method="post" action="/standard/{{ s.id
}}/delete"
                      onsubmit="return confirm('确定删除「{{
s.province }} {{ s.name }}」吗？\n该文件下的 {{ s.rules|length
}} 条规则将一并删除，且无法恢复。');">
                  {{ form.hidden_tag() }}
                  <button type="submit" class="link-btn
link-btn--danger">删除</button>
                </form>
              </span>
            </td>
          </tr>
          {% endfor %}
        </tbody>
      </table>
      {% else %}
      <p class="muted">暂无政策文件。</p>
      {% endif %}
    </div>
    {% endblock %}
    # ==== app/templates/standard/edit.html ====
    {% extends "base.html" %}
    {% block title %}{{ "编辑标准" if standard else "添加标准" }}{%
endblock %}
    {% block content %}
    <div class="card" style="max-width:800px;">
      <h2>{{ "编辑标准" if standard else "添加标准" }}</h2>
      <form method="post">
        {{ form.hidden_tag() }}
        <div class="field">
          <label class="field__label" for="province">省份</label>
          <input id="province" type="text" name="province"
style="width:160px;" required
                 value="{{ standard.province if standard else
'' }}">
        </div>
        <div class="field">
          <label class="field__label" for="std-name">名称</label>
          <input id="std-name" type="text" name="name"
style="width:100%;" required
                 value="{{ standard.name if standard else ''
}}">
        </div>
        <div class="field">
```

```
  <label class="field__label" for="doc-number">标准编号 /
文号</label>
  <input id="doc-number" type="text" name="doc_number"
style="width:320px;"
         value="{{ standard.doc_number or '' if
standard else '' }}">
</div>
<div class="field">
  <label class="field__label" for="category">类别</label>
  <select id="category" name="category">
    {% for c in ["地方标准", "政策文件"] %}
    <option value="{{ c }}" {% if standard and
standard.category == c %}selected{% endif %}>{{ c
}}</option>
    {% endfor %}
  </select>
</div>
<div class="field">
  <label class="field__label" for="publish-date">发布时间</l
abel>
  <input id="publish-date" type="text"
name="publish_date" style="width:200px;"
         placeholder="如 2023-03-30"
         value="{{ standard.publish_date or '' if
standard else '' }}">
</div>
<div class="field">
  <label class="field__label" for="implement-date">实施时间<
/label>
  <input id="implement-date" type="text"
name="implement_date" style="width:200px;"
         placeholder="如 2023-04-30（未明确可留空）"
         value="{{ standard.implement_date or '' if
standard else '' }}">
</div>
<div class="field">
  <label class="field__label" for="summary">特点 /
判定标准</label>
  <textarea id="summary" name="summary"
style="min-height:96px;">{{ standard.summary or '' if
standard else '' }}</textarea>
</div>
<p class="actions" style="margin-top:18px;">
  <button type="submit" class="btn">保存</button>
  {% if standard %}
  <form method="post" action="/standard/{{ standard.id
}}/delete" style="display:inline;"
        onsubmit="return confirm('确定删除「{{
standard.province }} {{ standard.name
}}」吗？\n该标准下的规则将一并删除，且无法恢复。');">
```

```
          {{ form.hidden_tag() }}                                    9051
          <button type="submit" class="btn                           9052
      btn--danger">删除该标准</button>                                9053
        </form>                                                      9054
        {% endif %}                                                  9055
        <a class="btn btn--ghost" href="/standard/">返回</a>         9056
      </p>                                                           9057
    </form>                                                          9058
</div>                                                               9059
{% endblock %}                                                       9060
# ==== app/templates/standard/rules.html ====                        9061
{% extends "base.html" %}                                            9062
{% block title %}结构化规则 · {{ standard.province }}{% endblock     9063
    %}                                                               9064
{% block content %}                                                  9065
<div class="card">                                                   9066
  <h2>结构化规则 · {{ standard.province }} {{ standard.name          9067
    }}</h2>                                                          9068
  {% if rules %}                                                     9069
  <table>                                                            9070
    <caption class="muted">共 {{ rules|length }}                     9071
    条规则</caption>                                                 9072
    <thead>                                                          9073
      <tr>                                                           9074
        <th scope="col">判定项</th>                                  9075
        <th scope="col">条件</th>                                    9076
        <th scope="col">条件类型</th>                                9077
        <th scope="col">适用分区</th>                                9078
        <th scope="col">备注</th>                                    9079
        <th scope="col">操作</th>                                    9080
      </tr>                                                          9081
    </thead>                                                         9082
    <tbody>                                                          9083
      {% for r in rules %}                                           9084
      <tr>                                                           9085
        <td style="text-align:left;">{{ r.item }}</td>               9086
        <td class="mono nowrap">{{ r.operator }} {{                  9087
    r.threshold or "" }}</td>                                        9088
        <td>{{ r.condition_type }}</td>                              9089
        <td>{{ r.region or "—" }}</td>                               9090
        <td style="text-align:left;">{{ r.note or "—"                9091
    }}</td>                                                          9092
        <td>                                                         9093
          <form method="post" action="/standard/rule/{{              9094
    r.id }}/delete"                                                  9095
                onsubmit="return confirm('确定删除规则「{{ r.item    9096
    }} {{ r.operator }} {{ r.threshold or '' }}」吗？');">           9097
            {{ form.hidden_tag() }}                                  9098
            <button type="submit" class="btn btn--sm                 9099
    btn--danger">删除</button>                                       9100
```

```
            </form>                                                    9101
          </td>                                                        9102
        </tr>                                                          9103
        {% endfor %}                                                   9104
      </tbody>                                                         9105
    </table>                                                           9106
    {% else %}                                                         9107
    <p class="muted">该标准暂无规则。请在下方添加第一条规则。</p>                  9108
    {% endif %}                                                        9109
    <h3>添加规则</h3>                                                  9110
    <form method="post">                                               9111
      {{ form.hidden_tag() }}                                          9112
      <div class="actions">                                            9113
        <div>                                                          9114
          <label class="field__label" for="r-item">判定项</label>      9115
          <input id="r-item" type="text" name="item"                   9116
      placeholder="如：植被覆盖度" required>                           9117
        </div>                                                         9118
        <div>                                                          9119
          <label class="field__label" for="r-op">比较符</label>        9120
          <select id="r-op" name="operator">                           9121
            {% for o in [">=", ">", "<=", "<", "=="]                   9122
      %}<option value="{{ o }}">{{ o }}</option>{% endfor %}           9123
          </select>                                                    9124
        </div>                                                         9125
        <div>                                                          9126
          <label class="field__label" for="r-thr">阈值</label>         9127
          <input id="r-thr" type="text" name="threshold"               9128
      placeholder="如：50" style="width:110px;">                       9129
        </div>                                                         9130
        <div>                                                          9131
          <label class="field__label" for="r-cond">条件类型</label     9132
      >                                                                9133
          <select id="r-cond" name="condition_type">                   9134
            <option value="必要条件">必要条件</option>                 9135
            <option value="充分条件">充分条件</option>                 9136
          </select>                                                    9137
        </div>                                                         9138
        <div>                                                          9139
          <label class="field__label" for="r-region">适用分区</lab     9140
      el>                                                              9141
          <input id="r-region" type="text" name="region"               9142
      placeholder="如宁夏分区" style="width:130px;">                   9143
        </div>                                                         9144
        <div>                                                          9145
          <label class="field__label" for="r-note">备注</label>        9146
          <input id="r-note" type="text" name="note"                   9147
      style="width:220px;">                                            9148
        </div>                                                         9149
        <div>                                                          9150
```

```
            <button type="submit" class="btn">添加规则</button>
          </div>
        </div>
      </form>
      <p class="actions" style="margin-top:20px;">
        <a class="btn btn--sm btn--ghost"
        href="/standard/">返回标准列表</a>
        <a class="btn btn--sm btn--ghost"
        href="/standard/compare">省际口径比对</a>
      </p>
</div>
{% endblock %}
# ==== app/templates/standard/compare.html ====
{% extends "base.html" %}
{% block title %}省际口径比对{% endblock %}
{% block content %}
<div class="card">
  <h2>省际口径比对</h2>
  {% if not groups %}
  <p class="muted">
    暂无规则数据。请先在「标准知识库」中为标准录入规则，
    或运行 <span class="mono">python seed_demo.py</span> 载入论文表
    1 数据。
  </p>
  {% else %}
  {% for item, rs in groups.items() %}
  <h3>{{ item }}</h3>
  <table>
    <thead>
      <tr>
        <th scope="col">省份</th>
        <th scope="col">标准 / 文件</th>
        <th scope="col">条件</th>
        <th scope="col">条件类型</th>
        <th scope="col">备注</th>
      </tr>
    </thead>
    <tbody>
      {% for r in rs %}
      <tr>
        <td>{{ r.standard.province }}</td>
        <td style="text-align:left;">{{ r.standard.name
    }}</td>
        <td class="mono nowrap">{{ r.operator }} {{
    r.threshold or "" }}</td>
        <td>{{ r.condition_type }}</td>
        <td style="text-align:left;">{{ r.note or "—"
    }}</td>
      </tr>
      {% endfor %}
```

```
      </tbody>                                                                  9201
    </table>                                                                    9202
    {% endfor %}                                                                9203
    {% endif %}                                                                 9204
    <p class="actions" style="margin-top:16px;">                                9205
      <a class="btn btn--ghost" href="/standard/">返回标准列表</a>              9206
    </p>                                                                        9207
</div>                                                                          9208
{% endblock %}                                                                  9209
# ==== app/templates/constraint/index.html ====                                 9210
{% extends "base.html" %}                                                       9211
{% from "_macros.html" import patch_guide %}                                    9212
{% block title %}限制性指标判定{% endblock %}                                   9213
{% block content %}                                                             9214
<div class="card">                                                              9215
    <h2>限制性指标判定</h2>                                                    9216
    {{ patch_guide(projects_without_patch) }}                                   9217
    {% if not patches %}                                                        9218
    <p class="muted">暂无矿山图斑。请先在「项目与矿山档案」中建立图斑。</p>   9219
    {% else %}                                                                  9220
    <table>                                                                     9221
      <thead>                                                                   9222
        <tr>                                                                    9223
          <th scope="col">所属项目</th>                                         9224
          <th scope="col">矿山图斑</th>                                         9225
          <th scope="col">判定结果</th>                                         9226
          <th scope="col">操作</th>                                             9227
        </tr>                                                                   9228
      </thead>                                                                  9229
      <tbody>                                                                   9230
        {% for p in patches %}                                                  9231
        {% set r = latest.get(p.id) %}                                          9232
        <tr>                                                                    9233
          <td>{{ p.project.name if p.project else "—" }}</td>                   9234
          <td style="text-align:left;">{{ p.name }}</td>                        9235
          <td style="text-align:left;">                                         9236
            {% if r %}                                                          9237
              {% if r.passed %}                                                 9238
              <span class="tag tag--ok">通过</span>                             9239
              {% else %}                                                        9240
              <span class="tag tag--warn">不通过</span>                         9241
              {% endif %}                                                       9242
              <span class="muted">{{ r.reason }}</span>                         9243
            {% else %}                                                          9244
            <span class="tag tag--muted">未判定</span>                          9245
            {% endif %}                                                         9246
          </td>                                                                 9247
          <td class="nowrap">                                                   9248
            <span class="row-actions">                                          9249
              <a class="link-btn" href="/constraint/{{ p.id                     9250
```

```
      }}">执行判定</a>                                                    9251
            </span>                                                      9252
          </td>                                                          9253
        </tr>                                                            9254
        {% endfor %}                                                     9255
      </tbody>                                                           9256
    </table>                                                             9257
    {% endif %}                                                          9258
</div>                                                                   9259
{% endblock %}                                                           9260
# ==== app/templates/constraint/check.html ====                          9261
{% extends "base.html" %}                                                9262
{% block title %}限制性条件判定 · {{ patch.name }}{% endblock %}          9263
{% block content %}                                                      9264
<div class="card" style="max-width:760px;">                              9265
    <h2>限制性条件判定 · {{ patch.name }}</h2>                           9266
    <form method="post">                                                 9267
      {{ form.hidden_tag() }}                                            9268
      {% for field, name, val in field_states %}                         9269
      <fieldset style="border:1px solid                                  9270
      var(--line);border-radius:6px;padding:10px                         9271
      14px;margin:0 0 10px;">                                            9272
        <legend style="padding:0 6px;font-weight:600;">{{                9273
      name }}</legend>                                                   9274
        <div class="actions">                                            9275
          <label class="checkbox">                                       9276
            <input type="radio" name="{{ field }}" value="1"             9277
                   {% if val is none or val %}checked{% endif            9278
      %}>                                                                9279
            <span>通过</span>                                            9280
          </label>                                                       9281
          <label class="checkbox">                                       9282
            <input type="radio" name="{{ field }}" value="0"             9283
                   {% if val is not none and not val                     9284
      %}checked{% endif %}>                                              9285
            <span>不通过</span>                                          9286
          </label>                                                       9287
        </div>                                                           9288
      </fieldset>                                                        9289
      {% endfor %}                                                       9290
      <p class="actions" style="margin-top:16px;">                       9291
        <button type="submit" class="btn">保存判定</button>              9292
        <a class="btn btn--ghost" href="/constraint/">返回</a>           9293
      </p>                                                               9294
    </form>                                                              9295
    {% if record %}                                                      9296
    <div style="margin-top:18px;padding:14px;border-radius:6px           9297
      ;border-left:4px solid;                                            9298
                 {% if record.passed %}background:var(--ok-soft          9299
      );border-left-color:var(--ok);                                     9300
```

```
                {% else %}background:var(--danger-soft);border
    -left-color:var(--danger);{% endif %}">
    <p style="margin:0 0 4px;font-weight:600;">
      判定结论：
      {% if record.passed %}
      <span class="ok">通过，可进入定量评价</span>
      {% else %}
      <span class="warn">不通过 —— 建议生态重建或人工辅助</span>
      {% endif %}
    </p>
    <p class="muted" style="margin:0;">{{ record.reason
    }}</p>
  </div>
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/ahp/index.html ====
{% extends "base.html" %}
{% block title %}AHP 权重计算{% endblock %}
{% block content %}
<div class="card" style="max-width:720px;">
  <h2>AHP 权重计算 · 选择指标层级</h2>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="{{ form.unit_type.id
    }}">微地质单元类型</label>
      {{ form.unit_type(id=form.unit_type.id) }}
    </div>
    <div class="field">
      <label class="field__label" for="{{ form.level.id
    }}">指标层级</label>
      {{ form.level(id=form.level.id) }}
    </div>
    <div class="field">
      <label class="field__label" for="{{ form.l1.id }}">
        一级指标（层级选「二级指标」时需选择）
      </label>
      {{ form.l1(id=form.l1.id) }}
    </div>
    <p class="actions" style="margin-top:16px;">
      {{ form.submit(class_="btn") }}
      <a class="btn btn--ghost" href="/ahp/indicators">指标定义维
    护</a>
      <a class="btn btn--ghost" href="/ahp/experts">多专家矩阵</a
    >
    </p>
  </form>
</div>
<div class="card">
```

```
  <h2>各微地质单元当前生效权重</h2>                                                9351
  <table>                                                                          9352
    <thead>                                                                        9353
      <tr>                                                                         9354
        <th scope="col">微地质单元</th>                                            9355
        <th scope="col">当前权重来源</th>                                          9356
        <th scope="col">操作</th>                                                  9357
      </tr>                                                                        9358
    </thead>                                                                       9359
    <tbody>                                                                        9360
      {% for r in weight_rows %}                                                   9361
      <tr>                                                                         9362
        <td style="text-align:left;">{{ r.unit_type }}</td>                        9363
        <td style="text-align:left;">                                              9364
          {% if r.custom %}                                                        9365
          <span class="tag tag--warn">{{ r.label }}</span>                         9366
          {% else %}                                                               9367
          <span class="tag tag--muted">{{ r.label }}</span>                        9368
          {% endif %}                                                              9369
        </td>                                                                      9370
        <td class="nowrap">                                                        9371
          <span class="row-actions">                                               9372
            <a class="link-btn" href="/ahp/matrix?unit_type=                       9373
    {{ r.unit_type }}&level=1">一级指标权重</a>                                    9374
            <a class="link-btn" href="/ahp/matrix?unit_type=                       9375
    {{ r.unit_type }}&level=1&calc=1">查看一致性</a>                               9376
            {% if r.custom %}                                                      9377
            <form method="post" action="/ahp/weights/reset"                        9378
                  onsubmit="return confirm('确定把「{{                             9379
    r.unit_type }}」恢复为论文表 3 的默认权重吗？');">                             9380
              <input type="hidden" name="csrf_token"                               9381
    value="{{ csrf_token() }}">                                                    9382
              <input type="hidden" name="unit_type"                                9383
    value="{{ r.unit_type }}">                                                     9384
              <button type="submit" class="link-btn                                9385
    link-btn--danger">恢复默认</button>                                            9386
            </form>                                                                9387
            {% endif %}                                                            9388
          </span>                                                                  9389
        </td>                                                                      9390
      </tr>                                                                        9391
      {% endfor %}                                                                 9392
    </tbody>                                                                       9393
  </table>                                                                         9394
</div>                                                                             9395
{% endblock %}                                                                     9396
# ==== app/templates/ahp/matrix.html ====                                          9397
{% extends "base.html" %}                                                          9398
{% block title %}判断矩阵录入{% endblock %}                                        9399
{% block content %}                                                                9400
```

```
<div class="card">
  <h2>
    判断矩阵录入
    · {{ unit_type }}
    · {% if level == 1 %}一级指标{% else %}二级指标（{{ l1 }}）{%
    endif %}
  </h2>
  <p class="muted">
    采用 1–9 标度：1 同等重要、3 稍微重要、5 明显重要、7 强烈重要、9 极端重要，
    2、4、6、8 为相邻判断的中间值。只需录入上三角，对角线为 1，下三角自动取倒数。
    矩阵已按论文表 3 的权重预填。
  </p>
  <form method="post">
    {{ form.hidden_tag() }}
    <table style="width:auto;">
      <thead>
        <tr>
          <th></th>
          {% for name in names %}<th scope="col">{{ name
    }}</th>{% endfor %}
        </tr>
      </thead>
      <tbody>
        {% for i in range(names|length) %}
        <tr>
          <th scope="row">{{ names[i] }}</th>
          {% for j in range(names|length) %}
            {% if i == j %}
            <td class="muted num">1</td>
            {% elif i < j %}
            <td>
              <label class="muted" style="position:absolute;
    left:-9999px;"
                     for="m-{{ i }}-{{ j }}">{{ names[i] }}
    相对 {{ names[j] }} 的重要性</label>
              <input id="m-{{ i }}-{{ j }}" type="number"
    step="0.01" min="0.01"
                     name="m_{{ i }}_{{ j }}" value="{{
    default[i][j] }}">
            </td>
            {% else %}
            <td class="muted num">{{ default[i][j] }}</td>
            {% endif %}
          {% endfor %}
        </tr>
        {% endfor %}
      </tbody>
    </table>
    <p class="actions">
      <button type="submit" class="btn" name="action"
```

```
    value="calc">计算权重</button>
      {% if result and result.consistent %}
      <button type="submit" class="btn" name="action"
    value="save">保存并生效</button>
      {% endif %}
      <a class="btn btn--ghost" href="/ahp/">重新选择</a>
    </p>
</form>
<p class="muted" style="margin-top:10px;">
    当前「{{ unit_type }}」生效权重：<strong>{{ source_label
    }}</strong>。
    点击「保存并生效」后，本单元类型的适宜性评价将采用此处算出的权重；
    如需回到论文取值，可在 <a href="/ahp/">权重首页</a> 恢复默认。
</p>
{% if result %}
<hr style="margin:22px 0;border:none;border-top:1px solid
    var(--line);">
<h3>计算结果</h3>
<table style="width:auto;">
    <thead>
      <tr>
        <th scope="col">指标</th>
        {% for name in names %}<th scope="col">{{ name
    }}</th>{% endfor %}
      </tr>
    </thead>
    <tbody>
      <tr>
        <th scope="row">权重</th>
        {% for w in result.weights %}<td class="mono">{{ w
    }}</td>{% endfor %}
      </tr>
    </tbody>
</table>
<div class="stat-row">
    <div>
      <div class="stat__label">最大特征值 λmax</div>
      <div class="stat__value mono">{{ result.lambda_max
    }}</div>
    </div>
    <div>
      <div class="stat__label">一致性指标 CI</div>
      <div class="stat__value mono">{{
    "%.4f"|format(result.ci) }}</div>
    </div>
    <div>
      <div class="stat__label">一致性比例 CR</div>
      <div class="stat__value mono">{{
    "%.4f"|format(result.cr) }}</div>
    </div>
```

```
      <div>                                                                    9501
        <div class="stat__label">一致性检验（要求 CR &lt; 0.1）</div>          9502
        <div class="stat__value">                                              9503
          {% if result.consistent %}<span class="ok">通过</span>               9504
          {% else %}<span class="warn">不通过</span>{% endif %}                9505
        </div>                                                                 9506
      </div>                                                                   9507
    </div>                                                                     9508
    {% if not result.consistent and result.most_inconsistent                   9509
      %}                                                                       9510
    <p style="margin-top:14px;padding:12px;border-radius:6px;b                 9511
      ackground:var(--danger-soft);                                            9512
              border-left:4px solid var(--danger);">                           9513
      <strong class="warn">建议优先调整：</strong>                             9514
      第 {{ result.most_inconsistent.i + 1 }} 行、第 {{                        9515
      result.most_inconsistent.j + 1 }} 列的比较                               9516
      （当前 <span class="mono num">{{ result.most_inconsistent.v              9517
      alue|round(3) }}</span>，                                                9518
      理想约 <span class="mono num">{{ result.most_inconsistent.e              9519
      xpected|round(3) }}</span>）                                             9520
    </p>                                                                       9521
    {% endif %}                                                                9522
  {% endif %}                                                                  9523
</div>                                                                         9524
{% endblock %}                                                                 9525
# ==== app/templates/ahp/indicators.html ====                                  9526
{% extends "base.html" %}                                                      9527
{% block title %}指标定义维护{% endblock %}                                    9528
{% block content %}                                                            9529
<div class="card">                                                             9530
  <h2>指标体系维护</h2>                                                        9531
  <p class="actions" style="margin-bottom:14px;">                              9532
    <a class="btn btn--sm btn--ghost"                                          9533
    href="/ahp/">返回权重计算</a>                                              9534
    <a class="btn btn--sm btn--ghost"                                          9535
    href="/ahp/experts">多专家矩阵</a>                                         9536
    <form method="post" action="/ahp/indicators/reset"                         9537
    style="display:inline;"                                                    9538
          onsubmit="return confirm('将按论文取值重置全部指标定义，自定义修改会丢失。 9539
    确定继续？');">                                                            9540
      {{ form.hidden_tag() }}                                                  9541
      <button type="submit" class="btn btn--sm                                 9542
    btn--danger">按论文取值重置</button>                                       9543
    </form>                                                                    9544
  </p>                                                                         9545
  {% for level, title in [(1, "一级指标"), (2, "二级指标"), (3,                9546
    "三级指标")] %}                                                            9547
  {% set level_items = all_ind|selectattr("level",                             9548
    "equalto", level)|list %}                                                  9549
  {% if level_items %}                                                         9550
```

```
  <h3>{{ title }} ({{ level_items|length }} 项)</h3>
  <table>
    <thead>
      <tr>
        <th scope="col">指标</th>
        <th scope="col">上级指标</th>
        <th scope="col">单位</th>
        <th scope="col">标准化类型</th>
        <th scope="col">理想取值</th>
        <th scope="col">启用单元</th>
        <th scope="col">限制性</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for i in level_items %}
      <tr>
        <td style="text-align:left;">
          <span class="mono muted">{{ i.code
}}</span><br>{{ i.name }}
        </td>
        <td>{{ (all_ind|selectattr("code", "equalto",
i.parent_code)|list|first).name if i.parent_code else
"—" }}</td>
        <td>{{ i.unit or "—" }}</td>
        <td>{{ type_label(i.ideal_type) }}</td>
        <td class="mono">{{ i.ideal_value or "—" }}</td>
        <td style="text-align:left;">{{ i.enabled_units or
"—" }}</td>
        <td>
          {% if i.is_constraint %}<span class="tag
tag--warn">是</span>
          {% else %}<span class="tag tag--muted">否</span>{%
endif %}
        </td>
        <td class="nowrap">
          <span class="row-actions">
            <a class="link-btn" href="/ahp/indicators/{{
i.code }}/edit">编辑</a>
          </span>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% endif %}
  {% endfor %}
</div>
{% endblock %}
# ==== app/templates/ahp/indicator_edit.html ====
```

```
{% extends "base.html" %}
{% block title %}编辑指标 · {{ ind.name }}{% endblock %}
{% block content %}
<div class="card" style="max-width:680px;">
  <h2>编辑指标 · {{ ind.name }}</h2>
  <p class="muted">
    指标编码 <span class="mono">{{ ind.code }}</span>、层级 {{ ind.level }} 级，
    由论文指标体系确定，不可修改。
  </p>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="name">指标名称</label>
      <input id="name" type="text" name="name" style="width:100%;" required
             value="{{ ind.name }}">
    </div>
    <div class="field">
      <label class="field__label" for="unit">计量单位</label>
      <input id="unit" type="text" name="unit" style="width:180px;"
             value="{{ ind.unit or '' }}">
    </div>
    <div class="field">
      <label class="field__label" for="ideal-type">标准化类型</label>
      <select id="ideal-type" name="ideal_type">
        {% for code, label in [("benefit","越大越优"),("cost","越小越优"),("mid","适中型"),("discrete","离散赋值")] %}
        <option value="{{ code }}" {% if ind.ideal_type == code %}selected{% endif %}>{{ label }}</option>
        {% endfor %}
      </select>
    </div>
    <div class="field">
      <label class="field__label" for="ideal-value">理想取值</label>
      <input id="ideal-value" type="text" name="ideal_value" style="width:100%;"
             value="{{ ind.ideal_value or '' }}">
      <p class="muted" style="margin:4px 0 0;">
        如土石比 0.8、土层厚度 20、pH 5~9、植被覆盖度 75。
      </p>
    </div>
    <fieldset style="border:1px solid var(--line);border-radius:6px;padding:10px 14px;margin:0 0 12px;">
      <legend style="padding:0 6px;font-weight:600;">启用该指标的微地质单元</legend>
```

```
        {% for t in unit_types %}                                          9651
        <label class="checkbox" style="margin-right:18px;">               9652
          <input type="checkbox" name="enabled_units"                      9653
      value="{{ t }}"                                                      9654
                 {% if t in ind.units_list() %}checked{%                   9655
      endif %}>                                                            9656
          <span>{{ t }}</span>                                             9657
        </label>                                                           9658
        {% endfor %}                                                       9659
        <p class="muted" style="margin:8px 0 0;">                          9660
          论文设定：岩质边坡不启用土壤指标，平台不启用地形指标。                  9661
        </p>                                                               9662
      </fieldset>                                                          9663
      <div class="field">                                                  9664
        <label class="checkbox" for="is-constraint">                       9665
          <input id="is-constraint" type="checkbox"                        9666
      name="is_constraint" value="1"                                       9667
                 {% if ind.is_constraint %}checked{% endif %}>             9668
          <span>同时作为限制性指标（参与一票否决判定）</span>                      9669
        </label>                                                           9670
      </div>                                                               9671
      <div class="field">                                                  9672
        <label class="field__label" for="description">取值依据说明</       9673
      label>                                                               9674
        <textarea id="description" name="description"                      9675
      style="min-height:88px;">{{ ind.description or ''                    9676
      }}</textarea>                                                        9677
      </div>                                                               9678
      <p class="actions" style="margin-top:16px;">                         9679
        <button type="submit" class="btn">保存</button>                    9680
        <a class="btn btn--ghost" href="/ahp/indicators">返回指标列         9681
      表</a>                                                               9682
      </p>                                                                 9683
    </form>                                                                9684
</div>                                                                     9685
{% endblock %}                                                             9686
# ==== app/templates/ahp/experts.html ====                                 9687
{% extends "base.html" %}                                                  9688
{% block title %}多专家判断矩阵{% endblock %}                                9689
{% block content %}                                                        9690
<div class="card">                                                         9691
  <h2>多专家判断矩阵 · 录入与汇总</h2>                                        9692
  <form method="get" class="actions" style="margin-bottom:14               9693
    px;">                                                                  9694
    <div>                                                                  9695
      <label class="field__label" for="unit-type">微地质单元类型</l        9696
    abel>                                                                  9697
      <select id="unit-type" name="unit_type"                              9698
    onchange="this.form.submit()">                                         9699
        {% for t in ["岩质边坡", "碎石边坡", "平台"] %}                        9700
```

```
          <option value="{{ t }}" {% if unit_type == t                    9701
    %}selected{% endif %}>{{ t }}</option>                                9702
          {% endfor %}                                                    9703
        </select>                                                         9704
    </div>                                                                9705
    <div>                                                                 9706
        <label class="field__label" for="level">指标层级</label>          9707
        <select id="level" name="level"                                   9708
    onchange="this.form.submit()">                                        9709
          <option value="1" {% if level == 1 %}selected{%                 9710
    endif %}>一级指标</option>                                            9711
          <option value="2" {% if level == 2 %}selected{%                 9712
    endif %}>二级指标</option>                                            9713
        </select>                                                         9714
    </div>                                                                9715
</form>                                                                   9716
<p class="muted">                                                         9717
    当前范围：<strong>{{ unit_type }}</strong> ·                          9718
    {% if level == 1 %}一级指标{% else %}二级指标（{{ l1 }}）{% endif     9719
    %}。                                                                  9720
    若选二级指标，请先到                                                  9721
    <a href="/ahp/">权重计算</a> 选择一级指标后进入本页。                 9722
</p>                                                                      9723
{% if rows %}                                                             9724
<table>                                                                   9725
    <caption class="muted">已录入 {{ rows|length }}                       9726
    位专家的矩阵</caption>                                                9727
    <thead>                                                               9728
        <tr>                                                              9729
          <th scope="col">专家</th>                                       9730
          <th scope="col">矩阵规模</th>                                   9731
          <th scope="col">该矩阵 CR</th>                                  9732
          <th scope="col">一致性</th>                                     9733
          <th scope="col">录入时间</th>                                   9734
          <th scope="col">操作</th>                                       9735
        </tr>                                                             9736
    </thead>                                                              9737
    <tbody>                                                               9738
        {% for r in rows %}                                               9739
        <tr>                                                              9740
          <td>{{ r.expert_name }}</td>                                    9741
          <td class="num">{{ n }} × {{ n }}</td>                          9742
          <td class="mono">{{ "%.4f"|format(r.cr) if r.cr is              9743
    not none else "—" }}</td>                                             9744
          <td>                                                            9745
            {% if r.cr is not none and r.cr < 0.1 %}<span                 9746
    class="tag tag--ok">通过</span>                                       9747
            {% else %}<span class="tag tag--warn">不通过</span>{%         9748
    endif %}                                                              9749
          </td>                                                           9750
```

```
        <td class="mono nowrap">{{ r.created_at | localdt }}</td>
        <td>
          <form method="post" action="/ahp/experts/{{ r.id }}/delete"
                onsubmit="return confirm('确定删除专家「{{ r.expert_name }}」的矩阵吗？');">
            {{ form.hidden_tag() }}
            <button type="submit" class="btn btn--sm btn--danger">删除</button>
          </form>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  <p class="actions" style="margin-bottom:10px;">
    <a class="btn" href="/ahp/experts/aggregate?unit_type={{ unit_type }}&level={{ level }}{% if l1 %}&l1={{ l1 }}{% endif %}">
      汇总计算（几何平均）
    </a>
  </p>
{% else %}
  <p class="muted">尚未录入任何专家矩阵。请在下方录入第一位专家的判断矩阵。</p>
{% endif %}

<h3>录入一位专家的判断矩阵</h3>
<form method="post">
  {{ form.hidden_tag() }}
  <input type="hidden" name="action" value="add">
  <input type="hidden" name="unit_type" value="{{ unit_type }}">
  <input type="hidden" name="level" value="{{ level }}">
  {% if l1 %}<input type="hidden" name="l1" value="{{ l1 }}">{% endif %}
  <div class="field">
    <label class="field__label" for="expert-name">专家姓名</label>
    <input id="expert-name" type="text" name="expert_name"
           style="width:220px;" placeholder="如：张工">
  </div>
  <table style="width:auto;">
    <thead>
      <tr>
        <th></th>
        {% for name in names %}<th scope="col">{{ name }}</th>{% endfor %}
      </tr>
    </thead>
    <tbody>
```

```
          {% for i in range(n) %}                                          9801
          <tr>                                                            9802
            <th scope="row">{{ names[i] }}</th>                           9803
            {% for j in range(n) %}                                       9804
              {% if i == j %}                                             9805
              <td class="muted num">1</td>                                9806
              {% elif i < j %}                                            9807
              <td>                                                        9808
                <label class="muted" style="position:absolute;            9809
    left:-9999px;"                                                        9810
                       for="e-{{ i }}-{{ j }}">{{ names[i] }}             9811
    相对 {{ names[j] }} 的重要性</label>                                  9812
                <input id="e-{{ i }}-{{ j }}" type="number"               9813
    step="0.01" min="0.01"                                                9814
                       name="m_{{ i }}_{{ j }}" value="{{                 9815
    default[i][j] }}">                                                    9816
              </td>                                                       9817
              {% else %}                                                  9818
              <td class="muted num">{{ default[i][j] }}</td>              9819
              {% endif %}                                                 9820
            {% endfor %}                                                  9821
          </tr>                                                           9822
          {% endfor %}                                                    9823
        </tbody>                                                          9824
    </table>                                                              9825
    <p class="actions" style="margin-top:12px;">                          9826
      <button type="submit" class="btn">录入该专家矩阵</button>           9827
      <a class="btn btn--ghost" href="/ahp/indicators">指标维护<          9828
    /a>                                                                   9829
      <a class="btn btn--ghost" href="/ahp/">返回权重计算</a>             9830
    </p>                                                                  9831
  </form>                                                                 9832
</div>                                                                    9833
{% endblock %}                                                            9834
# ==== app/templates/ahp/experts_aggregate.html ====                      9835
{% extends "base.html" %}                                                 9836
{% block title %}多专家矩阵汇总{% endblock %}                             9837
{% block content %}                                                       9838
<div class="card">                                                        9839
  <h2>多专家矩阵汇总 · {{ unit_type }} · {% if level == 1 %}一级指标{%   9840
    else %}二级指标（{{ l1 }}）{% endif %}</h2>                           9841
  {% if not result %}                                                     9842
  <p class="muted">                                                       9843
    该范围内尚未录入专家矩阵。                                           9844
    <a href="/ahp/experts?unit_type={{ unit_type                          9845
    }}&level={{ level }}{% if l1 %}&l1={{ l1 }}{% endif                   9846
    %}">去录入</a>                                                       9847
  </p>                                                                    9848
  {% else %}                                                              9849
  <div class="stat-row">                                                  9850
```

```
    <div>                                                                9851
      <div class="stat__label">参与汇总的专家</div>                        9852
      <div class="stat__value">{{ result.count }} 位</div>               9853
    </div>                                                               9854
    <div>                                                                9855
      <div class="stat__label">λmax</div>                                9856
      <div class="stat__value mono">{{ result.lambda_max                 9857
    }}</div>                                                             9858
    </div>                                                               9859
    <div>                                                                9860
      <div class="stat__label">CI</div>                                  9861
      <div class="stat__value mono">{{                                   9862
    "%.4f"|format(result.ci) }}</div>                                    9863
    </div>                                                               9864
    <div>                                                                9865
      <div class="stat__label">CR（要求 &lt; 0.1）</div>                   9866
      <div class="stat__value mono">{{                                   9867
    "%.4f"|format(result.cr) }}</div>                                    9868
    </div>                                                               9869
    <div>                                                                9870
      <div class="stat__label">一致性检验</div>                            9871
      <div class="stat__value">                                          9872
        {% if result.consistent %}<span class="ok">通过</span>           9873
        {% else %}<span class="warn">不通过</span>{% endif %}            9874
      </div>                                                             9875
    </div>                                                               9876
  </div>                                                                 9877
  <h3>汇总后权重</h3>                                                    9878
  <table style="width:auto;">                                            9879
    <thead>                                                              9880
      <tr>                                                               9881
        <th scope="col">指标</th>                                        9882
        {% for name in names %}<th scope="col">{{ name                   9883
    }}</th>{% endfor %}                                                  9884
      </tr>                                                              9885
    </thead>                                                             9886
    <tbody>                                                              9887
      <tr>                                                               9888
        <th scope="row">权重</th>                                        9889
        {% for w in result.weights %}<td class="mono">{{ w               9890
    }}</td>{% endfor %}                                                  9891
      </tr>                                                              9892
    </tbody>                                                             9893
  </table>                                                               9894
  <h3>各专家矩阵的一致性</h3>                                            9895
  <table style="width:auto;">                                            9896
    <thead>                                                              9897
      <tr><th scope="col">专家</th><th scope="col">该矩阵                9898
    CR</th><th scope="col">判定</th></tr>                                9899
    </thead>                                                             9900
```

```
    <tbody>
      {% for r in rows %}
      <tr>
        <td>{{ r.expert_name }}</td>
        <td class="mono">{{ "%.4f"|format(r.cr) if r.cr is
    not none else "—" }}</td>
        <td>
          {% if r.cr is not none and r.cr < 0.1 %}<span
    class="ok">通过</span>
          {% else %}<span class="warn">不通过</span>{% endif %}
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% if not result.consistent and result.most_inconsistent
    %}
  <p style="padding:12px;border-radius:6px;background:var(--
    danger-soft);
              border-left:4px solid var(--danger);">
    <strong class="warn">汇总矩阵一致性不通过，建议优先调整：</strong>
    第 {{ result.most_inconsistent.i + 1 }} 行、第 {{
    result.most_inconsistent.j + 1 }} 列的比较
    （当前 <span class="mono">{{ result.most_inconsistent.value
    |round(3) }}</span>，
    理想约 <span class="mono">{{ result.most_inconsistent.expec
    ted|round(3) }}</span>）。
    该偏差通常来自个别专家打分，可先核对一致性较差的专家矩阵。
  </p>
  {% endif %}
  {% endif %}
  <p class="actions" style="margin-top:16px;">
    <form method="post"
          action="/ahp/experts?unit_type={{ unit_type
    }}&level={{ level }}{% if l1 %}&l1={{ l1 }}{% endif %}"
          style="display:inline;"
          onsubmit="return confirm('保存后，「{{ unit_type
    }}」的适宜性评价将改用这组汇总权重计算。确定继续吗？');">
      {{ form.hidden_tag() }}
      <input type="hidden" name="action" value="save_agg">
      <button type="submit" class="btn" {% if not
    result.consistent %}disabled{% endif %}>
        保存为生效权重
      </button>
    </form>
    <a class="btn btn--ghost" href="/ahp/experts?unit_type={
    { unit_type }}&level={{ level }}{% if l1 %}&l1={{ l1
    }}{% endif %}">返回录入</a>
    <a class="btn btn--ghost" href="/ahp/">返回权重计算</a>
  </p>
```

```
</div>
{% endblock %}
# ==== app/templates/indicator/index.html ====
{% extends "base.html" %}
{% block title %}指标数据采集{% endblock %}
{% block content %}
<div class="card">
  <h2>指标数据采集与管理</h2>
  <p class="actions" style="margin-bottom:12px;">
    <a class="btn btn--sm" href="/indicator/import">批量导入（CSV / Excel）</a>
  </p>
  {% if not patches %}
  <p class="muted">
    暂无矿山图斑。请先在「项目与矿山档案」中建立图斑，
    或运行 <span class="mono">python seed_demo.py</span>
    载入论文案例。
  </p>
  {% else %}
  {% for p in patches %}
  <div class="patch-panel">
    <div class="patch-panel__head">
      <h3 class="patch-panel__name">{{ p.name }}</h3>
      <span class="patch-panel__meta">
        所属项目：{{ p.project.name if p.project else "—" }}
      </span>
      <span class="patch-panel__meta">
        微地质单元：{{ p.units.count() }} 个
      </span>
      {% if p.units.count() %}
      <span class="patch-panel__meta patch-panel__tail">
        <a class="link-btn" href="/indicator/import">批量导入本图斑数据</a>
      </span>
      {% endif %}
    </div>
    {% if not p.units.count() %}
    <div class="notice">
      <p class="notice__title">该图斑尚未划分微地质单元，暂不能录入指标数据。</p>
      <p style="margin:0;">
        指标数据是按微地质单元（岩质边坡 / 碎石边坡 / 平台）分别组织的，
        请先完成划分，再回到本页录入。
      </p>
      <p style="margin:10px 0 0;">
        <a class="btn btn--sm" href="/unit/{{ p.id }}">去划分微地质单元</a>
      </p>
    </div>
    {% else %}
    <table>
```

```
        <thead>                                                                  10001
          <tr>                                                                   10002
            <th scope="col">微地质单元</th>                                      10003
            <th scope="col">面积占比</th>                                        10004
            <th scope="col">数据状态</th>                                        10005
            <th scope="col">操作</th>                                            10006
          </tr>                                                                  10007
        </thead>                                                                 10008
        <tbody>                                                                  10009
          {% for u in p.units %}                                                 10010
          <tr>                                                                   10011
            <td>{{ u.unit_type }}</td>                                           10012
            <td class="num">                                                     10013
              {% if u.ratio is not none %}{{                                     10014
      "%.2f%%"|format(u.ratio * 100) }}{% else %}—{% endif %}                    10015
            </td>                                                                10016
            <td>                                                                 10017
              {% if u.indicator_data %}                                          10018
              <span class="tag tag--ok">已录入</span>                            10019
              {% else %}                                                         10020
              <span class="tag tag--muted">未录入</span>                         10021
              {% endif %}                                                        10022
            </td>                                                                10023
            <td class="nowrap">                                                  10024
              <span class="row-actions">                                         10025
                <a class="link-btn" href="/indicator/{{ u.id                     10026
      }}">录入 / 编辑</a>                                                        10027
              </span>                                                            10028
            </td>                                                                10029
          </tr>                                                                  10030
          {% endfor %}                                                           10031
        </tbody>                                                                 10032
      </table>                                                                   10033
      {% endif %}                                                                10034
    </div>                                                                       10035
    {% endfor %}                                                                 10036
    {% endif %}                                                                  10037
</div>                                                                           10038
{% endblock %}                                                                   10039
# ==== app/templates/indicator/edit.html ====                                    10040
{% extends "base.html" %}                                                        10041
{% block title %}指标数据录入 · {{ unit.unit_type }}{% endblock %}               10042
{% block content %}                                                              10043
<div class="card" style="max-width:820px;">                                      10044
  <h2>指标数据录入 · {{ unit.unit_type }}</h2>                                   10045
  <p class="muted">                                                              10046
    除标注「可选」的项外，本页字段均为必填：填齐后评价才能执行，缺项会在评价页逐条列出。      10047
    <br>标注「未检测」的重金属可以留空，按论文注记以「未受污染」计入污染指标，              10048
    评价结果与导出报告里会如实写明是哪几项未检测 —— 它不等于实测达标。                    10049
    岩质边坡不设土壤指标、平台不设地形指标，系统已按单元类型隐藏不相关字段。              10050
```

```
</p>
<form method="post">
  {{ form.hidden_tag() }}
  {% for group_name, fields, has_hm in groups %}
  <h3>{{ group_name }}</h3>
  {% for fname, cname, unit_ in fields %}
  <div class="field">
    <label class="field__label" for="f-{{ fname }}">
      {{ cname }}{% if unit_ %} ({{ unit_ }}) {% endif %}
    </label>
    {% if fname == "aspect" %}
    <select id="f-{{ fname }}" name="aspect">
      <option value="">— 不适用 —</option>
      {% for a in aspect_types %}
      <option value="{{ a }}" {% if current.get("aspect")
== a %}selected{% endif %}>{{ a }}</option>
      {% endfor %}
    </select>
    {% else %}
    <input id="f-{{ fname }}" type="number" step="any"
name="{{ fname }}"
           value="{{ current.get(fname) if
current.get(fname) is not none else '' }}">
    {% endif %}
  </div>
  {% endfor %}
  {% endfor %}
  {% for group_name, fields, has_hm in groups %}
  {% if has_hm %}
  <h3>重金属含量（mg/kg）<span class="muted">（可选，未检测可留空）</span>
</h3>
  <p class="muted">未检测的金属可留空：按论文注释以「未受污染」计入污染指标；
    评价结果页与导出报告中会写明未检测的是哪几项，避免被误读成实测达标。</p>
  <table style="width:auto;">
    <thead>
      <tr>
        {% for m in heavy_metals %}<th scope="col">{{ m
}}</th>{% endfor %}
      </tr>
    </thead>
    <tbody>
      <tr>
        {% for m in heavy_metals %}
        <td>
          <label class="muted" style="position:absolute;
left:-9999px;"
                 for="hm-{{ m }}">{{ m }} 含量</label>
          <input id="hm-{{ m }}" type="number"
step="any" name="hm_{{ m }}"
                 value="{{ current.get('heavy',
```

```
      {}).get(m) if current.get('heavy', {}).get(m) is not      10101
      none else '' }}">                                         10102
                </td>                                           10103
                {% endfor %}                                    10104
              </tr>                                             10105
            </tbody>                                            10106
          </table>                                              10107
          {% endif %}                                           10108
      {% endfor %}                                              10109
      <p class="actions" style="margin-top:18px;">              10110
        <button type="submit" class="btn">保存</button>         10111
        <a class="btn btn--ghost" href="/indicator/">返回</a>   10112
      </p>                                                      10113
    </form>                                                     10114
</div>                                                          10115
{% endblock %}                                                  10116
# ==== app/templates/indicator/import.html ====                 10117
{% extends "base.html" %}                                       10118
{% block title %}批量导入指标数据{% endblock %}                  10119
{% block content %}                                             10120
<div class="card" style="max-width:860px;">                     10121
  <h2>批量导入指标数据</h2>                                      10122
  <p class="muted">                                             10123
    支持 CSV 与 Excel（.xlsx/.xls）。文件每行对应一个微地质单元，  10124
    系统按「单元类型」匹配图斑内已划分的单元后写入数据。          10125
    先下载模板按格式填写，可减少出错。                          10126
  </p>                                                          10127
  <p class="actions" style="margin-bottom:14px;">               10128
    <a class="btn btn--sm btn--ghost"                           10129
    href="/indicator/import/template">下载导入模板（CSV）</a>    10130
    <a class="btn btn--sm btn--ghost"                           10131
    href="/indicator/">返回指标数据</a>                          10132
  </p>                                                          10133
  <form method="post" enctype="multipart/form-data">            10134
    {{ form.hidden_tag() }}                                     10135
    <div class="field">                                         10136
      <label class="field__label" for="patch-id">目标矿山图斑</lab10137
    el>                                                         10138
      <select id="patch-id" name="patch_id" required>           10139
        <option value="">— 请选择 —</option>                    10140
        {% for p in patches %}                                  10141
        <option value="{{ p.id }}">{{ p.name }} ({{            10142
    p.units.count() }} 个单元）</option>                        10143
        {% endfor %}                                            10144
      </select>                                                 10145
    </div>                                                      10146
    <div class="field">                                         10147
      <label class="field__label" for="datafile">数据文件</label10148
    >                                                           10149
      <input id="datafile" type="file" name="datafile"          10150
```

```
                accept=".csv,.xlsx,.xls" required>
      </div>
      <p class="actions" style="margin-top:16px;">
        <button type="submit" class="btn">开始导入</button>
      </p>
    </form>
    <h3>模板列说明</h3>
    <p class="muted">
      共 {{ headers|length }} 列。留空表示该指标「不适用」。
      坡向须填「阳坡 / 阴阳坡 / 阴坡」之一。未检测的重金属留空即可
      （按论文注释视为未受污染）。
    </p>
    <p class="mono muted" style="line-height:1.9;">{{
      headers|join(" , ") }}</p>
    <h3>导入规则</h3>
    <ul class="muted" style="padding-left:20px;line-height:1.9
      ;">
      <li>「单元类型」必须与图斑内已划分的单元一致，否则该行会被跳过并在结果中提示。</li>
      <li>同一单元类型若在文件中出现多行，以最后一行为准。</li>
      <li>已有数据的单元会被覆盖更新；没有数据的单元将新建记录。</li>
      <li>数值列请填纯数字，不要带单位（如填 <span class="mono">10</span> 而非
      <span class="mono">10cm</span>）。</li>
    </ul>
</div>
{% if report %}
<div class="card" style="max-width:860px;">
    <h2>导入结果</h2>
    <div class="stat-row">
      <div>
        <div class="stat__label">目标图斑</div>
        <div style="font-size:15px;">{{ report.patch }}</div>
      </div>
      <div>
        <div class="stat__label">成功导入</div>
        <div class="stat__value ok">{{ report.saved }}
      个单元</div>
      </div>
      <div>
        <div class="stat__label">问题行</div>
        <div class="stat__value {% if report.errors %}warn{%
      endif %}">{{ report.errors|length }}</div>
      </div>
    </div>
    {% if report.errors %}
    <h3>未导入的行</h3>
    <ul class="muted" style="padding-left:20px;line-height:1.9
      ;">
      {% for e in report.errors %}<li>{{ e }}</li>{% endfor %}
    </ul>
    {% else %}
```

```
    <p class="ok" style="margin-top:12px;">全部数据行均已成功导入。</p>
    {% endif %}
    <p class="actions" style="margin-top:16px;">
      <a class="btn btn--ghost" href="/indicator/">查看指标数据</a>
    </p>
  </div>
{% endif %}
{% endblock %}
#  ==== app/templates/evaluate/index.html ====
{% extends "base.html" %}
{% block title %}适宜性定量评价{% endblock %}
{% block content %}
<div class="card">
  <h2>适宜性定量评价</h2>
  {% if not patches %}
  <p class="muted">暂无矿山图斑。请先到「项目与矿山档案」建立图斑。</p>
  {% else %}
  <table>
    <thead>
      <tr>
        <th scope="col">所属项目</th>
        <th scope="col">矿山图斑</th>
        <th scope="col">前置条件</th>
        <th scope="col">最近评价结果</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for p in patches %}
      {% set pr = progress.get(p.id) %}
      <tr>
        <td>{{ p.project.name if p.project else "—" }}</td>
        <td style="text-align:left;">{{ p.name }}</td>
        <td style="text-align:left;">
          {% if pr and pr.ready %}
          <span class="tag tag--ok">已具备</span>
          {% elif pr %}
          <span class="tag tag--warn">缺：{{
pr.blocking[0].label }}</span>
          <span class="muted">{{ pr.blocking[0].reason
}}</span>
          {% endif %}
        </td>
        <td>
          {% if latest.get(p.id) %}
          F = <span class="mono num">{{
latest[p.id].final_score }}</span>,
          <span class="tag tag--ok">{{ latest[p.id].grade
}}</span>
          {% else %}
```

```
            <span class="tag tag--muted">未评价</span>                    10251
            {% endif %}                                                   10252
          </td>                                                           10253
          <td class="nowrap">                                             10254
            <span class="row-actions">                                    10255
              <a class="link-btn" href="/evaluate/{{ p.id                 10256
    }}">执行评价</a>                                                      10257
              {% if latest.get(p.id) %}                                   10258
              <a class="link-btn" href="/evaluate/result/{{               10259
    latest[p.id].id }}">查看结果</a>                                      10260
              {% endif %}                                                 10261
            </span>                                                       10262
          </td>                                                           10263
        </tr>                                                             10264
        {% endfor %}                                                      10265
      </tbody>                                                            10266
    </table>                                                              10267
    {% endif %}                                                           10268
</div>                                                                    10269
{% endblock %}                                                            10270
# ==== app/templates/evaluate/run.html ====                               10271
{% extends "base.html" %}                                                 10272
{% import "_macros.html" as ui %}                                         10273
{% block title %}执行评价 · {{ patch.name }}{% endblock %}                10274
{% block content %}                                                       10275
<div class="card" style="max-width:880px;">                               10276
  <h2>执行适宜性评价 · {{ patch.name }}</h2>                              10277
  {{ ui.workflow_steps(progress) }}                                       10278
  {{ ui.workflow_blocking(progress) }}                                    10279
  {% if progress.ready %}                                                 10280
  <form method="post">                                                    10281
    {{ form.hidden_tag() }}                                               10282
    <p class="actions">                                                   10283
      <button type="submit" class="btn">开始评价</button>                 10284
      <a class="btn btn--ghost" href="/evaluate/">返回</a>                10285
    </p>                                                                  10286
  </form>                                                                 10287
  {% else %}                                                              10288
  <p class="actions">                                                     10289
    <button type="button" class="btn" disabled>开始评价</button>          10290
    <a class="btn btn--ghost" href="/evaluate/">返回</a>                  10291
  </p>                                                                    10292
  {% endif %}                                                             10293
</div>                                                                    10294
<div class="card" style="max-width:880px;">                               10295
  <h2>当前流程状态</h2>                                                   10296
  <table>                                                                 10297
    <thead>                                                               10298
      <tr><th scope="col">步骤</th><th scope="col">状态</th><th           10299
    scope="col">说明</th></tr>                                            10300
```

```
      </thead>                                                          10301
      <tbody>                                                           10302
        <tr>                                                            10303
          <td>限制性条件判定</td>                                       10304
          <td>                                                          10305
            {% if progress.constraint.passed %}<span                    10306
      class="tag tag--ok">已通过</span>                                 10307
            {% elif progress.constraint.done %}<span                    10308
      class="tag tag--warn">未通过</span>                               10309
            {% else %}<span class="tag tag--muted">待完成</span>{       10310
      % endif %}                                                        10311
          </td>                                                         10312
          <td style="text-align:left;">{{                               10313
      progress.constraint.detail }}</td>                                10314
        </tr>                                                           10315
        <tr>                                                            10316
          <td>划分微地质单元</td>                                       10317
          <td>                                                          10318
            {% if progress.units.done %}<span class="tag                10319
      tag--ok">已完成</span>                                            10320
            {% else %}<span class="tag tag--muted">待完成</span>{       10321
      % endif %}                                                        10322
          </td>                                                         10323
          <td style="text-align:left;">{{                               10324
      progress.units.detail }}</td>                                     10325
        </tr>                                                           10326
        <tr>                                                            10327
          <td>录入指标数据</td>                                         10328
          <td>                                                          10329
            {% if progress.data.ready %}<span class="tag                10330
      tag--ok">已完成</span>                                            10331
            {% else %}<span class="tag tag--muted">待完成</span>{       10332
      % endif %}                                                        10333
          </td>                                                         10334
          <td style="text-align:left;">{{                               10335
      progress.data.detail }}</td>                                      10336
        </tr>                                                           10337
      </tbody>                                                          10338
    </table>                                                            10339
</div>                                                                  10340
{% if weight_sources %}                                                 10341
<div class="card" style="max-width:880px;">                             10342
    <h2>本次将采用的指标权重</h2>  <table style="max-width:720px;">     10343
      <thead>                                                           10344
        <tr><th scope="col">微地质单元</th><th                          10345
      scope="col">权重来源</th></tr>                                    10346
      </thead>                                                          10347
      <tbody>                                                           10348
        {% for ut, label in weight_sources.items() %}                   10349
        <tr>                                                            10350
```

```
            <td style="text-align:left;">{{ ut }}</td>
            <td style="text-align:left;">
              {% if "自定义" in label %}
              <span class="tag tag--warn">{{ label }}</span>
              {% else %}
              <span class="tag tag--muted">{{ label }}</span>
              {% endif %}
            </td>
          </tr>
          {% endfor %}
        </tbody>
      </table>
</div>
{% endif %}
{% endblock %}
# ==== app/templates/evaluate/result.html ====
{% extends "base.html" %}
{% block title %}评价结果{% endblock %}
{% block content %}
<div class="card">
  <h2>适宜性评价结果 · {{ patch.name if patch else "" }}</h2>
  <div class="stat-row">
    <div>
      <div class="stat__label">最终评价因子 F</div>
      <div class="stat__value mono">{{ r.final_score
    }}</div>
    </div>
    <div>
      <div class="stat__label">适宜性等级</div>
      <div class="stat__value ok">{{ r.grade }}</div>
    </div>
    <div style="flex:1;min-width:240px;">
      <div class="stat__label">推荐措施</div>
      <div style="font-size:15px;">{{ r.measures }}</div>
    </div>
  </div>
  {% if unit_scores %}
  <h3>各微地质单元得分</h3>
  <table style="max-width:860px;">
    <thead>
      <tr>
        <th scope="col">微地质单元</th>
        <th scope="col">面积占比</th>
        <th scope="col">单元评价因子 F'</th>
        <th scope="col">加权贡献</th>
        <th scope="col">权重来源</th>
      </tr>
    </thead>
    <tbody>
      {% for u in unit_scores %}
```

```
      <tr>                                                                  10401
        <td>{{ u.unit_type }}</td>                                          10402
        <td class="num">{{ "%.2f%%"|format(u.ratio * 100)                   10403
  }}</td>                                                                   10404
        <td class="mono">{{ u.unit_score }}</td>                            10405
        <td class="mono">{{ "%.4f"|format(u.ratio *                         10406
  u.unit_score) }}</td>                                                     10407
        <td class="muted">{{ u.weight_source or "论文表 3                   10408
  默认权重" }}</td>                                                         10409
      </tr>                                                                 10410
      {% endfor %}                                                          10411
    </tbody>                                                                10412
  </table>                                                                  10413
  {% endif %}                                                               10414
  {% if weak %}                                                             10415
  <h3>短板提示</h3>                                                         10416
  <p class="muted" style="margin-top:-4px;">                                10417
    按「权重 ×（1 − 标准化值）」排序，提升这些指标对总分的拉动最大：       10418
  </p>                                                                      10419
  <table style="max-width:760px;">                                          10420
    <thead>                                                                 10421
      <tr>                                                                  10422
        <th scope="col">微地质单元</th>                                     10423
        <th scope="col">指标</th>                                           10424
        <th scope="col">实测值</th>                                         10425
        <th scope="col">标准化值</th>                                       10426
        <th scope="col">提升空间</th>                                       10427
      </tr>                                                                 10428
    </thead>                                                                10429
    <tbody>                                                                 10430
      {% for w in weak %}                                                   10431
      <tr>                                                                  10432
        <td>{{ w.unit_type }}</td>                                          10433
        <td>{{ w.l2 }}</td>                                                 10434
        <td class="mono">{{ w.raw if w.raw is not none else                 10435
  "—" }}</td>                                                               10436
        <td class="mono">{{ "%.4f"|format(w.std_value)                      10437
  }}</td>                                                                   10438
        <td class="mono">{{ "%.4f"|format(w.gain) }}</td>                   10439
      </tr>                                                                 10440
      {% endfor %}                                                          10441
    </tbody>                                                                10442
  </table>                                                                  10443
  {% endif %}                                                               10444
  <p class="muted" style="margin-top:14px;">                                10445
    计算人：{{ operator or "—" }} ·                                         10446
    计算时间：{{ r.created_at | localdt("%Y-%m-%d %H:%M:%S") }} ·           10447
    参数来源：{{ param_sources }}                                           10448
  </p>                                                                      10449
  {% if params %}                                                           10450
```

```
  <details style="margin-top:8px;">                                    10451
    <summary class="muted">本次计算使用的参数快照</summary>              10452
    <table style="max-width:520px;">                                   10453
      <thead>                                                          10454
        <tr><th scope="col">参数</th><th                               10455
    scope="col">取值</th><th scope="col">来源</th></tr>                10456
      </thead>                                                         10457
      <tbody>                                                          10458
        {% for code, item in params.items() %}                         10459
        <tr>                                                           10460
          <td class="mono">{{ code }}</td>                             10461
          <td class="mono">{{ item.value }}</td>                       10462
          <td class="muted">{{ item.source }}</td>                     10463
        </tr>                                                          10464
        {% endfor %}                                                   10465
      </tbody>                                                         10466
    </table>                                                           10467
  </details>                                                           10468
  {% endif %}                                                          10469
  <p class="actions" style="margin-top:18px;">                         10470
    <a class="btn btn--ghost" href="/report/{{ r.patch_id              10471
    }}">查看得分明细</a>                                               10472
    <a class="btn btn--ghost" href="/evaluate/">返回评价列表</a>       10473
  </p>                                                                 10474
</div>                                                                 10475
{% endblock %}                                                         10476
# ==== app/templates/report/index.html ====                            10477
{% extends "base.html" %}                                              10478
{% block title %}报告输出{% endblock %}                                10479
{% block content %}                                                    10480
<div class="card">                                                     10481
  <h2>报告与结果输出</h2>                                              10482
  {% if not patches %}                                                 10483
  <p class="muted">暂无矿山图斑。请先到「项目与矿山档案」建立图斑。</p>  10484
  {% else %}                                                           10485
  <table>                                                              10486
    <thead>                                                            10487
      <tr>                                                             10488
        <th scope="col">所属项目</th>                                  10489
        <th scope="col">矿山图斑</th>                                  10490
        <th scope="col">可生成报告</th>                                10491
        <th scope="col">操作</th>                                      10492
      </tr>                                                            10493
    </thead>                                                           10494
    <tbody>                                                            10495
      {% for p in patches %}                                           10496
      {% set pr = progress.get(p.id) %}                                10497
      <tr>                                                             10498
        <td>{{ p.project.name if p.project else "—" }}</td>            10499
        <td style="text-align:left;">{{ p.name }}</td>                 10500
```

```
          <td style="text-align:left;">                                    10501
            {% if pr and pr.ready %}                                       10502
            <span class="tag tag--ok">可以</span>                          10503
            {% elif pr %}                                                  10504
            <span class="tag tag--warn">暂不可</span>                      10505
            <span class="muted">{{ pr.blocking[0].reason                   10506
    }}</span>                                                              10507
            {% endif %}                                                    10508
          </td>                                                            10509
          <td class="nowrap">                                              10510
            {% if pr and pr.ready %}                                       10511
            <span class="row-actions">                                     10512
              <a class="link-btn" href="/report/{{ p.id                    10513
    }}">查看得分明细</a>                                                  10514
              <a class="link-btn" href="/report/{{ p.id                    10515
    }}/pdf">导出 PDF</a>                                                  10516
              <a class="link-btn" href="/report/{{ p.id                    10517
    }}/word">导出 Word</a>                                                10518
            </span>                                                        10519
            {% else %}                                                     10520
            <span class="muted">—</span>                                   10521
            {% endif %}                                                    10522
          </td>                                                            10523
        </tr>                                                              10524
        {% endfor %}                                                       10525
      </tbody>                                                             10526
    </table>                                                               10527
    {% endif %}                                                            10528
</div>                                                                     10529
{% endblock %}                                                             10530
# ==== app/templates/report/detail.html ====                               10531
{% extends "base.html" %}                                                  10532
{% block title %}得分明细 · {{ patch.name }}{% endblock %}                 10533
{% block content %}                                                        10534
<div class="card">                                                         10535
  <h2>得分明细 · {{ patch.name }}</h2>                                     10536
  <div class="stat-row">                                                   10537
    <div>                                                                  10538
      <div class="stat__label">最终评价因子 F</div>                        10539
      <div class="stat__value mono">{{                                     10540
    "%.4f"|format(result.final_score) }}</div>                             10541
    </div>                                                                 10542
    <div>                                                                  10543
      <div class="stat__label">适宜性等级</div>                            10544
      <div class="stat__value ok">{{ result.grade }}</div>                 10545
    </div>                                                                 10546
    <div style="flex:1;min-width:240px;">                                  10547
      <div class="stat__label">推荐措施</div>                              10548
      <div style="font-size:15px;">{{                                      10549
    result.grade_info.measure }}</div>                                     10550
```

```
      </div>
    </div>
    <p class="muted" style="margin-top:12px;">
      本页与导出报告均取自
      <strong>{% if record %}{{ record.created_at |
      localdt("%Y-%m-%d %H:%M:%S") }}{% endif %}
      记录的那次评价结果</strong>（评价记录编号 {{ record.id if record else
      "—" }}），
      不会因事后修改指标数据而变化。
    </p>
    {% if undetected %}
    <p class="muted" style="margin-top:8px;">
      数据完整度说明：以下重金属项<strong>未检测</strong>，按论文注记以「未受污染」计入污染指标，
      并非实测达标 ——
      {% for unit_type, metals in undetected %}
        {{ unit_type }}：{{ metals|join("、") }}{% if not
      loop.last %}；{% endif %}
      {% endfor %}。
    </p>
    {% endif %}
    <p class="actions" style="margin-top:16px;">
      <a class="btn" href="/report/{{ patch.id }}/pdf">导出 PDF
      报告</a>
      <a class="btn btn--ghost" href="/report/{{ patch.id
      }}/word">导出 Word 报告</a>
      <a class="btn btn--ghost" href="/report/">返回</a>
    </p>
  </div>
  <div class="card">
    <h2>结果图表</h2>
    <h3>各微地质单元得分对比</h3>
    <div>{{ chart_units|safe }}</div>
    <h3>各一级指标平均得分</h3>
    <p class="muted">反映各一级指标对评价结果的实际影响，便于识别关键控制项。</p>
    <div>{{ chart_weight|safe }}</div>
  </div>
  {% for u in result.unit_results %}
  <div class="card">
    <h2>{{ u.unit_type }} · 占比 {{ "%.2f%%"|format(u.ratio *
      100) }} · 单元得分 {{ "%.4f"|format(u.unit_score) }}</h2>
    {% if u.weight_source %}
    <p class="muted" style="margin-top:-6px;">权重来源：{{
      u.weight_source }}</p>
    {% endif %}
    <table>
      <thead>
        <tr>
          <th scope="col">一级指标</th>
          <th scope="col">二级指标</th>
          <th scope="col">实测值</th>
```

```
          <th scope="col">标准化值</th>                                    10601
          <th scope="col">权重</th>                                        10602
          <th scope="col">贡献</th>                                        10603
        </tr>                                                              10604
      </thead>                                                             10605
      <tbody>                                                              10606
        {% for d in u.details %}                                           10607
        <tr>                                                               10608
          <td>{{ d.l1 }}</td>                                              10609
          <td style="text-align:left;">{{ d.l2 }}</td>                     10610
          <td class="mono" style="text-align:left;">{{ "—" if 10611
      d.raw is none else d.raw }}</td>                                     10612
          <td class="mono">{{ "—" if d.std_value is none else 10613
      "%.4f"|format(d.std_value) }}</td>                                   10614
          <td class="mono">{{ "%.4f"|format(d.weight) }}</td> 10615
          <td class="mono">{{ "—" if d.contribution is none  10616
      else "%.4f"|format(d.contribution) }}</td>                           10617
        </tr>                                                              10618
        {% endfor %}                                                       10619
      </tbody>                                                             10620
    </table>                                                               10621
    <p class="muted" style="margin:0;">                                    10622
      一级指标得分：                                                       10623
      {% for k, v in u.l1_scores.items() %}{{ k }} = <span     10624
      class="num">{{ "%.4f"|format(v) }}</span>{% if not       10625
      loop.last %};{% endif %}{% endfor %}                                 10626
    </p>                                                                   10627
</div>                                                                     10628
{% endfor %}                                                               10629
{% endblock %}                                                             10630
# ==== app/templates/report/print.html ====                                10631
<!DOCTYPE html>                                                            10632
<html lang="zh-CN">                                                        10633
<head>                                                                     10634
<meta charset="UTF-8">                                                     10635
<title>自然恢复适宜性评价报告 · {{ patch.name }}</title>                   10636
<style>                                                                    10637
  @page { size: A4; margin: 18mm 16mm; }                                   10638
  * { box-sizing: border-box; }                                            10639
  body {                                                                   10640
    margin: 0; color: #23262b; background: #fff;                           10641
    font-family: "Microsoft YaHei", "SimSun", sans-serif;                  10642
    font-size: 10.5pt; line-height: 1.65;                                  10643
  }                                                                        10644
  h1 { font-size: 17pt; text-align: center; margin: 0 0      10645
    6px; }                                                                 10646
  h2 { font-size: 13pt; color: #1f3a5f; margin: 20px 0 8px; 10647
       border-bottom: 1px solid #dfe3e8; padding-bottom:     10648
    4px; }                                                                 10649
  h3 { font-size: 11.5pt; color: #1f3a5f; margin: 16px 0    10650
```

```
    6px; }                                                                  10651
  .sub { text-align: center; color: #6b7280; font-size:                     10652
    9.5pt; margin-bottom: 18px; }                                           10653
  table { width: 100%; border-collapse: collapse; margin:                   10654
    8px 0 14px;                                                             10655
          font-variant-numeric: tabular-nums; }                             10656
  th, td { border: 1px solid #c9ccd1; padding: 4px 8px;                     10657
    text-align: center; }                                                   10658
  th { background: #eef2f7; font-weight: 600; }                             10659
  .left { text-align: left; }                                               10660
  .mono { font-family: Consolas, "Courier New", monospace; }                10661
  .kv { width: auto; }                                                      10662
  .kv th { width: 30%; text-align: left; }                                  10663
  .kv td { text-align: left; }                                              10664
  ul { margin: 6px 0 12px; padding-left: 20px; }                            10665
  .note { color: #6b7280; font-size: 9.5pt; }                               10666
  .chart { margin: 10px 0 16px; }                                           10667
  .sig { margin-top: 28px; font-size: 10pt; }                               10668
  .avoid-break { break-inside: avoid; }                                     10669
  svg { max-width: 100%; }                                                  10670
</style>                                                                    10671
</head>                                                                     10672
<body>                                                                      10673
  <h1>历史遗留废弃矿山自然恢复适宜性评价报告</h1>                                  10674
  <p class="sub">                                                           10675
    {{ patch.name }}                                                        10676
    {% if patch.admin_region %} · {{ patch.admin_region                     10677
    }}{% endif %}                                                           10678
    <br>历史遗留废弃矿山自然恢复适宜性评价系统 Ｖ1.0 ·                              10679
    生成时间 {{ generated_at | localdt }}                                   10680
  </p>                                                                      10681
  <h2>一、图斑基本信息</h2>                                                    10682
  <table class="kv">                                                        10683
    <tr><th>图斑名称</th><td>{{ patch.name }}</td></tr>                       10684
    <tr><th>所在行政区</th><td>{{ patch.admin_region or "—"                  10685
    }}</td></tr>                                                            10686
    <tr><th>图斑面积（公顷）</th><td>{{ patch.area_ha if                      10687
    patch.area_ha is not none else "—" }}</td></tr>                         10688
    <tr><th>开采矿种</th><td>{{ patch.mineral_type or "—"                   10689
    }}</td></tr>                                                            10690
    <tr><th>开采方式</th><td>{{ patch.mining_method or "—"                  10691
    }}</td></tr>                                                            10692
    <tr><th>责任主体缺失</th><td>{{ "是" if patch.responsible_entit          10693
    y_missing else "否" }}</td></tr>                                        10694
    {% if reference %}                                                      10695
    <tr><th>参照生态系统</th>                                                10696
        <td>植被覆盖度基准 {{ reference.coverage_benchmark }}%,              10697
            多样性指数基准 {{ reference.shannon_benchmark                   10698
    }}</td></tr>                                                            10699
    {% endif %}                                                             10700
```

```
</table>
<h2>二、评价结论</h2>
<table class="kv">
  <tr><th>最终评价因子 F</th><td class="mono"><strong>{{
  "%.4f"|format(result.final_score) }}</strong></td></tr>
  <tr><th>适宜性等级</th><td><strong>{{ result.grade
  }}</strong></td></tr>
  <tr><th>判定依据</th><td>{{ result.grade_info.meaning
  }}</td></tr>
  <tr><th>推荐措施</th><td>{{ result.grade_info.measure
  }}</td></tr>
</table>
<h2>三、各微地质单元得分</h2>
<table>
  <thead>
    <tr><th>微地质单元</th><th>面积占比</th><th>单元评价因子
  F'</th><th>加权贡献</th></tr>
  </thead>
  <tbody>
    {% for u in result.unit_results %}
    <tr>
      <td>{{ u.unit_type }}</td>
      <td class="mono">{{ "%.2f%%"|format(u.ratio * 100)
  }}</td>
      <td class="mono">{{ "%.4f"|format(u.unit_score)
  }}</td>
      <td class="mono">{{ "%.4f"|format(u.weighted)
  }}</td>
    </tr>
    {% endfor %}
  </tbody>
</table>
<div class="chart">{{ chart_units|safe }}</div>
<h2>四、一级指标权重分布</h2>
<div class="chart">{{ chart_weight|safe }}</div>
<h2>五、得分明细</h2>
{% for u in result.unit_results %}
<h3>{{ u.unit_type }}（占比 {{ "%.2f%%"|format(u.ratio *
  100) }}，单元得分 {{ "%.4f"|format(u.unit_score) }}）</h3>
<table>
  <thead>
    <tr>
      <th>一级指标</th><th>二级指标</th><th>实测值</th>
      <th>标准化值</th><th>权重</th><th>贡献</th>
    </tr>
  </thead>
  <tbody>
    {% for d in u.details %}
    <tr>
      <td>{{ d.l1 }}</td>
```

```
            <td class="left">{{ d.l2 }}</td>
            <td class="mono left">{{ "—" if d.raw is none else
      d.raw }}</td>
            <td class="mono">{{ "—" if d.std_value is none else
      "%.4f"|format(d.std_value) }}</td>
            <td class="mono">{{ "%.4f"|format(d.weight) }}</td>
            <td class="mono">{{ "—" if d.contribution is none
      else "%.4f"|format(d.contribution) }}</td>
          </tr>
          {% endfor %}
        </tbody>
      </table>
      <p class="note">
        一级指标得分：
        {% for k, v in u.l1_scores.items() %}{{ k }} = {{
        "%.4f"|format(v) }}{% if not loop.last %};{% endif %}{%
        endfor %}
      </p>
      {% endfor %}
      <h2>六、评价依据说明</h2>
      <ul class="note">
        <li>指标标准化按论文式（8） 执行，按指标性质分为越大越优、越小越优、
            适中型与离散赋值四类。</li>
        <li>单元评价因子按论文式（9） 由二级指标权重加权求得；
            整矿评价因子由各微地质单元按面积占比加权合成。</li>
        <li>适宜性等级按论文表 4 划分：F&lt;0.3 不适宜、0.3≤F≤0.7 基本适宜、F≥0.7
        适宜。</li>
        <li>指标权重取自论文表 3，按岩质边坡、碎石边坡、平台三类微地质单元分别确定。</li>
      </ul>
      <p class="sig">
        评价人（签字）：_____　　日期：_____
      </p>
    </body>
    </html>
    # ==== app/templates/search.html ====
    {% extends "base.html" %}
    {% from "_macros.html" import icon %}
    {% block title %}搜索 · 历史遗留废弃矿山自然恢复适宜性评价系统{% endblock %}
    {% block content %}
    <div class="card">
      <h2>搜索结果</h2>
      {% if not q %}
      <p class="muted">请输入关键词。可检索项目名称与编号、图斑名称与行政区 / 矿种、
        标准名称与文号。</p>
      {% else %}
      <p class="muted">
        关键词「<strong>{{ q }}</strong>」共找到
        <span class="num">{{ total }}</span> 条结果。
      </p>
      {% if not total %}
```

```
    <p class="muted">没有匹配的内容。换个词试试，或检查是否有错别字。</p>
    {% endif %}
    {% endif %}
</div>
{% if projects %}
<div class="card">
    <h2>评价项目（{{ projects|length }}）</h2>
    <table>
      <thead>
        <tr>
          <th scope="col">项目名称</th>
          <th scope="col">编号</th>
          <th scope="col">状态</th>
          <th scope="col">图斑数</th>
          <th scope="col">操作</th>
        </tr>
      </thead>
      <tbody>
        {% for p in projects %}
        <tr>
          <td style="text-align:left;">{{ p.name }}</td>
          <td class="mono">{{ p.code or "—" }}</td>
          <td><span class="tag tag--muted">{{ p.status
    }}</span></td>
          <td class="num">{{ p.patches.count() }}</td>
          <td class="nowrap">
            <span class="row-actions">
              <a class="link-btn" href="/project/{{ p.id
    }}/patches">矿山图斑</a>
              <a class="link-btn" href="/project/{{ p.id
    }}/edit">编辑</a>
            </span>
          </td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
</div>
{% endif %}
{% if patches %}
<div class="card">
    <h2>矿山图斑（{{ patches|length }}）</h2>
    <table>
      <thead>
        <tr>
          <th scope="col">所属项目</th>
          <th scope="col">图斑名称</th>
          <th scope="col">行政区</th>
          <th scope="col">矿种</th>
          <th scope="col">操作</th>
```

```
        </tr>
      </thead>
      <tbody>
        {% for m in patches %}
        <tr>
          <td>{{ m.project.name if m.project else "—" }}</td>
          <td style="text-align:left;">{{ m.name }}</td>
          <td>{{ m.admin_region or "—" }}</td>
          <td>{{ m.mineral_type or "—" }}</td>
          <td class="nowrap">
            <span class="row-actions">
              <a class="link-btn" href="/constraint/{{ m.id
}}">限制性判定</a>
              <a class="link-btn" href="/unit/{{ m.id
}}">微地质单元</a>
              <a class="link-btn" href="/evaluate/{{ m.id
}}">执行评价</a>
            </span>
          </td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
</div>
{% endif %}
{% if standards %}
<div class="card">
    <h2>标准与政策文件（{{ standards|length }}）</h2>
    <table>
      <thead>
        <tr>
          <th scope="col">类别</th>
          <th scope="col">省份</th>
          <th scope="col">名称</th>
          <th scope="col">文号</th>
          <th scope="col">操作</th>
        </tr>
      </thead>
      <tbody>
        {% for s in standards %}
        <tr>
          <td>{{ s.category }}</td>
          <td>{{ s.province }}</td>
          <td style="text-align:left;">{{ s.name }}</td>
          <td class="mono">{{ s.doc_number or "—" }}</td>
          <td class="nowrap">
            <span class="row-actions">
              <a class="link-btn" href="/standard/{{ s.id
}}/rules">规则</a>
            </span>
```

```
            </td>                                                                    10901
          </tr>                                                                      10902
          {% endfor %}                                                               10903
        </tbody>                                                                     10904
      </table>                                                                       10905
</div>                                                                               10906
{% endif %}                                                                          10907
{% endblock %}                                                                       10908
#  ==== seed_demo.py ====                                                           10909
#  -*- coding: utf-8 -*-                                                            10910
"""内置玉田县案例：把论文第 5 节案例数据写入数据库。                                  10911
用于功能验证与操作演示。数据逐字取自论文表 5 与第 5 节。                             10912
运行：python seed_demo.py                    幂等，只补写缺失的演示数据，            10913
                                                 不会删除用户自建的项目与图斑        10914
      python seed_demo.py --reset            连同标准库与指标定义一并重建            10915
"""                                                                                  10916
import json                                                                          10917
import sys                                                                           10918
sys.stdout.reconfigure(encoding="utf-8")                                             10919
from app import create_app                                                           10920
from app.models import (db, User, Project, MinePatch,                                10921
    ReferenceEcosystem,                                                              10922
                            MicroUnit, IndicatorData, Standard,                      10923
    StandardRule,                                                                    10924
                            ConstraintCheck, EvaluationResult,                       10925
    ScoreDetail)                                                                     10926
app = create_app()                                                                   10927
#  内置演示案例的项目编号，用于区分「演示数据」与「用户自建数据」                     10928
DEMO_CODE = "DEMO-001"                                                               10929
#  论文表 1：11 项地方标准与政策文件                                                 10930
STANDARDS = [                                                                        10931
    dict(category="地方标准", province="辽宁",                                       10932
    name="矿山废弃地植被自然恢复技术规程",                                           10933
        doc_number="DB21/T 3702—2023",                                               10934
    publish_date="2023-03-30",                                                       10935
        implement_date="2023-04-30",                                                 10936
        summary="首个正式发布的矿山自然恢复地方标准；提出必要条件和充分条件",       10937
        rules=[("植被覆盖度", ">=", "50", "必要条件", None),                         10938
               ("无地质灾害", "==", "是", "必要条件", None),                         10939
               ("无水土污染", "==", "是", "必要条件", None),                         10940
               ("与参照系差距", "<", "20", "充分条件", "%")]),                       10941
    dict(category="地方标准", province="湖北",                                       10942
    name="废弃露天矿山生态环境自然恢复标准",                                         10943
        doc_number="DB42/T 2421—2025",                                               10944
    publish_date="2025-08-20",                                                       10945
        implement_date="2025-10-20",                                                 10946
        summary="结合矿山生态需求进行自然恢复适宜性分级判断",                        10947
        rules=[("自然恢复适宜性分级", "==", "是", "必要条件", None)]),               10948
    dict(category="地方标准", province="内蒙古",                                     10949
    name="历史遗留废弃矿山自然恢复判定规范",                                         10950
```

```
        doc_number="DB15/T 4280—2026",                                  10951
    publish_date="2026-01-30",                                          10952
        implement_date="2026-03-01",                                    10953
        summary="调查内容包含参照生态系统调查",                                  10954
        rules=[("地质灾害发育程度", "<=", "弱发育", "必要条件", None),          10955
               ("地形地貌与周边协调", "==", "是", "必要条件", None),            10956
               ("有利于植被恢复的表层土壤", "==", "是", "必要条件", None),      10957
               ("已形成稳定植被覆盖", "==", "是", "必要条件", None)]),          10958
    dict(category="地方标准", province="江西",                              10959
    name="废弃矿山自然恢复技术规范",                                            10960
        doc_number="DB36/T 2221—2026",                                  10961
    publish_date="2026-02-02",                                          10962
        implement_date="2026-08-01",                                    10963
        summary="对参照生态系统进行了要求；提出自然恢复适宜性指数",                      10964
        rules=[("参照生态系统调查", "==", "是", "必要条件", None),              10965
               ("自然恢复适宜性指数", ">=", "阈值", "充分条件", None)]),        10966
    dict(category="地方标准", province="宁夏", name="矿山自然恢复认定规程",10967
        doc_number=None, publish_date="2026-06-18",                     10968
    implement_date=None,                                                10969
        summary="分区差异化：三个一级生态修复分区及 22 个二级分区",                 10970
        rules=[("无地质灾害风险", "==", "是", "必要条件", None),                10971
               ("植被覆盖度趋势", "==", "稳定或正增长", "必要条件", None),      10972
               ("与参照系差距", "<", "20", "充分条件", "%")]),                 10973
    dict(category="政策文件", province="安徽",                              10974
        name="安徽省自然资源厅关于规范废弃露天矿山采取自然恢复方式进行治理的通知",      10975
        doc_number="皖自然资修函[2020]34号",                                10976
    publish_date="2020-03-03", implement_date=None,                     10977
        summary="自然恢复条件：责任主体为政府、无地质灾害、无水土污染、不在三线三边、水土条件良好",10978
        rules=[("责任主体", "==", "政府", "必要条件", None),                    10979
               ("无地质灾害", "==", "是", "必要条件", None),                   10980
               ("无水土污染", "==", "是", "必要条件", None)]),                 10981
    dict(category="政策文件", province="山东",                              10982
        name="山东省自然资源厅关于进一步规范历史遗留废弃露天开采矿山采取自然恢复方式进行治理的通知",10983
        doc_number="鲁自然资字[2022]104号",                                 10984
    publish_date="2022-08-02", implement_date=None,                     10985
        summary="条件与安徽省相同，另增加自然恢复认定标准",                          10986
        rules=[("无地质灾害", "==", "是", "必要条件", None),                    10987
               ("无水土污染", "==", "是", "必要条件", None)]),                 10988
    dict(category="政策文件", province="湖南",                              10989
        name="关于印发《湖南省历史遗留矿山自然恢复指导意见（试行）》的通知",          10990
        doc_number="湘自资办发[2023]58号",                                  10991
    publish_date="2023-07-20", implement_date=None,                     10992
        summary="自然恢复认定标准：无地质灾害或无威胁对象、无水土污染、设备拆除、符合规划、植被覆盖度10993
    >75%、连续3年正向增长",                                                   10994
        rules=[("植被覆盖度", ">", "75", "必要条件", "%"),                     10995
               ("植被覆盖率趋势", "==", "连续3年正向增长", "必要条件", None),10996
               ("无地质灾害", "==", "是", "必要条件", None),                   10997
               ("符合国土空间规划", "==", "是", "必要条件", None)]),           10998
    dict(category="政策文件", province="新疆",                              10999
        name="自治区历史遗留废弃矿山自然恢复方式销号图斑工作指南（试行）",              11000
```

```
            doc_number="新自然资源办函〔2024〕219号",
        publish_date="2024-12-20", implement_date=None,
            summary="荒漠区沙地/裸土地/裸岩石砾地满足无地质灾害、地形协调、无水土污染即可判定",
            rules=[("植被覆盖度", ">=", "周边30%", "必要条件", None),
                   ("无地质灾害", "==", "是", "必要条件", None),
                   ("地形平缓地貌稳定", "==", "是", "必要条件", None)]),
        dict(category="政策文件", province="广西",
            name="广西壮族自治区历史遗留矿山自然恢复技术指南",
            doc_number="桂自然资办〔2024〕250号",
        publish_date="2024-12-30", implement_date=None,
            summary="除高陡边坡和裸露岩体外，水土水热条件良好、植被自然恢复趋势明显",
            rules=[("无地质灾害", "==", "是", "必要条件", None),
                   ("植被自然恢复趋势", "==", "明显", "必要条件", None),
                   ("生产设备拆除完毕", "==", "是", "必要条件", None)]),
        dict(category="政策文件", province="西藏",
            name="西藏自治区历史遗留废弃矿山生态自然恢复认定指导意见（试行）",
            doc_number="藏自然资发〔2025〕19号",
        publish_date="2025-09-24", implement_date=None,
            summary="含修复成本高、生态修复效益不显著等适用条件",
            rules=[("责任主体", "==", "政府", "必要条件", None),
                   ("地形地貌完整与周边协调", "==", "是", "必要条件", None),
                   ("无地质灾害", "==", "是", "必要条件", None)]),
]
CASE = {
    "name": "玉田县典型历史遗留废弃矿山案例",
    "patch_name": "玉田县孤树镇白云岩采石场",
    "area_ha": 5.6,
    "mineral_type": "白云岩",
    "units": [
        {
            "unit_type": "岩质边坡", "ratio": 0.1386,
            "data": {"coverage": 68.88, "shannon": 0.00,
                     "slope": 63.33, "aspect": "阴阳坡"},
        },
        {
            "unit_type": "碎石边坡", "ratio": 0.6295,
            "data": {"soil_ratio": 0.34, "thickness": 10,
    "ph": 9.24,
                     "organic_matter": 4.89,
                     "heavy_metals": {"Hg": 0.05, "As":
    5.17, "Ni": 12.60,
                                      "Cu": 11.70, "Cd":
    0.04, "Pb": 6.50},
                     "coverage": 87.92, "shannon": 0.60,
                     "slope": 28.75, "aspect": "阳坡"},
        },
        {
            "unit_type": "平台", "ratio": 0.2319,
            "data": {"soil_ratio": 0.56, "thickness": 30,
    "ph": 8.72,
```

```
                        "organic_matter": 10.92,                    11051
                        "heavy_metals": {"Hg": 0.02, "As":          11052
    3.13, "Ni": 12.33,                                              11053
                                                "Cu": 11.14, "Cd":  11054
    0.09, "Pb": 9.98},                                              11055
                        "coverage": 58.42, "shannon": 0.72},        11056
        },                                                          11057
    ],                                                              11058
}                                                                   11059
def clean_case(reset=False):                                        11060
    """清理数据。                                                   11061
    默认只清理内置演示案例（code = DEMO-001）自身的数据，**不触碰   11062
    用户自己建立的项目与图斑**。加 --reset 时才额外清空标准库与指标定义，  11063
    以便把演示环境恢复到初始状态。                                  11064
    注意：不要在此处执行 Project.query.delete()，否则用户新建的项目  11065
    会在每次启动时被删除。                                          11066
    """                                                             11067
    from app.models import EvaluationResult, ScoreDetail,           11068
    ConstraintCheck                                                 11069
    demo_projects = Project.query.filter_by(code=DEMO_CODE)         11070
    all()                                                           11071
    demo_ids = [p.id for p in demo_projects]                        11072
    patch_ids = []                                                  11073
    if demo_ids:                                                    11074
        patch_ids = [p.id for p in MinePatch.query.filter(          11075
            MinePatch.project_id.in_(demo_ids)).all()]              11076
    if patch_ids:                                                   11077
        unit_ids = [u.id for u in MicroUnit.query.filter(           11078
            MicroUnit.patch_id.in_(patch_ids)).all()]               11079
        if unit_ids:                                                11080
            IndicatorData.query.filter(                             11081
                IndicatorData.unit_id.in_(unit_ids)).delete(        11082
                synchronize_session=False)                          11083
        result_ids = [r.id for r in EvaluationResult.query.f        11084
    ilter(                                                          11085
            EvaluationResult.patch_id.in_(patch_ids)).all()]        11086
        if result_ids:                                              11087
            ScoreDetail.query.filter(                               11088
                ScoreDetail.result_id.in_(result_ids)).delet        11089
    e(                                                              11090
                synchronize_session=False)                          11091
        EvaluationResult.query.filter(                              11092
            EvaluationResult.patch_id.in_(patch_ids)).delete        11093
    (                                                               11094
            synchronize_session=False)                              11095
        ConstraintCheck.query.filter(                               11096
            ConstraintCheck.patch_id.in_(patch_ids)).delete(        11097
            synchronize_session=False)                              11098
        ReferenceEcosystem.query.filter(                            11099
            ReferenceEcosystem.patch_id.in_(patch_ids)).dele        11100
```

```
    te(
            synchronize_session=False)
        MicroUnit.query.filter(MicroUnit.patch_id.in_(patch_
    ids)).delete(
            synchronize_session=False)
        MinePatch.query.filter(MinePatch.id.in_(patch_ids)).
    delete(
            synchronize_session=False)
    if demo_ids:
        Project.query.filter(Project.id.in_(demo_ids)).delet
    e(
            synchronize_session=False)
    if reset:
        StandardRule.query.delete()
        Standard.query.delete()
    #  兜底：清理指向已不存在评价记录的孤儿得分明细，
    #  避免长期运行后 score_details 表堆积无效数据。
    _purge_orphan_details()
    db.session.commit()
    return len(demo_ids)
def demo_patch():
    """取内置演示案例的图斑。
    库中可能存在用户自建项目或验证算例（编号 VERIFY-），
    故不能简单用 MinePatch.query.first()，否则取到的对象不确定。
    没有演示案例时才退回第一个图斑。
    """
    proj = Project.query.filter_by(code=DEMO_CODE).first()
    if proj is not None:
        m = MinePatch.query.filter_by(project_id=proj.id).fi
    rst()
        if m is not None:
            return m
    return MinePatch.query.first()
def _purge_orphan_details():
    """清理指向已不存在评价记录的孤儿得分明细。
    长期运行后 score_details 表可能堆积无效明细，启动时顺带清一次。
    """
    live_ids = [r.id for r in EvaluationResult.query.all()]
    if live_ids:
        ScoreDetail.query.filter(~ScoreDetail.result_id.in_(
    live_ids)).delete(
            synchronize_session=False)
    else:
        ScoreDetail.query.delete()
    db.session.commit()
def seed(reset=False):
    with app.app_context():
        db.create_all()          #  全新环境下先建表，保证本脚本可独立运行
        _purge_orphan_details()
        admin = User.query.filter_by(username="admin").first
```

```
()                                                                  11151
    if not admin:                                                   11152
        from werkzeug.security import                               11153
generate_password_hash                                              11154
        admin = User(username="admin",                              11155
                     password_hash=generate_password_has            11156
h("admin123"),                                                      11157
                     role="admin")                                  11158
        db.session.add(admin)                                       11159
        db.session.commit()                                         11160
    #  幂等：演示案例已存在就不重复写入，更不清空用户自己的项目                  11161
    existing = Project.query.filter_by(code=DEMO_CODE).f            11162
irst()                                                              11163
    if existing and not reset:                                      11164
        print("演示案例已存在，跳过写入（用户自建项目不受影响）")                  11165
        _report()                                                   11166
        _seed_reference_data(reset=False)                           11167
        return                                                      11168
    clean_case(reset=reset)                                         11169
    project = Project(name=CASE["name"], code=DEMO_CODE,            11170
                      description="论文第 5 节典型案例",                    11171
created_by=admin.id)                                                11172
    db.session.add(project)                                         11173
    db.session.flush()                                              11174
    patch = MinePatch(project_id=project.id,                        11175
name=CASE["patch_name"],                                            11176
                      admin_region="河北省唐山市玉田县",                    11177
                      area_ha=CASE["area_ha"],                      11178
                      mineral_type=CASE["mineral_type"],            11179
                      mining_method="露天开采",                       11180
stop_years=None,                                                    11181
                      responsible_entity_missing=True)              11182
    db.session.add(patch)                                           11183
    db.session.flush()                                              11184
    ref = ReferenceEcosystem(patch_id=patch.id,                     11185
                             coverage_benchmark=75.0,               11186
shannon_benchmark=0.7)                                              11187
    db.session.add(ref)                                             11188
    for u in CASE["units"]:                                         11189
        unit = MicroUnit(patch_id=patch.id,                         11190
unit_type=u["unit_type"],                                           11191
                         ratio=u["ratio"], area=None)               11192
        db.session.add(unit)                                        11193
        db.session.flush()                                          11194
        d = u["data"]                                               11195
        ind = IndicatorData(                                        11196
            unit_id=unit.id,                                        11197
            soil_ratio=d.get("soil_ratio"),                         11198
            thickness=d.get("thickness"),                           11199
            ph=d.get("ph"),                                         11200
```

```
                    organic_matter=d.get("organic_matter"),
                    heavy_metals=json.dumps(d["heavy_metals"],
    ensure_ascii=False)
                                        if d.get("heavy_metals") else
    None,
                    coverage=d.get("coverage"),
                    shannon=d.get("shannon"),
                    slope=d.get("slope"),
                    aspect=d.get("aspect"),
                )
                db.session.add(ind)
            #  限制性条件判定记录：演示案例按论文第 5 节描述属「无地质灾害隐患点」、
            #  具备自然恢复条件，故五项全部通过。缺此记录会使后续评价被流程门禁拦住。
            db.session.add(ConstraintCheck(
                patch_id=patch.id,
                policy_pass=True, location_pass=True,
    pollution_pass=True,
                geo_pass=True, farmland_pass=True,
                passed=True, reason="全部通过"))
            db.session.commit()
            print("已写入案例：", CASE["name"])
            print("   图斑：", CASE["patch_name"])
            print("   微地质单元：岩质边坡 13.86% / 碎石边坡 62.95% / 平台
    23.19%")
            print("   管理员账号：admin / admin123")
        _seed_reference_data(reset=reset)
        _report()
def _seed_reference_data(reset=False):
    """写入标准库与指标定义。
    参考资料只在库为空（或显式 --reset）时写入，避免覆盖用户自行维护
    的标准条目与指标定义。
    """
    if reset or Standard.query.count() == 0:
        seed_standards()
    from app.services.indicator_service import
    seed_indicators
    n = seed_indicators()
    print("指标定义：%d 项" % n)
def _report():
    total_p = Project.query.count()
    total_patch = MinePatch.query.count()
    print()
    print("当前库内：项目 %d 个、图斑 %d 个" % (total_p, total_patch))
    if total_p > total_patch:
        print("   提示：项目是归档单位，评价针对的是图斑。")
        print("         新项目需先建立图斑，才会出现在后续各模块中。")
def seed_standards():
    """写入论文表 1 的 11 项标准与政策文件及其结构化规则。"""
    StandardRule.query.delete()
    Standard.query.delete()
```

```
        db.session.commit()                                                    11251
        for s in STANDARDS:                                                    11252
            std = Standard(category=s["category"],                             11253
        province=s["province"],                                                11254
                           name=s["name"],                                     11255
        doc_number=s["doc_number"],                                            11256
                           publish_date=s["publish_date"],                     11257
                           implement_date=s["implement_date"],                 11258
                           summary=s["summary"])                               11259
            db.session.add(std)                                                11260
            db.session.flush()                                                 11261
            for item, op, thr, ctype, note in s["rules"]:                      11262
                db.session.add(StandardRule(standard_id=std.id,                11263
        item=item,                                                             11264
                                                operator=op,                   11265
        threshold=thr,                                                         11266
                                                                               11267
        condition_type=ctype, note=note))                                      11268
        db.session.commit()                                                    11269
        print("已写入标准/政策文件：%d 项，规则 %d 条"                                  11270
              % (len(STANDARDS), sum(len(s["rules"]) for s in                  11271
        STANDARDS)))                                                           11272
if __name__ == "__main__":                                                     11273
        # --reset：把演示环境恢复到初始状态（清空标准库与指标定义后重建）。                    11274
        # 无论是否 --reset，都不会删除用户自建的项目与图斑。                              11275
        seed(reset="--reset" in sys.argv[1:])                                  11276
#  ==== 测试/test_ahp.py ====                                                  11277
#  -*- coding: utf-8 -*-                                                       11278
"""地基自检：验证 AHP 引擎与应用可启动。"""                                              11279
import sys                                                                     11280
sys.stdout.reconfigure(encoding="utf-8")                                       11281
import numpy as np                                                             11282
import os                                                                      11283
#  本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，                              11284
#  使 `from app import ...` 与 app/app.db 的定位不受影响。                       11285
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__fil                   11286
        e__)))                                                                 11287
if ROOT not in sys.path:                                                       11288
        sys.path.insert(0, ROOT)                                               11289
from app.services import ahp_service as ahp                                    11290
LINE = "=" * 66                                                                11291
#  ------------------------------------------------------------------          11292
        ------------------                                                     11293
#  1） 完全一致矩阵 → 权重应精确还原，CR=0                                          11294
#  ------------------------------------------------------------------          11295
        ------------------                                                     11296
print(LINE)                                                                    11297
print("测试 1  完全一致判断矩阵的权重还原与一致性")                                        11298
print(LINE)                                                                    11299
#  由权重构造完全一致矩阵：a_ij = w_i / w_j                                       11300
```

```
w_true = np.array([0.44, 0.31, 0.25])
A = np.outer(w_true, 1.0 / w_true)
r = ahp.weights_and_consistency(A)
print("  判断矩阵 A：")
for row in A:
    print("    " + "  ".join(f"{v:8.4f}" for v in row))
print()
print("  计算权重 :", "  ".join(f"{v:.4f}" for v in
    r["weights"]))
print("  真实权重 :", "  ".join(f"{v:.4f}" for v in w_true))
print("  λmax = %.6f   CI = %.3e   CR = %.3e   一致: %s"
      % (r["lambda_max"], r["ci"], r["cr"],
    r["consistent"]))
assert np.allclose(r["weights"], w_true, atol=1e-6),
    "权重未精确还原"
assert r["consistent"], "完全一致矩阵应通过检验"
# ------------------------------------------------------------------
    ----------------
# 2） 上三角输入 → 完整矩阵 → 一致性定位
# ------------------------------------------------------------------
    ----------------
print()
print(LINE)
print("测试 2  上三角录入与最不一致项定位")
print(LINE)
# 一个不太一致的 3 阶矩阵：上三角 [2, 5, 3]（a12=2, a13=5, a23=3）
M = ahp.matrix_from_upper([2, 5, 3])
r2 = ahp.weights_and_consistency(M)
bad = ahp.find_most_inconsistent(M)
print("  由 [a12=2, a13=5, a23=3] 构造矩阵：")
for row in M:
    print("    " + "  ".join(f"{v:8.4f}" for v in row))
print("  权重:", "  ".join(f"{v:.4f}" for v in r2["weights"]))
      "  CR = %.3f  一致: %s" % (r2["cr"], r2["consistent"]))
print("  最不一致比较项: 第(%d,%d) 项，当前 %.3f，理想 %.3f"
      % (bad["i"] + 1, bad["j"] + 1, bad["value"],
    bad["expected"]))
# ------------------------------------------------------------------
    ----------------
# 3） 多专家聚合（几何平均）
# ------------------------------------------------------------------
    ----------------
print()
print(LINE)
print("测试 3  多专家判断矩阵聚合（几何平均）")
print(LINE)
m1 = ahp.matrix_from_upper([2, 4, 3])
m2 = ahp.matrix_from_upper([3, 5, 2])
agg = ahp.aggregate_matrices([m1, m2])
print("  专家1 上三角 [2,4,3]，专家2 上三角 [3,5,2]")
```

```
print("   聚合后上三角：  a12=%.4f  a13=%.4f  a23=%.4f"                    11351
      % (agg[0, 1], agg[0, 2], agg[1, 2]))                                 11352
assert abs(agg[0, 1] - np.sqrt(2 * 3)) < 1e-9, "几何平均错误"               11353
# ------------------------------------------------------------------------11354
    ------------------                                                     11355
# 4） 论文表 3 权重可还原性                                                11356
# ------------------------------------------------------------------------11357
    ------------------                                                     11358
print()                                                                  11359
print(LINE)                                                              11360
print("测试 4  论文表 3 碎石边坡一级权重 （0.44, 0.31, 0.25）")           11361
print(LINE)                                                              11362
j = ahp.judge_from_weights([0.44, 0.31, 0.25])                           11363
print("   λmax = %.6f    CR = %.3e    一致：%s" %                          11364
    (j["lambda_max"], j["cr"], j["consistent"]))                         11365
assert j["consistent"], "论文权重应能作为完全一致矩阵主特征向量"          11366
# ------------------------------------------------------------------------11367
    ------------------                                                     11368
# 5） 应用可启动 + 建表                                                    11369
# ------------------------------------------------------------------------11370
    ------------------                                                     11371
print()                                                                  11372
print(LINE)                                                              11373
print("测试 5  Flask 应用启动与建表")                                     11374
print(LINE)                                                              11375
from app import create_app                                               11376
from app.models import db                                                11377
app = create_app()                                                       11378
with app.app_context():                                                  11379
    db.create_all()                                                      11380
    tables = sorted(db.metadata.tables.keys())                           11381
print("   已创建 %d 张表：" % len(tables))                                11382
for t in tables:                                                         11383
    print("     -", t)                                                  11384
print()                                                                  11385
print(LINE)                                                              11386
print("全部测试通过 √")                                                  11387
print(LINE)                                                              11388
# ==== 测试/test_evaluate.py ====                                        11389
# -*- coding: utf-8 -*-                                                  11390
"""评价引擎自检：用论文玉田县案例跑通完整计算链。"""                      11391
import sys                                                               11392
sys.stdout.reconfigure(encoding="utf-8")                                 11393
import os                                                                11394
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，                11395
# 使 `from app import ...` 与 app/app.db 的定位不受影响。                  11396
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__fil             11397
    e__)))                                                               11398
if ROOT not in sys.path:                                                 11399
    sys.path.insert(0, ROOT)                                             11400
```

```
from app.services import evaluation_service as ev
from app.services import standardization as std
LINE = "=" * 70
# ------------------------------------------------------------------
# 论文第 5 节案例数据（表 5，逐字抄录）
# ------------------------------------------------------------------
# 三类单元面积占比：岩质边坡 13.86%、碎石边坡 62.95%、平台 23.19%
# 注：岩质边坡无土壤指标、平台无地形指标，对应"不适用"。
case = [
    {
        "unit_type": "岩质边坡",
        "ratio": 0.1386,
        "data": {
            "coverage": 68.88, "shannon": 0.00,
            "slope": 63.33, "aspect": "阴阳坡",
        },
    },
    {
        "unit_type": "碎石边坡",
        "ratio": 0.6295,
        "data": {
            "soil_ratio": 0.34, "thickness": 10, "ph":
    9.24, "organic_matter": 4.89,
            "heavy_metals": {"Hg": 0.05, "As": 5.17, "Ni":
    12.60,
                             "Cu": 11.70, "Cd": 0.04, "Pb":
    6.50},
            "coverage": 87.92, "shannon": 0.60,
            "slope": 28.75, "aspect": "阳坡",
        },
    },
    {
        "unit_type": "平台",
        "ratio": 0.2319,
        "data": {
            "soil_ratio": 0.56, "thickness": 30, "ph":
    8.72, "organic_matter": 10.92,
            "heavy_metals": {"Hg": 0.02, "As": 3.13, "Ni":
    12.33,
                             "Cu": 11.14, "Cd": 0.09, "Pb":
    9.98},
            "coverage": 58.42, "shannon": 0.72,
        },
    },
]
# ------------------------------------------------------------------
# 逐单元输出标准化与得分
```

```
# ------------------------------------------------------------------------11451
    ----------------                                                    11452
print(LINE)                                                             11453
print("玉田县案例 · 评价引擎完整计算")                                  11454
print(LINE)                                                             11455
for u in case:                                                          11456
    r = ev.score_unit(u["unit_type"], u["data"])                        11457
    print()                                                             11458
    print("  【%s】  (面积占比 %.2f%%)" % (u["unit_type"],              11459
    u["ratio"] * 100))                                                  11460
    print("  " + "-" * 60)                                             11461
    for l1, score in r["l1_scores"].items():                            11462
        print("    %s 得分 = %.4f" % (l1, score))                      11463
    print("    单元评价因子 F' = %.4f" % r["unit_score"])              11464
print()                                                                 11465
print(LINE)                                                             11466
print("面积加权合成（式(9)）")                                          11467
print(LINE)                                                             11468
res = ev.score_mine(case)                                               11469
total = res["final_score"]                                              11470
print()                                                                 11471
for ur in res["unit_results"]:                                          11472
    print("  %s  %.2f%% × %.4f = %.4f"                                 11473
          % (ur["unit_type"], ur["ratio"] * 100,                        11474
    ur["unit_score"], ur["weighted"]))                                  11475
print("  " + "-" * 60)                                                 11476
print("  最终评价因子 F = %.4f" % total)                               11477
print()                                                                 11478
print("  适宜性等级：%s" % res["grade"])                               11479
print("  判定依据  ：%s" % res["grade_info"]["meaning"])               11480
print("  推荐措施  ：%s" % res["grade_info"]["measure"])               11481
print()                                                                 11482
print(LINE)                                                             11483
print("结论")                                                           11484
print(LINE)                                                             11485
print("  - 评价引擎按论文式(8)(9) 与表 3 权重完整跑通")                 11486
print("  - 坡度标准化采用论文 25° 分界的线性折减默认值（见 standardization.py）") 11487
print("  - 最终 F ≈ %.2f，判定为「%s」，与论文「最终适宜自然恢复」的结论一致"  11488
      % (total, res["grade"]))                                          11489
print("  - 论文表 5 所载最终得分为 0.76；差异集中在坡度、坡向的标准化口径，")  11490
print("    该口径在 standardization.std_slope / aspect_score           11491
    中集中定义、可配置")                                                11492
print(LINE)                                                             11493
# ==== 测试/test_web_ahp.py ====                                        11494
# -*- coding: utf-8 -*-                                                 11495
"""AHP 网页界面自检：走通选择→录入→计算 全流程。"""                      11496
import sys                                                              11497
sys.stdout.reconfigure(encoding="utf-8")                                11498
import os                                                               11499
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，              11500
```

```
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
LINE = "=" * 66
# 测试配置：关闭 CSRF 以简化测试客户端（真实运行仍开启 CSRF）
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///:memory:"
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
# 建立表并创建测试用户，登录后再访问业务页面
with app.app_context():
    from app.models import db, User
    from werkzeug.security import generate_password_hash
    db.create_all()
    if not User.query.filter_by(username="admin").first():
        db.session.add(User(username="admin",
                            password_hash=generate_password_hash("admin123"),
                            role="admin"))
        db.session.commit()
client.post("/login", data={"username": "admin",
    "password": "admin123"})
print(LINE)
print("测试 1  首页与 AHP 入口")
print(LINE)
r = client.get("/")
print("  首页状态码：", r.status_code, "（应 200）")
assert r.status_code == 200
r = client.get("/ahp/")
print("  AHP 入口状态码：", r.status_code, "（应 200）")
assert r.status_code == 200
assert "微地质单元类型" in r.get_data(as_text=True)
print()
print(LINE)
print("测试 2  选择碎石边坡一级指标 → 跳转矩阵页")
print(LINE)
r = client.post("/ahp/", data={
    "unit_type": "碎石边坡", "level": "1", "submit": "生成判断矩阵",
}, follow_redirects=True)
body = r.get_data(as_text=True)
print("  跳转后页面含「判断矩阵录入」：", "判断矩阵录入" in body)
print("  含三个一级指标（土壤/植被/地形）：", all(n in body for n in
    ("土壤指标", "植被指标", "地形指标")))
```

```
assert "判断矩阵录入" in body
assert all(n in body for n in ("土壤指标", "植被指标", "地形指标"))
print()
print(LINE)
print("测试 3　提交论文表 3 一致矩阵 → 还原权重")
print(LINE)
# 碎石边坡一级权重 (0.44, 0.31, 0.25) 对应的完全一致矩阵上三角：
# a12 = 0.44/0.31 = 1.4194, a13 = 0.44/0.25 = 1.76, a23 =
    0.31/0.25 = 1.24
r = client.post("/ahp/matrix?unit_type=碎石边坡&level=1", data={
    "m_0_1": "1.4194", "m_0_2": "1.76", "m_1_2": "1.24",
}, follow_redirects=True)
body = r.get_data(as_text=True)
print("  结果页含「计算结果」:", "计算结果" in body)
print("  含权重 0.44:", "0.44" in body, " 含 0.31:", "0.31" in
    body, " 含 0.25:", "0.25" in body)
print("  含一致性检验通过:", "通过" in body)
assert "计算结果" in body
assert "0.44" in body and "0.31" in body and "0.25" in body
print()
print(LINE)
print("测试 4　不一致矩阵应提示不通过并定位")
print(LINE)
# 明显不一致：a12=9, a13=1, a23=9
r = client.post("/ahp/matrix?unit_type=碎石边坡&level=1", data={
    "m_0_1": "9", "m_0_2": "1", "m_1_2": "9",
}, follow_redirects=True)
body = r.get_data(as_text=True)
print("  含「不通过」:", "不通过" in body)
print("  含「建议优先调整」:", "建议优先调整" in body)
assert "不通过" in body
assert "建议优先调整" in body
print()
print(LINE)
print("网页界面全部测试通过 √")
print(LINE)
# ==== 测试/test_web_modules.py ====
# -*- coding: utf-8 -*-
"""模块自检：登录、项目档案、微地质单元、标准库、限制性判定。"""
import os
import sys
sys.stdout.reconfigure(encoding="utf-8")
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__
    )))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
LINE = "=" * 66
```

```
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        ROOT, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
print(LINE)
print("测试 1  未登录应被重定向到登录页")
print(LINE)
r = client.get("/project/")
print("  /project/ 状态码:", r.status_code, " 重定向到:",
    r.headers.get("Location"))
assert r.status_code == 302 and "login" in
    r.headers.get("Location", "")
print()
print(LINE)
print("测试 2  登录")
print(LINE)
r = client.post("/login", data={"username": "admin",
    "password": "admin123"},
                follow_redirects=True)
body = r.get_data(as_text=True)
#  工作台：页首横幅 + 评价流程分区 + 数据概览
print("  登录后含「评价流程」:", "评价流程" in body)
print("  含账户 admin:", "admin" in body)
print("  含页首横幅:", 'class="hero"' in body)
print("  含数据概览:", 'class="stat-card"' in body)
assert "评价流程" in body and 'class="hero"' in body
print()
print(LINE)
print("测试 3  项目与矿山档案")
print(LINE)
r = client.get("/project/")
body = r.get_data(as_text=True)
print("  项目列表状态码:", r.status_code)
print("  含案例项目:", "玉田县典型历史遗留废弃矿山案例" in body)
assert "玉田县典型历史遗留废弃矿山案例" in body
#  新建项目
client.post("/project/add", data={"name": "测试项目A", "code":
    "T-001"})
r = client.get("/project/")
print("  新建项目后含「测试项目A」:", "测试项目A" in r.get_data(as_text=True
    ))
assert "测试项目A" in r.get_data(as_text=True)
#  清理测试数据，避免污染演示库
with app.app_context():
    from app.models import db, Project
```

```
    for p in Project.query.filter_by(name="测试项目A").all():
        db.session.delete(p)
    db.session.commit()
print("  已清理测试项目")
print()
print(LINE)
print("测试 4  微地质单元（面积占比自动计算）")
print(LINE)
from app.models import MinePatch
with app.app_context():
    patch = MinePatch.query.filter_by(name="玉田县孤树镇白云岩采石场").first()
    pid = patch.id
r = client.get("/unit/%d" % pid)
body = r.get_data(as_text=True)
print("  单元列表含三类单元:", all(t in body for t in ("岩质边坡",
    "碎石边坡", "平台")))
print("  含占比合计:", "占比合计" in body)
assert "岩质边坡" in body and "占比合计" in body
print()
print(LINE)
print("测试 5  标准知识库与省际口径比对")
print(LINE)
r = client.get("/standard/")
body = r.get_data(as_text=True)
print("  含辽宁/湖南/新疆标准:", all(p in body for p in ("辽宁", "湖南",
    "新疆")))
assert "辽宁" in body
r = client.get("/standard/compare")
body = r.get_data(as_text=True)
print("  口径比对页含「植被覆盖度」分组:", "植被覆盖度" in body)
print("  含多省份并列:", "湖南" in body and "新疆" in body)
assert "植被覆盖度" in body
print()
print(LINE)
print("测试 6  限制性指标判定（一票否决）")
print(LINE)
r = client.get("/constraint/%d" % pid)
body = r.get_data(as_text=True)
print("  含五项条件:", all(x in body for x in ("政策约束", "区位约束",
    "土壤污染", "地质灾害", "耕地属性")))
assert all(x in body for x in ("政策约束", "区位约束", "土壤污染",
    "地质灾害", "耕地属性"))
# 全部通过
client.post("/constraint/%d" % pid, data={
    "policy_pass": "1", "location_pass": "1",
    "pollution_pass": "1",
    "geo_pass": "1", "farmland_pass": "1"})
r = client.get("/constraint/%d" % pid)
body = r.get_data(as_text=True)
```

```
print("  全通过后显示「通过，可进入定量评价」：", "可进入定量评价" in body)
assert "可进入定量评价" in body
# 一项不通过
client.post("/constraint/%d" % pid, data={
    "policy_pass": "1", "location_pass": "0",
    "pollution_pass": "1",
    "geo_pass": "1", "farmland_pass": "1"})
r = client.get("/constraint/%d" % pid)
body = r.get_data(as_text=True)
print("  一项不通过后显示「不通过」：", "不通过" in body)
print("  含失败项「区位约束」：", "区位约束" in body)
assert "不通过" in body
# 恢复为全部通过，避免把演示数据留在「不通过」状态影响后续步骤
client.post("/constraint/%d" % pid, data={
    "policy_pass": "1", "location_pass": "1",
    "pollution_pass": "1",
    "geo_pass": "1", "farmland_pass": "1"})
r = client.get("/constraint/%d" % pid)
print("  已恢复为全部通过：", "可进入定量评价" in r.get_data(as_text=True))
print()
print(LINE)
print("全部测试通过 √")
print(LINE)
# ==== 测试/test_web_evaluate.py ====
# -*- coding: utf-8 -*-
"""端到端自检：录数据 → 执行评价 → 查看结果。"""
import os
import sys
sys.stdout.reconfigure(encoding="utf-8")
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
LINE = "=" * 66
# 与真实配置共用同一个 app.db，但关闭 CSRF 便于测试客户端
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        ROOT, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
# 登录（案例种子已创建 admin 用户）
client.post("/login", data={"username": "admin",
    "password": "admin123"})
```

```
print(LINE)
print("测试 1  指标数据列表")
print(LINE)
r = client.get("/indicator/")
body = r.get_data(as_text=True)
print("  状态码：", r.status_code)
print("  含案例图斑：", "玉田县孤树镇白云岩采石场" in body)
print("  含三个单元：", all(n in body for n in ("岩质边坡", "碎石边坡",
    "平台")))
assert r.status_code == 200
assert "岩质边坡" in body and "碎石边坡" in body and "平台" in body
print()
print(LINE)
print("测试 2  指标数据录入页（按单元类型显示字段）")
print(LINE)
from app.models import MicroUnit, MinePatch
with app.app_context():
    unit_rock = MicroUnit.query.filter_by(unit_type="岩质边坡").
    first()
    unit_gravel = MicroUnit.query.filter_by(unit_type="碎石边坡"
    ).first()
r = client.get("/indicator/%d" % unit_rock.id)
b_rock = r.get_data(as_text=True)
print("  岩质边坡页：无「土石比」字段：", "土石比" not in b_rock, "（应 True）")
print("  岩质边坡页：含「植被覆盖度」：", "植被覆盖度" in b_rock)
assert "土石比" not in b_rock and "植被覆盖度" in b_rock
r = client.get("/indicator/%d" % unit_gravel.id)
b_gravel = r.get_data(as_text=True)
print("  碎石边坡页：含「土石比」字段：", "土石比" in b_gravel)
print("  碎石边坡页：含 8 种重金属：", all(m in b_gravel for m in
    ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu", "Zn")))
assert "土石比" in b_gravel and all(m in b_gravel for m in
    ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu", "Zn"))
print()
print(LINE)
print("测试 3  执行评价")
print(LINE)
with app.app_context():
    import seed_demo
    patch = seed_demo.demo_patch()
r = client.post("/evaluate/%d" % patch.id, data={},
    follow_redirects=True)
body = r.get_data(as_text=True)
print("  跳转至结果页：", "适宜性评价结果" in body)
print("  含最终评价因子 F：", "最终评价因子 F" in body)
print("  含等级「适宜」：", "适宜" in body)
print("  含推荐措施「自然恢复」：", "自然恢复" in body)
assert "最终评价因子 F" in body and "自然恢复" in body
print()
print(LINE)
```

```
print("测试 4  结果持久化")
print(LINE)
from app.models import EvaluationResult
with app.app_context():
    res = EvaluationResult.query.order_by(EvaluationResult.id.desc()).first()
    print("  最终得分 F =", res.final_score)
    print("  等级 =", res.grade)
    assert res.grade == "适宜"
    assert 0.7 <= res.final_score < 1.0
print()
print(LINE)
print("端到端全部测试通过 √")
print(LINE)
#  ==== 测试/test_web_report.py ====
#  -*- coding: utf-8 -*-
"""报告输出自检：得分明细页 + Word 导出。"""
import os
import sys
import io
sys.stdout.reconfigure(encoding="utf-8")
#  本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
#  使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
LINE = "=" * 66
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        ROOT, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
#  登录（案例种子已创建 admin 用户）
client.post("/login", data={"username": "admin",
    "password": "admin123"})
from app.models import MinePatch, EvaluationResult
with app.app_context():
    import seed_demo
    patch = seed_demo.demo_patch()
    pid = patch.id
#  报告取自已保存的评价记录，因此先按网页路径真实执行一次评价
from app.views.evaluate import load_patch_input,
    save_evaluation
with app.app_context():
```

```
        units = load_patch_input(pid)
        rec = save_evaluation(pid, units)
        rid = rec.id
        print("前置：已执行评价并保存记录 id=%d，F=%.4f（%s）"
              % (rid, rec.final_score, rec.grade))
        from app.models import ScoreDetail
        n_detail = ScoreDetail.query.filter_by(result_id=rid).co
        unt()
        print("         逐层得分明细已落库：%d 条" % n_detail)
print()
print(LINE)
print("测试 1  报告列表")
print(LINE)
r = client.get("/report/")
print("  状态码：", r.status_code)
body = r.get_data(as_text=True)
print("  含「所属项目」列：", "所属项目" in body)
assert r.status_code == 200
print()
print(LINE)
print("测试 2  得分明细页")
print(LINE)
r = client.get("/report/%d" % pid)
body = r.get_data(as_text=True)
print("  状态码：", r.status_code)
print("  含「得分明细」：", "得分明细" in body)
print("  含「最终评价因子 F」：", "最终评价因子 F" in body)
print("  含二级指标明细（植被覆盖度/坡度/坡向）：", all(n in body for n in
    ("植被覆盖度", "坡度", "坡向")))
print("  含权重/标准化值列：", "标准化值" in body and "贡献" in body)
assert r.status_code == 200
assert "植被覆盖度" in body and "标准化值" in body
print()
print(LINE)
print("测试 3  Word 报告导出")
print(LINE)
r = client.get("/report/%d/word" % pid)
print("  状态码：", r.status_code)
print("  Content-Type:", r.headers.get("Content-Type",
    "")[:60])
data = r.data
print("  文件大小：", len(data), "字节")
print("  是否为有效 docx（PK 头）：", data[:2] == b"PK")
assert r.status_code == 200 and data[:2] == b"PK"
# 用 python-docx 打开验证内容
import docx
d = docx.Document(io.BytesIO(data))
text = "\n".join(p.text for p in d.paragraphs)
print("  含报告标题：", "自然恢复适宜性评价报告" in text)
print("  含评价结论章节：", "评价结论" in text)
```

```
print("   含得分明细章节：", "得分明细" in text)
print("   段落数：", len(d.paragraphs), " 表格数：", len(d.tables))
assert "自然恢复适宜性评价报告" in text and len(d.tables) >= 3
print()
print(LINE)
print("报告输出全部测试通过 √")
print(LINE)
#  ==== 测试/test_ui_fixes.py ====
#  -*- coding: utf-8 -*-
"""界面修复自检：「记住我」与各模块删除功能。"""
import io
import os
import re
import sys
#  本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
#  使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
sys.stdout.reconfigure(encoding="utf-8")
LINE = "=" * 70
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        ROOT, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
    REMEMBER_COOKIE_DURATION = __import__("datetime").timede
    lta(days=30)
app = create_app(TestConfig)
client = app.test_client()
#  ----------------------------------------------------------------
    ----------
print(LINE)
print("一、「记住我」")
print(LINE)
html = client.get("/login").get_data(as_text=True)
#  1） 复选框必须有可点击关联的 label（这是原先失效的根因）
#      只判断「复选框与文字是否落在同一个 <label> 内」，不绑定具体类名 ——
#      否则换个样式类就会误报，而真正的回归点其实与类名无关。
labels = re.findall(r"<label[^>]*>(.*?)</label>", html,
    re.S)
wrapped = any(('name="remember"' in b and "记住我" in b) for b
    in labels)
print("   %s 「记住我」文字与复选框包在同一 label 内" % ("[符合]" if wrapped
    else "[不符合]"))
assert wrapped, "复选框未与文字关联，点击文字不会勾选"
```

```
has_for = 'for="remember"' in html
print("  %s label 带 for 属性指向复选框" % ("[符合]" if has_for else
    "[不符合]"))
# 2) 勾选后应下发带过期时间的 remember_token
c = app.test_client()
r = c.post("/login", data={"username": "admin", "password":
    "admin123",
                           "remember": "y"},
    follow_redirects=False)
cookies = r.headers.getlist("Set-Cookie")
rt = [h for h in cookies if h.startswith("remember_token=")]
print("  %s 勾选后下发 remember_token" % ("[符合]" if rt else
    "[不符合]"))
assert rt, "未下发 remember_token"
exp = re.search(r"Expires=([^;]+)", rt[0])
print("  %s 该 cookie 带过期时间：%s"
      % ("[符合]" if exp else "[不符合]", exp.group(1) if exp
    else "无（会话 cookie）"))
assert exp, "remember_token 无过期时间，关浏览器即失效"
# 3) 不勾选时不应下发 remember_token
c2 = app.test_client()
r2 = c2.post("/login", data={"username": "admin",
    "password": "admin123"},
             follow_redirects=False)
rt2 = [h for h in r2.headers.getlist("Set-Cookie") if
    h.startswith("remember_token=")]
print("  %s 未勾选时不下发 remember_token" % ("[符合]" if not rt2
    else "[不符合]"))
assert not rt2
# 4) 携带 remember_token 的新会话应自动登录
c3 = app.test_client()
c3.set_cookie("remember_token",
              rt[0].split(";")[0].split("=", 1)[1],
    domain="localhost")
r3 = c3.get("/", follow_redirects=False)
print("  %s 仅凭 remember_token 访问首页可自动登录（状态码 %d）"
      % ("[符合]" if r3.status_code == 200 else "[不符合]",
    r3.status_code))
assert r3.status_code == 200
# ------------------------------------------------------------------
    ----------------
print()
print(LINE)
print("二、删除功能")
print(LINE)
client.post("/login", data={"username": "admin",
    "password": "admin123"})
# 项目：列表页须有删除按钮
html = client.get("/project/").get_data(as_text=True)
has_del = "/delete" in html and "btn--danger" in html
```

```
print("  %s 项目列表含删除按钮" % ("[符合]" if has_del else "[不符合]"))
assert has_del, "项目列表缺少删除按钮"
print("  %s 删除前有二次确认" % ("[符合]" if "confirm(" in html else
    "[不符合]"))
assert "confirm(" in html
# 图斑列表：地址用的是项目主键，不是图斑主键
with app.app_context():
    from app.models import MinePatch, Project
    import seed_demo
    patch = seed_demo.demo_patch()
    patch_id = patch.id
    project_id = patch.project_id
html = client.get("/project/%d/patches" %
    project_id).get_data(as_text=True)
print("  %s 图斑列表含删除按钮" % ("[符合]" if "/project/patch/" in
    html and "btn--danger" in html else "[不符合]"))
assert "/project/patch/" in html and "btn--danger" in html
# 微地质单元
html = client.get("/unit/%d" % patch_id).get_data(as_text=Tr
    ue)
print("  %s 微单元列表含删除按钮" % ("[符合]" if "/unit/delete/" in
    html else "[不符合]"))
assert "/unit/delete/" in html
print("  %s 微单元编辑为独立页面（表单不再嵌在表格行内）"
      % ("[符合]" if "/unit/edit/" in html else "[不符合]"))
# 标准
html = client.get("/standard/").get_data(as_text=True)
print("  %s 标准列表含删除按钮" % ("[符合]" if "btn--danger" in html
    else "[不符合]"))
assert "btn--danger" in html
# 实际执行一次删除：新建项目 → 删除 → 确认消失
client.post("/project/add", data={"name": "待删除测试项目",
    "code": "DEL-1"})
with app.app_context():
    from app.models import db, Project
    tmp = Project.query.filter_by(name="待删除测试项目").first()
    tmp_id = tmp.id
html = client.get("/project/").get_data(as_text=True)
print("  %s 新建项目成功" % ("[符合]" if "待删除测试项目" in html else
    "[不符合]"))
r = client.post("/project/%d/delete" % tmp_id,
    follow_redirects=True)
html = r.get_data(as_text=True)
gone = "待删除测试项目" not in html
print("  %s 删除项目后列表中不再出现" % ("[符合]" if gone else "[不符合]"))
assert gone
print("  %s 有删除成功提示" % ("[符合]" if "已删除" in html else
    "[不符合]"))
print()
print(LINE)
```

```
print("六、顶栏宽度在各页面保持一致")
print(LINE)
#  滚动条出现在  .app-content  内部：内容高的页面会因此把顶栏挤窄约  10px，
#  于是「顶栏在不同页面不一样宽」。scrollbar-gutter: stable  预留槽位后各页一致。
base_src = io.open(os.path.join(ROOT, "app", "templates",
    "base.html"),
                   encoding="utf-8").read()
has_gutter = "scrollbar-gutter: stable" in base_src
print("   %s  内容区预留滚动条槽位（顶栏宽度不再随页面变化）"
      % ("[符合]" if has_gutter else "[不符合]"))
assert has_gutter, "缺少 scrollbar-gutter:
    stable，顶栏宽度会因滚动条而不一致"
print()
print(LINE)
print("界面修复自检全部通过 √")
print(LINE)
#  ==== 测试/test_new_features.py ====
#  -*- coding: utf-8 -*-
"""补齐模块自检：操作日志与用户维护、遥感影像、指标与多专家矩阵、
批量导入、图表与 PDF 导出。
对应计划书功能清单与系统实现的差异核对结论。
"""
import io
import json
import os
import re
import sys
sys.stdout.reconfigure(encoding="utf-8")
import fitz
#  本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
#  使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file
    __)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
from app.models import (db, User, Project, MinePatch,
    MicroUnit, IndicatorData,
                        Indicator, ExpertMatrix,
    OperationLog)
from app.utils.schema import ensure_schema
from app.utils import browser, charts
LINE = "=" * 72
BASE = ROOT
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
```

```
    TESTING = True
    UPLOAD_FOLDER = os.path.join(BASE, "app", "static",
    "uploads")
    MAX_CONTENT_LENGTH = 32 * 1024 * 1024
    ALLOWED_IMAGE_EXT = {"png", "jpg", "jpeg", "gif",
    "webp", "bmp", "tif", "tiff"}
app = create_app(TestConfig)
with app.app_context():
    db.create_all()
    ensure_schema(db)
    ExpertMatrix.query.delete()
    db.session.commit()
    # 固定取内置演示案例的图斑，避免受库中用户自建项目 / 验证算例的影响
    import seed_demo
    patch_id = seed_demo.demo_patch().id
c = app.test_client()
c.post("/login", data={"username": "admin", "password":
    "admin123"})
def ok(flag, label, extra=""):
    print("  %s %-44s %s" % ("[符合]" if flag else "[不符合]",
    label, extra))
    assert flag, label
# ------------------------------------------------------------
    ------------------
print(LINE)
print("操作日志与用户维护")
print(LINE)
before = None
with app.app_context():
    before = OperationLog.query.count()
c.post("/project/add", data={"name": "日志核对项目", "code":
    "LOG-T"})
with app.app_context():
    after = OperationLog.query.count()
    last = OperationLog.query.order_by(OperationLog.id.desc(
    )).first()
ok(after == before + 1, "写操作被写入操作日志", "%d -> %d" % (before,
    after))
ok(last.action == "新建项目", "日志记录了可读动作名", last.action)
ok("project" in (last.target_type or ""), "日志记录了对象类型",
    last.target_type)
r = c.get("/profile?tab=logs")
body = r.get_data(as_text=True)
ok(r.status_code == 200 and "操作日志" in body, "操作日志分类可访问")
ok("新建项目" in body, "日志页显示操作记录")
# 账号编辑 / 删除（职务只有管理员与操作员两种）
with app.app_context():
    u = User(username="tmp_user", role="operator",
             password_hash=User.query.first().password_hash)
    db.session.add(u); db.session.commit()
```

```
    uid = u.id                                                          12151
r = c.get("/users/%d/edit" % uid)                                       12152
ok(r.status_code == 200, "账号编辑页可访问")                              12153
#  保护校验要趁系统里只有一名管理员时做：                                  12154
#  先降级唯一的 admin 应被拒绝，再把临时账号提为管理员看修改是否生效       12155
with app.app_context():                                                 12156
    admin_id = User.query.filter_by(role="admin").first().id            12157
c.post("/users/%d/edit" % admin_id,                                     12158
       data={"role": "operator", "active": "1"})                        12159
with app.app_context():                                                 12160
    ok(db.session.get(User, admin_id).role == "admin",                  12161
    "拒绝把最后一个管理员降级")                                          12162
c.post("/users/%d/edit" % uid, data={"role": "admin",                   12163
    "active": "1"})                                                     12164
with app.app_context():                                                 12165
    ok(db.session.get(User, uid).role == "admin",                       12166
    "职务已修改并保存")                                                  12167
c.post("/users/%d/delete" % uid, follow_redirects=True)                 12168
with app.app_context():                                                 12169
    ok(db.session.get(User, uid) is None, "账号删除生效")               12170
#  清理日志核对用的项目                                                 12171
with app.app_context():                                                 12172
    for p in Project.query.filter_by(name="日志核对项目").all():        12173
        db.session.delete(p)                                            12174
    db.session.commit()                                                 12175
#  ------------------------------------------------------------------   12176
    ------------------                                                  12177
print()                                                                 12178
print(LINE)                                                             12179
print("遥感影像挂接")                                                    12180
print(LINE)                                                             12181
doc = fitz.open()                                                       12182
pg = doc.new_page(width=320, height=200)                                12183
pg.insert_text((20, 60), "RS image", fontname="china-s",                12184
    fontsize=16)                                                        12185
png = doc[0].get_pixmap(dpi=100).tobytes("png")                         12186
doc.close()                                                             12187
r = c.post("/project/patch/%d/image" % patch_id,                        12188
           data={"image": (io.BytesIO(png), "影像.png"),                12189
                 "image_note": "2025年6月无人机正射影像"},              12190
           content_type="multipart/form-data",                          12191
    follow_redirects=True)                                              12192
body = r.get_data(as_text=True)                                         12193
ok("已挂接" in body and "<img" in body, "影像上传并显示")                12194
with app.app_context():                                                 12195
    rel = db.session.get(MinePatch, patch_id).image_path                12196
ok(bool(rel), "数据库记录影像路径", rel)                                 12197
abs_path = os.path.join(TestConfig.UPLOAD_FOLDER,                       12198
    rel.replace("uploads/", "", 1))                                     12199
ok(os.path.isfile(abs_path), "影像文件落盘")                             12200
```

```
ok(c.get("/static/" + rel).status_code == 200, "影像可经 URL 访问")
r = c.post("/project/patch/%d/image" % patch_id,
           data={"image": (io.BytesIO(b"x"), "bad.txt")},
           content_type="multipart/form-data",
           follow_redirects=True)
ok("不支持" in r.get_data(as_text=True), "拒绝非图片格式")
c.post("/project/patch/%d/image/delete" % patch_id,
       follow_redirects=True)
with app.app_context():
    ok(db.session.get(MinePatch, patch_id).image_path is None, "影像移除后字段清空")
ok(not os.path.isfile(abs_path), "影像文件已删除")
# ------------------------------------------------------------------
print()
print(LINE)
print("指标定义维护与多专家矩阵")
print(LINE)
r = c.get("/ahp/indicators")
body = r.get_data(as_text=True)
with app.app_context():
    n_ind = Indicator.query.count()
ok(r.status_code == 200 and n_ind > 0, "指标定义已入库并可查看", "%d 项" % n_ind)
ok(all(k in body for k in ("一级指标", "二级指标", "三级指标")),
   "指标树按层级展示")
ok("岩质边坡" in body and "平台" in body, "显示各指标启用的微地质单元")
c.post("/ahp/indicators/coverage/edit",
       data={"name": "植被覆盖度", "unit": "%", "ideal_type": "benefit",
             "ideal_value": "75", "enabled_units": ["岩质边坡"],
             "description": "自检修改"})
with app.app_context():
    ok(Indicator.query.filter_by(code="coverage").first().description == "自检修改",
       "指标定义可编辑")
c.post("/ahp/indicators/reset")
with app.app_context():
    ok("自检修改" not in (Indicator.query.filter_by(code="coverage").first().description or ""),
       "可按论文取值重置")
base = {"m_0_1": "1.4194", "m_0_2": "1.76", "m_1_2": "1.24"}
for name in ("张工", "李工", "王工"):
    d = dict(base); d.update({"action": "add", "unit_type": "碎石边坡",
                              "level": "1", "expert_name": name})
    c.post("/ahp/experts", data=d)
with app.app_context():
```

```
        ok(ExpertMatrix.query.filter_by(unit_type="碎石边坡",
        level=1).count() == 3,
            "录入 3 位专家的判断矩阵")
r = c.get("/ahp/experts/aggregate?unit_type=碎石边坡&level=1")
body = r.get_data(as_text=True)
ok("汇总后权重" in body and "各专家矩阵的一致性" in body,
    "多专家矩阵几何平均汇总可用")
ws = re.search(r"<th scope=\"row\">权重</th>\s*(.*?)</tr>",
        body, re.S)
vals = re.findall(r">([0-9.]+)</td>", ws.group(1)) if ws
        else []
ok(len(vals) == 3 and abs(sum(map(float, vals)) - 1.0) <
        0.01,
    "汇总权重归一", "、".join(vals))
ok(abs(float(vals[0]) - 0.44) < 0.02, "汇总权重接近论文表 3 取值",
        vals[0] if vals else "")
# ------------------------------------------------------------------
        ------------------
print()
print(LINE)
print("批量导入")
print(LINE)
r = c.get("/indicator/import/template")
tpl = r.data.decode("utf-8-sig")
ok(r.status_code == 200 and "单元类型" in tpl, "导入模板可下载")
ok("土石比" in tpl and "植被覆盖度" in tpl, "模板包含指标列")
csv_text = (
        "单元类型,土石比,土层厚度,pH,有机质,Hg,As,Ni,Cu,Cd,Pb,植被覆盖度,Shannon,坡度
        ,坡向\n"
        "碎石边坡,0.34,10,9.24,4.89,0.05,5.17,12.60,11.70,0.04,6.50,
        87.92,0.60,28.75,阳坡\n"
        "平台,0.56,30,8.72,10.92,0.02,3.13,12.33,11.14,0.09,9.98,5
        8.42,0.72,,,\n"
        "不存在单元,1,1,7,1,,,,,,,,,\n"
)
r = c.post("/indicator/import",
           data={"patch_id": str(patch_id),
                 "datafile": (io.BytesIO(csv_text.encode("ut
        f-8")), "d.csv")},
           content_type="multipart/form-data",
        follow_redirects=True)
body = r.get_data(as_text=True)
ok("导入结果" in body, "导入结果页展示")
ok("不存在单元" in body, "错误行被报告")
with app.app_context():
        u = MicroUnit.query.filter_by(patch_id=patch_id,
        unit_type="碎石边坡").first()
        ind = IndicatorData.query.filter_by(unit_id=u.id).first(
        )
        ok(abs(ind.soil_ratio - 0.34) < 1e-6 and
```

```
    abs(ind.coverage - 87.92) < 1e-6,                                  12301
        "导入数据已写入数据库")                                          12302
r = c.post("/indicator/import",                                        12303
           data={"patch_id": str(patch_id),                            12304
                 "datafile": (io.BytesIO(b"x"), "d.pdf")},             12305
           content_type="multipart/form-data",                         12306
    follow_redirects=True)                                             12307
ok("仅支持" in r.get_data(as_text=True), "拒绝不支持的导入格式")          12308
# ------------------------------------------------------------------12309
    ----------------                                                   12310
print()                                                                12311
print(LINE)                                                            12312
print("图表与 PDF 导出")                                               12313
print(LINE)                                                            12314
svg = charts.hbars([("岩质边坡", 0.54, "0.5401"), ("平台", 0.84,       12315
    "0.8376")])                                                        12316
ok("<svg" in svg and "0.8376" in svg, "条形图生成且带数值标签")         12317
svg2 = charts.donut([("植被指标", 0.6, "0.6000"), ("土壤指标", 0.4,12318
    "0.4000")])                                                        12319
ok("<svg" in svg2 and "占比" in svg2, "环形图生成且带图例")              12320
# 报告是评价结论的呈现，需先真实执行一次评价才能打开                    12321
with app.app_context():                                                12322
    from app.views.evaluate import load_patch_input,                   12323
    save_evaluation                                                    12324
    save_evaluation(patch_id, load_patch_input(patch_id))              12325
r = c.get("/report/%d" % patch_id)                                     12326
body = r.get_data(as_text=True)                                        12327
ok(body.count("<svg") >= 2, "得分明细页含图表", "%d 个 SVG" %          12328
    body.count("<svg"))                                                12329
exe = browser.browser_path()                                           12330
if exe is None:                                                        12331
    print("  [跳过] 未找到浏览器，PDF 导出无法验证")                    12332
else:                                                                  12333
    r = c.get("/report/%d/pdf" % patch_id)                             12334
    data = r.data                                                      12335
    ok(data[:4] == b"%PDF", "PDF 报告导出成功", "%d 字节" %            12336
    len(data))                                                         12337
    d = fitz.open(stream=data, filetype="pdf")                         12338
    txt = "".join(d[i].get_text() for i in                             12339
    range(d.page_count))                                               12340
    ok("自然恢复适宜性评价报告" in txt, "PDF 含报告标题")              12341
    ok("评价结论" in txt and "得分明细" in txt, "PDF 含结论与明细章节")  12342
    ok(d.page_count >= 2, "PDF 页数合理", "%d 页" % d.page_count)      12343
    d.close()                                                          12344
print()                                                                12345
print(LINE)                                                            12346
print("补齐模块自检全部通过 √")                                        12347
print(LINE)                                                            12348
# ==== 测试/test_workflow.py ====                                      12349
# -*- coding: utf-8 -*-                                                12350
```

```
"""流程门禁自检：前置步骤未完成时，后续模块必须拦得住。
论文　4．1　节确定的评价流程存在先后关系，缺任何一环都不应产出评价结论。
本测试逐个构造「缺前置」的场景，验证系统不会放行。
"""
import os
import sys
sys.stdout.reconfigure(encoding="utf-8")
#  本脚本位于　测试/　子目录，先把仓库根目录加入模块搜索路径，
#  使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
from app.models import (db, Project, MinePatch, MicroUnit,
    IndicatorData,
                        ConstraintCheck, EvaluationResult)
from app.services import workflow as wf
LINE = "=" * 72
BASE = ROOT
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
c = app.test_client()
c.post("/login", data={"username": "admin", "password":
    "admin123"})
_fails = []
def ok(flag, label, extra=""):
    print("  %s %-46s %s" % ("[符合]" if flag else "[不符合]",
    label, extra))
    if not flag:
        _fails.append(label)
def purge_fixtures():
    """开跑前先清掉上一次可能残留的自检项目。
    自检项目按　code　前缀命名（WF-）；如果上一次跑到一半被打断（报错、被中断），
    清理函数没执行到，残留的项目就会出现在工作台与项目列表里　——
    生成软著截图时这些测试数据会被拍进去。
    """
    with app.app_context():
        leftovers = Project.query.filter(Project.code.like("
    WF-%")).all()
        for p in leftovers:
            for m in p.patches.all():
                db.session.delete(m)
            db.session.delete(p)
```

```
        db.session.commit()
        return len(leftovers)
def new_patch(tag):
    with app.app_context():
        p = Project(name="门禁-" + tag, code="WF-" + tag)
        db.session.add(p); db.session.flush()
        patch = MinePatch(project_id=p.id, name="门禁图斑-" +
    tag)
        db.session.add(patch); db.session.flush()
        pid = patch.id
        db.session.commit()
    return pid
def add_units(pid):
    with app.app_context():
        db.session.add(MicroUnit(patch_id=pid,
    unit_type="碎石边坡",
                                 area=1.0, ratio=1.0))
        db.session.commit()
def add_constraint(pid, passed=True):
    with app.app_context():
        db.session.add(ConstraintCheck(
            patch_id=pid, policy_pass=True,
    location_pass=passed,
            pollution_pass=True, geo_pass=True,
    farmland_pass=True,
            passed=passed, reason="全部通过" if passed else
    "未通过：区位约束"))
        db.session.commit()
def add_data(pid):
    with app.app_context():
        u = MicroUnit.query.filter_by(patch_id=pid).first()
        db.session.add(IndicatorData(unit_id=u.id,
    soil_ratio=0.34, thickness=10,
                                     ph=9.24,
    organic_matter=4.89, coverage=87.92,
                                     shannon=0.60,
    slope=28.75, aspect="阳坡"))
        db.session.commit()
def cleanup(pid, tag):
    with app.app_context():
        from app.models import ScoreDetail
        # 先删得分明细，再删评价记录，避免留下孤儿明细
        rids = [r.id for r in EvaluationResult.query.filter_
    by(patch_id=pid)]
        if rids:
            ScoreDetail.query.filter(ScoreDetail.result_id.i
    n_(rids)).delete(
                synchronize_session=False)
        EvaluationResult.query.filter_by(patch_id=pid).delet
    e()
```

```
        ConstraintCheck.query.filter_by(patch_id=pid).delete
    ()
        uids = [u.id for u in MicroUnit.query.filter_by(pat
    h_id=pid)]
        if uids:
            IndicatorData.query.filter(IndicatorData.unit_id
    .in_(uids)).delete(
                synchronize_session=False)
        MicroUnit.query.filter_by(patch_id=pid).delete()
        m = db.session.get(MinePatch, pid)
        if m:
            db.session.delete(m)
        pr = Project.query.filter_by(name="门禁-" +
    tag).first()
        if pr:
            db.session.delete(pr)
        db.session.commit()
def results_of(pid):
    with app.app_context():
        return EvaluationResult.query.filter_by(patch_id=pid
    ).count()
def post_eval(pid):
    return c.post("/evaluate/%d" % pid, data={},
    follow_redirects=False)
def get_report(pid):
    return c.get("/report/%d" % pid, follow_redirects=False)
# ----------------------------------------------------------------
    ----------------
print(LINE)
print("一、前置条件缺失时，评价必须被拦住")
print(LINE)
print("  （开跑前清掉 %d 个上一次残留的自检项目）" % purge_fixtures())
print()
print("场景 1：连微地质单元都没划分")
pid = new_patch("1")
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "未划分单元 → 不产生评价结果",
   "HTTP %d，结果数 %d" % (r.status_code, n))
with app.app_context():
    pr = wf.patch_progress(pid)
steps = [b["step"] for b in pr["blocking"]]
ok(not pr["ready"] and "units" in steps,
   "拦原因包含「划分微地质单元」", "、".join(steps))
ok(steps == ["constraint", "units"],
   "拦截顺序符合论文 4.1 节：限制性判定在前，划分单元在后",
   "、".join(steps))
cleanup(pid, "1")
print()
print("场景 2：有单元，但未做限制性条件判定")
```

```
pid = new_patch("2"); add_units(pid); add_data(pid)
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "未做限制性判定 → 不产生评价结果",
   "结果数 %d" % n)
with app.app_context():
    pr = wf.patch_progress(pid)
ok(any(b["step"] == "constraint" for b in pr["blocking"]),
   "拦原因指向「执行限制性条件判定」")
cleanup(pid, "2")
print()
print("场景 3：限制性条件判定【未通过】")
pid = new_patch("3"); add_units(pid); add_data(pid);
    add_constraint(pid, passed=False)
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "判定未通过 → 不进入定量评价",
   "结果数 %d" % n)
with app.app_context():
    pr = wf.patch_progress(pid)
ok(not pr["constraint"]["passed"], "流程状态显示判定未通过")
blocked = [b for b in pr["blocking"] if b["step"] ==
    "constraint"]
# 提示只写「缺什么」：label 说明未通过，reason 直接引用判定记录里没过的项
ok(bool(blocked) and "未通过" in blocked[0]["label"],
   "拦提示指明限制性条件判定未通过", blocked[0]["label"] if blocked else
    "")
ok(bool(blocked) and "区位约束" in (blocked[0]["reason"] or ""),
   "并把具体没通过的条件带出来", blocked[0]["reason"] if blocked else "")
cleanup(pid, "3")
print()
print("场景 4：判定通过，但指标数据全空")
pid = new_patch("4"); add_units(pid); add_constraint(pid,
    passed=True)
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "数据未录入 → 不产生评价结果",
   "结果数 %d" % n)
with app.app_context():
    pr = wf.patch_progress(pid)
ok(any(b["step"] == "data" for b in pr["blocking"]),
   "拦原因指向「录入指标数据」")
ok(pr["data"]["missing"] == ["碎石边坡"],
   "指出具体缺哪个单元的数据", "、".join(pr["data"]["missing"]))
cleanup(pid, "4")
print()
print("场景 5：单元面积占比合计不为 100%")
pid = new_patch("5"); add_constraint(pid, passed=True)
with app.app_context():
    u = MicroUnit(patch_id=pid, unit_type="碎石边坡", area=1.0,
```

```
        ratio=0.5)                                                    12551
        db.session.add(u); db.session.commit()                        12552
        db.session.add(IndicatorData(unit_id=u.id,                    12553
        soil_ratio=0.3, thickness=10,                                 12554
                                              ph=8.0,                 12555
        organic_matter=5.0, coverage=80.0,                            12556
                                              shannon=0.6, slope=20.0,12557
        aspect="阳坡"))                                               12558
        db.session.commit()                                           12559
r = post_eval(pid)                                                    12560
ok(results_of(pid) == 0, "占比不足 100% → 不产生评价结果")                12561
cleanup(pid, "5")                                                     12562
print()                                                               12563
print(LINE)                                                           12564
print("二、前置条件齐备时才放行")                                        12565
print(LINE)                                                           12566
pid = new_patch("OK"); add_units(pid); add_constraint(pid,            12567
        passed=True); add_data(pid)                                   12568
with app.app_context():                                               12569
        pr = wf.patch_progress(pid)                                   12570
ok(pr["ready"], "单元 + 判定通过 + 数据齐备 → 判定为可执行")              12571
r = post_eval(pid)                                                    12572
n = results_of(pid)                                                   12573
ok(n == 1, "此时执行评价成功产生结果", "结果数 %d" % n)                   12574
print("   重定向到:", r.headers.get("Location"))                       12575
print()                                                               12576
print(LINE)                                                           12577
print("三、报告不得先于评价产生")                                        12578
print(LINE)                                                           12579
pid2 = new_patch("R")     # 什么都不做                                12580
r = get_report(pid2)                                                  12581
ok(r.status_code == 302 and "/report/" ==                             12582
        (r.headers.get("Location") or ""),                            12583
    "前置缺失时 /report/<id> 被退回列表", "HTTP %d" % r.status_code)   12584
for suffix in ("/pdf", "/word"):                                      12585
        rr = c.get("/report/%d%s" % (pid2, suffix),                   12586
        follow_redirects=False)                                       12587
        ok(rr.status_code == 302, "导出 %s 同样被拦住" % suffix, "HTTP12588
        %d" % rr.status_code)                                         12589
cleanup(pid2, "R")                                                    12590
cleanup(pid, "OK")                                                    12591
print()                                                               12592
print(LINE)                                                           12593
print("四、前置齐备但从未执行评价时，报告仍不得产生")                    12594
print(LINE)                                                           12595
#  报告是评价结论的呈现：即使前置步骤都做完了，只要没真正执行过定量评价，   12596
#  也不应凭空推算出结论。                                               12597
pid3 = new_patch("N")                                                 12598
add_units(pid3); add_constraint(pid3, passed=True);                   12599
        add_data(pid3)                                                12600
```

```
with app.app_context():
    pr = wf.report_progress(pid3)
ok(not pr["ready"], "无评价记录 → 报告流程判定为未就绪")
ok(bool(pr["blocking"]) and pr["blocking"][0]["step"] ==
    "evaluate",
   "拦原因指向「执行适宜性评价」")
r = get_report(pid3)
ok(r.status_code == 302 and results_of(pid3) == 0,
   "直接访问 /report/<id> 被退回，且不产生任何评价结果",
   "HTTP %d" % r.status_code)
#  真正执行评价之后，报告才放行，且取值与评价记录一致
r = post_eval(pid3)
ok(results_of(pid3) == 1, "先执行评价 → 产生评价记录")
with app.app_context():
    pr = wf.report_progress(pid3)
    rec = pr["result"]
ok(pr["ready"] and rec is not None, "有评价记录后，报告流程就绪")
rr = get_report(pid3)
body = rr.get_data(as_text=True)
ok(rr.status_code == 200, "此时可打开得分明细页", "HTTP %d" %
    rr.status_code)
ok(("%.4f" % rec.final_score) in body,
   "报告数值取自评价记录，而非重新计算",
   "F=%.4f" % rec.final_score)
cleanup(pid3, "N")
print()
print(LINE)
print("五、内置演示数据的清理不得波及用户自建数据")
print(LINE)
#  回归保护：seed_demo.clean_case() 曾执行 Project.query.delete(),
#  导致用户新建的项目在每次启动时被连带删除。
import seed_demo as sd
with app.app_context():
    keep_p = Project(name="用户自建项目-守护", code="KEEP-001")
    db.session.add(keep_p); db.session.flush()
    keep_patch = MinePatch(project_id=keep_p.id,
    name="用户自建图斑-守护")
    db.session.add(keep_patch); db.session.flush()
    keep_pid = keep_p.id
    keep_patch_id = keep_patch.id
    db.session.commit()
with app.app_context():
    sd.clean_case(reset=False)
with app.app_context():
    still_p = db.session.get(Project, keep_pid)
    still_patch = db.session.get(MinePatch, keep_patch_id)
ok(still_p is not None, "清理演示数据后，用户自建项目仍存在")
ok(still_patch is not None, "清理演示数据后，用户自建图斑仍存在")
#  演示数据清掉后应能重新写入，保证演示环境可恢复
sd.seed()
```

```
with app.app_context():
    demo_back = Project.query.filter_by(code=sd.DEMO_CODE).f
    irst()
ok(demo_back is not None, "演示案例可重新写入（幂等恢复）")
with app.app_context():
    m = db.session.get(MinePatch, keep_patch_id)
    if m:
        db.session.delete(m)
    p = db.session.get(Project, keep_pid)
    if p:
        db.session.delete(p)
    db.session.commit()
print()
print(LINE)
print("六、同一图斑不得重复划分同类型微地质单元")
print(LINE)
#  论文 4.1 节把图斑分为平台、碎石边坡、岩质边坡三类，每类至多一个。
#  允许重复会让面积占比、按类型组织的指标数据与批量导入对不上号。
pid4 = new_patch("D")
c.post("/unit/%d/add" % pid4, data={"unit_type": "平台",
    "area": "1.0"},
       follow_redirects=True)
with app.app_context():
    n1 = MicroUnit.query.filter_by(patch_id=pid4).count()
ok(n1 == 1, "首次添加「平台」成功", "单元数 %d" % n1)
r = c.post("/unit/%d/add" % pid4, data={"unit_type": "平台",
    "area": "2.0"},
              follow_redirects=True)
with app.app_context():
    n2 = MicroUnit.query.filter_by(patch_id=pid4).count()
ok(n2 == 1, "重复添加同类型被拒绝", "单元数 %d" % n2)
ok("同一类型只能划分一个" in r.get_data(as_text=True), "给出重复提示")
r = c.post("/unit/%d/add" % pid4, data={"unit_type": "火山口",
    "area": "1.0"},
              follow_redirects=True)
with app.app_context():
    n3 = MicroUnit.query.filter_by(patch_id=pid4).count()
ok(n3 == 1, "非法单元类型被拒绝", "单元数 %d" % n3)
c.post("/unit/%d/add" % pid4, data={"unit_type": "碎石边坡",
    "area": "3.0"},
       follow_redirects=True)
with app.app_context():
    uid = MicroUnit.query.filter_by(patch_id=pid4,
                                    unit_type="碎石边坡").first(
    ).id
r = c.post("/unit/edit/%d" % uid,
              data={"unit_type": "平台", "area": "3.0"},
    follow_redirects=True)
with app.app_context():
    types = sorted(u.unit_type for u in
```

```
                    MicroUnit.query.filter_by(patch_id=pid4).
    all())
ok(types == ["平台", "碎石边坡"], "改类型时同样拦截重复", "、".join(types))
cleanup(pid4, "D")
print()
print(LINE)
if _fails:
    print("存在未通过项：")
    for f in _fails:
        print("  -", f)
    sys.exit(1)
print("流程门禁自检全部通过 √")
print(LINE)
# ==== 测试/test_weights.py ====
# -*- coding: utf-8 -*-
"""权重生效链路自检：AHP 算出的权重必须真正参与评价计算。
此前 weight_schemes / weight_items 两张表从未被写入，评价引擎也只用
代码里的论文表 3 默认权重，相当于权重模块算完即弃。本测试守住这条链路：
    默认 → 论文表 3
    保存 AHP 权重 → 评价改用新权重，结果随之变化
    恢复默认 → 结果回到论文取值
"""
import os
import sys
sys.stdout.reconfigure(encoding="utf-8")
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
from app.models import (db, Project, MinePatch, MicroUnit,
    IndicatorData,
                        ConstraintCheck, EvaluationResult,
    WeightScheme,
                        WeightItem)
from app.services import workflow as wf
from app.services import weight_service as ws
LINE = "=" * 72
BASE = ROOT
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
c = app.test_client()
```

```
c.post("/login", data={"username": "admin", "password":
    "admin123"})
_fails = []
def ok(flag, label, extra=""):
    print("  %s %-46s %s" % ("[符合]" if flag else "[不符合]",
    label, extra))
    if not flag:
        _fails.append(label)
TAG = "W"
UNIT_TYPE = "碎石边坡"
def build_case():
    """建一个只含单一微地质单元的图斑，便于观察权重变化对得分的影响。"""
    with app.app_context():
        p = Project(name="权重测试-%s" % TAG, code="WEIGHT-%s"
    % TAG)
        db.session.add(p); db.session.flush()
        patch = MinePatch(project_id=p.id, name="权重测试图斑-%s"
    % TAG)
        db.session.add(patch); db.session.flush()
        db.session.add(MicroUnit(patch_id=patch.id,
    unit_type=UNIT_TYPE,
                                 area=1.0, ratio=1.0))
        db.session.flush()
        u = MicroUnit.query.filter_by(patch_id=patch.id).fir
    st()
        db.session.add(IndicatorData(
            unit_id=u.id, soil_ratio=0.34, thickness=10,
    ph=9.24,
            organic_matter=4.89, coverage=87.92,
    shannon=0.60,
            slope=28.75, aspect="阳坡"))
        db.session.add(ConstraintCheck(
            patch_id=patch.id, policy_pass=True,
    location_pass=True,
            pollution_pass=True, geo_pass=True,
    farmland_pass=True,
            passed=True, reason="全部通过"))
        db.session.commit()
        return patch.id
def purge_fixtures():
    """开跑前清掉上一次可能残留的自检项目（code 前缀 WEIGHT-）。
    上一次跑到一半被打断时清理函数执行不到，残留项目会出现在工作台与
    项目列表里，生成软著截图时会被拍进去。
    """
    with app.app_context():
        from app.models import Project, ReferenceEcosystem
        leftovers = Project.query.filter(Project.code.like("
    WEIGHT-%")).all()
        for p in leftovers:
            for m in p.patches.all():
```

```
                uids = [u.id for u in
    MicroUnit.query.filter_by(patch_id=m.id)]
                if uids:
                    IndicatorData.query.filter(
                        IndicatorData.unit_id.in_(uids)).del
    ete(
                        synchronize_session=False)
                EvaluationResult.query.filter_by(patch_id=m
    id).delete()
                ConstraintCheck.query.filter_by(patch_id=m.
    d).delete()
                ReferenceEcosystem.query.filter_by(patch_id=
    m.id).delete()
                MicroUnit.query.filter_by(patch_id=m.id).del
    ete()
                db.session.delete(m)
            db.session.delete(p)
        db.session.commit()
        return len(leftovers)
def cleanup(pid):
    with app.app_context():
        from app.models import ScoreDetail
        rids = [r.id for r in EvaluationResult.query.filter_
    by(patch_id=pid)]
        if rids:
            ScoreDetail.query.filter(ScoreDetail.result_id.i
    n_(rids)).delete(
                synchronize_session=False)
        EvaluationResult.query.filter_by(patch_id=pid).delet
    e()
        ConstraintCheck.query.filter_by(patch_id=pid).delete
    ()
        uids = [u.id for u in MicroUnit.query.filter_by(patc
    h_id=pid)]
        if uids:
            IndicatorData.query.filter(IndicatorData.unit_id
    .in_(uids)).delete(
                synchronize_session=False)
        MicroUnit.query.filter_by(patch_id=pid).delete()
        m = db.session.get(MinePatch, pid)
        if m:
            db.session.delete(m)
        p = Project.query.filter_by(code="WEIGHT-%s" %
    TAG).first()
        if p:
            db.session.delete(p)
        db.session.commit()
def run_eval(pid):
    c.post("/evaluate/%d" % pid, data={},
    follow_redirects=False)
```

```
    with app.app_context():
        return EvaluationResult.query.filter_by(patch_id=pid
    ) \
            .order_by(EvaluationResult.id.desc()).first().fi
    nal_score
print(LINE)
print("一、默认应使用论文表 3 的权重")
print(LINE)
print("  （开跑前清掉 %d 个上一次残留的自检项目）" % purge_fixtures())
with app.app_context():
    ws.reset_weights(UNIT_TYPE)
    w, label, scheme = ws.effective_weights(UNIT_TYPE)
ok(scheme is None and "论文表" in label, "未配置时权重来源为论文表 3",
    label)
ok(abs(w["l1"]["土壤指标"] - 0.44) < 1e-9, "一级权重取自论文表 3",
   "土壤指标 %.2f" % w["l1"]["土壤指标"])
pid = build_case()
base_score = run_eval(pid)
print("  默认权重下的评价值 F = %.4f" % base_score)
print()
print(LINE)
print("二、保存 AHP 算出的权重后，评价应改用新权重")
print(LINE)
with app.app_context():
    # 模拟 AHP 计算结果：把土壤指标权重提高、植被与地形降低（仍和为 1）
    ws.save_weights(UNIT_TYPE, [("soil", 0.70), ("veg",
    0.20),
                                ("terrain", 0.10)])
    n_items = WeightItem.query.count()
ok(n_items >= 3, "weight_items 表已写入权重条目", "%d 条" % n_items)
with app.app_context():
    w2, label2, scheme2 = ws.effective_weights(UNIT_TYPE)
ok(scheme2 is not None and "自定义" in label2, "权重来源变为自定义方案",
    label2)
ok(abs(w2["l1"]["土壤指标"] - 0.70) < 1e-9, "新权重已生效",
   "土壤指标 %.2f" % w2["l1"]["土壤指标"])
ok(abs(w2["l1"]["植被指标"] - 0.20) < 1e-9, "同批保存的其他项也已生效")
ok(abs(w2["l2"]["土壤指标"]["土石比"] - 0.25) < 1e-9,
   "未覆盖的二级权重仍保留论文取值",
   "土石比 %.2f" % w2["l2"]["土壤指标"]["土石比"])
new_score = run_eval(pid)
print("  新权重下的评价值 F = %.4f" % new_score)
ok(abs(new_score - base_score) > 1e-4,
   "权重变化确实改变了评价结果",
   "%.4f → %.4f" % (base_score, new_score))
print()
print(LINE)
print("三、结果应记录所用权重来源")
print(LINE)
import json
```

```
with app.app_context():
    rec = EvaluationResult.query.filter_by(patch_id=pid) \
        .order_by(EvaluationResult.id.desc()).first()
    stored = json.loads(rec.unit_scores)
ok(any(u.get("weight_source") for u in stored),
   "评价记录中标注了权重来源",
   stored[0].get("weight_source") or "")
print()
print(LINE)
print("四、恢复默认后应回到论文取值")
print(LINE)
with app.app_context():
    ws.reset_weights(UNIT_TYPE)
    w3, label3, scheme3 = ws.effective_weights(UNIT_TYPE)
ok(scheme3 is None and "论文表" in label3, "已恢复为论文表 3 权重",
    label3)
back_score = run_eval(pid)
print("  恢复后评价值 F = %.4f（默认时 %.4f）" % (back_score,
    base_score))
ok(abs(back_score - base_score) < 1e-9, "评价结果与默认时一致")
print()
print(LINE)
print("五、权重首页概览")
print(LINE)
with app.app_context():
    rows = ws.summary()
ok(len(rows) == 3, "三种微地质单元均有权重来源说明", "%d 行" % len(rows))
r = c.get("/ahp/")
body = r.get_data(as_text=True)
ok(r.status_code == 200 and "当前生效权重" in body, "权重首页正常展示")
print()
print(LINE)
print("六、一致性不通过的矩阵不得保存生效")
print(LINE)
with app.app_context():
    ws.reset_weights(UNIT_TYPE)
n = 3
# 明显矛盾的矩阵：a12=9, a13=1/9, a23=9 —— CR 必然超限
mats = [(0, 1, "9"), (0, 2, "0.1111"), (1, 2, "9")]
data = {"csrf_token": "", "action": "save"}
for i, j, v in mats:
    data["m_%d_%d" % (i, j)] = v
r = c.post("/ahp/matrix?unit_type=%s&level=1" % UNIT_TYPE,
    data=data,
           follow_redirects=True)
with app.app_context():
    _, label4, scheme4 = ws.effective_weights(UNIT_TYPE)
ok(scheme4 is None, "CR 超限的矩阵未被保存生效", label4)
ok("一致性" in r.get_data(as_text=True), "页面给出了未通过一致性的提示")
cleanup(pid)
```

```
    print()
    print(LINE)
    if _fails:
        print("存在未通过项：")
        for f in _fails:
            print("  -", f)
        sys.exit(1)
    print("权重生效链路自检全部通过 √")
    print(LINE)
# ==== app/services/__init__.py ====
# -*- coding: utf-8 -*-
# ==== app/services/cleanup.py ====
# -*- coding: utf-8 -*-
"""级联清理：删图斑、删项目时，把挂在它下面的数据一并清掉。
为什么单独抽出来：`Project.patches` 与 `MinePatch.units` 上有 ORM 级联，
但图斑下面的**指标数据、限制性判定、评价结果、得分明细、参照生态系统**
都没有级联关系。原先只有「删除图斑」那条路径里手写了清理，
于是「删除项目」会留下一批孤儿行——自检实测：删一个项目后，
上面五张表各残留 1 行（项目与图斑本身都删掉了，数据却还在库里）。
两条删除路径都调用这里，才不会漏。
"""
from app.models import (db, MinePatch, MicroUnit,
    IndicatorData,
                        EvaluationResult, ScoreDetail,
    ConstraintCheck,
                        ReferenceEcosystem)
def delete_patch_children(patch_id):
    """清掉挂在某个图斑下的全部附属数据（不含图斑本身）。
    返回各表清理条数，便于调用方打印或断言。
    """
    counts = {}
    unit_ids = [u.id for u in MicroUnit.query.filter_by(patch_id=patch_id).all()]
    if unit_ids:
        counts["indicator_data"] = IndicatorData.query.filter(
            IndicatorData.unit_id.in_(unit_ids)).delete(synchronize_session=False)
    result_ids = [r.id for r in
                  EvaluationResult.query.filter_by(patch_id=patch_id).all()]
    if result_ids:
        counts["score_details"] = ScoreDetail.query.filter(
            ScoreDetail.result_id.in_(result_ids)).delete(synchronize_session=False)
    counts["evaluation_results"] = EvaluationResult.query.filter_by(
        patch_id=patch_id).delete(synchronize_session=False)
    counts["constraint_checks"] = ConstraintCheck.query.filter_by(
```

```
            patch_id=patch_id).delete(synchronize_session=False)
        counts["micro_units"] = MicroUnit.query.filter_by(
            patch_id=patch_id).delete(synchronize_session=False)
        counts["reference_ecosystems"] = ReferenceEcosystem.quer
        y.filter_by(
            patch_id=patch_id).delete(synchronize_session=False)
        return counts
def delete_patch(patch_id):
    """删除一个图斑及其全部附属数据。图斑不存在时返回 (None, {})。"""
    patch = db.session.get(MinePatch, patch_id)
    if patch is None:
        return None, {}
    counts = delete_patch_children(patch_id)
    db.session.delete(patch)
    db.session.commit()
    return patch, counts
def delete_project(project):
    """删除一个项目：先清掉每个图斑的附属数据，再删项目本身。
    项目下的图斑由 `Project.patches` 的 ORM 级联删除，但图斑下面那五张表
    必须先手工清，否则会留孤儿。
    """
    cleaned = 0
    for patch in project.patches.all():
        counts = delete_patch_children(patch.id)
        cleaned += sum(counts.values())
    name = project.name
    db.session.delete(project)
    db.session.commit()
    return name, cleaned
# ==== app/services/unit_config.py ====
# -*- coding: utf-8 -*-
"""标准化参数与必填项：让界面上能改的东西真正参与计算。
以前有个说不通的地方：图斑的「参照生态系统基准」、指标定义里的「理想取值」
和「启用单元」在界面上都能改、也能落库，**但计算链只读代码常量**
（`app/utils/constants.py`），从不查这两张表。于是报告里写着「基准 60%」，
实际按常量 75% 算，同一份报告两个口径。
现在统一从这里取：
- 基准值（植被覆盖度、Shannon）来自图斑的 `reference_ecosystems`；
- 各指标的理想取值来自 `indicators.ideal_value`（能解析成数字的才覆盖，
  像「5~9」「25° 分界」这种区间 / 规则型仍按代码里的算法处理）；
- 启用单元来自 `indicators.enabled_units`（岩质边坡无土壤、平台无地形）。
库里没有行、或值不是数字时一律回落到代码常量 —— 所以历史数据与
`验证数据/` 里的 8 个算例结果不会因为这次改动而变化。
"""
from app.models import Indicator, ReferenceEcosystem
from app.utils.constants import (
    UNIT_TYPES, IDEAL_SOIL_RATIO, IDEAL_THICKNESS,
    IDEAL_ORGANIC,
    IDEAL_COVERAGE, IDEAL_SHANNON, HEAVY_METAL_LIMIT,
    HEAVY_METALS,
```

```
)
# 二级指标编码 → 标准化参数名（同时也是 IndicatorData 的字段名）
NUMERIC_IDEALS = {
    "soil_ratio": ("soil_ratio", IDEAL_SOIL_RATIO),
    "thickness": ("thickness", IDEAL_THICKNESS),
    "organic_matter": ("organic_matter", IDEAL_ORGANIC),
    "coverage": ("coverage", IDEAL_COVERAGE),
    "shannon": ("shannon", IDEAL_SHANNON),
}
# 重金属编码 → 金属名
METAL_CODES = {"hm_" + m: m for m in HEAVY_METALS}
# 必填的实测字段（污染指标按论文注记允许「未检测」，故不在必填之列）
REQUIRED_CODES = ["soil_ratio", "thickness", "ph",
    "organic_matter",
                  "coverage", "shannon", "slope", "aspect"]
# 字段 → 界面上的中文名，用于提示「缺什么」
FIELD_LABELS = {
    "soil_ratio": "土石比", "thickness": "土层厚度", "ph": "pH",
    "organic_matter": "有机质", "coverage": "植被覆盖度",
    "shannon": "生物多样性（Shannon）", "slope": "坡度", "aspect":
    "坡向",
}
def _numeric(text):
    """把理想取值的文本解析成数字；区间、规则型返回 None。"""
    if text is None:
        return None
    try:
        return float(str(text).strip())
    except (TypeError, ValueError):
        return None
class UnitSettings:
    """一个图斑上下文的标准化参数。空对象表示「全部按代码常量」。"""
    def __init__(self, params=None, enabled=None):
        self.params = params or {}              # 覆盖用的标准值
        self.enabled = enabled or {}            # 单元类型 →
    启用的二级指标编码集合
    def ideal(self, code, fallback):
        """取某指标的理想取值：库里有数字就用库里的，否则用常量。"""
        value = self.params.get(code)
        return fallback if value is None else value
    def group_enabled(self, unit_type, l1_name, l1_code):
        """该单元类型下某个一级指标是否启用（库里没配就按代码常量）。"""
        codes = self.enabled.get(unit_type)
        if codes is None:
            return None                         # 交给调用方按常量判断
        return l1_code in codes
    def enabled_codes(self, unit_type):
        """该单元类型启用的全部二级指标编码；库里没配返回 None。"""
        return self.enabled.get(unit_type)
    def required_codes(self, unit_type):
```

```
        """该单元类型必填的实测字段。                                                13101
        按「启用的二级指标」推导：岩质边坡不启用土壤，平台不启用地形。                13102
        库里没配该项时用代码常量里的结构性规则。                                    13103
        """                                                                         13104
        from app.utils.constants import                                             13105
    UNIT_ENABLED_INDICATORS                                                         13106
        codes = self.enabled.get(unit_type)                                         13107
        if codes is None:                                                           13108
            groups = UNIT_ENABLED_INDICATORS.get(unit_type,                         13109
    ())                                                                             13110
            l2_by_group = {                                                         13111
                "土壤指标": ["soil_ratio", "thickness", "ph",                       13112
    "organic_matter"],                                                              13113
                "植被指标": ["coverage", "shannon"],                                13114
                "地形指标": ["slope", "aspect"],                                    13115
            }                                                                       13116
            codes = {c for g in groups for c in                                     13117
    l2_by_group.get(g, [])}                                                         13118
        return [c for c in REQUIRED_CODES if c in codes]                            13119
    def missing_fields(self, unit_type, data):                                      13120
        """返回该单元缺哪些必填项（中文名列表）。"""                                13121
        missing = []                                                                13122
        for code in self.required_codes(unit_type):                                 13123
            value = data.get(code)                                                  13124
            if value is None or (isinstance(value, str) and                         13125
    value.strip() == ""):                                                           13126
                missing.append(FIELD_LABELS.get(code, code))                        13127
        return missing                                                              13128
    def undetected_metals(self, data):                                              13129
        """按论文注记，未检测的重金属按「未受污染」计入；这里列出是哪些。"""        13130
        measured = data.get("heavy_metals") or {}                                   13131
        return [m for m in HEAVY_METALS if measured.get(m)                          13132
    is None]                                                                        13133
def for_patch(patch_id):                                                            13134
    """读某个图斑的标准化参数；库里有什么就用什么，没有就用常量。                    13135
    优先级：图斑自己的参照生态系统基准  **高于**  指标定义里的全局理想取值  ——      13136
    前者说的是「这个图斑的参照生态系统是什么样」，后者是「一般情况下的理想值」，    13137
    越具体的越应该说了算。两者都优先于代码常量。                                    13138
    """                                                                             13139
    params = {}                                                                     13140
    enabled = {}                                                                    13141
    for ind in Indicator.query.all():                                               13142
        code = ind.code                                                             13143
        if code in NUMERIC_IDEALS:                                                  13144
            value = _numeric(ind.ideal_value)                                       13145
            if value is not None:                                                   13146
                params[code] = value                                                13147
        elif code in METAL_CODES:                                                   13148
            #  重金属的「理想值」存的是限值，作为越小越优的极值使用                13149
            value = _numeric(ind.ideal_value)                                       13150
```

```
                if value is not None:
                    params[code] = value
            units = (ind.enabled_units or "").strip()
            if units:
                for unit in UNIT_TYPES:
                    if unit in units:
                        enabled.setdefault(unit,
        set()).add(code)
        #  再用该图斑的参照生态系统基准覆盖：它是「这个图斑的参照生态系统」，
        #  比指标定义里的全局理想取值更具体，应该它说了算。
        ref = ReferenceEcosystem.query.filter_by(patch_id=patch_
        id).first()
        if ref is not None:
            if ref.coverage_benchmark is not None:
                params["coverage"] = float(ref.coverage_benchmar
        k)
            if ref.shannon_benchmark is not None:
                params["shannon"] = float(ref.shannon_benchmark)
        return UnitSettings(params=params, enabled=enabled)
def metal_limits(settings):
        """重金属的极值表：库里有配置的用库里的，其余用常量。"""
        limits = dict(HEAVY_METAL_LIMIT)
        for code, metal in METAL_CODES.items():
            value = settings.params.get(code) if settings else
        None
            if value is not None:
                limits[metal] = value
        return limits
#  ==== app/utils/__init__.py ====
#  -*- coding: utf-8 -*-
#  ==== app/utils/parsing.py ====
#  -*- coding: utf-8 -*-
"""表单数值的安全解析。
背景：页面上的数字输入框原来写成 `float(request.form.get("area"))`，
用户填「约 3.5」或全角数字就会撞上 ValueError —— 页面上是默认英文 500 页。
真人演示时这是最容易踩到的一类。统一走这里：
没填返回 None，填了但不是数字抛 FieldError，让视图 flash 一句中文再回退。
"""
import math
class FieldError(ValueError):
        """表单里某个字段填得不对，携带给用户看的中文提示。"""
def parse_float(raw, field, *, minimum=None, maximum=None):
        """把表单值解析成 float。
        raw       表单原始值（可能是 None / "" / "abc" / "1e400"）
        field     字段名，用于拼提示
        minimum / maximum   可选的取值范围，越界同样抛 FieldError
        """
        if raw is None:
            return None
        text = str(raw).strip()
```

```
    if text == "":
        return None
    try:
        value = float(text)
    except (TypeError, ValueError):
        raise FieldError("「%s」请填数字" % field)
    # nan / inf 能通过 float()，但存进库里会把标准化算成满分，必须挡住
    if not math.isfinite(value):
        raise FieldError("「%s」请填正常的数字" % field)
    if minimum is not None and value < minimum:
        raise FieldError("「%s」不能小于 %g" % (field, minimum))
    if maximum is not None and value > maximum:
        raise FieldError("「%s」不能大于 %g" % (field, maximum))
    return value
def parse_int(raw, field, *, default=None, minimum=None,
    maximum=None):
    """解析整数，用于查询参数与计数类字段。"""
    if raw is None:
        return default
    text = str(raw).strip()
    if text == "":
        return default
    try:
        value = int(text)
    except (TypeError, ValueError):
        raise FieldError("「%s」请填整数" % field)
    if minimum is not None and value < minimum:
        value = minimum
    if maximum is not None and value > maximum:
        value = maximum
    return value
# ==== app/utils/timefmt.py ====
# -*- coding: utf-8 -*-
"""时间显示：库里一律存 UTC，展示时换成服务器本地时间。
为什么不在写库时直接用本地时间：一个系统里时间基准只能有一个。
`app/models.py` 里所有 `created_at` / `checked_at` 都用
    `datetime.utcnow`，
存的是 UTC；如果写库改成本地时间，已经存进库的几千条记录会新旧混着差 8 小时。
所以统一在「展示」这一层换算，老数据不用动。
用法（Jinja）：
    {{ row.created_at | localdt }}                  →
    2026-09-26 23:33
    {{ row.created_at | localdt("%Y-%m-%d %H:%M:%S") }} →
    2026-09-26 23:33:28
"""
from datetime import datetime, timezone
DEFAULT_FORMAT = "%Y-%m-%d %H:%M"
def to_local(value, fmt=DEFAULT_FORMAT, empty="—"):
    """把 UTC 时间换算成服务器本地时间并格式化。
    传进来的值按「无时区即 UTC」处理；带时区的原样换算，便于以后改用
```

```
    带时区的时间戳。空值返回占位符，免得页面上出现 None。
    """
    if value is None:
        return empty
    if value.tzinfo is None:
        value = value.replace(tzinfo=timezone.utc)
    return value.astimezone().strftime(fmt)
def now_utc():
    """取当前 UTC 时间（带时区），用于只作展示、不落库的时间戳。"""
    return datetime.now(timezone.utc)
def utcnow():
    """取当前 UTC 时间（无时区），用于数据库默认值。
    `datetime.utcnow()` 在 Python 3.12 已标记弃用，导入时会刷告警；
    这里用带时区的 now() 再摘掉时区，得到的仍是 naive-UTC，行为一致。
    """
    return datetime.now(timezone.utc).replace(tzinfo=None)
# ==== app/views/__init__.py ====
# -*- coding: utf-8 -*-
```